



Optimistic Rollup L2 기술분석보고서

- OP Stack 을 중심으로 -

2026.04

Gurufin Research Center

조광현

Tillo Tulkinbekov

목차

OP STACK	4
서론 (INTRODUCTION)	4
모듈형 아키텍처 (MODULAR ARCHITECTURE)	4
핵심 기술 메커니즘 (KEY TECHNICAL MECHANISMS)	4
OPTIMISM 노드 아키텍처	5
CANNON 증명 엔진	7
데이터 최적화: EIP-4844 와 BLOB 도입	7
슈퍼체인 비전 (THE SUPERCHAIN VISION)과 상호운용성 (INTEROPERABILITY)	7
생태계 및 활용 사례 (ECOSYSTEM & USE CASES)	8
결론 및 향후 전망: ZK-OP STACK 으로의 진화	8
OPTIMISM 저장소의 핵심 구성요소	8
OP-NODE - 실행	11
용어: “실행”의 위치	11
전체 역할 요약	11
ENGINE CONTROLLER 의 EVENT SYSTEM	14
SYNCSTATUS	17
OP-SEQUENCER	21
설명	21
OP-SEQUENCER 가 강제로 블록 앞에 삽입하는 트랜잭션	21
OP-GETH 의 USER TXS 를 포함여부	22
SEQUENCER 와 ENGINCONTROLLER 의 역할 분담	23
GOSSIP 의 역할	23
OP-NODE 에서 L2 블록 생성	24
L2 체인에서 사용자 트랜잭션 수신	34
FINDL1ORIGIN 함수 역할	37
OP-CONDUCTOR 를 이용한 멀티 시퀀서 구성	40
DEPOSITTXS 생성 과정	43
DEPOSITTRANSACTION 에서 ETH 처리와 ETHLOCKBOX 구현	47
OP_NODE: OP_BATCHER	51
설명	51
유도 (DERIVATION)	52
FINDL1ORIGIN 함수의 역할	52
OP-BATCHER L1 트랜잭션 전송 흐름	52
OP_NODE: DERIVATION	57
설명	57
상세 프로세스	58
PIPELINEDERIVER	67
DERIVATIONPIPELINE 이 ATTRIBUTESWITHPARENT 를 만들어 EL(OP-GETH)에 전달	69
SEQUENCER 와 DERIVATIONPIPELINE 의 ATTRIBUTESWITHPARENT 생성 비교	72
BLOB 데이터로부터 ETH.PAYLOADATTRIBTTES 생성 과정	75
ATTRIBUTES 생성 과정	76
CONSOLIDATION	78
L1 BLOCK 의 SAFE, FINALIZED 상태에 따라 L2 블록 상태 변경	82
OP_PROPOSER	85

개요	85
핵심 기능	85
DISPUTE GAME 구현체	89
DISPUTE GAME PROXY	92
DISPUTE GAME PROXY CLONE (CLONES WITH IMMUTABLE ARGS)	93
메인 루프: 폴링 → 제안 여부 → L1 전송	96
OP-PROPOSER 에서 DISPUTE GAME FACTORY 의 역할과 작동 순서	99
OUTPUT ROOT 형성: 왜 OP-NODE RPC 인가	103
7 일 후 변경되는 값과 프로세스	106
전체 WITHDRAWAL 프로세스 타임라인	107
OP-PROPOSER 가 제출하는 트랜잭션	108
BOND 제출 방법과 금액	111
OP-CHALLENGER	112
설명	112
OP-CHALLENGER 의 동작 흐름	113
METADATA 와 CLAIM DATA []를 로드하는 방법	118
챌린지 게임 엔진 CANNON	122
OP-CHALLENGER 의 RESOLVE 프로세스	123
CANNON: MIPS 온체인 에뮬레이터	133
CANNON 의 역할과 위치	133
전체 아키텍처 개요	134
이분법 분쟁 게임 (BISECTION GAME)	135
OP-PROGRAM: 폴트 증명 프로그램	136
오프체인 에뮬레이터: MIPSEVM	138
온체인 에뮬레이터: MIPS64.SOL	139
WITNESS 데이터의 3 가지 구성요소	141
PREIMAGE ORACLE	142
OP-CHALLENGER 와의 연동	143
CANNON CLI 사용 흐름	144
요약: CANNON 의 핵심 아이디어	145
WITHDRAW	146
개요	146
출금 프로세스 5 단계 요약	146
핵심 소스 코드	148
PROVE WITHDRAWAL TRANSACTION 4 개 인자 획득 방법	155
DISPUTE GAME 인덱스 조회 방법	166
BLOCKNUMBER: L2 TO L1 MESSAGE PASSER. INITIATE WITHDRAWAL VS EXTRA DATA	177
ETH_GETPROOF 응답의 STORAGE HASH 와 STORAGE PROOF	179
WITHDRAWAL 에 해당하는 DISPUTE GAME 을 찾는 방법	183
RESPECTED GAME TYPE	186
INTEROP	190
설명	190
IDENTIFIER — 메시지 포인터	190
메시지 전달 방식: SOURCE → DESTINATION	191
OP_SUPERVISOR	192
전체 흐름 요약 (코드 레벨)	197
ANVIL (L1) + OPSTACK (L2) INSTALLATION	198
SETUP	198

OP-DEPLOYER	200
SEQUENCER	220
BATCHER	229
PROPOSER	235
CHALLENGER	245
CLI	247
FUND L2	247
TX ON L2	250

OP Stack

서론 (Introduction)

OP Stack 은 Optimism 생태계를 지탱하는 오픈 소스 모듈형 소프트웨어 스택입니다. 이는 단순히 하나의 레이어 2(L2)를 구축하는 도구를 넘어, 수많은 L2 체인들이 표준화된 규격을 공유하며 거대한 '슈퍼체인(Superc chain)'으로 통합될 수 있도록 설계되었습니다. OP Stack 의 핵심 철학은 모듈성(Modularity), 단순성(Simplicity), 그리고 확장성(Extensibility)에 있습니다.

모듈형 아키텍처 (Modular Architecture)

OP Stack 은 기능을 계층별로 분리하여, 개발자가 필요에 따라 특정 컴포넌트를 교체할 수 있는 유연한 구조를 가집니다.

- 데이터 가용성 계층 (Data Availability Layer): L2 트랜잭션의 원시 데이터를 저장하는 곳입니다. 기본적으로 이더리움 L1 을 사용하지만, 비용 절감을 위해서나 EIP-4844 이후에는 한시적으로만 저장되므로 Celestia 나 EigenDA 같은 외부 DA 솔루션을 선택적으로 결합할 수 있습니다.
 - 시퀀싱 계층 (Sequencing Layer): 사용자 트랜잭션을 수집하고 순서를 정해 L2 블록을 생성합니다. 현재는 단일 시퀀서 모델이 주를 이루지만, 향후 공유 시퀀싱(Shared Sequencing)을 통해 여러 체인이 트랜잭션을 공동 처리할 수 있도록 설계되었습니다.
유도 계층 (Derivation Layer): DA 계층에 기록된 데이터를 읽어와 실행 계층이 이해할 수 있는 포맷으로 변환합니다. 이 과정에서 L1 의 데이터를 기반으로 L2 의 상태를 재구성(Reconstruction)할 수 있는 결정론적(Deterministic) 특성을 보장합니다.
 - 실행 계층 (Execution Layer): 실제 상태 변화가 일어나는 곳입니다. op-geth 를 통해 이더리움 가상 머신(EVM)과 100% 호환되는 환경을 제공하며, 스마트 컨트랙트 실행 및 계정 상태 관리를 담당합니다.
 - 결제 계층 (Settlement Layer): L2 의 상태 루트(State Root)를 L1 에 기록하고, 출금 시 발생하는 분쟁을 해결합니다. 낙관적 롤업의 경우, 이 계층에서 사기 증명(Fraud Proof) 메커니즘이 작동합니다.
-

핵심 기술 메커니즘 (Key Technical Mechanisms)

- Bedrock 업그레이드와 이더리움 등가성 (Ethereum Equivalence): Bedrock 은 OP Stack 의 1 세대 아키텍처로, 이더리움 코드베이스와의 차이를 최소화했습니다. 이를 통해 이더리움의 인프라(Geth 등)를 그대로 활용하며 유지보수 비용을 획기적으로 줄였습니다.
- 낙관적 롤업 및 사기 증명 (Optimistic Rollup & Cannon): 모든 트랜잭션이 유효하다고 가정하고 즉시 처리하되, 도전 기간(Challenge Period) 동안 무효 실행을 이의 제기할 수 있습니다. Cannon 은 온체인에서 MIPS 명령어를 실행해 분쟁을 해결하며, 향후 증명 방식 확장을 고려한 설계입니다.

- L1-L2 브릿징 아키텍처: L1CrossDomainMessenger 와 L2CrossDomainMessenger 컨트랙트를 통해 안전한 메시지 전송을 지원합니다. 입금(Deposit)은 L1 에서 이벤트를 감지하여 L2 에서 민팅하는 방식이며, 출금(Withdrawal)은 7 일의 대기 시간 후 L1 에서 증명을 제출하여 완료됩니다.

Optimism 노드 아키텍처

Optimism 노드 종류

1. Sequencer (시퀀서 / 리더 노드)

- 네트워크당 1 개만 존재
- 새로운 L2 블록 생성 및 트랜잭션 순서 결정
- L1 에 배치 데이터 제출
- "Unsafe" 블록을 먼저 생성

2. Follower/Verifier Nodes (팔로워/검증 노드)

- 무제한으로 운영 가능
- L1 데이터로부터 L2 상태를 독립적으로 유도(derive)
- 시퀀서의 작업을 검증
- DerivationPipeline 을 통해 L1→L2 상태 재구성

3. Replica Nodes (레플리카 노드)

- P2P gossip 을 통해 시퀀서로부터 빠르게 동기화
- L1 derivation 없이 시퀀서 데이터 직접 수신 (빠른 동기화)

1. Sequencer (시퀀서) - 단일 리더

네트워크에서 1 개만 존재하며, 다음 역할을 수행합니다:

- 새로운 L2 블록 생성 (Unsafe blocks)
- 트랜잭션 순서 결정
- P2P gossip 으로 다른 노드에 블록 전파
- L1 에 배치 데이터 제출

2. Verifier/Follower Nodes (검증 노드) - 무제한

누구나 운영 가능하며:

- L1 데이터로부터 L2 상태를 독립적으로 유도(derivation)
- 시퀀서의 작업을 검증
- DerivationPipeline 을 통해 L1→L2 상태 재구성

3. P2P Gossip 을 통한 동기화

모든 노드는 P2P gossip 네트워크를 통해 블록 및 트랜잭션 데이터를 전파하고 동기화합니다.

- 각 노드는 새로 생성되거나 수신한 블록을 libp2p 기반 gossip 프로토콜을 통해 인접 노드들에게 브로드캐스트합니다.
- gossip 프로토콜은 메시지를 확률적으로 확산시키는 방식으로, 네트워크 전체에 빠르게 전파되면서도 과도한 중복 전송을 줄입니다.
- 노드는 수신한 블록의 유효성(Execution 결과, 상태 루트 등)을 검증한 뒤 로컬 체인에 반영합니다.
- 일부 노드가 일시적으로 오프라인 상태가 되더라도, 재접속 시 gossip 및 백필(backfill) 메커니즘을 통해 최신 상태로 빠르게 복구할 수 있습니다.
- 이 구조는 중앙 브로드캐스터 없이도 네트워크 전체의 데이터 일관성을 유지하도록 설계되어 있습니다.

4. HA (High Availability) 모드 - 다중 시퀀서

op-conductor 를 활용하면 다중 시퀀서 환경에서 고가용성(HA)을 구현할 수 있으며, 여러 시퀀서 노드 중 하나만 리더(leader)로 활성화됩니다.

- op-conductor 는 리더 선출(leader election) 및 상태 동기화를 관리하여 단일 활성 시퀀서 구조를 유지합니다.
- 리더 시퀀서는 트랜잭션을 수집하고 L2 블록을 생성하며, 이를 네트워크에 전파합니다.
- 나머지 시퀀서 노드들은 팔로워(follower)로 동작하며, 리더의 상태를 지속적으로 추적하고 동기화합니다.
- 리더 노드에 장애가 발생할 경우, op-conductor 가 자동으로 새로운 리더를 선출하여 서비스 중단을 최소화합니다.
- 리더 전환 시에도 체인 상태의 일관성을 유지하기 위해, 시퀀서 간 상태(unsafe/safe head 등)가 사전에 공유되고 검증됩니다.
- 이 구조는 단일 시퀀서 장애로 인한 네트워크 중단을 방지하고, 운영 안정성을 크게 향상시킵니다.

요약 표

노드 유형	개수	블록 생성	L1 Derivation	P2P Sync
Sequencer (리더)	1 개	✓	✓	발신
Verifier (팔로워)	무제한	✗	✓	수신
Standby Sequencer (HA)	N 개	대기 중	✓	수신

결론: Optimism 네트워크에는 1 개의 활성 시퀀서와 다수의 검증 노드가 존재합니다. 검증 노드들은 L1 데이터를 통해 독립적으로 L2 상태를 유도하고 시퀀서의 작업을 검증합니다.

Cannon 증명 엔진

OP Stack의 차세대 사기 증명(Fraud Proof) 시스템인 Cannon은 단순한 검증 도구를 넘어, '온체인 미니 컴퓨터' 역할을 수행합니다.

- MIPS 가상 머신 (MIPS VM): Cannon은 이더리움 L1(EVM) 위에서 동작하는 MIPS 아키텍처 에뮬레이터입니다. L2 트랜잭션 실행 중 분쟁이 발생하면, 실행 엔진(op-geth) 전체를 L1에서 돌리는 대신, 문제가 된 단 하나의 명령어(Instruction)를 MIPS 단위로 쪼개어 실행합니다.
- 이분법 게임 (Bisection Game): 공격자와 방어자는 수백만 개의 실행 단계 중 의견이 갈리는 지점을 찾기 위해 '이분법' 방식으로 범위를 좁혀 나갑니다. 최종적으로 단 하나의 명령어 단계에 도달하면, L1 컨트랙트가 해당 명령어를 직접 실행하여 누구의 상태 루트(State Root)가 맞는지 판결합니다.
- 기술적 의의: 이 방식은 실행 엔진의 복잡도와 상관없이 증명이 가능하게 합니다. 즉, 향후 실행 엔진이 geth에서 다른 언어로 바뀌더라도 MIPS 컴파일만 가능하다면 동일한 증명 시스템을 사용할 수 있는 '미래 지향적(Future-proof)' 설계입니다.

데이터 최적화: EIP-4844와 Blob 도입

이더리움의 Dencun 업그레이드(EIP-4844) 이후, OP Stack은 데이터 가용성(DA) 비용을 획기적으로 절감하는 구조로 진화했습니다.

- Blob-carrying Transactions: 과거에는 L2 데이터를 L1의 calldata 영역에 저장하여 높은 가스비를 지불했습니다. 이제 OP Stack은 이더리움의 새로운 데이터 저장 공간인 'Blob'을 활용합니다. Blob은 일정 기간(약 18일) 후 삭제되므로 저장 비용이 매우 저렴합니다.
- Batchers의 역할 변화: OP Stack의 op-batcher 컴포넌트는 이제 L2 트랜잭션 묶음을 Blob 형태로 패키징하여 L1에 제출합니다. 이를 통해 L2 사용자들의 가스 수수료가 기존 대비 90% 이상 절감되었습니다.
- Point Evaluation Precompile: L1 결제 계층에서는 Blob 데이터 전체를 읽지 않고도, 해당 데이터가 올바르게 제출되었는지 확인하기 위해 '포인트 평가(Point Evaluation)' 프리컴파일을 사용합니다. 이는 데이터의 무결성을 보장하면서도 검증 비용을 최소화하는 핵심 기술입니다.

슈퍼체인 비전 (The Superchain Vision)과 상호운용성 (Interoperability)

OP Stack의 궁극적인 목표는 개별 체인들의 파편화를 막는 것입니다.

- 표준화된 체인 ID 및 설정: 모든 OP Stack 체인은 동일한 L1StandardBridge 규격을 공유합니다. 이는 서로 다른 L2 간의 자산 이동 시 별도의 신뢰 레이어가 필요 없는 '네이티브 브릿징' 환경을 구축합니다.

- 공유 시퀀싱의 기술적 도전: 여러 OP Stack 체인이 하나의 시퀀서 세트를 공유함으로써, 체인 A의 트랜잭션과 체인 B의 트랜잭션을 하나의 묶음으로 처리하는 '원자적 상호작용(Atomic Interop)'을 연구 중입니다.

생태계 및 활용 사례 (Ecosystem & Use Cases)

- Base (Coinbase): 코인베이스가 구축한 L2로, OP Stack의 안정성과 확장성을 증명한 대표적 사례입니다.
- World Chain (Worldcoin): 신원 인증 기반의 앱 특화 체인으로, OP Stack의 커스터마이징 기능을 활용해 사용자 경험을 최적화하고 있습니다.
- Zora & Mode: 창작자 경제 및 DeFi 특화 체인으로, OP Stack의 모듈성을 활용해 자체적인 인센티브 구조(Sequencer Fee Sharing)를 실험하고 있습니다.
- Manta Pacific
- Unichain
- 모듈형 변주: 개발자는 실행 엔진을 수정하여 게임 전용 체인을 만들거나, 가스비 결제 수단을 자체 토큰으로 변경하는 등 특정 목적에 맞는 'App-specific Chain'을 쉽게 구축할 수 있습니다.

결론 및 향후 전망: ZK-OP Stack으로의 진화

OP Stack의 가장 큰 강점은 '증명 추상화(Proof Abstraction)'입니다. 현재는 Cannon을 통한 낙관적 증명을 사용하지만, 아키텍처를 바꾸지 않고도 결제 계층의 증명 모듈만 ZK-VM(예: RISC Zero, Succinct)으로 교체하면 즉시 ZK-Rollup으로 전환될 수 있습니다. 이는 OP Stack이 단순한 유행이 아니라, 이더리움 확장성의 장기적인 표준이 될 것임을 시사합니다.

optimism 저장소의 핵심 구성요소

모노레포: github.com/ethereum-optimism/optimism.

1 한눈에 보는 디렉터리

이 optimism 모노레포는 OP Stack을 구동하는 클라이언트·도구·온체인 코드를 한곳에 둡니다.

구분	경로	역할 요약
롤업 합의	op-node	L1에서 배치·출력을 읽어 L2체인 헤드·유도(derivation)을 담당하는 롤업 합의 계층 클라이언트
실행	(별도) op-geth	EVM 호환 실행 클라이언트; 본 저장소와 함께 배포되는 실행 계층
L1 제출	op-batcher	L2 트랜잭션 배치를 L1에 제출(calldata-Blob-AltDA 등)

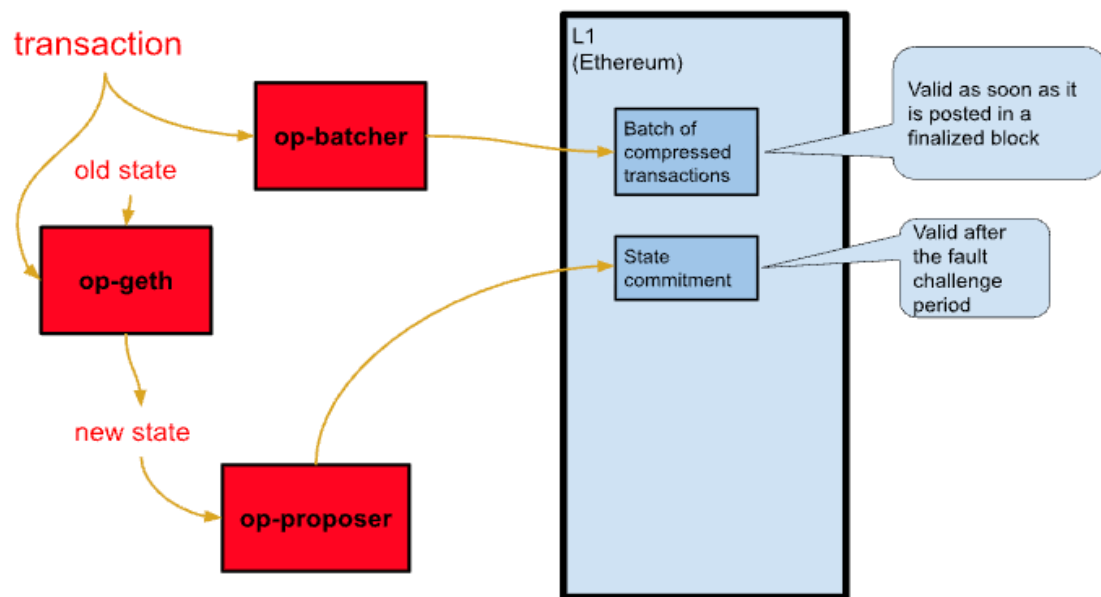
출력 제안	op-proposer	L2 상태 출력(Output Root)을 L1 에 제안
분쟁	cannon, op-challenger, op-program	MIPS 온체인 에뮬레이터, 분쟁 게임 챌린저, 폴트 프루프 프로그램
스마트 컨트랙트	packages/contracts-bedrock	Bedrock 세대 브릿지·출력·분쟁 등 L1/L2 온체인 로직
대안 DA	op-alt-da	이더리움 외 DA 모드(베타) 등 확장 옵션
배포·검증	op-deployer, op-validator	체인 배포·업그레이드 CLI, 설정·배포 검증

2 코드로 읽는 순서 (시리즈 들)

데이터가 L2 에서 L1 으로 올라가고, 다시 검증자에 의해 읽히는 흐름을 기준으로 보면 다음 순서가 자연스럽습니다.

순서	컴포넌트	다를 관점
1	op-batcher	시퀀서가 만든 L2 블록 내용을 압축·프레이밍해 L1 BatchInboxAddress 에 기록(DA)
2	op-node	L1 에서 배치 인박스 데이터를 읽어 유도(derivation), unsafe→safe 전환. L1→L2 Deposit Transaction 도 같은 유도 파이프라인에서 L1 블록·로그로부터 합성된다(배치가 보내는 배치와는 별도 경로).
3	op-proposer	L2 출력을 L1 에 제안(롤업 컨트랙트와 연동)
4	packages/contracts-bedrock	배치 인박스·출력·브릿지·분쟁 게임 등 온체인 인터페이스. 입금의 L1 발생 지점(예: Optimism Portal·L1 브릿지)은 여기서 보고, "L2 에 어떤 deposit tx 로 들어가는지"는 op-node 유도과 스펙으로 연결해 읽는 것이 자연스럽다.
5	cannon/op-challenger/op-program	출력·실행에 대한 폴트 프루프 경로
6	op-alt-da (선택)	L1 에는 커밋만, 본문은 외부

3 op_stack 구조



op-node - 실행

용어: "실행"의 위치

EVM 상태 전이(트랜잭션 실행)는 별도 저장소의 실행 클라이언트(op-geth 등)가 담당한다. op-node는 그 위에서 L1 데이터로부터 L2 블록 입력을 유도(derive)하고, 시퀀서 모드에서는 Engine API(예: engine_forkchoiceUpdated, engine_getPayload, engine_newPayload)로 실행 엔진을 조종한다. 즉 이 절의 "실행"은 롤업 노드가 EL(Execution Layer)을 구동·동기화한다는 의미에 가깝다.

전체 역할 요약

- 유도(Derivation): op-batcher가 L1에 올린 배치(calldata/blob/AltDA 커밋)를 읽어 프레임·채널·배치를 복원하고, PayloadAttributes로 바꾼 뒤 EL에 안전(safe) 체인을 맞춘다. Ecotone 이후 blob은 L1 트랜잭션에서 blob hash를 얻고 비콘 등에서 sidecar를 받아 복원하는 경로가 붙는다.
- 시퀀서(선택): 활성화 시 타이머·이벤트 루프로 블록 빌딩 작업을 돌리며, FindL1Origin → PreparePayloadAttributes(deposit-l1Info·업그레이드 트랜잭션 포함) → BuildStartEvent → EL이 payload를 조립·봉인합니다. MaxSequencerDrift를 넘으면 NoTxPool로 빈 블록(시스템/입금만)을 강제하는 등 L1과의 시간 관계를 맞춘다.
- 검증자(Verifier): 시퀀서 없이 유도 결과만 EL에 넣어 L2를 재생(replay)한다. 사용자 트랜잭션은 L1 배치에서 복원된 것만 포함되고, EL txpool 경로는 쓰지 않는다.
- L1→L2 입금: 포털 등 L1 로그에서 TransactionDeposited를 해석해 L2 DepositTx로 넣는 경로는 유도·속성 빌더 쪽과 맞물린다.

구성요소 배치(드라이버 관점)

NewDriver가 L1 추적, EngineController, DerivationPipeline, 동기화·파이널리티, (옵션) Sequencer를 이벤트 레지스트리에 올려 한 루프에서 함께 돌게 한다.

```
// op-node/rollup/driver/driver.go
// NewDriver composes an events handler that tracks L1 state, triggers L2 Derivation, and optionally sequences
new L2 blocks.
func NewDriver(
    sys event.Registry,
    drain Drain,
    driverCfg *Config,
    cfg *rollup.Config,
    l1ChainConfig *params.ChainConfig,
    depSet derive.DependencySet,
    l2 L2Chain,
```

```

l1 L1Chain,
upstreamFollowSource UpstreamFollowSource,
l1Blobs derive.L1BlobsFetcher,
altSync AltSync,
network Network,
log log.Logger,
metrics Metrics,
sequencerStateListener sequencing.SequencerStateListener,
safeHeadListener rollup.SafeHeadListener,
syncCfg *sync.Config,
sequencerConductor conductor.SequencerConductor,
altDA AltDAInterface,
indexingMode bool,
) *Driver {
    driverCtx, driverCancel := context.WithCancel(context.Background())

    statusTracker := status.NewStatusTracker(log, metrics)
    sys.Register("status", statusTracker)

    l1Tracker := status.NewL1Tracker(l1)

    l1 = metered.NewMeteredL1Fetcher(l1Tracker, metrics)
    verifyConfDepth := confdepth.NewConfDepth(driverCfg.VerifierConfDepth, statusTracker.L1Head, l1)

    ec := engine.NewEngineController(driverCtx, l2, log, metrics, cfg, syncCfg, l1, sys.Register("engine-controller",
nil))
    // TODO(#17115): Refactor dependency cycles
    ec.SetCrossUpdateHandler(statusTracker)

    var finalizer Finalizer
    if cfg.AltDAEnabled() {
        finalizer = finality.NewAltDAFinalizer(driverCtx, log, cfg, driverCfg.Finalizer, l1, altDA, ec)
    } else {
        finalizer = finality.NewFinalizer(driverCtx, log, cfg, driverCfg.Finalizer, l1, ec)
    }
    sys.Register("finalizer", finalizer)

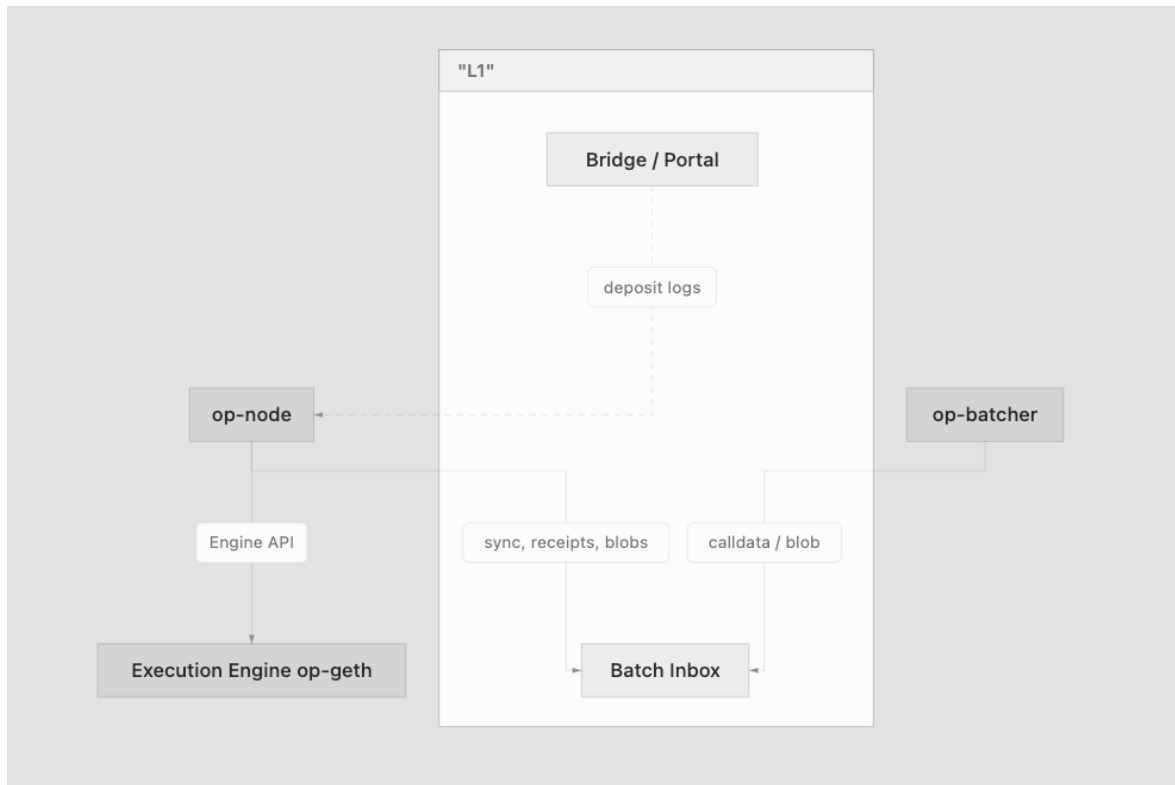
    attrHandler := attributes.NewAttributesHandler(log, cfg, driverCtx, l2, ec)
    sys.Register("attributes-handler", attrHandler)

    derivationPipeline := derive.NewDerivationPipeline(log, cfg, depSet, verifyConfDepth, l1Blobs, altDA, l2,
metrics, indexingMode, l1ChainConfig)

    pipelineDeriver := derive.NewPipelineDeriver(driverCtx, derivationPipeline)

```

```
sys.Register("pipeline", pipelineDriver)
```



유도 파이프라인 스테이지

L1 을 한 블록씩 밀어 넣으며 끝단에서 PayloadAttributes 를 뽑는 pull 기반 스테이지 체인입니다. NewDerivationPipeline 에서 연결 순서가 한눈에 드러난다.

```
// op-node/rollup/derive/pipeline.go
// NewDerivationPipeline creates a DerivationPipeline, to turn L1 data into L2 block-inputs.
func NewDerivationPipeline(log log.Logger, rollupCfg *rollup.Config, depSet DependencySet, l1Fetcher
L1Fetcher, l1Blobs L1BlobsFetcher,
    altDA AltDAInputFetcher, l2Source L2Source, metrics Metrics, managedBySupervisor bool, l1ChainConfig
*params.ChainConfig,
) *DerivationPipeline {
    spec := rollup.NewChainSpec(rollupCfg)
    // Stages are strung together into a pipeline,
    // results are pulled from the stage closed to the L2 engine, which pulls from the previous stage, and so
    on.
    var l1Traversal l1TraversalStage
    if managedBySupervisor {
        l1Traversal = NewL1TraversalManaged(log, rollupCfg, l1Fetcher)
    } else {
        l1Traversal = NewL1Traversal(log, rollupCfg, l1Fetcher)
    }
}
```

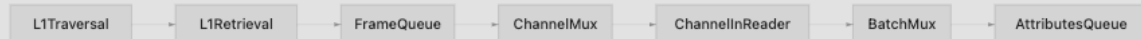
```

}
dataSrc := NewDataSourceFactory(log, rollupCfg, l1Fetcher, l1Blobs, altDA) // auxiliary stage for L1Retrieval
l1Src := NewL1Retrieval(log, dataSrc, l1Traversal)
frameQueue := NewFrameQueue(log, rollupCfg, l1Src)
channelMux := NewChannelMux(log, spec, frameQueue, metrics)
chInReader := NewChannelInReader(rollupCfg, log, channelMux, metrics)
batchMux := NewBatchMux(log, rollupCfg, chInReader, l2Source)
attrBuilder := NewFetchingAttributesBuilder(rollupCfg, l1ChainConfig, depSet, l1Fetcher, l2Source)
attributesQueue := NewAttributesQueue(log, rollupCfg, attrBuilder, batchMux)

// Reset from ResetEngine then up from L1 Traversal. The stages do not talk to each other during
// the ResetEngine, but after the ResetEngine, this is the order in which the stages could talk to each other.
// Note: The ResetEngine is the only reset that can fail.
stages := []ResetableStage{l1Traversal, l1Src, altDA, frameQueue, channelMux, chInReader, batchMux,
attributesQueue}

```

AttributesQueue 가 만드는 트랜잭션 리스트는 보통 L1 정보·입금·(필요 시) 업그레이드·배치에서 온 사용자 트랜잭션 순서를 따릅니다. 유도로만 확정된 블록은 pending safe / local safe 로 승격되는 계열과 엮인다.



Engine Controller 의 Event System

EngineController.OnEvent 실행 시점

EngineController.OnEvent 는 이벤트 시스템(Event System)에 의해 호출됩니다. 전체 흐름을 설명하겠다.

1. 이벤트 시스템 구조

op-node 는 이벤트 기반 아키텍처를 사용한다:

```

// op-service/event/events.go
type Deriver interface {
    // OnEvent runs the event with the context that was used to emit the event.
    // The context is managed by the emitter,
    // and may be used to continue work after OnEvent if compatible with the emitter.
    // OnEvent returns true if it recognizes the event as "processed",
    // for tracing/metrics primarily.
    OnEvent(ctx context.Context, ev Event) bool
}

```

EngineController 는 event.Deriver 인터페이스를 구현한다:

```

// op-node/rollup/engine/engine_controller.go

```

```
var _event.Deriver = (*EngineController)(nil)
```

2. 시스템 등록

드라이버 초기화 시 EngineController 가 이벤트 시스템에 등록된다:

```
// op-node/rollup/driver/driver.go
ec := engine.NewEngineController(driverCtx, l2, log, metrics, cfg, syncCfg, l1, sys.Register("engine-controller",
nil))
// ...
sys.Register("engine", ec)
```

3. 이벤트 발행 → 큐에 추가

어떤 컴포넌트가 emitter.Emit(ctx, event)를 호출하면:

```
// op-service/event/system.go
func (s *Sys) emit(name string, derivContext uint64, ctx context.Context, ev Event, emitPriority Priority) {
    emitContext := s.emitContext.Add(1)
    annotated := AnnotatedEvent{
        Ctx:      ctx,
        Event:     ev,
        EmitContext: emitContext,
        EmitPriority: emitPriority,
        // ...
    }
    // ...
    err := s.executor.Enqueue(annotated) // 큐에 추가
}
```

4. 이벤트 루프에서 Drain 호출

Driver.eventLoop 에서 이벤트가 있으면 Drain()을 호출한다:

```
// op-node/rollup/driver/driver.go
case <-s.drain.Await():
    if err := s.drain.Drain(); err != nil {
        // ...
    }
```

5. Drain → 모든 Deriver 에 이벤트 전달

Drain()은 큐의 이벤트를 꺼내서 등록된 모든 Deriver 에게 전달한다:

```
// op-service/event/executor_global.go
func (gs *GlobalSyncExec) Drain() error {
    for {
        // ...
        ev := gs.events.Pop()
    }
```

```

    if ev.Event == nil {
        return nil
    }
    gs.processEvent(ev) // 모든 핸들러에 전달
}
}

```

```

// op-service/event/executor_global.go
func (gs *GlobalSyncExec) processEvent(ev AnnotatedEvent) {
    gs.handlesLock.RLock()
    defer gs.handlesLock.RUnlock()
    for _, h := range gs.handles {
        h.onEvent(ev) // 각 Deriver 의 RunEvent 호출
    }
}

```

6. 최종적으로 OnEvent 호출

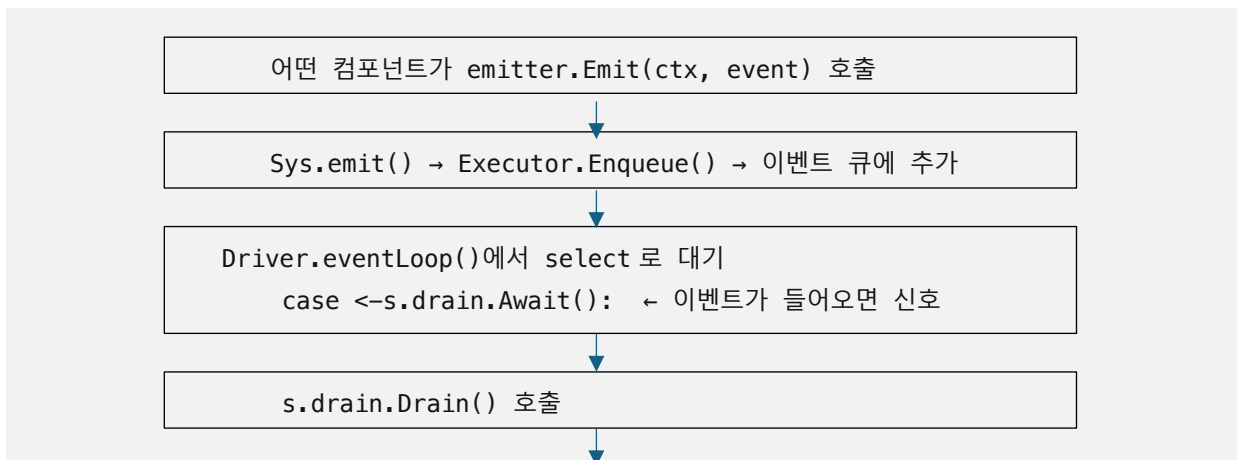
각 systemActor.RunEvent 가 실제 OnEvent 를 호출한다:

```

// op-service/event/system.go
func (r *systemActor) RunEvent(ev AnnotatedEvent) {
    if r.deriv == nil {
        return
    }
    // ...
    effect := r.deriv.OnEvent(ev.Ctx, ev.Event) // ← 여기서 EngineController.OnEvent 호출!
    // ...
}

```

흐름 요약



GlobalSyncExec.processEvent() → 모든 등록된 Deriver 에게
브로드캐스트

- EngineController.OnEvent(ctx, ev)
- Sequencer.OnEvent(ctx, ev)
- Finalizer.OnEvent(ctx, ev)
- ... (다른 모든 등록된 컴포넌트들)

핵심 포인트

- 모든 이벤트는 모든 Deriver 에게 브로드캐스트된다
- 각 OnEvent 는 자신이 처리할 이벤트 타입만 처리하고 나머지는 무시 (return false)
- 이벤트 처리는 동기적으로 순차 실행된다 (같은 스레드에서)
- 우선순위(priority)에 따라 중요한 이벤트가 먼저 처리될 수 있다

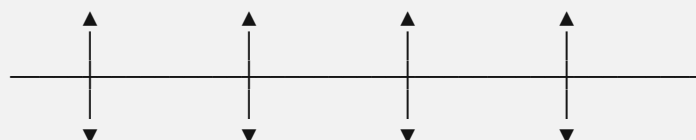
SyncStatus

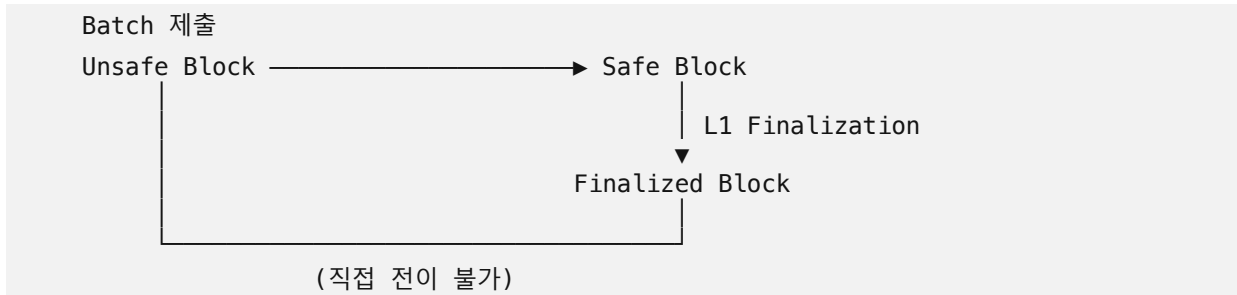
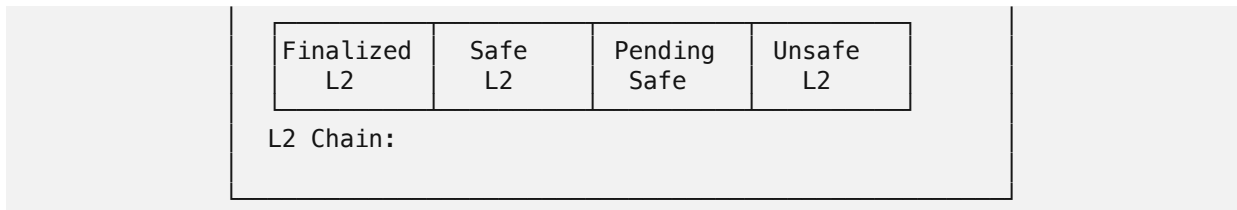
SyncStatus 는 op-node 의 동기화 상태 스냅샷으로, L1 과 L2 의 상태를 추적한다.

```
type SyncStatus struct {  
    CurrentL1      L1BlockRef // 현재 처리 중인 L1 블록 <- Derivation Pipeline 에서 설정  
    CurrentL1Finalized L1BlockRef // 현재 finalized L1 <- L1 Finalized 이벤트에서 설정  
    // L1 finality 업데이트 수신 시  
    // Casper FFG finality 에 의해 결정  
    HeadL1      L1BlockRef // L1 head  
    SafeL1      L1BlockRef // L1 safe  
    FinalizedL1 L1BlockRef // L1 finalized  
    UnsafeL2    L2BlockRef // L2 unsafe head  
    SafeL2      L2BlockRef // L2 safe head  
    FinalizedL2 L2BlockRef // L2 finalized head  
    // ... 추가 필드들  
}
```

SyncStatus

L1 Chain:





L2 블록 상태 변화의 의미

전체 상태 흐름

Unsafe → PendingSafe → LocalSafe → (CrossSafe) → Finalized

각 상태 전환은 “L1 에 대한 신뢰 수준이 얼마나 확보되었는가” 를 의미합니다.

1. Unsafe

의미: Sequencer 가 방금 만든 블록. L1 과 아무런 검증 관계가 없음.

- Sequencer 가 mempool 트랜잭션을 가져와 블록 생성 (NoTxPool = false)
- P2P gossip 으로 다른 노드에 전파
- AttributesWithParent.DerivedFrom 이 비어 있음 (eth.L1BlockRef{})
- 아직 L1 배치에 제출되지 않았거나, 제출됐더라도 Derivation 이 확인하지 않은 상태
- reorg 가능성이 가장 높음

2. PendingSafe (로컬 확인 진행 중)

의미: Derivation Pipeline 이 L1 배치 데이터를 읽어, 해당 블록의 트랜잭션이 L1 에 제출된 것을 확인한 상태.

전환 조건:

- DerivationPipeline 이 L1 에서 배치를 파싱
- AttributesWithParent.DerivedFrom 에 실제 L1 블록 참조가 채워짐
- 기존 Unsafe 블록과 비교(Consolidation)하여 일치하면 재빌드 없이 격상

```
// payload_success.go
```

```
if DerivedFrom != (eth.L1BlockRef{}): // L1 에서 derive 된 경우만
    e.tryUpdatePendingSafe(ctx, ref) // → PendingSafe 로 격상
```

- 아직 LocalSafe 가 아님: epoch(L1 블록)의 마지막 L2 블록까지 모두 확인되지 않은 상태

3. LocalSafe (로컬 완전 확인)

의미: 해당 L1 epoch 에 속한 L2 블록들이 모두 Derivation 에 의해 확인된 상태(Concluding = true).

전환 조건:

- AttributesWithParent.Concluding == true (L1 epoch 의 마지막 L2 블록)
- TryUpdateLocalSafe() 호출

```
// 이 체인이 interop 아닌 경우
// LocalSafe → Safe(CrossSafe)로 바로 승격
e.PromoteSafe(ctx, ref)
```

- Interop(다중 체인) 모드에서는 cross-chain validation 이 추가로 필요
- 단일 체인 모드에서는 LocalSafe ≈ Safe

4. Safe (CrossSafe)

의미: L1 on-chain 데이터로 완전히 검증됨. L1 이 safe 상태인 블록에 포함된 배치로부터 유도된 L2 블록.

- L1 의 safe 개념(Casper FFG 기반, 2/3 검증자 투표 완료)을 L2 에 반영
- op-node 가 L1 safe 블록을 폴링하여 감지

```
// node.go
onL1Safe := func(ctx context.Context, sig eth.L1BlockRef) {
    node.l2Driver.StatusTracker.OnL1Safe(sig)
}
l1SafeSub := eth.PollBlockChanges(node.log, node.l1Source, onL1Safe, eth.Safe, ...)
```

- Interop 환경에서는 다른 L2 체인과의 cross-chain 검증도 완료된 상태

5. Finalized

의미: Ethereum L1 finality 달성. 절대 변경 불가능.

- L1 의 finalized 블록(Casper FFG 완전 확정, ~13 분 소요)에 배치가 포함된 L2 블록
- Finalizer 컴포넌트가 L1↔L2 derivation 관계를 추적하여 결정

```
// node.go
onL1Finalized := func(ctx context.Context, sig eth.L1BlockRef) {
    node.l2Driver.Finalizer.OnL1Finalized(sig)
    // → EngineController.PromoteFinalized(ctx, finalized)
    // → SetFinalized(ref) → tryUpdateEngine()
}
```

상태별 비교 요약

상태	안전도	전환 트리거	reorg 가능성
Unsafe	매우 낮음	Sequencer 블록 생성 직후	높음
PendingSafe	낮음	L1 배치에서 트랜잭션 확인 (Derivation)	낮음
LocalSafe	중간	L1 epoch 내 L2 블록 전체 확인 완료	매우 낮음

Safe (CrossSafe)	높음	L1 safe 상태 블록에 포함됨	거의 없음
Finalized	최고	L1 finality 달성 (~13 분)	불가

핵심 원리

Unsafe 는 Sequencer 를 신뢰해야 하는 상태이고, PendingSafe 이후부터는 L1 데이터(누구나 검증 가능)에 근거하는 상태입니다. 이것이 OP Stack 의 보안 모델 핵심으로, Sequencer 가 악의적이더라도 Derivation 이 올바른 상태를 강제(reorg)할 수 있습니다.

op-sequencer

설명

시퀀서(Sequencer)는 Engine Controller 와 협력하여 L2 블록 생성의 중심 조율자다. 실제 블록을 만드는 것은 op-geth 이지만, 언제·무엇을·어떻게 만들지는 시퀀서가 결정한다. startBuildingBlock()에서 다음 블록이 참조할 L1 origin 을 결정하고, L1 deposit·업그레이드 트랜잭션 등 강제 포함 트랜잭션 목록과 user txpool 포함 여부(NoTxPool)를 설정한다.

이후 ForkchoiceUpdate API 로 op-geth 의 블록 빌딩을 시작시키고, 봉인(seal) 타이밍이 되면 GetPayload 를 호출한다. op-geth 는 이 시점에 진행 중이던 txpool 수집을 중단(interruptBuilding)하고 완성된 블록을 반환한다. 시퀀서는 이 블록을 받아 P2P 네트워크로 다른 노드(Verifier 등)에 먼저 비동기로 전파(gossip)한 뒤, NewPayload API 로 op-geth 의 L2 체인에 정식 삽입한다. 이 시점 블록의 상태는 unsafe 이다.

트랜잭션 순서 측면에서, 블록 앞부분의 L1 Info-deposit·업그레이드 트랜잭션은 시퀀서가 순서를 고정한다. 그 뒤에 오는 user txpool 트랜잭션들 사이의 순서는 op-geth 의 txpool 이 가스 가격 기준으로 정렬하며, 시퀀서는 이를 직접 재정렬하지 않는다. 다만 NoTxPool 플래그와 블록 봉인 타이밍을 통해 해당 블록에 어떤 트랜잭션을 얼마나 담을지를 제어한다.

op-sequencer 가 강제로 블록 앞에 삽입하는 트랜잭션

1. L1 InfoTx - L1-info attributes transaction
2. depositTxs: L1 -> L2 deposit txs
3. upgradeTxs: 업그레이드 관련 txs

depositTxs

depositTxs 는 L1 → L2 방향의 자산/메시지 전달을 위한 트랜잭션입니다. L1 의 컨트랙트 이벤트로부터 L2 트랜잭션이 자동으로 파생된다. L1 에서 OptimisticPortal contract 의 depositTransaction 을 실행하면서 시작된다. 아래 depositTxs 생성과정을 참고하기 바란다.

upgradeTxs

upgradeTxs 는 OP Stack 네트워크 업그레이드(하드포크) 시 자동으로 실행되는 시스템 트랜잭션들을 담는 변수이다.

지원하는 업그레이드들

업그레이드	주요 변경사항
Ecotone	L1Block/GasPriceOracle 업그레이드, EIP-4788 배포
Fjord	GasPriceOracle 업그레이드 (RIP-7212)
Isthmus	새로운 시스템 컨트랙트 업그레이드

Jovian	추가 기능 업그레이드
Interop	CrossL2Inbox 등 인터옵 관련 컨트랙트

왜 중요한가?

1. 자동화된 업그레이드: 하드포크 시점에 모든 노드가 동일한 시스템 트랜잭션 실행
2. 결정론적: Sequencer와 Verifier가 동일한 트랜잭션 생성
3. 프록시 패턴: L2 프리컴파일 컨트랙트가 프록시로 되어 있어 업그레이드 가능
4. Deposit 트랜잭션 형태: L1 deposit과 동일한 형태로 실행되어 가스비 없이 실행됨

op-geth의 user txs를 포함여부

상황	NoTxPool	효과
정상 시퀀싱	false	txpool user tx 포함
Drift 초과	true	시스템 tx만 포함 (빈 블록)
하드포크 블록	true	업그레이드 tx만 실행
리커버리 모드	true	user tx 없이 복구

1. Drift 초과

L2는 2초마다 블록을 생성하지만, L1은 12초마다 블록을 생성합니다. 하나의 L1(L1 origin)에 여러개의 L2가 생성되는 데, L1이 12초마다 들어오기로 되어 있지만 만약 그 블록이 생성되는 시간이 SequencerDrift보다 커지게 되면 더 이상 유저 트랜잭션을 포함하지 않고 빈 블록만 생성하도록 한다. 그 뒤 L1이 정상적인 블록 생성이 감지되는 경우 L1 Origin을 변경하고 유저 트랜잭션을 포함하는 정상프로세스를 돌아간다. 만약, 시퀀서가 maxSequenceDriftFjord를 무시하고 유저 트랜잭션을 끼워 넣더라도 Verifier가 L1에서 재현(Derivation)할 때 해당 배치를 강제로 Drop한다.

MaxSequencerDrift 값

- Bedrock~Canyon: 설정값 (보통 600초 = 10분)
- Fjord 이후: 고정 1800초 (30분)

2. 리커버리 모드

op_batcher에 의한 L1으로의 배치가 SeqWindowSize만큼 L1 블록이 지났는데도 없는 경우 Verifier는 L2 슬롯들을 빈 블록으로 강제로 생성한다. 이 때 시퀀서가 user txs가 포함된 블록을 만들게 되면 verifier와의 체인이 서로 불일치하게 됩니다. 따라서 recovery 모드에 들어가게 되면 시퀀서도 빈 블록을 만들게 되고, 다음 L1 origin이 정확히 지정된 후 Verifier와 Sequencer는 정상적으로 작동한다.

	정상 모드	Recovery Mode
user tx	포함	제외 (NoTxPool=true)
L1 Origin 선택	캐시 활용, 유연하게	매번 직접 fetch, 강제로 다음 블록

목적	최대 처리량	Verifier 와 동일한 빈 블록 생성
활성화	기본	--sequencer.recover 또는 Admin RPC

Recovery Mode 는 “Sequencer 가 Verifier 처럼 행동”하는 모드입니다. Sequencing Window Timeout 상황에서 Verifier 가 강제로 만드는 빈 블록과 완전히 동일한 블록을 Sequencer 도 만들어서, 두 체인이 갈라지지 않도록 맞추는 비상 메커니즘이다.

SeqWindowSize

sequenceWindowSize = 3600 (단위 L1 블록 수) → $3600 \times 12 \text{ 초} = 43,200 \text{ 초} = 12 \text{ 시간}$ Sequencer 는 12 시간 안에 반드시 L1 에 배치를 제출해야 한다.

Sequencer 와 EngineController 의 역할 분담

단계	담당 컴포넌트	하는 일	매개이벤트
블록 빌딩 타이밍/속성 결정	Sequencer	startBuildingBlock()	
ForkchoiceUpdate (빌딩 시작)	EngineController	onBuildStart() → op-geth RPC	BuildStartEvent (S)
GetPayload (Sealing)	EngineController	onBuildSeal() → op-geth RPC	BuildSealEvent (S)
P2P gossip	Sequencer	asyncGossip.Gossip()	
NewPayload (체인 삽입)	EngineController	onPayloadProcess() → op-geth RPC	PayloadProcessEvent (S)
ForkchoiceUpdate (head 확정)	EngineController	tryUpdateEngineInternal() → op-geth RPC	PayloadSuccessEvent (E)

매개 이벤트: S - Sequencer 가 EngineController 에게 전달하기 위해 발생 이벤트

E - EngineController 가 발생하는 이벤트

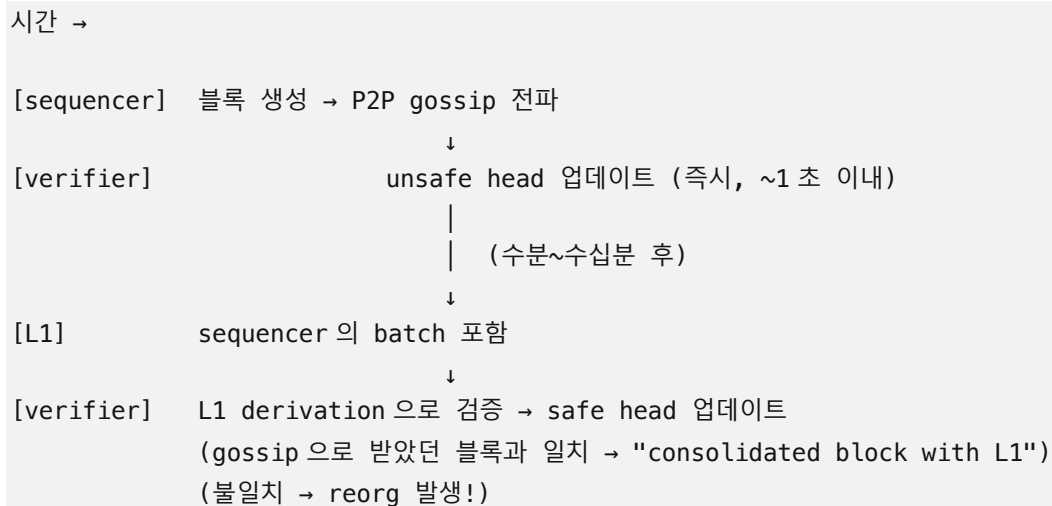
Gossip 의 역할

```
// op-node/rollup/engine/engine_controller.go
// onForkchoiceUpdate refreshes unsafe payload queue and peeks at the next applicable unsafe payload, if
any,
// to apply on top of the received forkchoice pre-state.
func (e *EngineController) onForkchoiceUpdate(ctx context.Context, event ForkchoiceUpdateEvent) {
    // ...
    // 현재 unsafe head 바로 다음 블록이 큐에 있으면 삽입
    if refParentHash != event.UnsafeL2Head.Hash || refBlockNumber != event.UnsafeL2Head.Number+1 {
        return // 아직 적용 불가
    }
    e.processUnsafePayload(ctx, nextEnvelope)
}
```

그리고 나중에 derivation 이 해당 블록을 L1 batch 에서 확인하면 "consolidation" 로그가 찍힙니다:

```
// op-node/rollup/engine/engine_controller.go
reason = "consolidated block with L1"
```

전체 타임라인



op-node 에서 L2 블록 생성

op-node 에서 L2 블록 생성은 Sequencer 와 Enging Conroller 가 이벤트 기반 아키텍처 하에서 협력하여 이루어집니다. L2 블록 생성에서 Sequencer 의 작동은 op-node 의 init 함수에서 l2Driver 를 생성하고, l2Driver 가 Start 될때 이벤트를 처리하는 무한루프가 동작하고, 이 중 sequencerCh 을 통해 SequencerActionEvent 가 발생하면 Sequencer 의 startBuildingBlock 함수가 실행되면서 블록생성이 시작된다. 주기적인 SequencerActionEvent 가 L2 블록의 주기적 생성을 이끍니다. 프로세스를 순서대로 정리하면 다음과 같다

1. 블록 타이밍 제어

```
// op-node/rollup/sequencing/sequencer.go
func (d *Sequencer) onForkchoiceUpdate(x engine.ForkchoiceUpdateEvent) {
    // ...
    if x.UnsafeL2Head.Number > d.latestHead.Number {
        d.nextActionOK = true
        now := d.timeNow()
        blockTime := time.Duration(d.rollupCfg.BlockTime) * time.Second
        payloadTime := time.Unix(int64(x.UnsafeL2Head.Time+d.rollupCfg.BlockTime), 0)
        remainingTime := payloadTime.Sub(now)
        if remainingTime > blockTime {
```

```

    // if we have too much time, then wait before starting the build
    d.nextAction = payloadTime.Add(-blockTime)
} else {
    // otherwise start instantly
    d.nextAction = now
}
}

```

- 새 unsafe head 를 받으면 다음 블록의 payload time 기준으로 빌딩 시작 시각을 계산
- 봉인(seal)은 payloadTime - 50ms(sealingDuration) 시점으로 스케줄

2. 시퀀서 startBuildingBlock() - 블록빌딩 시작

startBuildingBlock 함수는 주어진 L2 head, safe, and finalized blocks 위에 l1Origin 을 얻어 블록 빌딩을 시작한다.

- 1) L1 origing 블록을 결정(FindL1Origin)
- 2) 페이로드 속성을 준비한 후(PreparePayloadAttributes)
- 3) NoTxPool 속성처리 - L2 의 블록이 L1 블록에 비해 너무 앞서게 되면(> MaxSequenceDrift) true 로 설정하여 사용자 트랜잭션이 포함되지 않은 블록을 생성한다.
- 4) 특수 블록 처리 (업그레이드 블록 등) - 특수 블록이 포함되면 사용자 블록은 포함되지 않는다 (NoTx Pool = true)
- 5) BuildStartEvent 이벤트 발행 → Engine Controller 로 전달한다.

```

// op-node/rollup/sequencing/sequencer.go
// StartBuildingBlock initiates a block building job on top of the given L2 head, safe and finalized blocks, and
// using the provided l1Origin.
func (d *Sequencer) startBuildingBlock() {
    ctx := d.ctx
    l2Head := d.latestHead

    // 1. L1 origin 블록 선택
    l1Origin, err := d.l1OriginSelector.FindL1Origin(ctx, l2Head)
    // ...

    // 2. 페이로드 속성(payload attributes) 준비
    attrs, err := d.attrBuilder.PreparePayloadAttributes(fetchCtx, l2Head, l1Origin.ID())
    // ...

    // 3. 특수 블록 처리 (업그레이드 블록 등)
    attrs.NoTxPool = uint64(attrs.Timestamp) > l1Origin.Time+d.spec.MaxSequencerDrift(l1Origin.Time)

    // 4. 블록 빌딩 시작 이벤트 발행
    withParent := &derive.AttributesWithParent{

```

```

    Attributes: attrs,
    Parent:      l2Head,
    Concluding: false,
    DerivedFrom: eth.L1BlockRef{},
}

d.emitter.Emit(d.ctx, engine.BuildStartEvent{
    Attributes: withParent,
})
}

```

3. 속성빌딩: PreparePayloadAttributes() - 블록 내용 준비

- 1) L2 시스템 설정 가져오기
- 2) 새 epoch 인 경우 L1 에서 deposit 트랜잭션 수입 (DepositTx) - 이 tx 생성과정은 아래 'depositTx 생성과정'을 참조한다.
- 3) UpgradeTx cnrk (Ecostone, Fjord, Isthmus 등)
- 4) l1InfoTx 생성
- 5) tx 순서 설정 : l1InfoTx, depositTx, upgradeTx
- 6) PayloadAttributes 구조체 생성

```

// op-node/rollup/derive/attributes.go
func (ba *FetchingAttributesBuilder) PreparePayloadAttributes(ctx context.Context, l2Parent eth.L2BlockRef,
epoch eth.BlockID) (attrs *eth.PayloadAttributes, err error) {
    // 1. L2 시스템 설정 가져오기
    sysConfig, err := ba.l2.SystemConfigByL2Hash(ctx, l2Parent.Hash)

    // 2. 새 epoch 인 경우 L1 에서 deposit 트랜잭션 수집
    if l2Parent.L1Origin.Number != epoch.Number {
        info, receipts, err := ba.l1.FetchReceipts(ctx, epoch.Hash)
        deposits, err := DeriveDeposits(receipts, ba.rollupCfg.DepositContractAddress)
        depositTxs = deposits
        seqNumber = 0
    }

    // 3. 다음 L2 블록 타임스탬프 계산
    nextL2Time := l2Parent.Time + ba.rollupCfg.BlockTime

    // 4. 업그레이드 트랜잭션 추가 (Ecotone, Fjord, Isthmus 등)
    if ba.rollupCfg.IsEcotoneActivationBlock(nextL2Time) {
        upgradeTxs, err = EcotoneNetworkUpgradeTransactions()
    }
}

```

```
// ...
}
```

4. Engine Controller: onBuildStart() 블록빌딩 요청

BuildStartEvent 는 EngineController 의 OnEvent 함수에 의해 onBuildStart 함수가 실행된다.
onBuildStart 함수는

- Forkchoice 상태를 구성하고
- Engine API 를 통해 페이로드 빌딩 시작 (Engine:startPayload)
- 성공시 BuildStartedEvent 발행한다.

```
// op-node/rollup/engine/build_start.go
func (e *EngineController) onBuildStart(ctx context.Context, ev BuildStartEvent) {
    // 1. Forkchoice 상태 구성
    fc := eth.ForkchoiceState{
        HeadBlockHash:    fcEvent.UnsafeL2Head.Hash,
        SafeBlockHash:    fcEvent.SafeL2Head.Hash,
        FinalizedBlockHash: fcEvent.FinalizedL2Head.Hash,
    }

    // 2. Engine API 를 통해 페이로드 빌딩 시작
    id, errType, err := e.startPayload(rpcCtx, fc, ev.Attributes.Attributes)

    // 3. 성공 시 BuildStartedEvent 발행
    e.emitter.Emit(ctx, BuildStartedEvent{
        Info:      eth.PayloadInfo{ID: id, Timestamp: uint64(ev.Attributes.Attributes.Timestamp)},
        BuildStarted: buildStartTime,
        // ...
    })
}
```

5. startPayload() - Execution Engine 에 블록 빌딩 요청

- 1) Engine API 의 ForkchoiceUpdate 호출 (EIP-4337)
 - PayloadAttributes.Transactions 를 먼저 포함
 - NoTxPool == false 이면 -> txpool 에서 우선순위가 높은 트랜잭션 가져오기 -> 가스 한도까지 트랜잭션 추가
- 2) 성공시 payloadID 반환

```
// op-node/rollup/engine/engine_controller.go
func (e *EngineController) startPayload(ctx context.Context, fc eth.ForkchoiceState, attrs *eth.PayloadAttributes) (id eth.PayloadID, errType BlockInsertionErrType, err error) {
```

```
// Engine API 의 ForkchoiceUpdate 호출 (EIP-4337)
fcRes, err := e.engine.ForkchoiceUpdate(ctx, &fc, attrs)
// ...

switch fcRes.PayloadStatus.Status {
case eth.ExecutionValid:
    return *fcRes.PayloadID, BlockInsertOK, nil // PayloadID 반환
// ...
}
```

핵심: engine.ForkchoiceUpdate()는 op-geth (Execution Engine)에 RPC 호출하여 블록 빌딩을 시작한다.

6. EngineController 블록 Sealing - onBuildSeal()

l2Driver 의 메인 이벤트 루프가 타이머 기반으로 다시 SequencerActionEvent 를 발령시키면 Sequencer.onSequencerAction() 함수내에서 이번에는 L2 블록을 빌딩중이기 때문에 startBuildingBlock()을 실행하지 않고 BuildSealEvent 를 발생한다. 이 이벤트는 EngineController.onBuildSeal 함수를 실행시키고 engine.GetPayload 를 실행하여 빌드된 payload 를 가져와 BuildSealedEvent 를 발생한다.

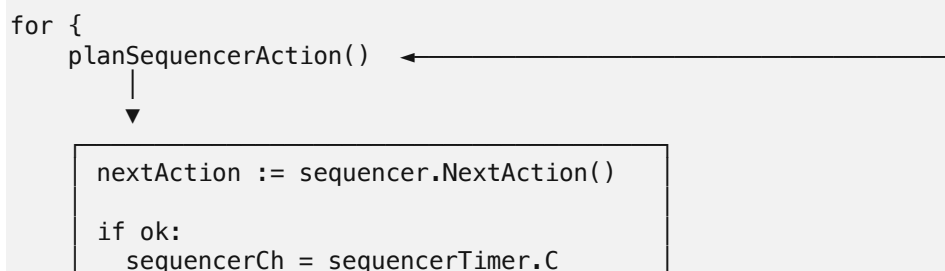
```
// op-node/rollup/engine/build_seal.go
func (e *EngineController) onBuildSeal(ctx context.Context, ev BuildSealEvent) {
    // Engine API 의 GetPayload 호출로 완성된 블록 가져오기
    envelope, err := e.engine.GetPayload(rpcCtx, ev.Info)
    // ...

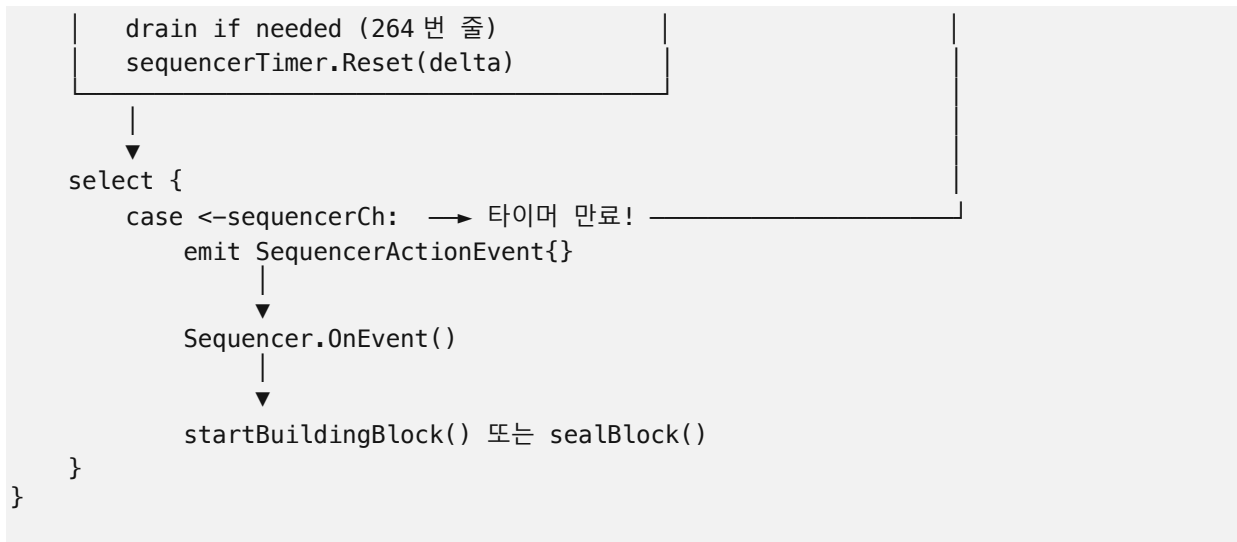
    // 블록 검증
    if err := sanityCheckPayload(envelope.ExecutionPayload); err != nil {
        // ...
    }

    // PayloadSealedEvent 발행
    ref, err := derive.PayloadToBlockRef(e.rollupCfg, envelope.ExecutionPayload)
    // ...
}
```

전체흐름도

Eventloop





7. Sequencer 가 Sealed 블록 처리 - onBuildSealed()

engine.PayloadProcessEvent 생성

8. EngineController: Payload 처리 - onPayloadProcess

engine.NewPayload()를 호출하여 execution engine 에 payload 를 삽입합니다. 성공시 (status.Status == eth.ExecutionValid) PayloadSuccessEvent 발생

9. EngineController: Unsafe Head 업데이트 - onPayloadSuccess()

```

// op-node/rollup/engine/payload_success.go
func (e *EngineController) onPayloadSuccess(ctx context.Context, ev PayloadSuccessEvent) {
    // Unsafe head 업데이트
    e.tryUpdateUnsafe(ctx, ev.Ref)

    // L1 에서 derive 된 경우 pending-safe, local-safe 로 승격
    if ev.DerivedFrom != (eth.L1BlockRef{}) {
        e.tryUpdatePendingSafe(ctx, ev.Ref, ev.Concluding, ev.DerivedFrom)
        e.tryUpdateLocalSafe(ctx, ev.Ref, ev.Concluding, ev.DerivedFrom)
    }

    // Forkchoice Update 호출
    err := e.tryUpdateEngineInternal(ctx)
}
  
```

[이 부분이 만약 Derive 이었다면 PendingSafe, LocalSafe 로 변경하는 부분임]

10. EngineController: tryUpdateUnsafe() - Unsafe Head 설정

```

// TryUpdateUnsafe updates the unsafe head and backs up the previous one if needed
func (e *EngineController) tryUpdateUnsafe(ctx context.Context, ref eth.L2BlockRef) {
    // Backup unsafeHead when new block is not built on original unsafe head.
  
```

```

    if e.unsafeHead.Number >= ref.Number {
        e.SetBackupUnsafeL2Head(e.unsafeHead, false)
    }
    e.SetUnsafeHead(ref)
    e.emitter.Emit(ctx, UnsafeUpdateEvent{Ref: ref})
}

```

11. SetUnsafeHead() - 실제 상태 변경

```

// SetUnsafeHead sets the local-unsafe head.
func (e *EngineController) SetUnsafeHead(r eth.L2BlockRef) {
    e.metrics.RecordL2Ref("l2_unsafe", r)
    e.unsafeHead = r
    e.needFCUCall = true
    e.needSafeHeadUpdate = false
    e.chainSpec.CheckForkActivation(e.log, r)
}

```

12. EngineController: Forkchoice Update 호출 - tryUpdateEngineInternal()

마지막으로 execution engine 에 새로운 forkchoice state 를 알림

```

func (e *EngineController) tryUpdateEngineInternal(ctx context.Context) error {
    fc := eth.ForkchoiceState{
        HeadBlockHash:    e.unsafeHead.Hash,
        SafeBlockHash:    e.safeHead.Hash,
        FinalizedBlockHash: e.finalizedHead.Hash,
    }
    logFn := e.logSyncProgressMaybe()
    defer logFn()
    fcRes, err := e.engine.ForkchoiceUpdate(ctx, &fc, nil)

    if fcRes.PayloadStatus.Status == eth.ExecutionValid {
        e.requestForkchoiceUpdate(ctx)
    }
    //...
    e.needFCUCall = false
}

```

13. EngineController.requestForkchoiceUpdate() - ForkchoiceUpdateEvent 발생

```

func (e *EngineController) requestForkchoiceUpdate(ctx context.Context) {
    e.emitter.Emit(ctx, ForkchoiceUpdateEvent{
        UnsafeL2Head:    e.unsafeHead,
        SafeL2Head:      e.safeHead,
        FinalizedL2Head: e.finalizedHead,
    })
}

```

```
}
```

14. Sequencer 가 ForkchoiceUpdateEvent 수신 -> nextAction 설정

```
func (d *Sequencer) onForkchoiceUpdate(x engine.ForkchoiceUpdateEvent) {
    d.log.Debug("Sequencer is processing forkchoice update", "unsafe", x.UnsafeL2Head, "latest", d.latestHead)

    if !d.active.Load() {
        d.setLatestHead(x.UnsafeL2Head)
        return
    }
    // ... safe lag 체크 ...

    // Drop stale block-building job
    if d.latest != (BuildingState{}) && d.latest.Onto.Number < x.UnsafeL2Head.Number {
        d.latest = BuildingState{} // ← 이전 빌딩 상태 클리어
    }

    if x.UnsafeL2Head.Number > d.latestHead.Number {
        d.nextActionOK = true // ← 액션 활성화!
        now := d.timeNow()
        blockTime := time.Duration(d.rollupCfg.BlockTime) * time.Second
        payloadTime := time.Unix(int64(x.UnsafeL2Head.Time+d.rollupCfg.BlockTime), 0)
        remainingTime := payloadTime.Sub(now)
        if remainingTime > blockTime {
            d.nextAction = payloadTime.Add(-blockTime)
        } else {
            d.nextAction = now // ← 즉시 또는 계산된 시간
        }
    }
    d.setLatestHead(x.UnsafeL2Head) // ← latestHead 업데이트
}
```

15. Driver 가 planSequencerAction() 호출 -> 타이머 설정

```
// driver.go
planSequencerAction := func() {
    nextAction, ok := s.sequencer.NextAction() // nextActionOK=true, nextAction 시간 반환
    // ... 타이머 설정 ...
    sequencerTimer.Reset(delta)
}
```

16. 타이머 만료 -> SequencerActionEvent 발생

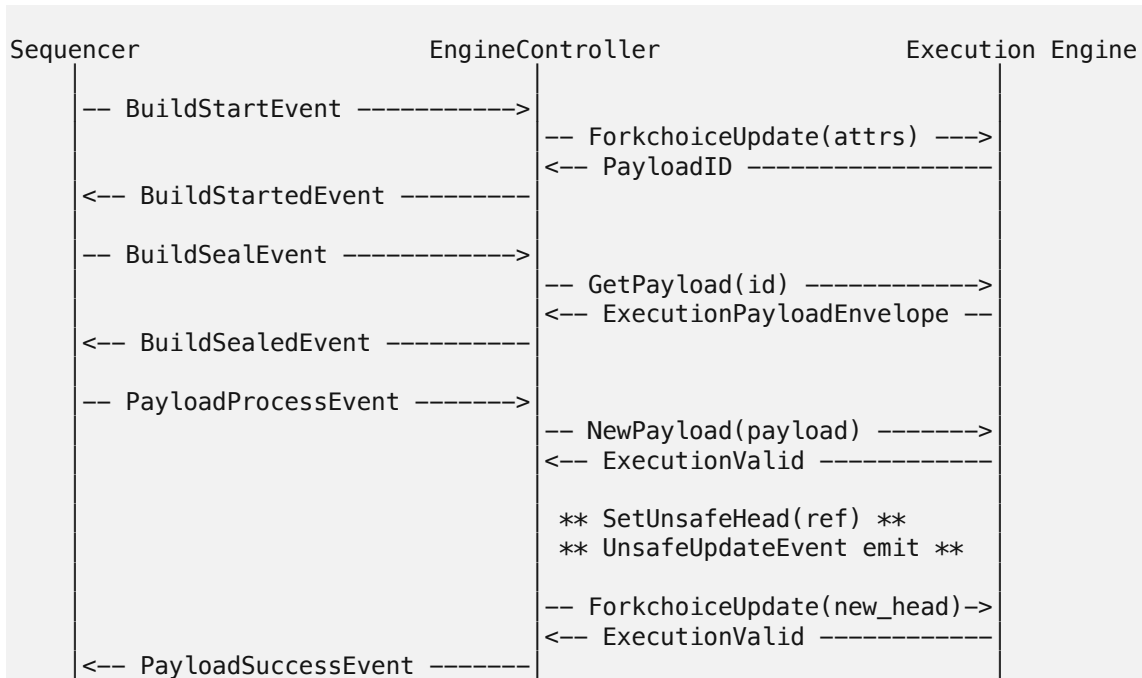
```
// driver.go
case <-sequencerCh:
    s.emitter.Emit(s.driverCtx, sequencing.SequencerActionEvent{})
```

17. Sequencer: onSequencerAction -> startBuildingBlock() 호출

```
// sequencer.go

func (d *Sequencer) onSequencerAction(ev SequencerActionEvent) {
    payload := d.asyncGossip.Get()
    if payload != nil {
        // ... 기존 payload 재처리 ...
    } else {
        if d.latest.Info != (eth.PayloadInfo{}) {
            // 빌딩 중 → seal
            d.emitter.Emit(d.ctx, engine.BuildSealEvent{...})
        } else if d.latest == (BuildingState{}) {
            // ★ 새 블록 빌딩 시작!
            d.startBuildingBlock()
        }
    }
}
```

전체플로우 다이어그램

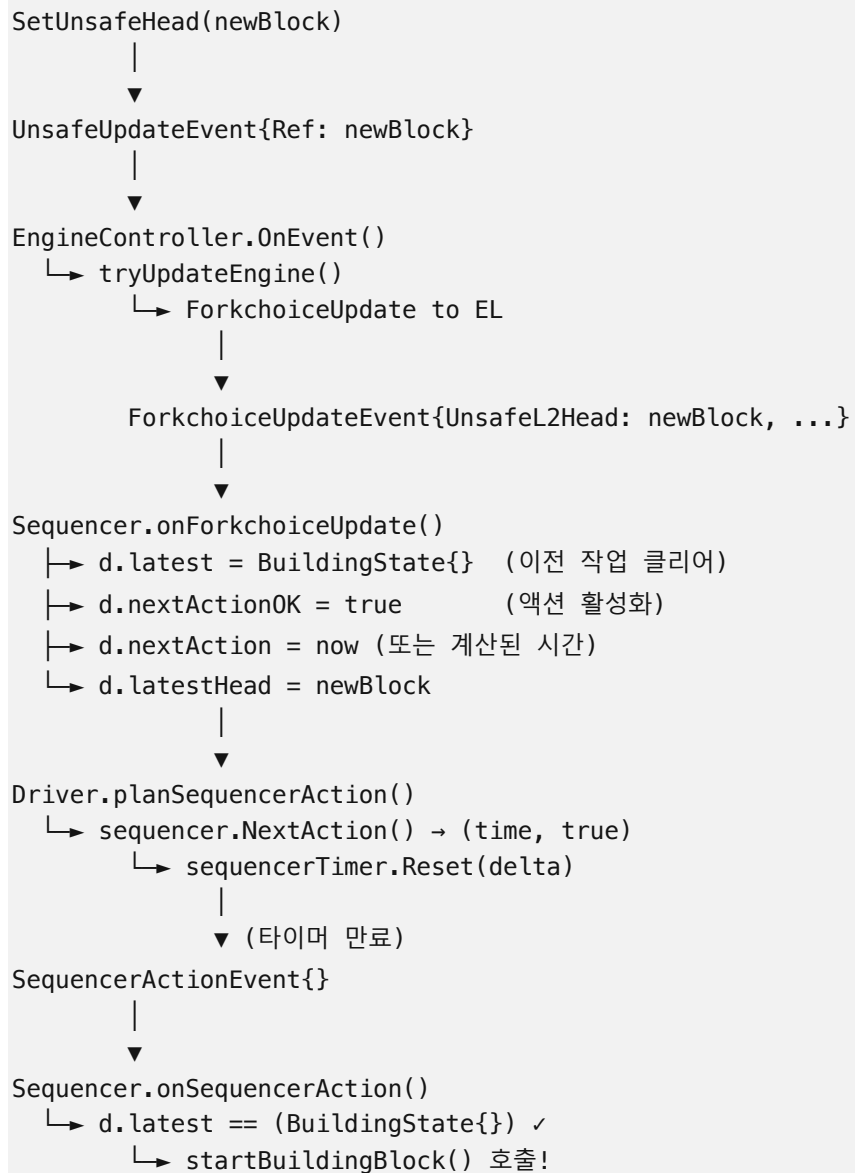


new Block 생성 즉 startBuildingBlock()으로 진입하기 위한 조건:

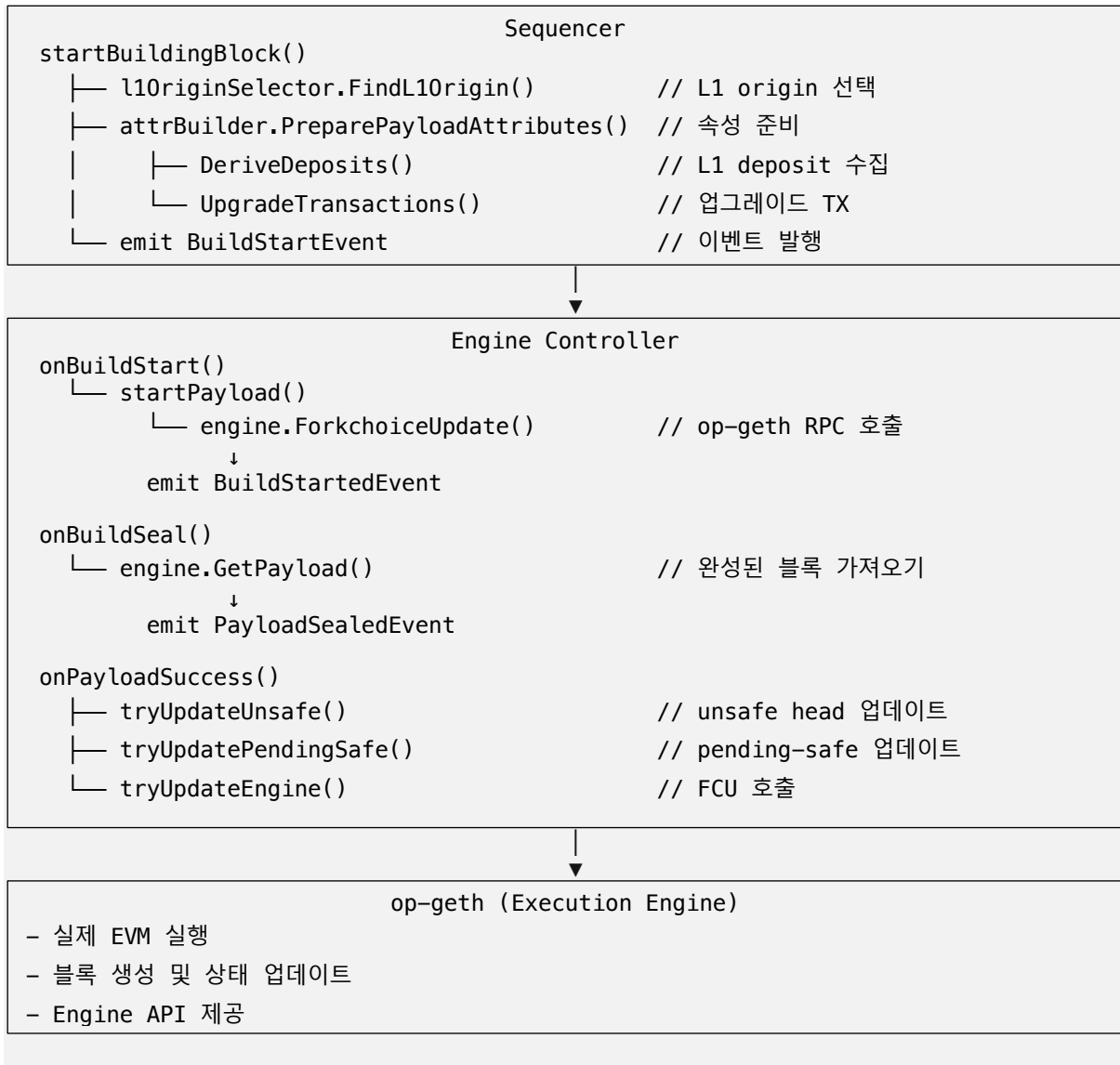
조건	설정 위치	설명
d.active.Load() == true	forceStart()	시퀀서가 활성화 상태
d.nextActionOK == true	onForkchoiceUpdate()	액션 스케줄 허용
d.latestHead != (eth.L2BlockRef{})	onForkchoiceUpdate()	최신 head 정보 있음

d.latest == (BuildingState{})	onForkchoiceUpdate()	현재 빌딩 작업 없음
asyncGossip.Get() == nil	-	재사용할 payload 없음

전체 이벤트 다이어그램



전체 흐름 요약



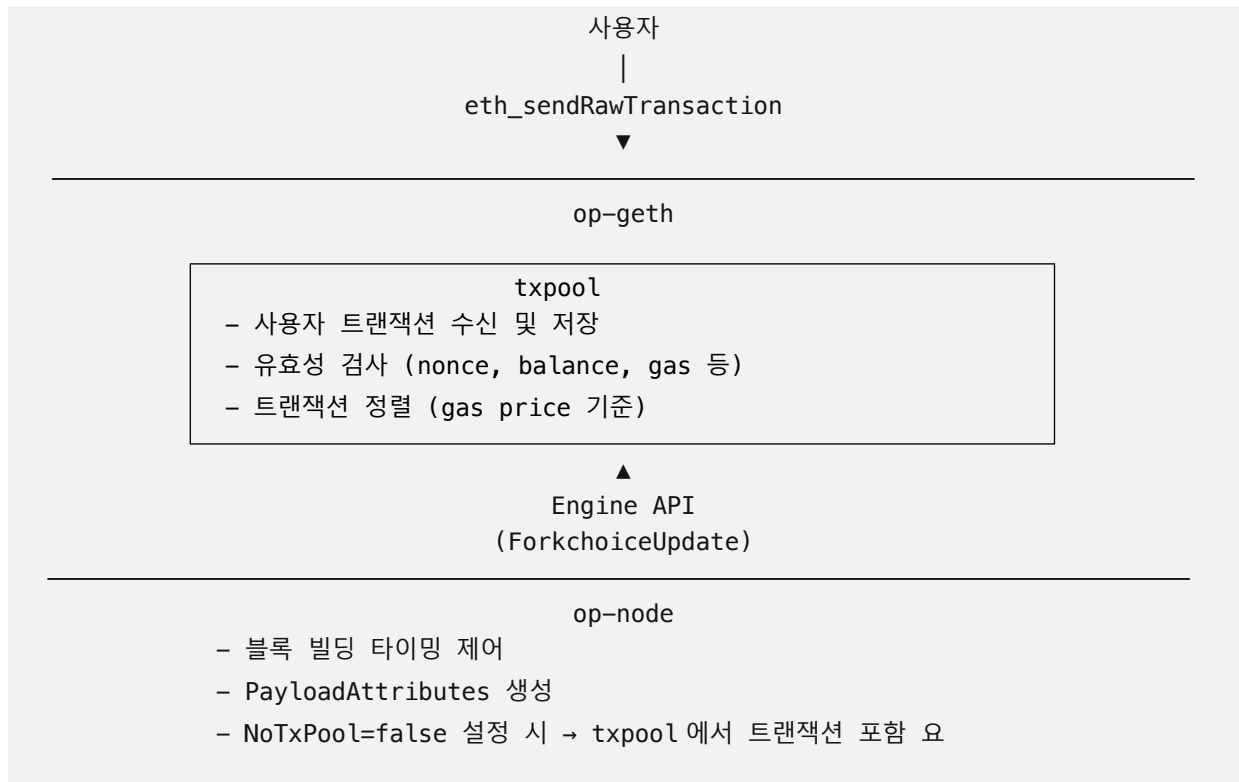
핵심 포인트

1. op-node 는 블록을 직접 생성하지 않음 - 대신 op-geth 에 Engine API 를 통해 요청
2. 이벤트 기반 아키텍처 - BuildStartEvent → BuildStartedEvent → BuildSealEvent → PayloadSuccessEvent
3. Engine API 사용 - ForkchoiceUpdate (빌딩 시작) + GetPayload (완성된 블록 가져오기)
4. 시퀀서가 타이밍 제어 - BlockTime 간격으로 새 블록 생성 트리거

L2 체인에서 사용자 트랜잭션 수신

OP Stack 에서 L2 사용자 트랜잭션은 op-geth (Execution Engine)가 담당합니다. op-node 는 블록 빌딩을 조율만 하고, 실제 트랜잭션 수신과 실행은 op-geth 에서 이루어진다.

전체 아키텍처



1. 사용자가 트랜잭션 전송

사용자는 `op-geth`의 RPC 엔드포인트로 트랜잭션을 전송한다:

표준 *Ethereum JSON-RPC*

- `eth_sendRawTransaction`
- `eth_sendTransaction`

`op-geth`는 표준 Ethereum 클라이언트처럼 작동하며, 트랜잭션을 `txpool`에 저장한다.

2. op-node 가 NoTxPool 제어

`op-node`의 시퀀서가 블록을 빌드할 때, `PayloadAttributes`를 생성합니다:

```
// op-service/eth/types.go
type PayloadAttributes struct {
    Timestamp Uint64Quantity `json:"timestamp"`
    PrevRandao Bytes32 `json:"prevRandao"`
    SuggestedFeeRecipient common.Address `json:"suggestedFeeRecipient"`
    // ...

    // Optimism additions
    // Transactions to force into the block (always at the start of the transactions
    list).
    Transactions []Data `json:"transactions,omitempty"`
    // NoTxPool to disable adding any transactions from the transaction-pool.
    NoTxPool bool `json:"noTxPool,omitempty"`
    // ...
}
```

핵심 필드:

- Transactions: 강제 포함될 트랜잭션 (L1 deposit, 시스템 트랜잭션)
- NoTxPool: false 면 txpool 에서 사용자 트랜잭션 포함

```
// op-node/rollup/sequencing/sequencer.go
// If our next L2 block timestamp is beyond the Sequencer drift threshold, then we must produce
// empty blocks (other than the L1 info deposit and any user deposits). We handle this by
// setting NoTxPool to true, which will cause the Sequencer to not include any transactions
// from the transaction pool.
attrs.NoTxPool = uint64(attrs.Timestamp) > l1Origin.Time+d.spec.MaxSequencerDrift(l1Origin.Time)
```

정상 시퀀싱 시: - NoTxPool = false → op-geth 가 txpool 에서 트랜잭션을 블록에 포함

특수 상황 (NoTxPool = true):

- 시퀀서 drift 초과
- 네트워크 업그레이드 블록 (Ecotone, Fjord, Isthmus 등)
- 리커버리 모드

4. Verifier 에서는 항상 NoTxPool = true

```
// op-node/rollup/derive/attributes_queue.go
// we are verifying, not sequencing, we've got all transactions and do not pull from the tx-pool
// (that would make the block derivation non-deterministic)
attrs.NoTxPool = true
attrs.Transactions = append(attrs.Transactions, batch.Transactions...)
```

Verifier 는 L1 batch 데이터에서 트랜잭션을 가져오므로 txpool 을 사용하지 않는다.

5. op-geth 에서 블록 빌딩

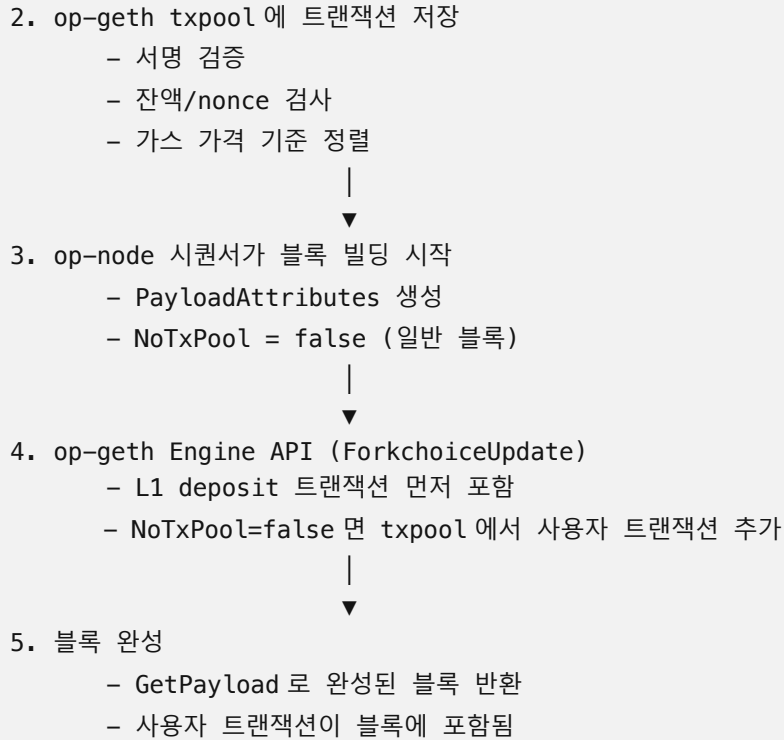
op-geth 가 ForkchoiceUpdate + PayloadAttributes 를 받으면:

- 1) PayloadAttributes.Transactions 를 먼저 포함 (deposit txs)
- 2) NoTxPool == false 이면:
 - txpool 에서 우선순위가 높은 트랜잭션 가져오기
 - 가스 한도까지 트랜잭션 추가
- 3) 블록 완성 후 PayloadID 반환

흐름 요약

1. 사용자가 op-geth RPC 로 트랜잭션 전송
POST /eth_sendRawTransaction





핵심 포인트

- 사용자 트랜잭션은 op-geth 가 수신 - 표준 Ethereum RPC 사용
- op-node 는 트랜잭션을 직접 받지 않음 - 블록 빌딩만 조율
- NoTxPool 플래그가 핵심 - false 일 때만 txpool 트랜잭션 포함
- 트랜잭션 순서: L1 deposit → 시스템 tx → 사용자 tx (txpool)
- Verifier 는 txpool 미사용 - L1 batch 데이터에서 트랜잭션 재생

FindL1Origin 함수 역할

FindL1Origin 은 다음 L2 블록을 빌드할 때 사용할 L1 origin 블록을 결정하는 함수이다.

핵심 개념: L1 Origin 이란?

L2 블록은 항상 특정 L1 블록을 참조(origin)합니다. 이는:

- L2 블록의 타임스탬프 기준점 역할
- L1 에서 발생한 deposit 트랜잭션의 소스
- Sequencer drift 계산의 기준점

함수 로직 분석

```
// op-node/rollup/sequencing/origin_selector.go
// FindL1Origin determines what the next L1 Origin should be.
// The L1 Origin is either the L2 Head's Origin, or the following L1 block
// if the next L2 block's time is greater than or equal to the L2 Head's Origin.
func (los *L1OriginSelector) FindL1Origin(ctx context.Context, l2Head eth.L2BlockRef) (eth.L1BlockRef, error) {
    currentOrigin, nextOrigin, err := los.CurrentAndNextOrigin(ctx, l2Head)
    if err != nil {
        return eth.L1BlockRef{}, err
    }

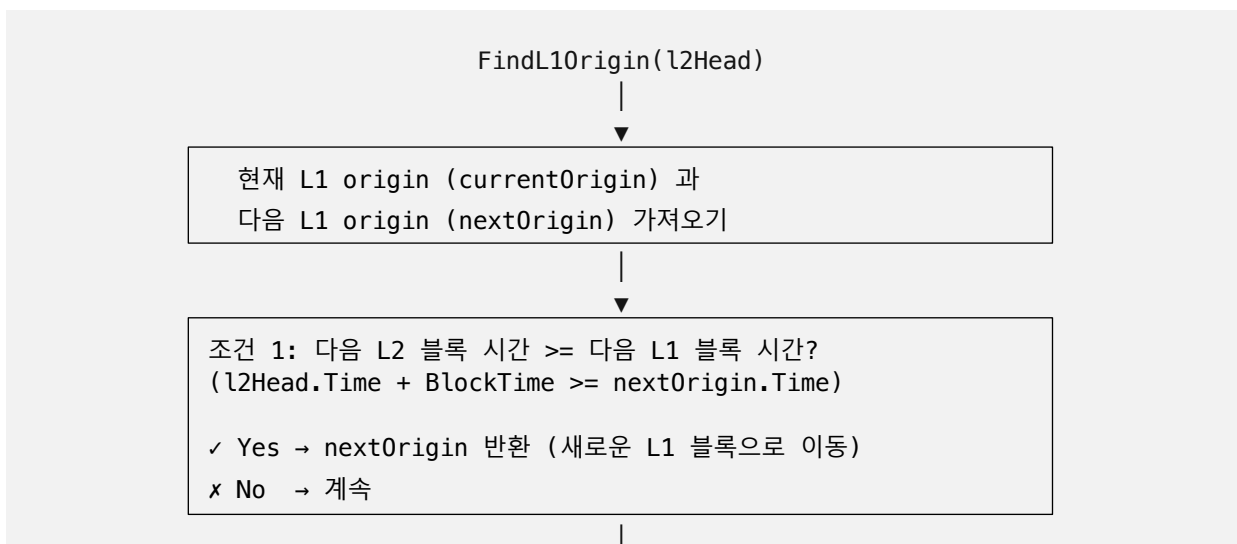
    // 조건 1: 다음 L1 블록으로 이동 가능한가?
    if nextOrigin != (eth.L1BlockRef{}) && l2Head.Time+los.cfg.BlockTime >= nextOrigin.Time {
        return nextOrigin, nil // 다음 L1 블록 사용
    }

    // 조건 2: Sequencer Drift 초과 여부 확인
    msd := los.spec.MaxSequencerDrift(currentOrigin.Time)
    pastSeqDrift := l2Head.Time+los.cfg.BlockTime-currentOrigin.Time > msd

    // Drift 내이면 현재 origin 유지
    if !pastSeqDrift {
        return currentOrigin, nil
    }

    // Drift 초과 시 다음 L1 블록 필요
    // ...
    return nextOrigin, nil
}
```

결정 로직 흐름도





예시 시나리오

설정:

- BlockTime = 2 초 (L2 블록 간격)
- MaxSequencerDrift = 600 초

상황 1: 정상 시퀀싱

L1 Block #100: time = 1000

L1 Block #101: time = 1012

L2 Head: time = 1006, L1origin = #100

다음 L2 time = 1006 + 2 = 1008

1008 < 1012 (nextOrigin.Time) → 아직 L1 #101 로 이동 안 함

1008 - 1000 = 8 < 600 (drift) → drift 내

결과: currentOrigin (#100) 반환

상황 2: 새 L1 블록으로 이동

L2 Head: time = 1010, L1origin = #100

다음 L2 time = 1010 + 2 = 1012

1012 >= 1012 (nextOrigin.Time) → L1 #101 로 이동 가능!

결과: nextOrigin (#101) 반환

상황 3: Drift 초과

L2 Head: time = 1600, L1origin = #100 (L1 #101 이 아직 없음)

다음 L2 time = 1600 + 2 = 1602

1602 - 1000 = 602 > 600 (drift 초과!) → 반드시 다음 L1 필요

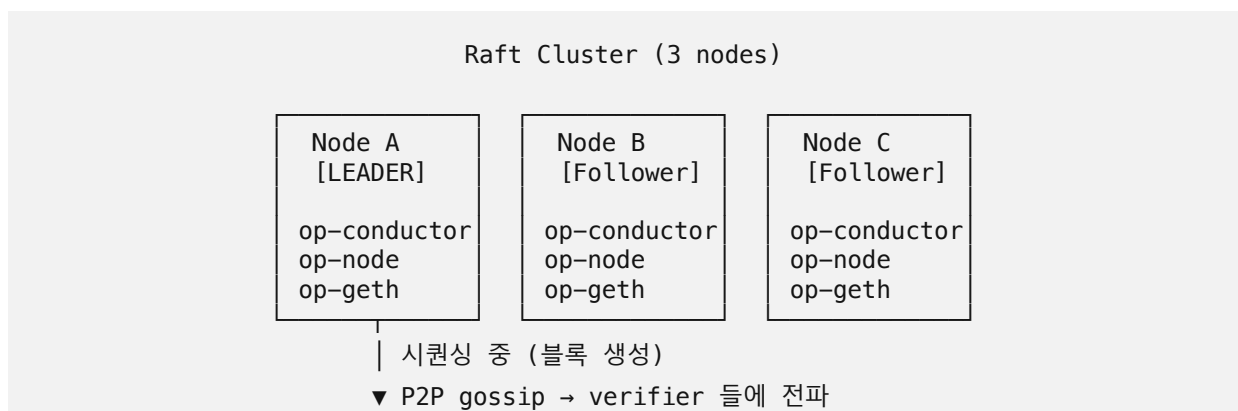
결과: 다음 L1 블록 fetch 후 반환 (실패 시 에러)

왜 중요한가?

- L1-L2 시간 동기화: L2 블록이 L1 블록보다 너무 앞서지 않도록 제한
- Deposit 처리: 새 L1 origin 으로 이동해야 해당 L1 블록의 deposit 포함 가능
- Sequencer Drift 제한: L2 가 L1 보다 MaxSequencerDrift 이상 앞서면 빈 블록 생성
- Derivation 결정론성: Verifier 가 동일한 L1 origin 선택 로직을 재현 가능

op-conductor 를 이용한 멀티 시퀀서 구성

기본 아키텍처



동작 원리: 3 가지 상태 조합

op-conductor 는 각 노드의 (leader, healthy, active) 3 가지 상태를 보고 8 가지 케이스를 자동 처리합니다:

```
// op-conductor/conductor/service.go
switch {
case status.leader && status.healthy && !status.active:
    // 리더인데 시퀀서가 꺼져있음 → 시퀀서 시작
    err = oc.startSequencer()

case status.leader && status.healthy && status.active:
    // 정상 리더 → 아무것도 안 함

case status.leader && !status.healthy && status.active:
    // 리더인데 비정상 → 시퀀서 중지 + 리더십 이전
    oc.stopSequencer()
    oc.transferLeader()

case !status.leader && status.healthy && status.active:
    // 리더가 아닌데 시퀀싱 중 → 시퀀서 중지
    err = oc.stopSequencer()
```

```
// ...
}
```

핵심 메커니즘 1: Raft 합의로 리더 선출

```
// op-conductor/consensus/raft.go
// RaftConsensus implements Consensus using raft protocol.
type RaftConsensus struct {
    serverID raft.ServerID
    r        *raft.Raft          // hashicorp/raft 라이브러리 사용
    transport *raft.NetworkTransport
    unsafeTracker *unsafeHeadTracker // 최신 unsafe head 공유
}
```

- hashicorp/raft 라이브러리 기반 Raft 합의
- 홀수 개의 노드 필요 (3 개 또는 5 개 권장, 과반수 투표)
- 리더 장애 시 자동으로 새 리더 선출

핵심 메커니즘 2: Raft FSM(Finite State Machine) 으로 최신 unsafe head 공유

```
// op-conductor/consensus/raft_fsm.go
// Apply implements raft.FSM - 새 블록이 생성될 때마다 호출됨
func (t *unsafeHeadTracker) Apply(l *raft.Log) interface{} {
    data := &eth.ExecutionPayloadEnvelope{}
    // ...
    // 최신 unsafe head 를 모든 follower 에 복제
    if t.unsafeHead == nil || t.unsafeHead.ExecutionPayload.BlockNumber < data.ExecutionPayload.BlockNumber
    {
        t.unsafeHead = data
    }
    return nil
}
```

리더 시퀀서가 블록을 만들 때마다 CommitUnsafePayload()를 호출하면:

```
// op-conductor/conductor/service.go
// CommitUnsafePayload commits an unsafe payload to the cluster FSM
// ensuring strong consistency by leveraging Raft consensus mechanisms.
func (oc *OpConductor) CommitUnsafePayload(_ context.Context, payload *eth.ExecutionPayloadEnvelope)
error {
    return oc.cons.CommitUnsafePayload(payload)
}
```

→ Raft 로그에 최신 unsafe head 가 기록되고 모든 follower 에 복제됩니다.

핵심 메커니즘 3: Health Monitor

```
// op-conductor/health/monitor.go
func NewSequencerHealthMonitor(log, metrics, interval, unsafeInterval, safeInterval,
```

minPeerCount uint64, ...) HealthMonitor

모니터링 항목:

- unsafe head 진행 속도 (블록이 계속 만들어지는지)
- safe head 진행 속도 (L1 배치 제출이 되는지)
- P2P peer 수 (최소 peer 수 미달 시 비정상)
- RPC 연결 (op-node, op-geth 연결 상태)

장애 감지 시 handleHealthUpdate() → action() → stopSequencer() + transferLeader() 자동 실행

장애 복구 시나리오

[정상 상태]

Node A (Leader, 시퀀싱 중) —블록 생성→ Raft log 에 커밋

Node B (Follower) ←복제— Raft log

Node C (Follower) ←복제— Raft log

[Node A 장애 발생]

Node A: conductor = unhealthy → stopSequencer + transferLeader 시도
(실패해도 무방: 리더십 없으면 블록 gossip/commit 불가)

Raft 투표: Node B, C 중 새 리더 선출 (과반수: 2 표 중 1 표)

[Node B가 새 리더]

Node B conductor: leader=true, healthy=true, active=false

→ action() → startSequencer()

→ Node B가 Raft log의 마지막 unsafe head 기준으로 시퀀싱 재개

장단점

장단점	내용
장점 ①	무중단 가용성: 리더 장애 시 수초 내 자동 failover
장점 ②	데이터 일관성: Raft FSM으로 최신 unsafe head를 모든 노드가 공유, 새 리더가 정확히 이어서 시퀀싱
장점 ③	이중 시퀀싱 방지: Raft 리더만 CommitUnsafePayload 가능, 리더 아닌 노드는 gossip/commit 차단
장점 ④	운영 편의: 관리자가 수동으로 TransferLeader 호출해 무중단 유지보수 가능
단점 ①	운영 복잡도: op-conductor 프로세스 추가, Raft 클러스터 관리 필요
단점 ②	홀수 노드 필요: 과반수 합의를 위해 최소 3 노드 (비용 증가)
단점 ③	Raft 합의 오버헤드: 블록 생성마다 CommitUnsafePayload Raft 로그 기록 → 약간의 지연
단점 ④	근본적 중앙화: 리더 1개만 시퀀싱하므로 분산 시퀀싱은 아님 (HA 일 뿐)

단일 vs 멀티 시퀀서 비교

	단일 시퀀서	멀티 시퀀서 (op-conductor)
장애 복구	수동 재시작	자동 failover (수초)
인프라 비용	낮음	높음 (3x)
운영 복잡도	단순	복잡 (Raft 관리)
블록 연속성	재시작 시 공백 가능	공백 최소화
이중 시퀀싱 위험	없음	Raft 로 방지

depositTxs 생성 과정

depositTxs 는 L1 → L2 방향의 자산/메시지 전달을 위한 트랜잭션입니다. L1 의 컨트랙트 이벤트로부터 L2 트랜잭션이 자동으로 파생됩니다.

전체 흐름

```
[사용자]
  | ETH + calldata
  ▼
[L1: OptimismPortal.depositTransaction()]
  | emit TransactionDeposited(from, to, version, opaqueData)
  ▼
[L1 블록체인에 이벤트 로그 기록]
  |
  ▼
[op-node: PreparePayloadAttributes()]
  | FetchReceipts(L1 block) → 영수증 스캔
  | DeriveDeposits(receipts) → 이벤트 파싱
  | UnmarshalDepositLogEvent(log) → DepositTx 생성
  ▼
[L2 블록 첫 번째 자리에 강제 포함]
  L1InfoTx → depositTxs → upgradeTx → user txs
```

1 단계: 사용자가 L1 에서 depositTransaction() 호출

```
// packages/contracts-bedrock/src/L1/OptimismPortal2.sol
function depositTransaction(
    address _to,      // L2 수신 주소
    uint256 _value,   // L2 에서 전송할 ETH
    uint64 _gasLimit, // L2 가스 한도
    bool _isCreation, // 컨트랙트 생성 여부
    bytes memory _data // calldata
) public payable {
    // ...
```

```
// opaqueData 패킹: mint + value + gasLimit + isCreation + data
bytes memory opaqueData = abi.encodePacked(msg.value, _value, _gasLimit, _isCreation, _data);

// L1 이벤트 발행 → op-node 가 감지
emit TransactionDeposited(from, _to, DEPOSIT_VERSION, opaqueData);
}
```

L1 블록에 TransactionDeposited 이벤트 로그가 기록됩니다.

2 단계: op-node 가 새 L1 epoch 에서 영수증 수집

PreparePayloadAttributes()에서 새로운 L1 origin 으로 바뀌는 첫 번째 L2 블록에서만 deposits 를 수집합니다:

```
// op-node/rollup/derive/attributes.go
// L1 origin 이 바뀐 경우에만 → epoch 의 첫 번째 L2 블록
if l2Parent.L1Origin.Number != epoch.Number {
    // L1 블록의 모든 트랜잭션 영수증 가져오기
    info, receipts, err := ba.l1.FetchReceipts(ctx, epoch.Hash)

    // 영수증에서 deposit 이벤트 파생
    deposits, err := DeriveDeposits(receipts, ba.rollupCfg.DepositContractAddress)

    depositTxs = deposits
    seqNumber = 0 // epoch 첫 번째 블록
} else {
    depositTxs = nil // 같은 epoch 의 두 번째 이후 L2 블록은 deposits 없음
    seqNumber = l2Parent.SequenceNumber + 1
}
```

3 단계: 영수증 스캔 → 이벤트 필터링

DeriveDeposits() → UserDeposits():

```
// op-node/rollup/derive/deposits.go
func UserDeposits(receipts []*types.Receipt, depositContractAddr common.Address) ([]*types.DepositTx, error) {
    for i, rec := range receipts {
        if rec.Status != types.ReceiptStatusSuccessful {
            continue // 실패한 트랜잭션은 무시
        }
        for j, log := range rec.Logs {
            // OptimismPortal 주소 + TransactionDeposited 이벤트 시그니처 필터
            if log.Address == depositContractAddr &&
                log.Topics[0] == DepositEventABIHash {
                dep, err := UnmarshalDepositLogEvent(log)
                // ...
            }
        }
    }
}
```

```

        out = append(out, dep)
    }
}
return out, result
}

```

- 성공한 L1 tx 만 대상 (실패한 tx 의 deposit 은 포함 안 함)
- depositContractAddr (OptimismPortal) 주소에서 발생한 TransactionDeposited 이벤트만 추출

4 단계: 이벤트 로그 → DepositTx 구조체 변환

UnmarshalDepositLogEvent():

```

// op-node/rollup/derive/deposit_log.go
func UnmarshalDepositLogEvent(ev *types.Log) (*types.DepositTx, error) {
    // Topics[1] = from 주소 (indexed)
    from := common.BytesToAddress(ev.Topics[1][12:])
    // Topics[2] = to 주소 (indexed)
    to := common.BytesToAddress(ev.Topics[2][12:])

    // opaqueData 파싱: mint, value, gasLimit, isCreation, data
    opaqueData := ev.Data[64 : 64+opaqueContentLength.Uint64()]

    var dep types.DepositTx
    // SourceHash: L1 블록해시 + 로그 인덱스로 고유 식별자 생성
    source := UserDepositSource{
        L1BlockHash: ev.BlockHash,
        LogIndex:     uint64(ev.Index),
    }
    dep.SourceHash = source.SourceHash()
    dep.From = from
    dep.IsSystemTransaction = false

    err = unmarshalDepositVersion0(&dep, to, opaqueData)
    return &dep, nil
}

func unmarshalDepositVersion0(dep *types.DepositTx, to common.Address, opaqueData []byte) error {
    dep.Mint = new(big.Int).SetBytes(opaqueData[0:32]) // L2 에서 민팅할 ETH
    dep.Value = new(big.Int).SetBytes(opaqueData[32:64]) // L2 에서 전송할 ETH
    dep.Gas = gas.Uint64() // L2 가스 한도
    dep.To = &to // L2 수신 주소
    dep.Data = opaqueData[73:] // calldata
}

```

5 단계: SourceHash — 중복 방지 핵심 메커니즘

```
// op-node/rollup/derive/deposit_source.go
func (dep *UserDepositSource) SourceHash() common.Hash {
    var input [32 * 2]byte
    copy(input[:32], dep.L1BlockHash[:]) // L1 블록 해시
    binary.BigEndian.PutUint64(input[32*2-8:], dep.LogIndex) // 로그 인덱스
    depositIDHash := crypto.Keccak256Hash(input[:])

    var domainInput [32 * 2]byte
    binary.BigEndian.PutUint64(domainInput[32-8:32], UserDepositSourceDomain) // domain=0
    copy(domainInput[32:], depositIDHash[:])
    return crypto.Keccak256Hash(domainInput[:])
}
```

SourceHash = keccak256(domain=0 || keccak256(L1BlockHash || LogIndex))

이 해시로 동일한 deposit 이 두 번 실행되는 것을 방지합니다.

6 단계: L2 블록에 강제 포함

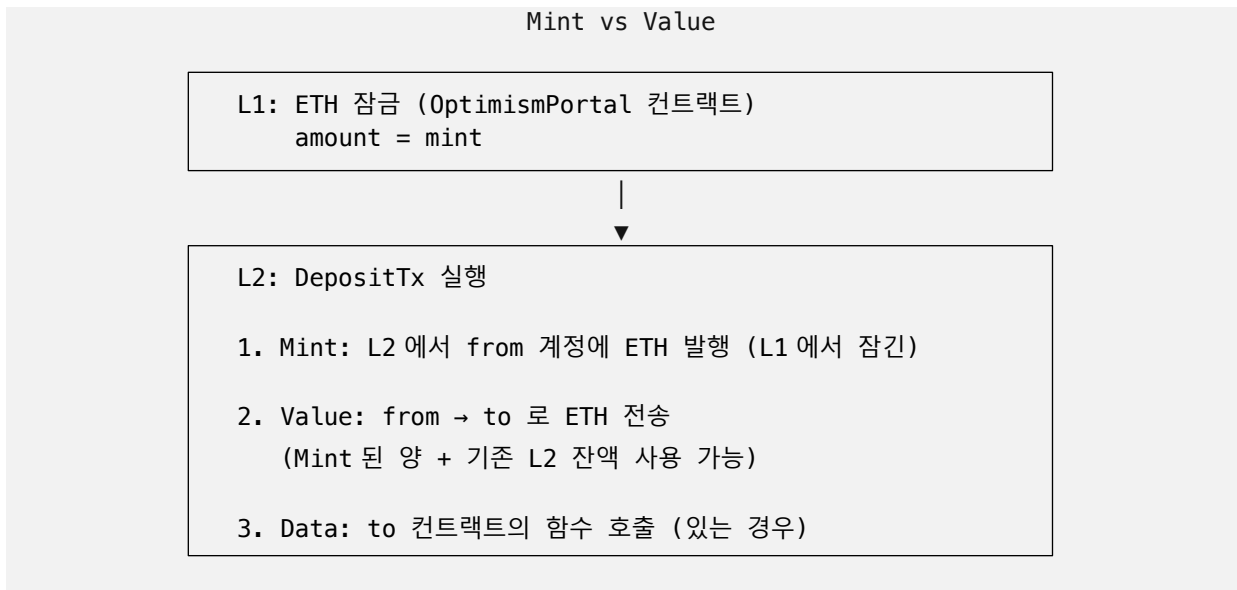
```
// op-node/rollup/derive/attributes.go
txs := make([]hexutil.Bytes, 0, 1+len(depositTxs)+len(upgradeTxs))
txs = append(txs, l1InfoTx) // 항상 첫 번째
txs = append(txs, depositTxs...) // user deposits
txs = append(txs, upgradeTxs...) // upgrade txs
```

전체 흐름 요약

L1 트랜잭션	L1 블록	L2 블록 (epoch 첫 번째)
사용자 → OptimismPortal depositTransaction() (ETH + calldata)	→ emit TransactionDeposited (from, to, opaqueData) ← FetchReceipts() ← ← DeriveDeposits() ← ← UnmarshalDepositLogEvent() ←	[0] L1InfoTx [1] DepositTx ← 여기! [2] DepositTx (여러개 가능) [3] user txpool txs

핵심 보장:

- 성공한 L1 tx 의 deposit 만 포함
- SourceHash 로 중복 실행 방지
- L1 origin 이 바뀌는 첫 L2 블록에만 포함
- 가스비 없이 실행 (IsSystemTransaction=false 이지만 L2 deposit 규칙 적용)



depositTransaction 에서 ETH 처리와 ETHLockbox 구현

1. msg.value 가 msg.sender 에서 차감되는 부분

Solidity/EVM 레벨에서 자동으로 처리됩니다.

```
function depositTransaction(
    address _to,
    uint256 _value,
    uint64 _gasLimit,
    bool _isCreation,
    bytes memory _data
)
public
payable // ← 이 키워드로 인해 msg.value 가 자동 전송됨
metered(_gasLimit)
{
```

EVM 동작 원리:

단계	설명
1	사용자가 트랜잭션에 value 필드 설정 (예: 1 ETH)
2	EVM 이 트랜잭션 실행 전에 msg.sender 잔액에서 msg.value 차감
3	차감된 ETH 가 호출된 컨트랙트 (OptimismPortal2)로 전송됨
4	컨트랙트 함수 로직 실행

핵심: msg.value 차감은 Solidity 코드에 없습니다. EVM 이 payable 함수 호출 시 자동으로 처리합니다.

2. ETHLockbox 로 보내는 부분

```
// packages/contracts-bedrock/src/L1/OptimismPortal2.sol
// If using ETHLockbox, lock the ETH in the ETHLockbox.
if (_isUsingLockbox()) {
    if (msg.value > 0) ethLockbox.lockETH{ value: msg.value }();
}
```

여기서 { value: msg.value }는 현재 컨트랙트(OptimismPortal2)가 보유한 ETH 를 lockETH 호출 시 함께 전송합니다.

3. ETHLockbox 구현

핵심 기능:

함수	설명
lockETH()	Deposit 시 Portal 이 호출, ETH 를 Lockbox 에 저장
unlockETH(uint256 _value)	Withdrawal finalize 시 Portal 이 호출, ETH 를 Portal 로 반환

lockETH 구현

```
// 141:150:packages/contracts-bedrock/src/L1/ETHLockbox.sol
/// @notice Locks ETH in the lockbox.
///      Called by an authorized portal on a deposit to lock the ETH value.
function lockETH() external payable {
    // Check that the sender is authorized to trigger this function.
    IOptimismPortal sender = IOptimismPortal(payable(msg.sender));
    if (!authorizedPortals[sender]) revert ETHLockbox_Unauthorized();

    // Emit the event.
    emit ETHLocked(sender, msg.value);
}
```

unlockETH 구현:

```
// packages/contracts-bedrock/src/L1/ETHLockbox.sol
/// @notice Unlocks ETH from the lockbox.
///      Called by an authorized portal when finalizing a withdrawal that requires ETH.
///      Cannot be called if the lockbox is paused.
/// @param _value The amount of ETH to unlock.
function unlockETH(uint256 _value) external {
    // Unlocks are blocked when paused, locks are not.
    if (paused()) revert ETHLockbox_Paused();

    // Check that the sender is authorized to trigger this function.
    IOptimismPortal sender = IOptimismPortal(payable(msg.sender));
    if (!authorizedPortals[sender]) revert ETHLockbox_Unauthorized();

    // Check that we have enough balance to process the unlock.
}
```

```

if (_value > address(this).balance) revert ETHLockbox_InsufficientBalance();

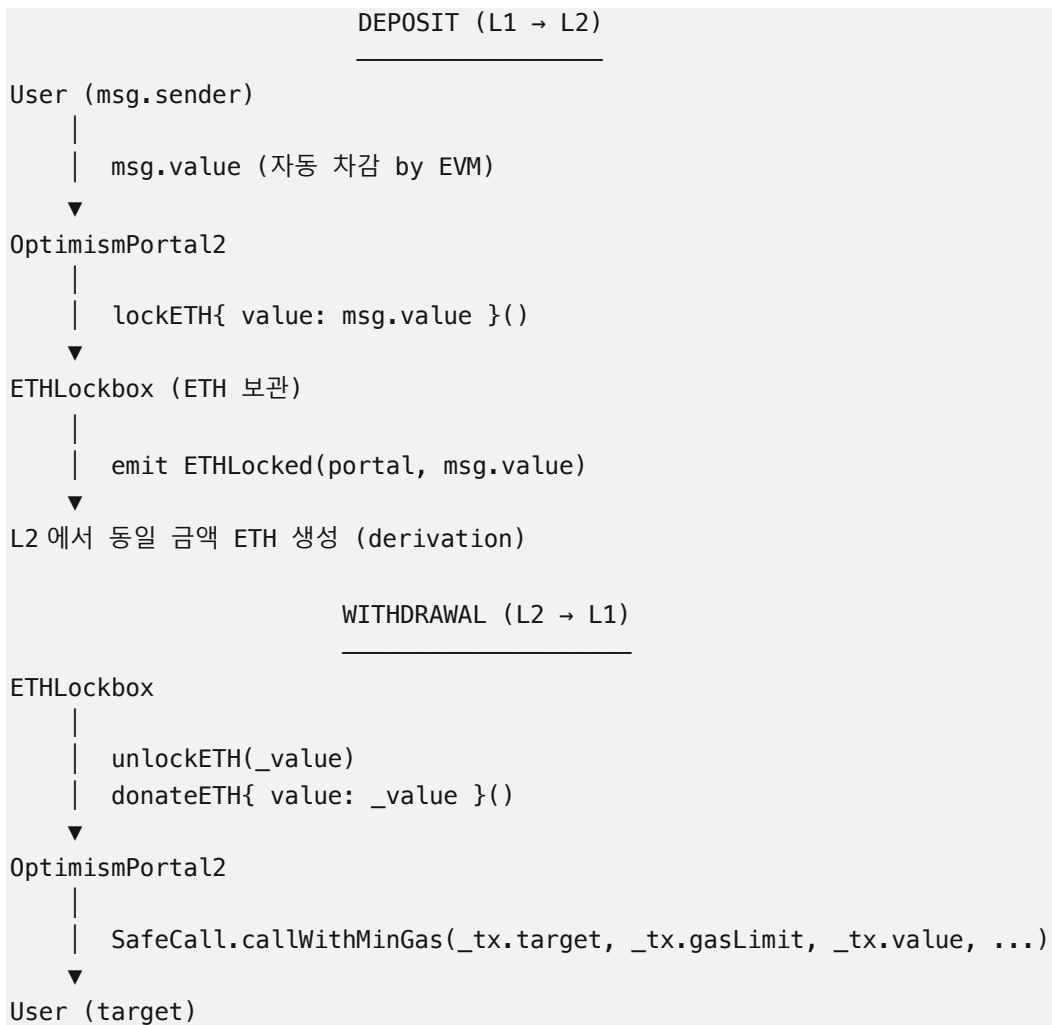
// Check that the sender is not executing a withdrawal transaction.
if (sender.l2Sender() != Constants.DEFAULT_L2_SENDER) {
    revert ETHLockbox_NoWithdrawalTransactions();
}

// Using donateETH to avoid triggering a deposit.
sender.donateETH{ value: _value }();

// Emit the event.
emit ETHUnlocked(sender, _value);
}

```

ETH 흐름 시각화



요약

질문	답변
----	----

msg.sender 차감 위치	EVM 이 payable 함수 호출 시 자동 처리 (코드에 없음)
ETHLockbox 위치	src/L1/ETHLockbox.sol
Lock 함수	lockETH() external payable - 그냥 ETH 받고 이벤트 emit
Unlock 함수	unlockETH(uint256) - 권한 검증 후 donateETH 로 Portal 에 전송

op_node: op_batcher

설명

batcher 는 sequencer 가 block 을 만들 때 포함되었던 트랜잭션들을 L1 에 게시하는 역할을 한다. 이를 통해 검증자가 게시된 데이터를 통해 블록을 재구성 할 수 있도록 하기 위함이다. 초기의 optimistic rollup 의 경우 비용을 줄이기 위해 여러 tx 데이터를 압축하여 calldata 형태로 L1 address(BatchInboxAddress)에 전송하도록 설계 되었다. 그 뒤에 EIP-4844 가 포함된 Execution Layer 의 Dencun 하드포크(EIP-4844 포함)이후 blob 트랜잭션이 생긴이후 보다 저렴한 비용의 blob-carrying tx(type-3)로 보낼 수 있게 되었다. 이 때 blob 의 최대 사이즈는 약 768kb 정도이다. Pectra 하드포크이후에는 calldata 비용이 2.5 배 증가하여 기존에 비해 상대적으로 비싸게 됨으로써 blob 을 더 많이 사용하도록 유도 되었다. 그러나 blob 모드의 단점이 존재하는 데, 그것은 blob 데이터의 Beacon Layer 에 보관되는 기간이 영구저장이 아니라 일시적이라는 것이다. 보통 보관되는 기간은 4,096 epochs ~ 약 18 일이며 이 기간이 지나면 pruning 된다. 이는 op_stack 이 challenge time 이 7 일인 것을 고려하면 충분한 시간이지만, 만약 롤업노드가 새로 합류한다거나 오래 오프라인이었다가 복귀하는 경우 에는 과거 데이터가 필요한 바 "historical blob"을 제공하는 Beacon endpoint 가 필요하다.

batcher 가 데이터를 보내는 L1 의 BatchInboxAddress 는 contract 가 아니라 가스비 절감을 위해 eoa 로 지정된다. 배치데이터는 calldata 또는 blob 에 저장되므로 컨트랙트 로직이 불필요하고 데이터만 L1 에 기록되므로, 실행할 필요가 없다

DA(data availability)를 calldata tx 를 이용할지 blob tx 를 이용할 지 결정하는 것은 평상시에는 비용(가스비)으로 결정하지만, 만약 throttling 상황(Sequencer 블록 중 아직 제출되지 않은 Unsafe 데이터가 과도하게 쌓일 경우)에서는 비용에 관계없이 무조건 blob 타입으로 설정하여 비용보다는 처리량을 우선하게 된다. throttling 상황에서는 batcher 가 sequencer 의 처리량에도 제한을 가할 수 있는데, sequencer 의 MaxBlockSize 와 MaxTxSize 를 줄여서 sequencer 가 생성하는 Unsafe L2 tx 의 양을 줄이게 하여, 결국은 batcher 에 들어오는 유입량을 줄이게 한다. 이 때의 정책은 step, linear, quadratic, pid 가 있다. 최근 AltDA(Alternative Data Availability)는 DA 를 L1 대신 외부 DA 서비스를 이용하는 모드가 생겼다. 이 때 Batchr 는 L1 에는 Commitment 만 저장하고 다른 DA Server 에 tx data 를 저장한다. blob 모드에 비해서도 비용절감의 효과가 있다. 하지만 외부 서버에 의존하는 단점도 존재한다. 외부서버로는 AWS S3 의 Storage 나 Avail, Celestia, EigenDA 등의 외부 DA 레이어가 존재한다.

L2 블록을 압축하는 데는 Compressor 를 사용한다. L2 block 들을 하나로 묶은 뒤 압축하여 여러 프레임으로 나누는 채널에서 Compressor 는 L1 에 제출할 데이터 크기를 줄이고, 채널이 언제 가득 찰는지 판단하는 역할을 한다. Channel 데이터는 Frame 으로 변환되어 L1 에 제출되는 데 L1 에는 여러 제약이 있다. 첫째로 Frame 크기 제한(Calldata ~ 129kb, Blob ~ 128kb), Channel Timeout(L1 블록 기준 일정시간 내 모든 Frame 제출 필요), Sequencing Window (L2 블록이 L1 에 포함되어야 하는 시간 제한) 등이다. 채널이 가득 찰는지 판단이 필요한 이유는 다음과 같다.

* 채널이 너무 크면?

프레임 수 증가 -> L1 TX 증가 -> 비용 증가

Timeout 전에 모든 Frame 제출 못할 위험

채널 전체 무효화 가능

* 채널이 너무 작으면?

압축 효율 저하 (적은데이터 = 낮은 압축률)

L1 Tx 오버헤드 증가

비용 비효율

따라서, Channel Full 감지후 현재 채널을 닫아 압축을 완료후 Frame 을 생성 후 Frame 을 제출한 후 L1 Tx 로 전송하고 새 채널을 생성한다. Compressor 의 방식에는 압축률 추정을 빠르게 할 수 있는 Ratio 와 이중버퍼로 정확한 크기를 보장하는 Shadow 방식이 있다. 압축 알고리즘으로는 Zlib 나 Brotli 를 사용할 수 있는데 압축률이 좋은 Brotli 를 최근 많이 사용한다. calldata 방식은 1 개의 Frame 을 보내고, blob 의 경우 여러 프레임 을 담아 보낸다.

유도(derivation)

유도는 L1 으로 부터 L2 tx 들을 다운로드 받아 그 tx 들을 검증자에서 재구성 하면서 unsafe 상태(sequencer 가 처음 블록을 만들 때의 상태)를 safe 상태로 업그레이드하는 것을 말한다. 이때 calldata 모드에서 발송된 배치데이터는 L1 의 해당 tx 들을 스캔해서 calldata 를 읽고, blob 모드의 경우 L1 tx 에서 참조 정보를 얻어 Beacon Layer 에서 blob sidecar 를 받아 유도한다.

FindL1Origin 함수의 역할

- L1-L2 시간 동기화: L2 블록이 L1 블록보다 너무 앞서지 않도록 제한
- Deposit 처리: 새 L1 origin 으로 이동해야 해당 L1 블록의 deposit 포함 가능
- Sequencer Drift 제한: L2 가 L1 보다 MaxSequencerDrift 이상 앞서면 빈 블록 생성 Derivation 결정
론성: Verifier 가 동일한 L1 origin 선택 로직을 재현 가능

op-batcher L1 트랜잭션 전송 흐름

op-batcher 에서 L1 으로 트랜잭션을 보내는 과정은 여러 함수가 단계별로 협력하여 이루어집니다. 주요 함수들을 다음과 같다.

1. 상위 레벨: publishStateToL1

```
// op-batcher/batcher/driver.go
// publishStateToL1 queues up all pending TxData to be published to the L1, returning when there is no more data to
// queue for publishing or if there was an error queuing the data.
func (l *BatchSubmitter) publishStateToL1(ctx context.Context, queue *txmgr.Queue[txRef], receiptsCh chan txmgr.TxReceipt[txRef], daGroup *errgroup.Group, pi pubInfo) {
    for {
        select {
```

```

    case <-ctx.Done():
        return
    default:
    }
    // ... txmgr 및 txpool 상태 확인 ...
    err := l.publishTxToL1(ctx, queue, receiptsCh, daGroup, pi)
    // ...
}
}

```

이 함수는 루프를 돌면서 대기 중인 모든 트랜잭션 데이터를 L1 에 게시한다.

2. 핵심 함수: publishTxToL1

```

// op-batcher/batcher/driver.go
// publishTxToL1 submits a single state tx to the L1
func (l *BatchSubmitter) publishTxToL1(ctx context.Context, queue *txmgr.Queue[txRef], receiptsCh chan
txmgr.TxReceipt[txRef], daGroup *errgroup.Group, pi pubInfo) error {
    // 1. L1 tip 블록 조회
    l1tip, isPectra, err := l.l1Tip(ctx)

    // 2. 채널 매니저에서 트랜잭션 데이터 가져오기
    txdata, err := l.channelMgr.TxData(l1tip.ID(), isPectra, params.IsThrottling(), pi)

    // 3. 트랜잭션 전송
    if err = l.sendTransaction(txdata, queue, receiptsCh, daGroup); err != nil {
        return fmt.Errorf("BatchSubmitter.sendTransaction failed: %w", err)
    }
    return nil
}

```

역할:

- L1 최신 블록(tip) 정보 조회
- channelMgr.TxData()로 전송할 배치 데이터 수집
- sendTransaction()을 호출하여 실제 전송

3. 트랜잭션 생성 및 전송: sendTransaction

sendTransaction 은 AltDA 면 외부 DA 에 올린 뒤 커밋만 calldata 로 L1 에 보내고, 그렇지 않으면 txdata.asBlob 에 따라 blob 후보 또는 calldata 후보를 만든다. 수신 주소는 항상 RollupConfig.BatchInboxAddress 이다.

```

// op-batcher/batcher/driver.go
// sendTransaction creates & queues for sending a transaction to the batch inbox address with the given
`txData`.

```

```

func (l *BatchSubmitter) sendTransaction(txdata txData, queue *txmgr.Queue[txRef], receiptsCh chan
txmgr.TxReceipt[txRef], daGroup *errgroup.Group) error {
    // AltDA 사용 시 별도 처리
    if l.Config.UseAltDA {
        l.publishToAltDAAndL1(txdata, queue, receiptsCh, daGroup)
        return nil
    }

    var candidate *txmgr.TxCandidate
    if txdata.asBlob {
        // Blob 트랜잭션 생성
        candidate, err = l.blobTxCandidate(txdata)
    } else {
        // Calldata 트랜잭션 생성
        candidate = l.calldataTxCandidate(txdata.CallData())
    }

    l.sendTx(txdata, false, candidate, queue, receiptsCh)
    return nil
}

```

역할:

- Blob 트랜잭션 또는 Calldata 트랜잭션 후보(TxCandidate) 생성
- 두 가지 전송 방식 지원:
 - ✓ Blob: EIP-4844 blob 트랜잭션 (더 저렴)
 - ✓ Calldata: 기존 calldata 방식

4. 트랜잭션 후보 생성 함수들

Blob 트랜잭션:

```

// op-batcher/batcher/driver.go
func (l *BatchSubmitter) blobTxCandidate(data txData) (*txmgr.TxCandidate, error) {
    blobs, err := data.Blobs()
    // ...
    return &txmgr.TxCandidate{
        To:    &l.RollupConfig.BatchInboxAddress, // BatchInbox 주소로 전송
        Blobs: blobs,
    }, nil
}

```

Calldata 트랜잭션:

```

// op-batcher/batcher/driver.go
func (l *BatchSubmitter) calldataTxCandidate(data []byte) *txmgr.TxCandidate {

```

```

return &txmgr.TxCandidate{
    To:    &l.RollupConfig.BatchInboxAddress, // BatchInbox 주소로 전송
    TxData: data,
}
}

```

5. 최종 전송: sendTx

```

// op-batcher/batcher/driver.go
// sendTx uses the txmgr queue to send the given transaction candidate
func (l *BatchSubmitter) sendTx(txdata txData, isCancel bool, candidate *txmgr.TxCandidate, queue
TxSender[txRef], receiptsCh chan txmgr.TxReceipt[txRef]) {
    floorDataGas, err := core.FloorDataGas(candidate.TxData)
    // ...
    candidate.GasLimit = floorDataGas

    queue.Send(txRef{id: txdata.ID(), isCancel: isCancel, isBlob: txdata.asBlob}, *candidate, receiptsCh)
}

```

역할:

- 가스 한도 계산
- txmgr.Queue.Send()를 통해 비동기로 L1 에 트랜잭션 전송

6. txmgr Queue (실제 L1 전송)

```

// op-service/txmgr/queue.go
func (q *Queue[T]) Send(id T, candidate TxCandidate, receiptCh chan TxReceipt[T]) {
    group, ctx := q.groupContext()
    responseChan := make(chan SendResponse, 1)
    // ...
    q.txMgr.SendAsync(ctx, candidate, responseChan) // 실제 L1 전송!
}

```

최종적으로 SimpleTxManager.SendAsync()가 호출되어:

- 트랜잭션 서명
- L1 에 브로드캐스트
- 컨펌 대기 및 재시도 (가스 가격 자동 bump)

전체 흐름 요약

```

publishStateToL1()
├─ publishTxToL1()           // 단일 트랜잭션 처리
│   └─ l1Tip()               // L1 최신 블록 조회
│   └─ channelMgr.TxData()   // 배치 데이터 수집
│   └─ sendTransaction()     // 트랜잭션 생성
│       └─ blobTxCandidate() 또는 calldataTxCandidate()

```

```
└─ sendTx()          // 큐에 추가
   └─ queue.Send()   // txmgr 를 통해 L1 전송
      └─ txMgr.SendAsync() // 실제 서명 및 브로드캐스트
```

목적지: 모든 트랜잭션은 RollupConfig.BatchInboxAddress 로 전송되며, 이는 L2 배치 데이터를 수신하는 L1 의 특수 주소입니다.

op_node: derivation

설명

OP Stack 에서 op-node 의 Derivation(도출) 과정은 L1(Ethereum)에 저장된 데이터를 바탕으로 L2(Optimism)의 상태를 재구성하고 동기화하는 핵심 메커니즘입니다. OP Stack 의 보안 모델 전체가 이 과정의 신뢰성에 기반합니다. 주요 역할과 세부 단계는 다음과 같습니다.

1. 주요 역할

- L2 상태의 결정론적 재구성 (Deterministic Reconstruction): L2 노드가 완전히 새로 생성되거나 장애가 발생하더라도, L1 에 기록된 데이터(Batcher 가 올린 데이터)만 있으면 누구나 동일한 L2 체인을 복구할 수 있게 합니다. 이는 시퀀서(Sequencer)를 완전히 신뢰하지 않아도 체인의 무결성을 보장하는 근거가 됩니다.
- L1 과 L2 의 동기화: L1 에서 발생하는 트랜잭션(예: L1 → L2 입금/메시지 전달)과 L1 에 저장된 L2 트랜잭션 배치를 읽어와 L2 엔진(op-geth)이 실행할 수 있는 형태로 전달합니다.
- 데이터 가용성(DA) 확인 calldata 나 blob data 를 이용하여 배치 데이터를 추출하는데, L1 tx 로 부터 calldata 의 경우 직접 데이터를 얻고, blob 의 경우 tx 로부터 blobhash 를 추출한 뒤, beacon api 로 실제 blob 을 다운로드 한다. L1 의 calldata 또는 blobs 에 저장된 바이너리 데이터를 추출하고 압축을 해제하여, 실제 L2 트랜잭션 데이터가 유효한지 검증합니다.

표로 정리하면 다음과 같다.

구분	Calldata (Pre-Ecotone)	Blob (Post-Ecotone)
데이터 위치	L1 트랜잭션 calldata	Beacon Chain sidecar
최대 크기	~130 KB (블록 가스 제한)	~126 KB per blob (최대 6 blobs)
비용	16 gas/byte (non-zero)	~1 gas/byte 수준
보존 기간	영구	~18 일 (4096 epochs)
데이터 소스	CalldataSource	BlobDataSource

2. Derivation 의 세부 단계

Derivation 파이프라인은 여러 단계를 거쳐 데이터를 정제합니다.

- 1) L1 Traversal: L1 블록을 순차적으로 탐색하며 배치(Batcher)가 보낸 데이터와 L1 시스템 이벤트를 수집합니다.
- 2) Data Extraction: L1 트랜잭션 내의 calldata 또는 EIP-4844 블록(Blobs)에서 바이너리 데이터를 추출합니다.

- 3) Decompression & Decoding: 압축된 데이터를 풀고, OP Stack 의 채널(Channel) 및 프레임(Frame) 구조에 맞게 디코딩하여 개별 'Batch' 단위로 만듭니다.
- 4) Batch Filtering: 중복되거나 유효하지 않은 배치, 혹은 시퀀싱 윈도우(Sequencing Window)를 벗어난 데이터를 걸러냅니다.
- 5) Payload Attributes Generation: 정제된 데이터를 바탕으로 L2 블록을 실행하기 위한 PayloadAttributes 를 생성합니다. 여기에는 L1 정보 삽입 트랜잭션(L1 Info Deposit)과 실제 사용자 트랜잭션들이 포함됩니다.
- 6) Engine API Communication: 생성된 속성들을 Engine API 를 통해 op-geth(Execution Engine)로 전달하여 실제로 L2 블록을 생성하고 상태를 업데이트합니다.

3. Derivation 의 중요성

- 검증자(Verifier)의 역할: 시퀀서가 생성한 블록이 정말 L1 에 기록된 데이터와 일치하는지 검증할 때 이 프로세스를 사용합니다.
- 검열 저항성: 시퀀서가 특정 사용자의 트랜잭션을 거부하더라도, 사용자가 L1 에 직접 트랜잭션을 강제 삽입(Force Inclusion)하면 Derivation 과정에서 이를 인식하여 L2 체인에 포함시킵니다.
- 신뢰 최소화: L2 의 보안성을 L1(Ethereum) 수준으로 끌어올리는 핵심 장치입니다.

Derivation(유도)은 L1 블록 데이터로부터 L2 블록을 결정론적으로 재구성하는 핵심 프로세스입니다. OP Stack 의 보안 모델 전체가 이 과정의 신뢰성에 기반합니다.

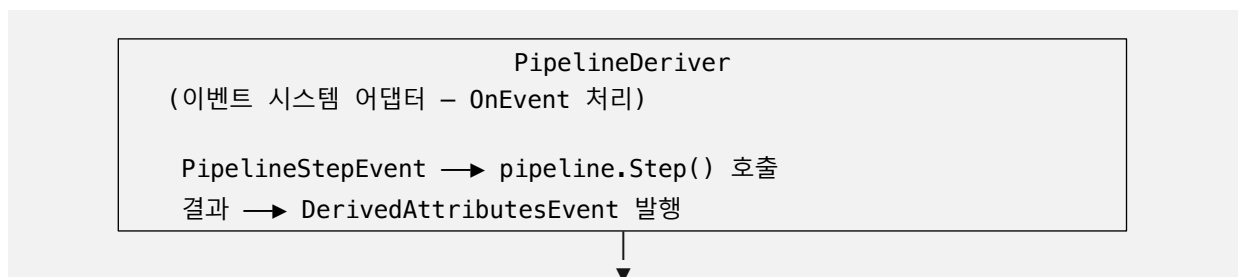
상세 프로세스

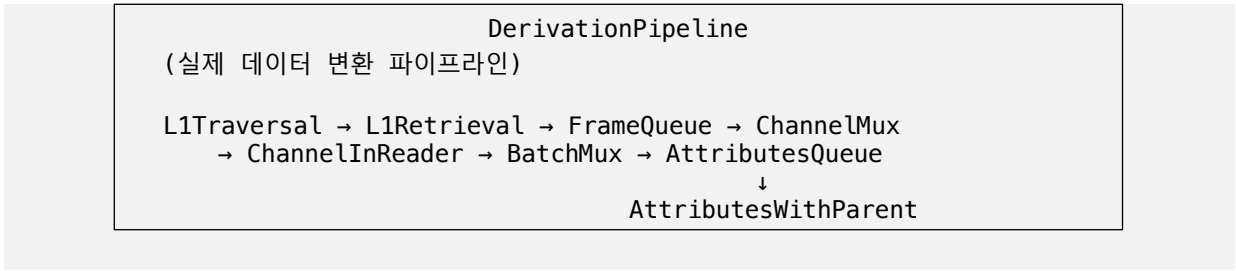
1. Derivation 의 존재 이유

OP Stack 은 "낙관적 롤업"이기 때문에:

- Sequencer 는 빠른 처리를 위해 L2 블록을 먼저 만들고 → L1 에 배치(batch)로 제출합니다
- 모든 노드(검증자 포함)는 L1 에 올라온 배치 데이터를 보고 L2 블록을 독립적으로 재구성할 수 있어야 합니다
- 이 재구성 과정이 바로 Derivation 이며, 결과가 일치하지 않으면 Fault Proof 로 이의 제기할 수 있습니다

2. 전체 아키텍처: 두 레이어





3. DerivationPipeline 스테이지별 상세 역할

스테이지	파일	역할
L1Traversal	l1_traversal.go	L1 블록을 번호 순서대로 하나씩 전진시킴. L1 reorg 감지
L1Retrieval	l1_retrieval.go	L1 블록에서 배치 데이터 원본 읽기 (calldata 또는 blob)
AltDA		Alternative DA (Plasma 등) 데이터 패칭
FrameQueue	frame_queue.go	데이터를 고정 크기의 프레임 단위로 파싱하여 버퍼링
ChannelMux	channel_mux.go	Frame 들을 Channel 로 조립 (pre/post Holocene)조립
ChannelInReader	channel_in_reader.go	채널 데이터를 압축 해제하여 Batch 로 디코딩
BatchMux	batch_mux.go	Batch 검증 및 큐잉 (Singular/Span batch 처리)
AttributesQueue	attributes_queue.go	배치를 PayloadAttributes 로 변환 (최종 산출물) 2 Engine 전송

PayloadAttributes 의 Txs 는 L1InfoTx, Deposits, BatchTxs 로 구성된다. Derivation 과정을 거친 L2 블록만이 Pending Safe/Local Safe 로 승격

핵심 데이터 흐름:

L1 블록 (calldata/blob)

- Frame (고정 크기 조각)
- Channel (압축된 배치 묶음)
- Batch (L2 트랜잭션 묶음)
- PayloadAttributes (L2 블록 생성 입력)

- 1) Driver 에서 Derivation Pipeline 설정
- driver.go 에서 NewDriver 함수가 호출될 때 파이프라인이 생성

derivationPipeline := derive.NewDerivationPipeline(log, cfg, depSet, verifConfDepth, l1Blobs, altDA, l2Source, metrics, indexingMode, l1ChainConfig)

pipelineDriver := derive.NewPipelineDriver(driverCtx, derivationPipeline)
sys.Register("pipeline", pipelineDriver)

2) DerivationPipeline 초기화

pipeline.go 에서 NewDerivationPipeline 함수가 각 스테이지를 생성하고 연결합니다:

```
// op-node/rollup/derive/pipeline.go
func NewDerivationPipeline(log log.Logger, rollupCfg *rollup.Config, depSet DependencySet, l1Fetcher
L1Fetcher, l1Blobs L1BlobsFetcher,
    altDA AltDAInputFetcher, l2Source L2Source, metrics Metrics, managedBySupervisor bool,
l1ChainConfig *params.ChainConfig,
) *DerivationPipeline {
    spec := rollup.NewChainSpec(rollupCfg)
    var l1Traversal L1TraversalStage
    if managedBySupervisor {
        l1Traversal = NewL1TraversalManaged(log, rollupCfg, l1Fetcher)
    } else {
        l1Traversal = NewL1Traversal(log, rollupCfg, l1Fetcher)
    }
    dataSrc := NewDataSourceFactory(log, rollupCfg, l1Fetcher, l1Blobs, altDA)
    l1Src := NewL1Retrieval(log, dataSrc, l1Traversal)
    frameQueue := NewFrameQueue(log, rollupCfg, l1Src)
    channelMux := NewChannelMux(log, spec, frameQueue, metrics)
    chInReader := NewChannelInReader(rollupCfg, log, channelMux, metrics)
    batchMux := NewBatchMux(log, rollupCfg, chInReader, l2Source)
    attrBuilder := NewFetchingAttributesBuilder(rollupCfg, l1ChainConfig, depSet, l1Fetcher, l2Source)
    attributesQueue := NewAttributesQueue(log, rollupCfg, attrBuilder, batchMux)

    // Reset from ResetEngine then up from L1 Traversal. The stages do not talk to each other during
    // the ResetEngine, but after the ResetEngine, this is the order in which the stages could talk to each
    other.
    // Note: The ResetEngine is the only reset that can fail.
    stages := []ResettableStage{l1Traversal, l1Src, altDA, frameQueue, channelMux, chInReader, batchMux,
attributesQueue}
    // ... pipeline 반환
```

L1Traversal 또는 L1TraversalManaged (interop 모드)가 첫 번째 스테이지로 생성됨 각 스테이지가 이전 스테이지를 참조하여 pull-based 파이프라인 구조 형성

3) L1Traversal 은 L1 체인을 순회하며 다음 L1 블록을 제공하는 가장 기본적인 스테이지입니다.

4) Derivation 흐름

① Reset 이벤트 흐름 시스템이 시작되거나 reorg 가 감지되면 ResetEvent 가 발생합니다.

```
// sync_deriver.go
func (s *SyncDeriver) onResetEvent(ctx context.Context, x rollup.ResetEvent) {
```

```

if s.ManagedBySupervisor {
    s.Log.Warn("Encountered reset when managed by op-supervisor, waiting for op-supervisor",
"err", x.Err)
    // IndexingMode will pick up the ResetEvent
    return
}
// If the system corrupts, e.g. due to a reorg, simply reset it
s.Log.Warn("Deriver system is resetting", "err", x.Err)
s.Emitter.Emit(ctx, engine.ResetEngineRequestEvent{})
s.StepDeriver.RequestStep(ctx, false)
}

```

- ② Engine Reset 실행 EngineController 가 FindL2Heads 를 호출하여 시작점을 찾고 강제 리셋을 수행합니다.

```

// engine_controller.go
// onResetEngineRequest handles the ResetEngineRequestEvent by finding L2 heads and performing a
force reset
func (e *EngineController) onResetEngineRequest(ctx context.Context) {
    result, err := sync.FindL2Heads(e.ctx, e.rollupCfg, e.l1, e.engine, e.log, e.syncCfg)
    if err != nil {
        e.emitter.Emit(ctx, rollup.ResetEvent{
            Err: fmt.Errorf("failed to find the L2 Heads to start from: %w", err),
        })
        return
    }
    e.forceReset(ctx, result.Unsafe, result.Unsafe, result.Safe, result.Safe, result.Finalized, false)
}

```

- ③ Pipeline Reset 파이프라인이 리셋되면 initialReset 에서 L1 origin 을 찾아 되감습니다.

```

// pipeline.go
// initialReset does the initial reset work of finding the L1 point to rewind back to
func (dp *DerivationPipeline) initialReset(ctx context.Context, resetL2Safe eth.L2BlockRef) error {
    dp.log.Info("Rewinding derivation-pipeline L1 traversal to handle reset")

    dp.metrics.RecordPipelineReset()
    spec := rollup.NewChainSpec(dp.rollupCfg)

    // Walk back L2 chain to find the L1 origin that is old enough to start buffering channel data from.
    pipelineL2 := resetL2Safe
    l1Origin := resetL2Safe.L1Origin
    // ... L1 origin 을 찾을 때까지 L2 체인을 되감음

```

```
}
```

- ④ L1Traversal Reset 각 스테이지가 순서대로 리셋됩니다.

```
// l1_traversal.go
// Reset sets the internal L1 block to the supplied base.
func (l1t *L1Traversal) Reset(ctx context.Context, base eth.L1BlockRef, cfg eth.SystemConfig) error {
    l1t.block = base
    l1t.done = false
    l1t.sysCfg = cfg
    l1t.log.Info("completed reset of derivation pipeline", "origin", base)
    return io.EOF
}
```

- 5) L1Travesal 진행 (AdvanceL1Block) 리셋 후, 파이프라인 Step 이 호출될 때 AdvanceL1Block 으로 다음 L1 블록으로 진행합니다. (Step 함수내에서 NextAttributes 가 io.EOF err 를 리턴할때)

```
// l1_traversal.go
// AdvanceL1Block advances the internal state of L1 Traversal
func (l1t *L1Traversal) AdvanceL1Block(ctx context.Context) error {
    origin := l1t.block
    nextL1Origin, err := l1t.l1Blocks.L1BlockRefByNumber(ctx, origin.Number+1)
    if errors.Is(err, ethereum.NotFound) {
        l1t.log.Debug("can't find next L1 block info (yet)", "number", origin.Number+1, "origin", origin)
        return io.EOF
    } else if err != nil {
        return NewTemporaryError(fmt.Errorf("failed to find L1 block info by number, at origin %s
next %d: %w", origin, origin.Number+1, err))
    }
    if l1t.block.Hash != nextL1Origin.ParentHash {
        return NewResetError(fmt.Errorf("detected L1 reorg from %s to %s with conflicting parent %s",
l1t.block, nextL1Origin, nextL1Origin.ParentID()))
    }

    // Parse L1 receipts of the given block and update the L1 system configuration
    _, receipts, err := l1t.l1Blocks.FetchReceipts(ctx, nextL1Origin.Hash)
    // ... SystemConfig 업데이트

    l1t.block = nextL1Origin
    l1t.done = false
    return nil
}
```

6) 전체 흐름 다이어그램

```
1. op-node 시작 / L1 reorg 감지
    ↓
2. ResetEvent 발생
    ↓
3. EngineController.onResetEngineRequest()
   - FindL2Heads() 호출하여 safe/unsafe/finalized 헤드 찾기
    ↓
4. forceReset()
   - Pipeline.Reset() 호출
   - 각 스테이지 순서대로 리셋
    ↓
5. L1Traversal.Reset(base L1BlockRef, sysCfg)
   - 시작점 L1 블록과 SystemConfig 설정
    ↓
6. SyncStep() → Pipeline.Step() 반복 호출
    ↓
7. AttributesQueue 에서 EOF 반환 시
   → L1Traversal.AdvanceL1Block() 호출
   → 다음 L1 블록으로 진행
    ↓
8. L1Retrieval.NextData()
   → L1Traversal.NextL1Block() 호출하여 L1 블록 획득
   → dataSrc.OpenData()로 해당 블록의 배치 데이터 열기
    ↓
9. 파이프라인 진행 → L2 Attributes 생성 → Engine 에 적용
```

이 과정을 통해 L1 데이터로부터 L2 블록이 유도(derivation)됩니다. L1Traversal 은 이 파이프라인의 데이터 소스 역할을 하며, L1 체인의 블록들을 순차적으로 탐색하여 다른 스테이지들에게 제공합니다.

4. Step() 함수 — 파이프라인의 심장

```
// op-node/rollup/derive/pipeline.go

// Step tries to progress the buffer.
// An EOF is returned if the pipeline is blocked by waiting for new L1 data.
// If ctx errors no error is returned, but the step may exit early in a state that can still be continued.
// Any other error is critical and the derivation pipeline should be reset.
// An error is expected when the underlying source closes.
// When Step returns nil, it should be called again, to continue the derivation process.

func (dp *DerivationPipeline) Step(ctx context.Context, pendingSafeHead eth.L2BlockRef) (outAttrib
*AttributesWithParent, outErr error) {
    defer dp.metrics.RecordL1Ref("l1_derived", dp.Origin())

    dp.metrics.SetDerivationIdle(false)
```

```

defer func() {
    if outErr == io.EOF || errors.Is(outErr, EngineELSyncing) {
        dp.metrics.SetDerivationIdle(true)
    }
}()

// if any stages need to be reset, do that first.
if dp.resetting < len(dp.stages) {
    if !dp.enginesReset {
        return nil, NewResetError(ErrEngineResetReq)
    }
    // ...reset 처리...
}

prevOrigin := dp.origin
newOrigin := dp.attrib.Origin()
if prevOrigin != newOrigin {
    // Check if the L2 unsafe head origin is consistent with the new origin
    if err := VerifyNewL1Origin(ctx, prevOrigin, dp.l1Fetcher, newOrigin); err != nil {
        return nil, fmt.Errorf("failed to verify L1 origin transition: %w", err)
    }
    dp.transformStages(prevOrigin, newOrigin)
    dp.origin = newOrigin
}

if attrib, err := dp.attrib.NextAttributes(ctx, pendingSafeHead); err == nil {
    return attrib, nil
} else if err == io.EOF {
    // If every stage has returned io.EOF, try to advance the L1 Origin
    return nil, dp.traversal.AdvanceL1Block(ctx)
} else if errors.Is(err, EngineELSyncing) {
    return nil, err
} else {
    return nil, fmt.Errorf("derivation failed: %w", err)
}
}

```

Step()은 한 번 호출될 때마다:

- 1) 리셋이 필요하면 리셋부터 처리
- 2) L1 origin 이 바뀌었으면 포크 변환(transformStages) 적용
- 3) AttributesQueue.NextAttributes()로 다음 attributes 시도
- 4) io.EOF 반환 시 → L1 블록을 한 칸 전진(AdvanceL1Block)

5. PipelineDriver — 이벤트 시스템과의 연결

```
// op-node/rollup/derive/deriver.go
func (d *PipelineDriver) OnEvent(ctx context.Context, ev event.Event) bool {
    switch x := ev.(type) {
    case PipelineStepEvent:
        if d.needAttributesConfirmation {
            d.pipeline.log.Debug("Previously sent attributes are unconfirmed to be received")
            return true
        }
        // ...
        attrib, err := d.pipeline.Step(d.ctx, x.PendingSafe)
        // ...
        if err == io.EOF {
            d.emitter.Emit(ctx, DeriverIdleEvent{Origin: d.pipeline.Origin()})
            d.emitter.Emit(ctx, ExhaustedL1Event{L1Ref: d.pipeline.Origin(), LastL2: x.PendingSafe})
        } else if errors.Is(err, ErrReset) {
            d.emitter.Emit(ctx, rollup.ResetEvent{Err: err})
        } else if attrib != nil {
            d.emitDerivedAttributesEvent(ctx, attrib)
        } else {
            d.emitter.Emit(ctx, DeriverMoreEvent{})
        }
    }
}
```

PipelineDriver 는 needAttributesConfirmation 플래그로 흐름 제어를 합니다. 이전에 발행한 attributes 가 엔진에서 확인되기 전까지는 새 attributes 를 생성하지 않습니다 (back-pressure).

6. AttributesWithParent 구조와 역할

```
// op-node/rollup/derive/attributes_queue.go
type AttributesWithParent struct {
    Attributes *eth.PayloadAttributes
    Parent     eth.L2BlockRef
    Concluding bool // Concluding indicates that the attributes conclude the pending safe phase

    DerivedFrom eth.L1BlockRef
}

// WithDepositsOnly return a shallow clone with all non-Deposit transactions
// stripped from the transactions of its attributes. The order is preserved.
func (a *AttributesWithParent) WithDepositsOnly() *AttributesWithParent {
    clone := *a
    clone.Attributes = clone.Attributes.WithDepositsOnly()
    return &clone
}
```

```
func (a *AttributesWithParent) IsDerived() bool {
    return a.DerivedFrom != (eth.L1BlockRef{})
}
```

필드	의미
Attributes	실제 L2 블록 생성 입력 (타임스탬프, 트랜잭션 목록 등)
Parent	이 블록의 부모 L2 블록
Concluding	현재 L1 epoch 의 마지막 L2 블록인지 여부
DerivedFrom	어느 L1 블록에서 유도되었는지 — safe 승격의 근거

DerivedFrom 이 비어있지 않으면(IsDerived() == true) → 이 블록은 L1 에 의해 검증된 것이므로 pending-safe → safe 로 승격될 수 있습니다.

7. AttributesQueue 에서 NoTxPool 강제 설정

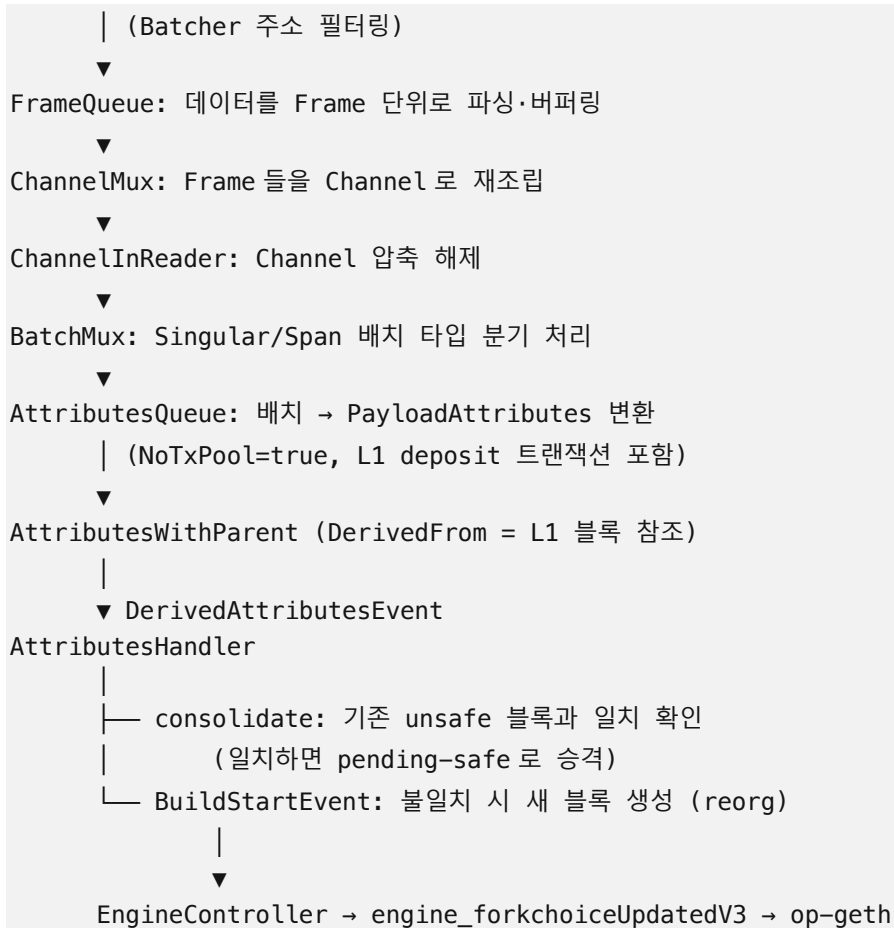
```
// op-node/rollup/derive/attributes_queue.go
func (aq *AttributesQueue) createNextAttributes(ctx context.Context, batch *SingularBatch, l2SafeHead
eth.L2BlockRef) (*eth.PayloadAttributes, error) {
    // sanity check parent hash
    if batch.ParentHash != l2SafeHead.Hash {
        return nil, NewResetError(fmt.Errorf("valid batch has bad parent hash %s, expected %s",
batch.ParentHash, l2SafeHead.Hash))
    }
    // ...
    attrs, err := aq.builder.PreparePayloadAttributes(fetchCtx, l2SafeHead, batch.Epoch())
    // ...

    // we are verifying, not sequencing, we've got all transactions and do not pull from the tx-pool
    // (that would make the block derivation non-deterministic)
    attrs.NoTxPool = true
    attrs.Transactions = append(attrs.Transactions, batch.Transactions...)
}
```

NoTxPool = true 는 매우 중요합니다. Derivation 은 “검증 모드”이기 때문에 mempool 에서 추가 트랜잭션을 가져오면 안 됩니다. L1 배치에 들어있는 트랜잭션만 포함해야 결정론적(deterministic)이 됩니다.

8. 전체 흐름 요약

```
[L1 블록 도착]
  |
  ▼
L1Traversal: L1 블록 번호순 순회
  | (L1 reorg 감지 시 → ResetEvent)
  ▼
L1Retrieval: calldata/blob 에서 배치 원본 데이터 추출
```



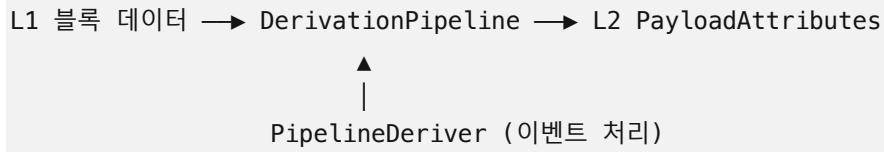
핵심 정리

관점	설명
Sequencer vs Derivation	Sequencer 는 체인을 "생성"(unsafe), Derivation 은 체인을 "검증"하여 safe 로 승격
결정론성	같은 L1 데이터 → 항상 동일한 L2 블록 (NoTxPool, 배치 데이터만 사용)
보안 모델	L1 배치 데이터가 공개되면 누구나 L2 상태를 독립 검증 가능 → Fault Proof 기반
리셋	L1 reorg 발생 시 파이프라인 전체를 안전한 시점으로 되감기
포크 처리	L1 origin 전환 시 transformStages()로 각 스테이지에 포크 활성화 알림

PipelineDeriver

PipelineDeriver 는 이벤트 기반의 wrapper 로, DerivationPipeline 을 이벤트 시스템과 연결하여 derivation 의 진행 상태를 추적하고 결과를 다른 컴포넌트들에게 알려주고, L1 데이터를 L2 블록 속성(attributes)으로 변환 하는 역할을 합니다.

핵심 역할 요약



구조체 정의

```
// op-node/rollup/derive/deriver.go
type PipelineDriver struct {
    pipeline *DerivationPipeline // 실제 derivation 로직
    ctx context.Context
    emitter event.Emitter // 이벤트 발행
    needAttributesConfirmation bool // attributes 수신 확인 대기 중
}
```

처리하는 이벤트들

```
// op-node/rollup/derive/deriver.go
func (d *PipelineDriver) OnEvent(ctx context.Context, ev event.Event) bool {
    switch x := ev.(type) {
    case PipelineStepEvent: // ← 주요 이벤트: derivation 단계 실행 (파이프라인 진행 요청)
    case DerivedAttributesEvent: // 새로운 PayloadAttributes 생성됨
    case DeriverIdleEvent: // 더 이상 처리할 L1 데이터 없음
    case ConfirmPipelineResetEvent: // 파이프라인 리셋 확인
    case ConfirmReceivedAttributesEvent: // attributes 수신 확인
    case DepositsOnlyPayloadAttributesRequestEvent: // deposits-only attributes 요청
    case ProvideL1Traversal: // L1 블록 제공
    }
}
```

주요 동작 흐름 (PipelineStepEvent 처리)



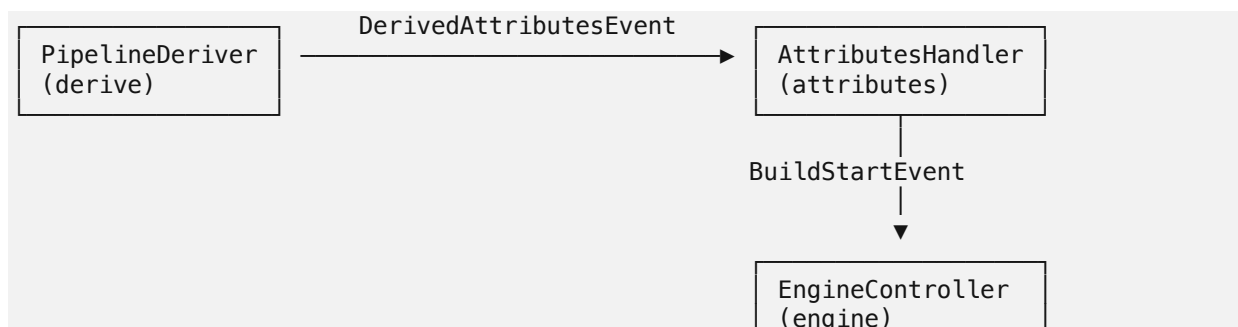
요약

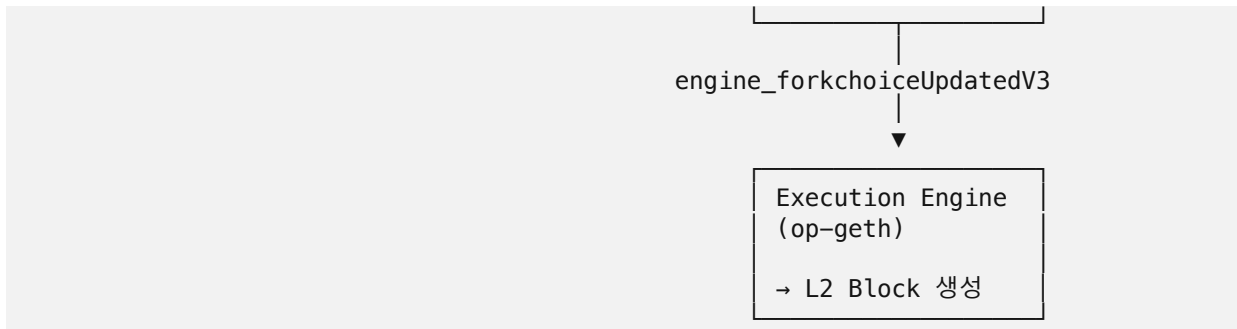
컴포넌트	역할
PipelineDriver	이벤트 수신 → DerivationPipeline 호출 → 결과 이벤트 발행
DerivationPipeline	L1 데이터를 여러 스테이지를 거쳐 L2 PayloadAttributes 로 변환

DerivationPipeline 이 AttributesWithParent 를 만들어 EL(op-geth)에 전달

DerivationPipeline 이 생성한 AttributesWithParent 는 L2 블록을 생성하기 위해 Execution Engine(op-geth)에 전달됩니다.

전체 흐름도





단계별 상세 설명

1 단계: PipelineDeriver → AttributesHandler

```
// op-node/rollup/derive/deriver.go
func (d *PipelineDeriver) emitDerivedAttributesEvent(ctx context.Context, attrib *AttributesWithParent) {
    d.needAttributesConfirmation = true
    d.emitter.Emit(ctx, DerivedAttributesEvent{Attributes: attrib})
}
```

2 단계: AttributesHandler 가 수신 및 저장

```
// op-node/rollup/attributes/attributes.go
case derive.DerivedAttributesEvent:
    eq.attributes = x.Attributes // ← 저장
    eq.sentAttributes = false
    eq.emitter.Emit(ctx, derive.ConfirmReceivedAttributesEvent{})
    eq.engineController.RequestPendingSafeUpdate(ctx)
```

3 단계: EngineController 에게 Build 요청

```
// op-node/rollup/attributes/attributes.go
if x.PendingSafe.Number < x.Unsafe.Number {
    // 기존 블록과 일치하는지 확인 (consolidation)
    eq.consolidateNextSafeAttributes(eq.attributes, x.PendingSafe)
} else {
    // 새 블록 생성 요청
    eq.sentAttributes = true
    eq.emitter.Emit(ctx, engine.BuildStartEvent{Attributes: eq.attributes})
}
```

4 단계: EngineController 가 Execution Engine 에 Payload 요청

```
// op-node/rollup/engine/build_start.go
func (e *EngineController) onBuildStart(ctx context.Context, ev BuildStartEvent) {
    // ForkchoiceState 설정
    fc := eth.ForkchoiceState{
        HeadBlockHash:    fcEvent.UnsafeL2Head.Hash,
        SafeBlockHash:    fcEvent.SafeL2Head.Hash,
        FinalizedBlockHash: fcEvent.FinalizedL2Head.Hash,
    }
}
```

```
// Execution Engine 에 payload building 시작 요청
id, errTyp, err := e.startPayload(rpcCtx, fc, ev.Attributes.Attributes)
// ...

// 성공 시 BuildStartedEvent 발행
e.emitter.Emit(ctx, BuildStartedEvent{...})
}
```

AttributesWithParent 구조

```
type AttributesWithParent struct {
    Attributes *eth.PayloadAttributes // 블록 생성에 필요한 실제 데이터
    Parent     eth.L2BlockRef           // 이 블록의 부모 L2 블록
    Concluding bool                     // L1 epoch 의 마지막 블록인지
    DerivedFrom eth.L1BlockRef          // 유도된 L1 블록 origin
}
```

PayloadAttributes 내용

```
type PayloadAttributes struct {
    Timestamp      uint64
    PrevRandao     common.Hash
    SuggestedFeeRecipient common.Address
    Withdrawals    []*Withdrawal
    ParentBeaconBlockRoot *common.Hash

    // OP Stack 특화 필드
    Transactions [][]byte // Deposit 트랜잭션 등
    NoTxPool     bool   // Sequencer 가 아닌 경우 true
    GasLimit     *uint64
}
```

요약

컴포넌트	역할
DerivationPipeline	L1 데이터로부터 AttributesWithParent 생성
AttributesHandler	Attributes 수신, 유효성 검증, 빌드 요청
EngineController	Execution Engine 과 통신하여 블록 생성
Execution Engine	실제 L2 블록 생성 (EVM 실행)

결론: AttributesWithParent 는 L2 블록을 생성하기 위한 모든 입력 데이터를 담고 있으며, 최종적으로 engine_forkchoiceUpdatedV3 RPC 를 통해 op-geth 에 전달되어 새로운 L2 블록이 생성됩니다.

Sequencer 와 DerivationPipeline 의 AttributesWithParent 생성 비교

두 프로세스는 모두 AttributesWithParent 를 생성하여 BuildStartEvent 를 발생시키지만, 목적과 데이터 소스가 완전히 다릅니다.

핵심 차이점 요약

구분	Sequencer	DerivationPipeline
역할	새로운 L2 블록 생성 (리더)	기존 L1 데이터로부터 L2 블록 재구성 (팔로워)
데이터 소스	L1 origin + 자체 결정	L1 블록/배치 데이터 파싱
트랜잭션	mempool 에서 가져옴 (NoTxPool=false)	L1 에서 유도된 것만 (NoTxPool=true)
DerivedFrom	eth.L1BlockRef{} (비어있음)	실제 L1 블록 참조
Safe 상태	unsafe 로만 처리됨	pending-safe → safe 로 승격

1. Sequencer 프로세스 (블록 생성)

```
// op-node/rollup/sequencing/sequencer.go
withParent := &derive.AttributesWithParent{
    Attributes: attrs,
    Parent:     l2Head,
    Concluding: false,
    DerivedFrom: eth.L1BlockRef{}, // ← 비어있음! L1 에서 유도되지 않음
}

d.emitter.Emit(d.ctx, engine.BuildStartEvent{
    Attributes: withParent,
})
```

프로세스:

```
Sequencer
├── l1OriginSelector.FindL1Origin() // L1 origin 선택
├── attrBuilder.PreparePayloadAttributes() // Attributes 생성
├── attrs.NoTxPool = false (기본값) // mempool 트랜잭션 포함
└── BuildStartEvent 발행
    │
    ▼
    EngineController → op-geth (새 블록 생성)
```

특징:

- Sequencer 가 직접 L1 origin 을 선택
- mempool 에서 사용자 트랜잭션 가져옴
- 생성된 블록은 unsafe 상태
- 네트워크에 gossip 으로 전파

2. DerivationPipeline 프로세스 (블록 재구성)

```
AttributesWithParent{
  Attributes: attrs,
  Parent:    pendingSafe,
  Concluding: true/false, // L1 epoch 종료 여부
  DerivedFrom: l1Origin,  // ← 실제 L1 블록 참조!
}
```

프로세스:

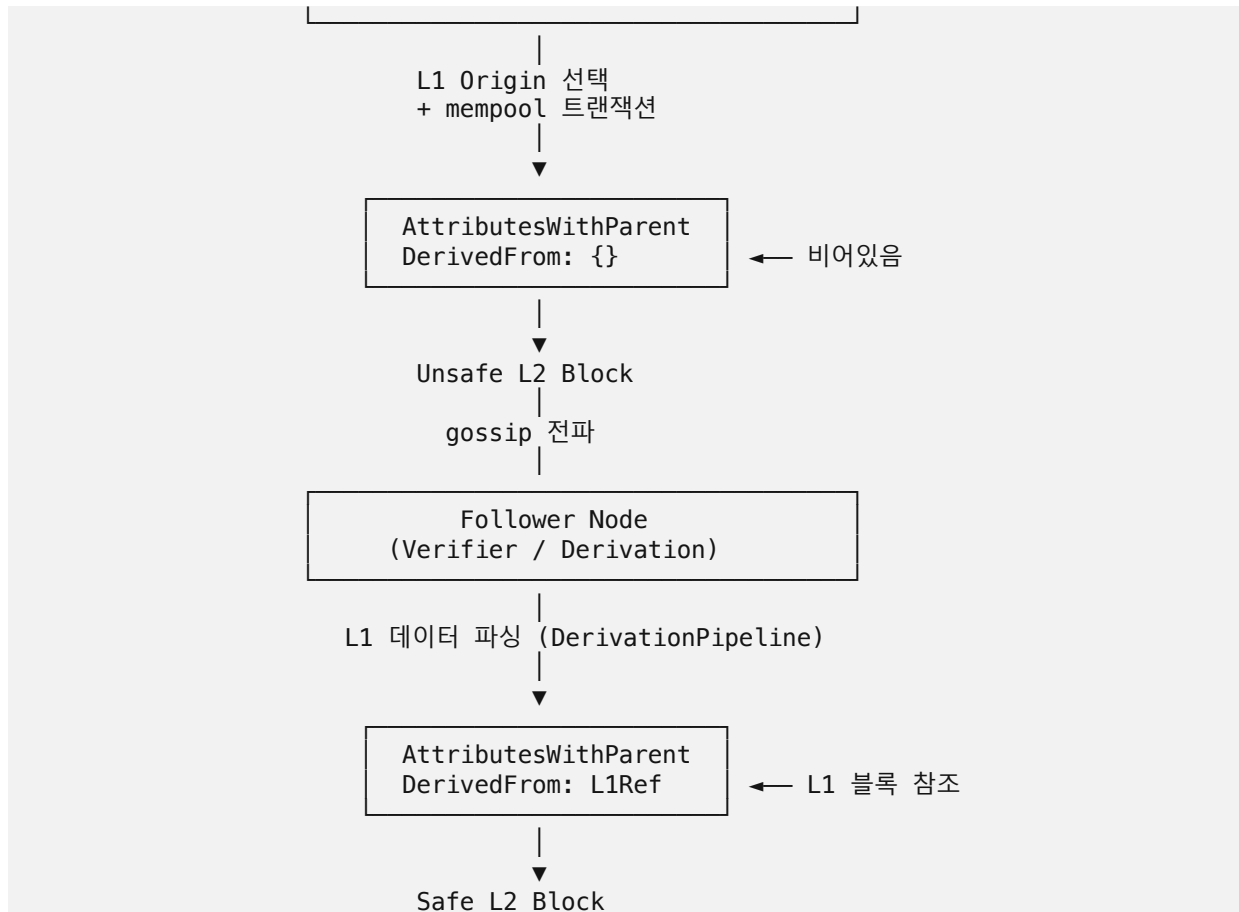
```
DerivationPipeline
├── L1Traversal → L1Retrieval    // L1 블록 데이터 가져오기
├── FrameQueue → ChannelMux → BatchMux  // 배치 파싱
├── AttributesQueue.NextAttributes() // Attributes 생성
│   (L1 에서 유도된 트랜잭션만 포함)
└── AttributesHandler
    ├── consolidate (기존 블록과 일치 확인)
    │   또는
    └── BuildStartEvent 발행 (불일치 시 reorg)
```

특징:

- L1 데이터를 순차적으로 파싱하여 블록 재구성
- deposits 트랜잭션만 포함 (L1 에서 온 것)
- 생성된 블록은 pending-safe → safe 로 승격
- Fault Proof 검증 가능

시각적 비교

Sequencer Node (Active Leader)



DerivedFrom 필드의 중요성

```

// Sequencer
DerivedFrom: eth.L1BlockRef{} // 비어있음 → safe 가 될 수 없음

// DerivationPipeline
DerivedFrom: l1Origin // L1 참조 → safe 로 승격 가능

```

이 필드는 AttributesHandler 에서 처리 방식을 결정합니다:

```

// op-node/rollup/engine/build_start.go
if ev.Attributes.DerivedFrom != (eth.L1BlockRef{}) &&
    e.pendingSafeHead.Hash != ev.Attributes.Parent.Hash {
    // Derived attributes 의 경우 pending-safe head 와 일치해야 함
}

```

요약

	Sequencer	DerivationPipeline
누가	리더 노드	모든 노드 (팔로워 포함)
언제	새 블록 생성 시	L1 데이터 동기화 시

왜	체인 확장	검증 & 안전한 상태 확인
결과	Unsafe 블록	Safe/Finalized 블록

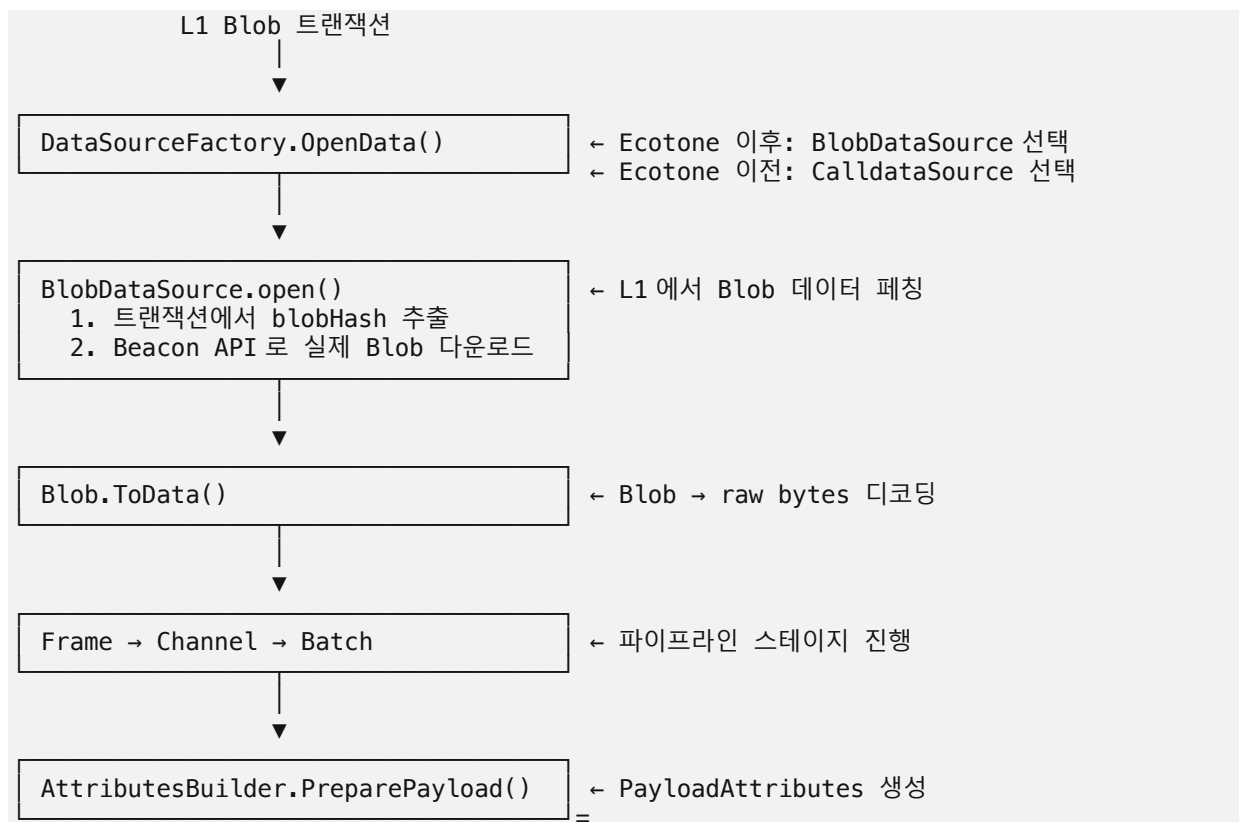
Sequencer 는 체인을 "생성"하고, DerivationPipeline 은 체인을 "검증"합니다.

```
withParent := &derive.AttributesWithParent{
    Attributes: attrs,
    Parent:     l2Head,
    Concluding: false,
    DerivedFrom: eth.L1BlockRef{}, // ← 비어있음! L1 에서 유도되지 않음
}

d.emitter.Emit(d.ctx, engine.BuildStartEvent{
    Attributes: withParent,
})
```

Blob 데이터로부터 eth.PayloadAttributes 생성 과정

전체 데이터 흐름 개요



dataAndHashesFromTxs 안의 isValidBatchTx 에서 BlobDataSource 를 사용하더라도 blob 과 calldata 를 모두 포함한다.

Blob 인코딩 구조

위치	크기	내용
byte[0]	6 bits	첫 번째 field element 의 상위 비트
byte[1]	1 byte	인코딩 버전 (현재 0)
byte[2:5]	3 bytes	데이터 길이 (big-endian uint24)
byte[5:32]	27 bytes	첫 번째 데이터 청크
byte[32:...]	나머지	4 field elements 씩 127 바이트 디코딩

Blob 크기: $4096 \times 32 = 131,072$ bytes

최대 데이터 크기: ~126 KB (MaxBlobDataSize)

attributes 생성 과정

1. PreparePayloadAttributes

- 2) 만약 L1 origin 이 변경되었다면,

```
info, receipts, err := ba.l1.FetchReceipts(ctx, epoch.Hash)
```

실제 구현된 곳은

```
EthClient.FetchReceipts info = BlockInfo UpdateSystemConfigWithL1Receipts
```

~ L1 시스템 설정 변경 감지 depositTx 를 생성 한다. seqNumber = 0 그렇지 않으면 l1info (l1BlockInfo) 만 가져오고 seqNumber = l1Parent.SequenceNumber + 1

- 3) upgradeTxs 생성

- 4) L1InfoDeposit Tx 생성

```
// Set a very large gas limit with `IsSystemTransaction` to ensure
// that the L1 Attributes Transaction does not run out of gas.
out := &types.DepositTx{
    SourceHash:    source.SourceHash(),
    From:          L1InfoDepositerAddress,
    To:            &L1BlockAddress,
    Mint:          nil,
    Value:         big.NewInt(0),
    Gas:           150_000_000,
    IsSystemTransaction: true,
    Data:          data,
}
```

- 5) txs 병합

```
txs := make([]hexutil.Bytes, 0, 1+len(depositTxs)+len(upgradeTxs))
txs = append(txs, l1InfoTx)
```

```
txs = append(tx, depositTx...)
txs = append(tx, upgradeTx...)
```

2. createNextAttributes

```
attrs := aq.builder.PreparePayloadAttributes(fetchCtx, l2SafeHead, batch.Epoch())
attrs.NoTxPool = true
attrs.Transactions = attrs.Transactions + batch.Transactions (사용자이외의 tx + 사용자 tx)
```

3 호출순서

NextAttributes 호출 체인



이벤트 흐름 요약

순서	컴포넌트	동작
1	Driver	PipelineStepEvent 발생
2	PipelineDeriver	이벤트 수신 → pipeline.Step() 호출
3	DerivationPipeline	attrib.NextAttributes() 호출
4	AttributesQueue	Batch 에서 PayloadAttributes 생성
5	PipelineDeriver	DerivedAttributesEvent 발생 → AttributesHandler 로 전달

NextAttributes 는 파이프라인의 마지막 단계로, 모든 이전 스테이지(L1Retrieval → FrameQueue → ChannelMux → BatchMux)를 통해 처리된 데이터를 최종 PayloadAttributes 로 변환합니다.

Consolidation

Sequencer 로부터 P2p 로 받은 unsafe 블록이 먼저 존재하는 경우

1. consolidateNextSafeAttributes 실행

```
// attributes.go
} else {
    // if there already exists a block we can just consolidate it
    if x.PendingSafe.Number < x.Unsafe.Number {
        eq.consolidateNextSafeAttributes(eq.attributes, x.PendingSafe)
    } else {
        // append to tip otherwise
        eq.sentAttributes = true
        eq.log.Debug("emitting build start event", "attributes", eq.attributes)
        eq.emitter.Emit(ctx, engine.BuildStartEvent{Attributes: eq.attributes})
    }
}
```

Consolidation 프로세스

consolidateNextSafeAttributes 함수에서 기존 unsafe 블록과 derived attributes 를 비교합니다:

```
// attributes.go

// consolidateNextSafeAttributes tries to match the next safe attributes against the existing unsafe chain,
// to avoid extra processing or unnecessary unwinding of the chain.
// However, if the attributes do not match, they will be forced to process the attributes.
func (eq *AttributesHandler) consolidateNextSafeAttributes(attributes *derive.AttributesWithParent, onto
eth.L2BlockRef) {
    ctx, cancel := context.WithTimeout(eq.ctx, time.Second*10)
    defer cancel()

    envelope, err := eq.l2.PayloadByNumber(ctx, attributes.Parent.Number+1)
    // ... error handling ...

    if err := AttributesMatchBlock(eq.cfg, attributes.Attributes, onto.Hash, envelope, eq.log); err != nil {
        eq.log.Warn("L2 reorg: existing unsafe block does not match derived attributes from L1",
            "err", err, "unsafe", envelope.ExecutionPayload.ID(), "pending_safe", onto)

        eq.sentAttributes = true
    }
}
```

```

// geth cannot wind back a chain without reorging to a new, previously non-canonical, block
eq.emitter.Emit(eq.ctx, engine.BuildStartEvent{Attributes: attributes})
return
} else {
    ref, err := derive.PayloadToBlockRef(eq.cfg, envelope.ExecutionPayload)
    // ...
    eq.engineController.TryUpdatePendingSafe(eq.ctx, ref, attributes.Concluding, attributes.DerivedFrom)
    eq.engineController.TryUpdateLocalSafe(eq.ctx, ref, attributes.Concluding, attributes.DerivedFrom)
}

// unsafe head stays the same, we did not reorg the chain.
}

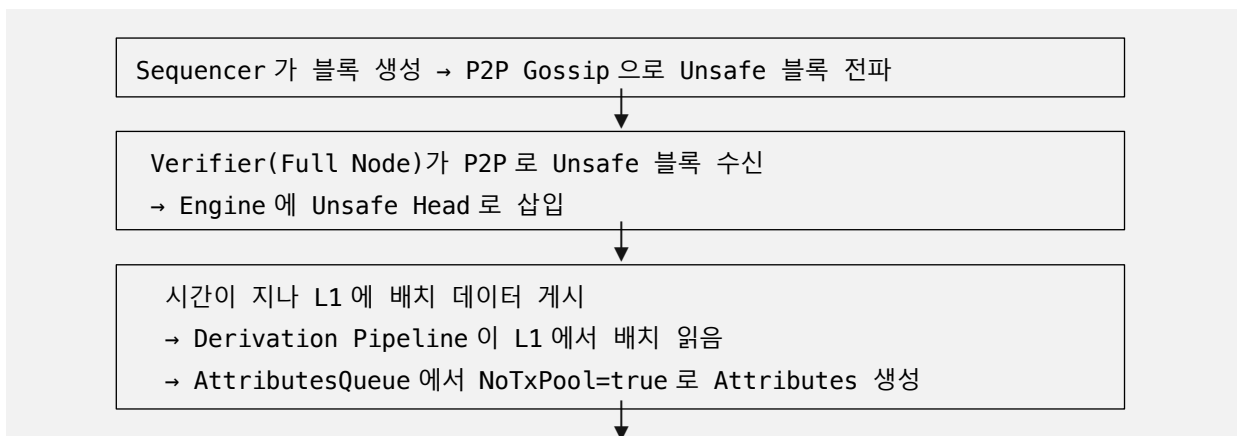
```

3. AttributesMatchBlock: 블록 비교

4. L2 블록 격상 (PendingSafe, LocalSafe)

- TryUpdatePendingSafe()
- TryUpdateLocalSafe()

5. 전체흐름



```

AttributesHandler.consolidateNextSafeAttributes()

if (PendingSafe.Number < Unsafe.Number) {
    // 이미 unsafe 블록이 존재함
    AttributesMatchBlock()로 비교

    if (일치) {
        // 기존 P2P unsafe 블록을 Safe 로 승격
        // 새 블록 빌드 필요 없음! ← 이게 핵심
        TryUpdatePendingSafe()
        TryUpdateLocalSafe()
    } else {
        // L2 Reorg 필요
        // Derived attributes 로 새 블록 빌드
        BuildStartEvent 발생
    }
} else {
    // Unsafe 블록 없음, 새로 빌드해야 함
    BuildStartEvent 발생
}

```

6. DerivedAttributesEvent → L2 Block 생성 → PendingSafe, LocalSafe 격상 전체 흐름

L2 Block 생성 및 Safe 격상 이벤트 흐름

1. PipelineDeriver: DerivedAttributesEvent 발생
(op-node/rollup/derive/deriver.go:212)

```
d.emitter.Emit(ctx, DerivedAttributesEvent{Attributes: attrib})
```

2. AttributesHandler: 이벤트 수신 및 저장
(op-node/rollup/attributes/attributes.go:91-

case derive.DerivedAttributesEvent:

```
eq.attributes = x.Attributes
```

// 속성 저장

```
eq.sentAttributes = false
```

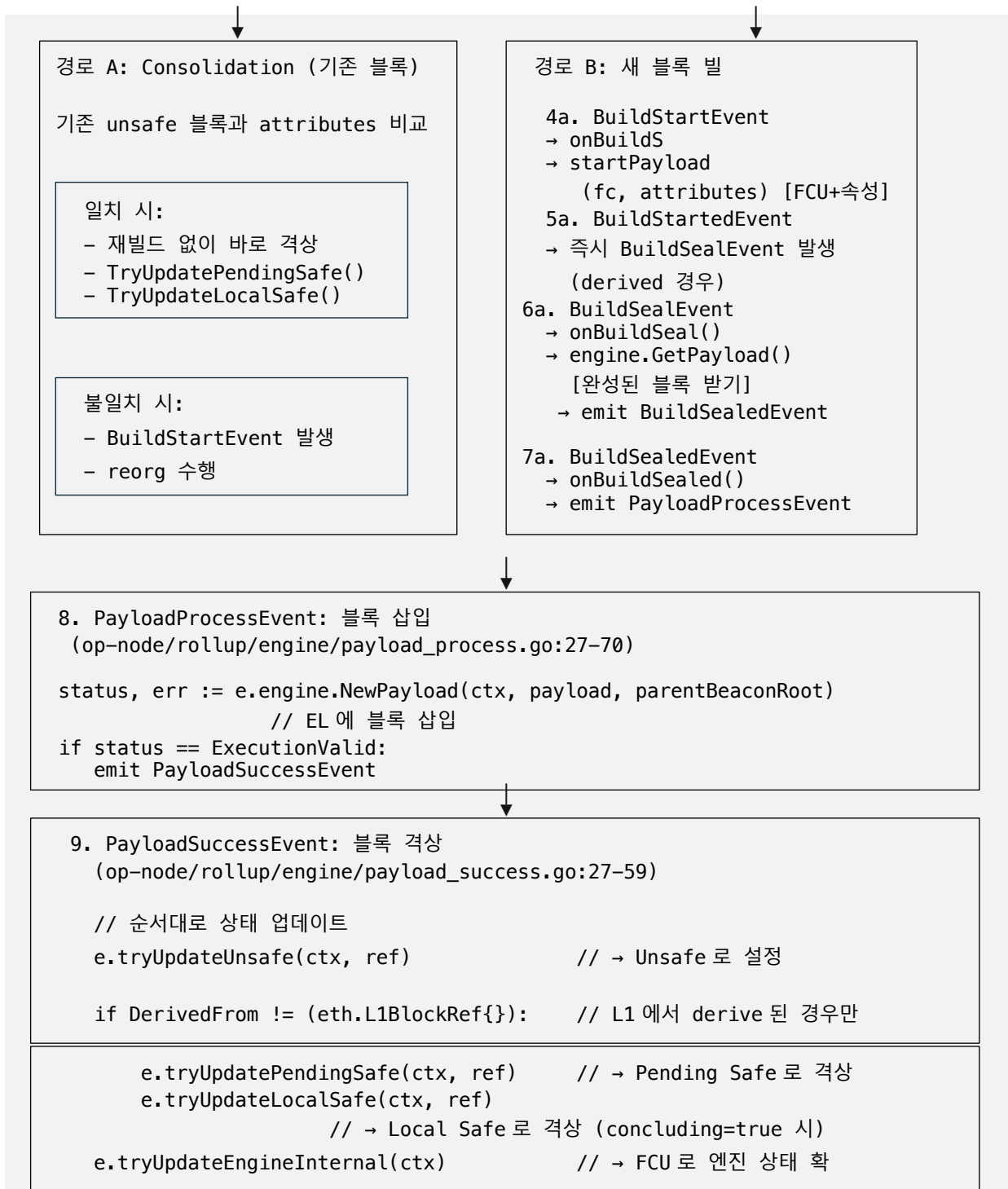
```
eq.emitter.Emit(ctx, ConfirmReceivedAttributesEvent{}) // 파이프라인에 확인 전송
```

```
eq.engineController.RequestPendingSafeUpdate(ctx) // 처리 시작 요청
```

3. AttributesHandler.onPendingSafeUpdate: 처리 분기

(op-node/rollup/attributes/attributes.go:138-193)

PendingSafe < Unsafe (이미 unsafe 블록 존재)	PendingSafe == Unsafe (새 블록 필요)
consolidateNextSafe- Attributes()	emit BuildStartEvent

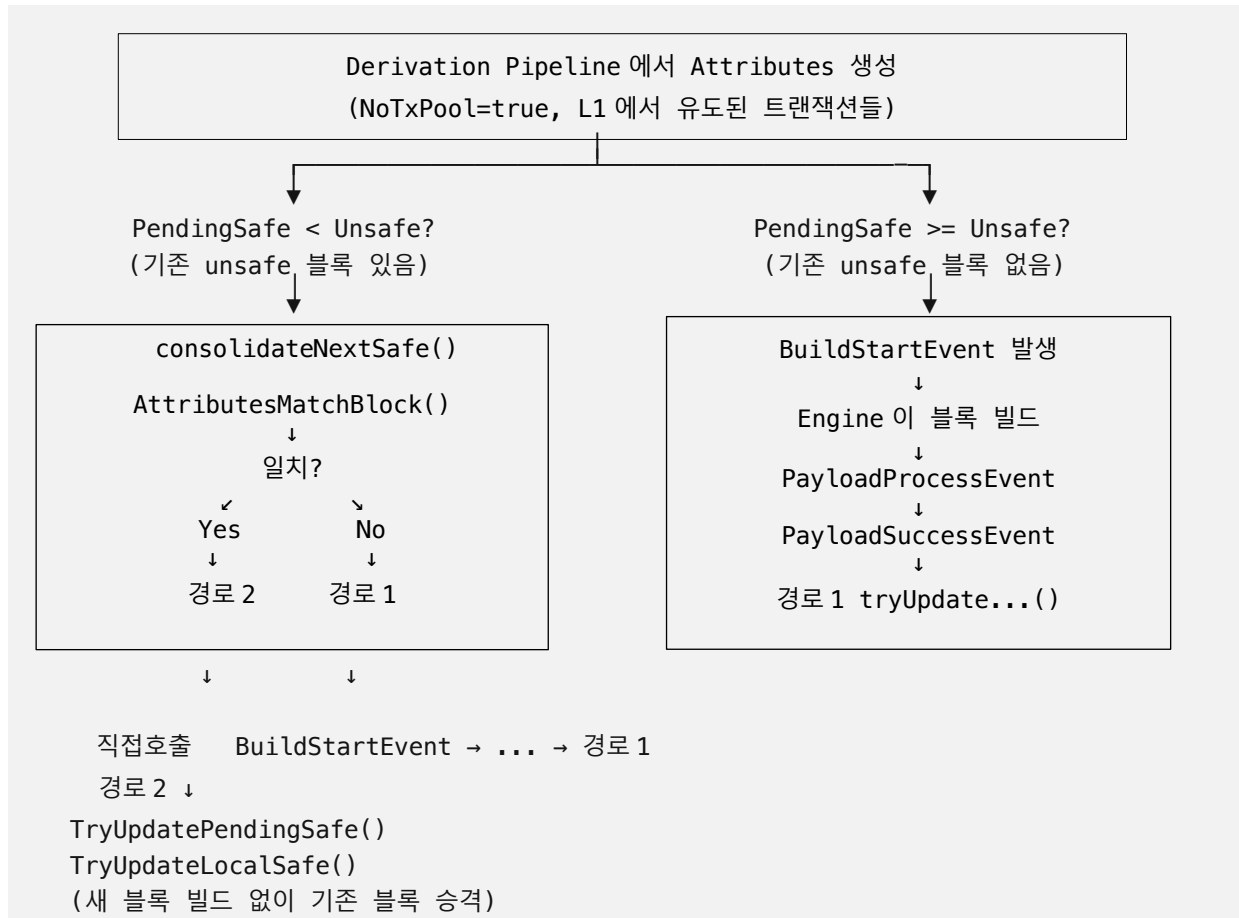


7. 위 두 플로우에서 본 격상이벤트의 두 배타적 흐름

경로 1 : 새 블록빌드 필요한 경우 (PayloadSuccessEvent -> tryUpdatePendingSafe): 이 경로는 기존 unsafe 블록이 없거나, 기존 블록이 derived attributes 와 일치하지 않을 때 실행됨

경로 2 : 기존 Unsafe 블록 통합 (Consolidation -> TryUpdatePendingSafe): 이 경로는 P2P 로 받은 unsafe 블록이 이미 존재하고, derived attributes 와 일치할 때 실행됨

8. 경로 2 를 따를 때, AttributeMatchBlock()에서 일치 하지 않은 결과가 발생할 때는 다시 경로 1 을 탄다.



** 위 세 다이어그램을 합칠 필요가 있음 **

tryUpdateLocalSafe 는 LocalSafeUpdateEvent 를 발행 LocalSafe 의 경우 rollupCfg 가 IsInterop 가 아닐 경우 e.PromoteSafe 를 통해 safeHead 설정

L1 block 의 safe, finalized 상태에 따라 L2 블록 상태 변경

Safe & Finalized State

```
// op-node/node/node.go

func initL1Handlers {

    onL1Safe := func(ctx context.Context, sig eth.L1BlockRef) {
        node.l2Driver.StatusTracker.OnL1Safe(sig)
    }

    onL1Finalized := func(ctx context.Context, sig eth.L1BlockRef) {
        // TODO(#16917) Remove Event System Refactor Comments
    }
}
```

```

// FinalizeL1Event fan out is updated to procedural method calls
node.L2Driver.StatusTracker.OnL1Finalized(sig)
node.L2Driver.Finalizer.OnL1Finalized(sig)
node.L2Driver.SyncDeriver.OnL1Finalized(ctx)
}

// Poll for the safe L1 block and finalized block,
// which only change once per epoch at most and may be delayed.
l1SafeSub := eth.PollBlockChanges(node.log, node.l1Source, onL1Safe, eth.Safe,
    cfg.L1EpochPollInterval, time.Second*10)
l1FinalizedSub := eth.PollBlockChanges(node.log, node.l1Source, onL1Finalized, eth.Finalized,
    cfg.L1EpochPollInterval, time.Second*10)
}

```

L1 의 Safe 와 Finalized 는 Ethereum Pos Casper FFG 합의 메커니즘에 유래하여, Optimism 은 이를 활용하여 L2 수준의 안전성 수준을 결정함

1) safe

```

```go
// op-node/rollup/status/status.go > StatusTracker
node.L2Driver.StatusTracker.OnL1Safe(sig)

```

#### 2) finalized

```

node.L2Driver.StatusTracker.OnL1Finalized(sig)
node.L2Driver.Finalizer.OnL1Finalized(sig)
-> engineController PromoteFinalized(ctx, finalized)
-> e.SetFinalized(ref)
-> e.tryUpdateEngine(ctx)
node.L2Driver.SyncDeriver.OnL1Finalized(sig)
-> s.StepDeriver.RequestStep(ctx,false)

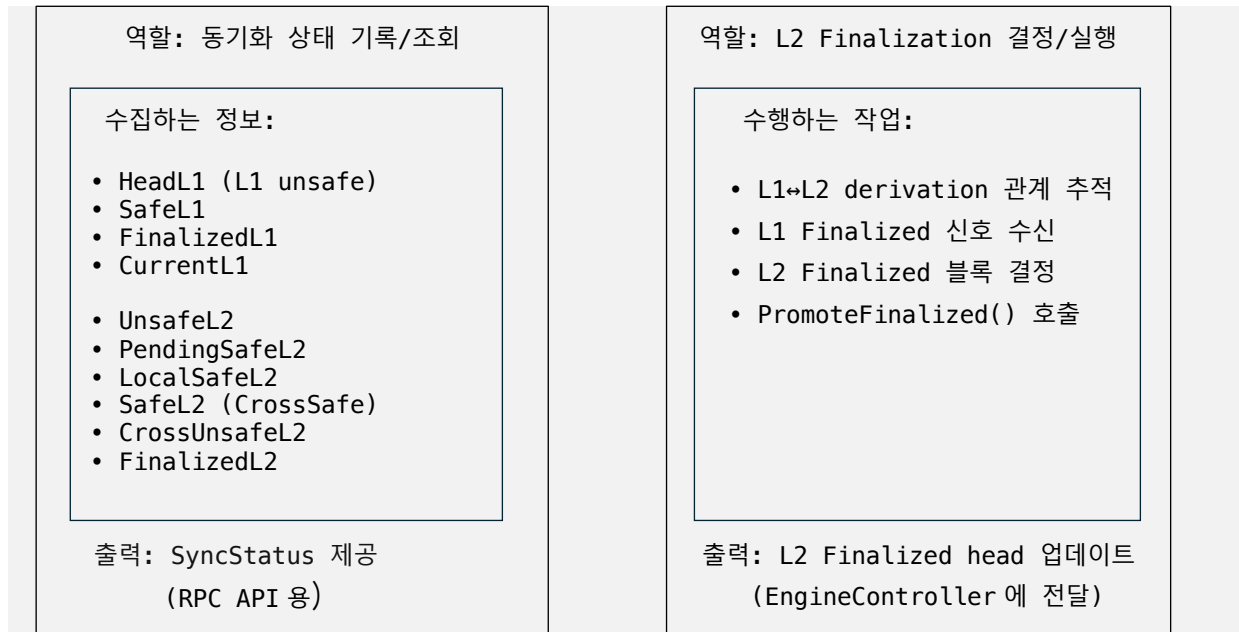
```

## StatusTracker vs Finalizer

### StatusTracker vs Finalizer 역할 비교

StatusTracker  
(상태추적자)

Finalizer  
(Finality 결정자)



## 상태별 의미

L2	안전도	의미
unsafeL2	매우 낮음	sequencer 가 만든 직후 블록, L1 과 무관
PendingSafeL2	낮음	L1
LocalSafeL2	중간	L1 에 게시되어 로컬에서 확인됨
CrossUnsafeL2	중상	L1 에는 있음, 하지만 cross validation 미완료
SafeL2 (CrossSafe)	높음	L1 에서 안정된 블록으로 인정됨
FinalizedL2	최고	Ethereum L1 finality 달성, 절대 변경 불가

unsafeL2 -> PendingSafeL2 Unsafe 블록이 L1 batch 데이터에 의해 "확인"되는 순간 (derivation pipeline 에서 해당 블록의 transactions 가 L1 에 제출된 것을 확인) PendingSafe 로 격상

## op\_proposer

---

### 개요

op-proposer 는 L2 상태를 L1 에 정기적으로 제안(proposal)하는 경량 서비스입니다. L2 체인의 output root 를 L1 에 제출하여 L2 상태에 대한 claim 을 생성합니다.

op\_proposer 는 주기적으로 L2 의 상태를 outputRoot 를 L1 의 DisputeGameFactory(예전 버전의 경우 L2OutputOracle)에 저장합니다.

### OutputRoot 의 구조

OutputRoot 의 구조는 버전바이트와 페이로드를 결합하여 Keccak256 해시를 취한 값입니다.

```
OutputRoot = keccak256(version_byte || payload)
```

이때 페이로드는 실제 L2 정보를 담고 있는 데이터 뭉치인데, 다음 세가지 핵심 데이터의 결합으로 이루어집니다.

```
payload = state_root || withdrawal_storage_root || latest_block_hash
```

- state\_root = L2 실행 레이어의 모든 계정 정보를 담고 있는 Merkle-Patricia-Trie(MPT)의 루트값입니다.
- withdrawal\_storage\_root(출금 스토리지 루트): L2 에서 L1 으로 메시지를 전달하는 L2ToL1MessagePasser 컨트랙트의 스토리지 루트입니다. 참고: Isthmus 하드포크 이후에는 블록 헤더의 withdrawalsRoot 값을 직접 사용할 수 있게 되어 검증 비용이 더욱 절감되었습니다.
- latest\_block\_hash (최신 블록 해시): 해당 Output Root 가 가리키는 특정 L2 블록의 해시값입니다.

### 제안과정

5. 조회: op-proposer 는 주기적으로 op-node 에 접속하여 최신 L2 상태에 대한 Output Root 를 요청합니다. 여기서 주기는 보통 1 시간입니다.
6. 제안: 획득한 OutputRoot 를 L1 에 배포된 DisputeGameFactory 컨트랙트의 proposeL2Output 함수를 통해 제출합니다.
7. 검증: 제안된 값은 보통 7 일간의 챌린지 기간동안 유효성을 검증 받으며, 이 기간이 지나면 확정되어 사용자가 L1 에서 자산을 출금할 수 있게 됩니다.

---

## 핵심 기능

### 1. Output Root 제안 (Proposal)

- finalized(확정된) L2 블록의 output root 를 L1 에 제출
- 기본적으로 약 1 시간 간격으로 제안 트랜잭션 발생

- 제안된 claim 은 withdrawal(출금) 검증에 사용됨

## 2. 두 가지 배포 모드 지원 : L2OutputOracle vs DisputeGameFactory

L2OutputOracle 은 더 이상 사용되지 않고 DisputeGameFactory(DGF)로 대체되었으며, 과거 호환성을 위해 NewL2OutputSubmitter 가 L1 주소 설정에 따라 구현을 고른다.

모드	대상 컨트랙트	설명
Pre-Fault-Proof	L2OutputOracle	레거시 방식, 직접 output root 제출
Fault Proof 활성화	DisputeGameFactory	각 claim 마다 dispute game 생성

```
// op-proposer/proposer/driver.go
// L2OutputSubmitter is responsible for proposing outputs
type L2OutputSubmitter struct {
 DriverSetup

 wg sync.WaitGroup
 done chan struct{}

 ctx context.Context
 cancel context.CancelFunc

 running atomic.Bool

 l2ooContract L2OOContract
 l2ooABI *abi.ABI

 dgfContract DGFCContract
}
```

```
// op-proposer/proposer/driver.go

// NewL2OutputSubmitter creates a new L2 Output Submitter
func NewL2OutputSubmitter(setup DriverSetup) (_ *L2OutputSubmitter, err error) {
 ctx, cancel := context.WithCancel(context.Background())
 // The above context is long-lived, and passed to the `L2OutputSubmitter` instance. This context is closed
 by
 // `StopL2OutputSubmitting`, but if this function returns an error or panics, we want to ensure that the
 context
 // doesn't leak.
 defer func() {
 if err != nil || recover() != nil {
 cancel()
 }
 }
}
```

```

}0

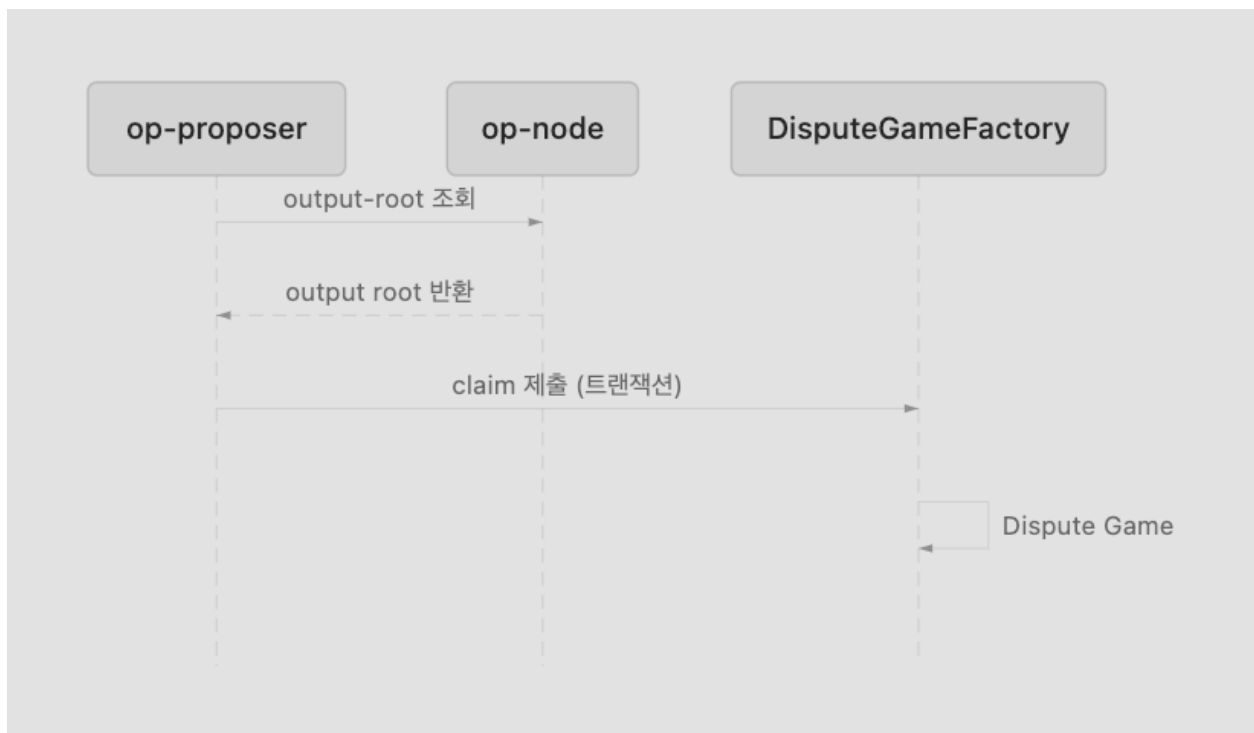
if setup.Cfg.L2OutputOracleAddr != nil {
 return newL2OOSubmitter(ctx, cancel, setup)
} else if setup.Cfg.DisputeGameFactoryAddr != nil {
 return newDGFSubmitter(ctx, cancel, setup)
} else {
 return nil, errors.New("neither the `L2OutputOracle` nor `DisputeGameFactory` addresses were provided")
}
}

```

### 3. 데이터 소스

소스	용도
Rollup RPC	단일 체인용 (Pre-Interop)
Supervisor RPC	Interop 체인용 (Post-Interop)

### 4. 주요 워크플로우



### 5. 안전성 메커니즘

```

// op-proposer/proposer/driver.go

// FetchCurrentBlockNumber gets the current block number from the [L2OutputSubmitter]'s [RollupClient]. If
the `AllowNonFinalized` configuration
// option is set, it will return the safe head block number, and if not, it will return the finalized head block
number.
func (l *L2OutputSubmitter) FetchCurrentBlockNumber(ctx context.Context) (uint64, error) {

```

```

status, err := l.ProposalSource.SyncStatus(ctx)
if err != nil {
 return 0, fmt.Errorf("getting sync status: %w", err)
}

// Use either the finalized or safe head depending on the config. Finalized head is default & safer.
if l.Cfg.AllowNonFinalized {
 return status.SafeL2, nil
}
return status.FinalizedL2, nil
}

```

- 기본: finalized L2 블록만 제안 (가장 안전)
- --allow-non-finalized: safe L2 블록도 제안 가능 (테스트용)

## 주요 설정 옵션

플래그	설명
--l1-eth-rpc	L1 RPC 엔드포인트
--rollup-rpc	op-node RPC 엔드포인트
--supervisor-rpc	Supervisor RPC (Interop 용)
--game-factory-address	DisputeGameFactory 컨트랙트 주소
--game-type	Dispute Game 타입
--proposal-interval	제안 간격 (기본 1 시간)
--poll-interval	체인 상태 폴링 간격 (기본 6 초)
--allow-non-finalized	finalized 아닌 블록도 제안 허용

## 역할과 중요성

- Withdrawal 지원: L1 에서 L2 출금을 검증하려면 proposal 이 필요
- 상태 증명: L2 상태에 대한 claim 을 L1 에 앵커링
- Fault Proof 연동: Dispute Game 을 통한 상태 검증 가능

## 설계 원칙

- 단순성 우선: 안전성이 중요하므로 복잡한 최적화보다 단순한 구현 선호
- 트랜잭션 관리 재사용: op-service 의 공통 txmgr 활용
- Liveness 장애 허용: 일시적 RPC 실패에 대한 복원력

## DisputeGame 구현체

### DisputeGame 구현체 종류

DisputeGameFactory 의 create() 함수는 gameImpls[\_gameType]에 등록된 구현체(proxy class)의 clone(EIP 1167)을 생성합니다:

```
// packages/contracts-bedrock/src/dispute/DisputeGameFactory.sol

// Grab the implementation contract for the given `GameType`.
IDisputeGame impl = gameImpls[_gameType];

// If there is no implementation to clone for the given `GameType`, revert.
if (address(impl) == address(0)) revert NoImplementation(_gameType);
```

### 구현체 목록

GameType ID	GameType 상수	구현 클래스	설명
0	CANNON	FaultDisputeGameV2	기본 Cannon VM 기반 (누구나 참여)
1	PERMISSIONED_CANNON	PermissionedDisputeGameV2	Cannon + 허가된 참여자만
2	ASTERISC	FaultDisputeGameV2	RISC-V VM 기반
3	ASTERISC_KONA	FaultDisputeGameV2	RISC-V + Kona
4	SUPER_CANNON	SuperFaultDisputeGame	Interop 용 Super Roots
5	SUPER_PERMISSIONED_CANNON	SuperPermissionedDisputeGame	Super Roots + 허가제
6	OP_SUCCINCT	OPSuccinctFaultDisputeGame	ZK Proof 기반
7	SUPER_ASTERISC_KONA	SuperFaultDisputeGame	Super Roots + Asterisc
8	CANNON_KONA	FaultDisputeGameV2	Cannon + Kona
9	SUPER_CANNON_KONA	SuperFaultDisputeGame	Super Roots + Cannon + Kona
254	FAST	테스트용	짧은 게임 기간
255	ALPHABET	테스트용	Alphabet VM
1337	KAILUA	RISC Zero 기반	ZK-STARK

### 구현 클래스 상세

#### 1. FaultDisputeGameV2 (기본)

```
// packages/contracts-bedrock/src/dispute/v2/FaultDisputeGameV2.sol

contract FaultDisputeGameV2 is Clone, ISemver {}
```

- Interactive Fault Proof 게임
- 누구나 참여 가능 (Permissionless)
- Bisection 방식으로 dispute 진행

## 2. *PermissionedDisputeGameV2*

```
// packages/contracts-bedrock/src/dispute/v2/PermissionedDisputeGameV2.sol

/// @title PermissionedDisputeGameV2
/// @notice PermissionedDisputeGameV2 is a contract that inherits from `FaultDisputeGameV2`, and contains
two roles:
/// - The `challenger` role, which is allowed to challenge a dispute.
/// - The `proposer` role, which is allowed to create proposals and participate in their game.
/// This contract exists as a way for networks to support the fault proof iteration of the OptimismPortal
/// contract without needing to support a fully permissionless system. Permissionless systems can
introduce
/// costs that certain networks may not wish to support. This contract can also be used as a fallback
mechanism
/// in case of a failure in the permissionless fault proof system in the stage one release.
contract PermissionedDisputeGameV2 is FaultDisputeGameV2 {}
```

- FaultDisputeGameV2 를 상속
- proposer 와 challenger 역할만 참여 가능

## 3. *SuperFaultDisputeGame*

```
// packages/contracts-bedrock/src/dispute/SuperFaultDisputeGame.sol

/// @title SuperFaultDisputeGame
/// @notice An implementation of the `IFaultDisputeGame` interface for interop.
contract SuperFaultDisputeGame is Clone, ISemver {}
```

- Interop (여러 L2 체인 통합)용
- Super Roots 검증

## 4. *SuperPermissionedDisputeGame*

```
// packages/contracts-bedrock/src/dispute/SuperPermissionedDisputeGame.sol

/// @title SuperPermissionedDisputeGame
/// @notice SuperPermissionedDisputeGame is a contract that inherits from `SuperFaultDisputeGame`, and
contains two
/// roles:
/// - The `challenger` role, which is allowed to challenge a dispute.
/// - The `proposer` role, which is allowed to create proposals and participate in their game.
/// This contract exists as a way for networks to support the fault proof iteration of the OptimismPortal
/// contract without needing to support a fully permissionless system. Permissionless systems can
introduce
/// costs that certain networks may not wish to support. This contract can also be used as a fallback
mechanism
```

```
/// in case of a failure in the permissionless fault proof system in the stage one release.
```

```
contract SuperPermissionedDisputeGame is SuperFaultDisputeGame {}
```

- SuperFaultDisputeGame 를 상속
- Interop + 허가제

##### 5. OPSuccinctFaultDisputeGame (ZK)

```
// packages/contracts-bedrock/src/dispute/zk/OPSuccinctFaultDisputeGame.sol
```

```
/// @title OPSuccinctFaultDisputeGame
```

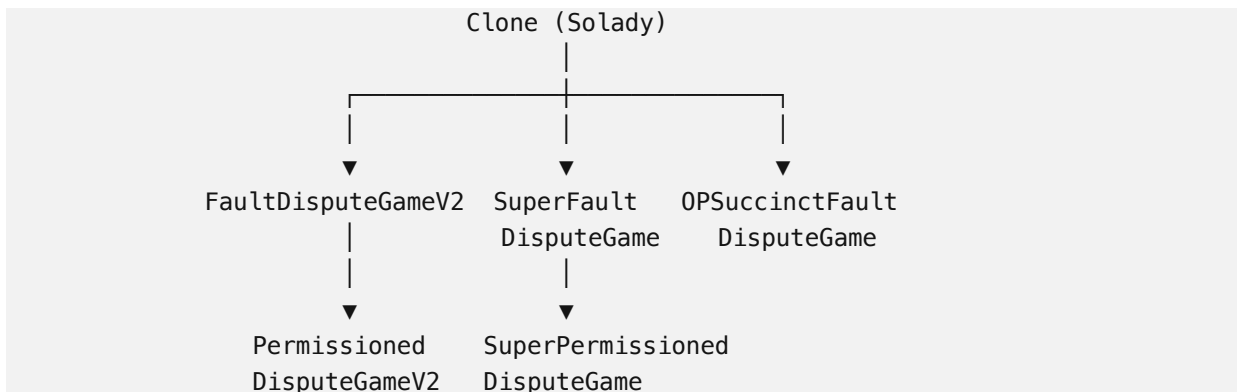
```
/// @notice An implementation of the `IFaultDisputeGame` interface.
```

```
/// @dev Derived from https://github.com/succinctlabs/op-succinct (at commit
c13844a9bbc330cca69eef2538d8f8ec123e1653)
```

```
contract OPSuccinctFaultDisputeGame is Clone, ISemver, IDisputeGame {}
```

- ZK Proof 기반 (Succinct SP1)
- Interactive bisection 없이 단일 proof 로 검증

### 상속 구조



### 요약

카테고리	클래스	특징
기본	FaultDisputeGameV2	단일 L2, Permissionless
허가제	PermissionedDisputeGameV2	단일 L2, Permissioned
Interop	SuperFaultDisputeGame	다중 L2 (Super Roots)
Interop + 허가제	SuperPermissionedDisputeGame	다중 L2 + Permissioned
ZK	OPSuccinctFaultDisputeGame	ZK Proof 기반

## Dispute Game Proxy

### DisputeGameFactory 모드 (ProposalInterval)

```
// op-proposer/proposer/driver.go

cutoff := time.Now().Add(-l.Cfg.ProposalInterval)
proposedRecently, proposalTime, claim, err := l.dgfContract.HasProposedSince(ctx, l.Txmgr.From(), cutoff,
l.Cfg.DisputeGameType)
```

- ProposalInterval 설정값에 따라 제안 (예: 1 시간, 6 시간 등)
- 마지막 제안 이후 interval 이 지나야 새 게임 생성

### L2OutputOracle 모드 (Legacy)

- 특정 블록 간격마다 제안 (예: 1800 블록  $\approx$  1 시간)
- L2OO 컨트랙트의 SUBMISSION\_INTERVAL 설정에 따름

### 실제 숫자 예시

설정	값	결과
L2 블록 시간	2 초	-
1 시간 동안 L2 블록 수	1,800 블록	-
ProposalInterval	1 시간	1 시간에 1 개 dispute game 생성
하루 생성 수	24 개 dispute game	(매우 적음)

### 비교

L2 블록 생성 빈도:  $\sim 43,200$  블록/일 (2 초마다)  
Dispute Game 생성:  $\sim 24$  개/일 (1 시간마다)  
비율: 1 Dispute Game : 1,800 L2 블록

### 왜 매 블록마다 생성하지 않는가?

이유	설명
Gas 비용	clone 생성 + initialize = 상당한 L1 gas
Init Bond	게임 생성 시 보증금 필요 (ETH 예치)
실용성	모든 블록을 검증할 필요 없음
Challenge 기간	7 일 동안 dispute 가능하므로 자주 제출할 필요 없음

### Clone 기술의 이점

생성 빈도가 낮더라도 clone 을 사용하는 이유:

// 일반 배포 대비 Gas 절약

```
proxy_ = IDisputeGame(address(impl).clone(...)); // ~45,000 gas
// vs new FaultDisputeGameV2(...) // ~2,000,000+ gas
```

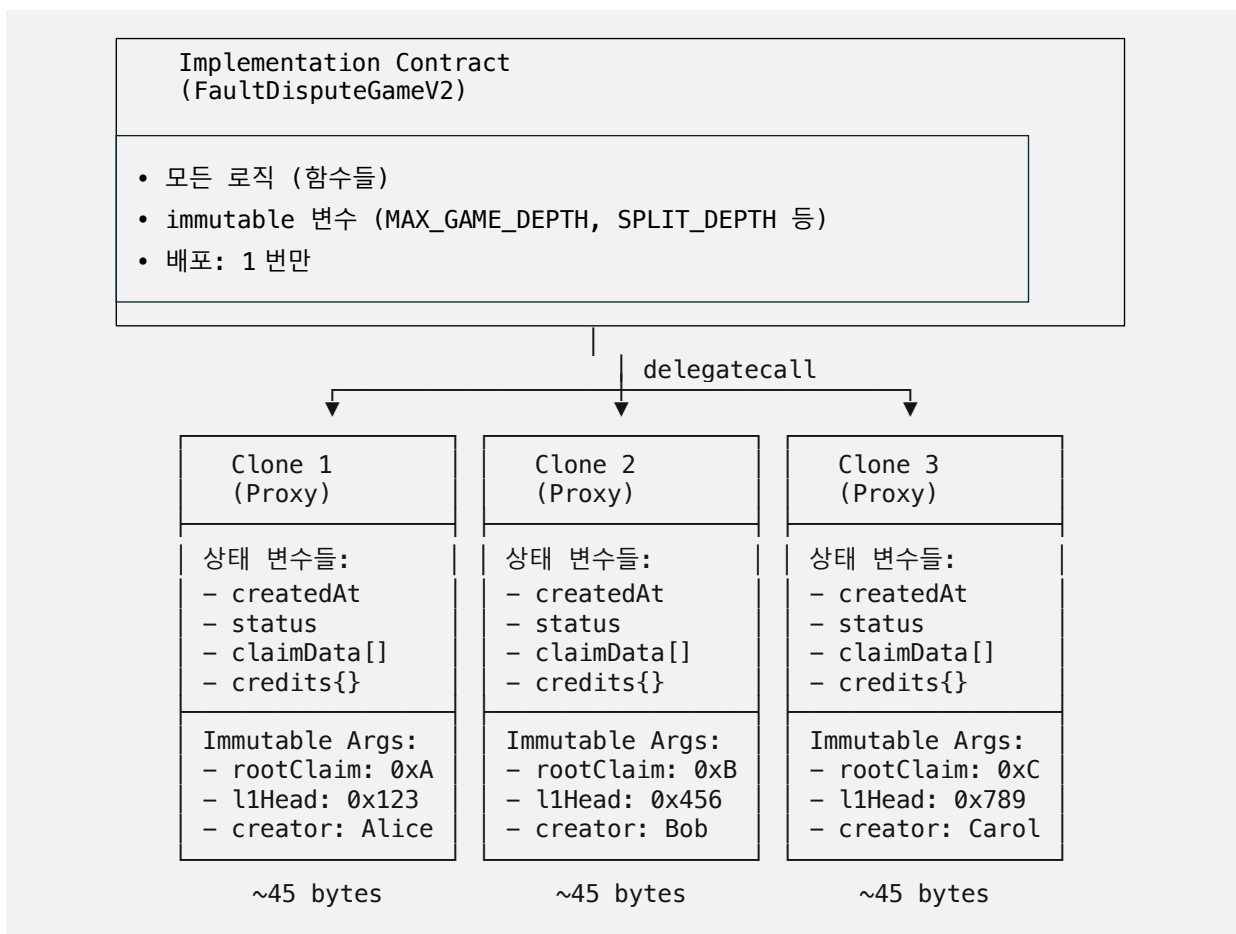
- Minimal Proxy Pattern (EIP-1167) 사용
- 로직은 implementation 에, 상태만 각 clone 에 저장
- 배포 비용 대폭 감소

## 요약

질문	답변
매 L2 블록마다 proxy 생성?	❌ 아니오
생성 빈도	ProposalInterval 설정값 (예: 1 시간)
하루 생성 수	약 24 개 (1 시간 간격 기준)
Clone 사용 이유	Gas 비용 절감

## Dispute Game Proxy Clone (Clones with Immutable Args)

### 핵심 개념



## 실제 코드 예시

### 1. Factory 에서 Clone 생성

```
// packages/contracts-bedrock/src/dispute/DisputeGameFactory.sol
if (gameArgs[_gameType].length == 0) {
 // Clone the implementation contract and initialize it with the given parameters.
 //
 // CWIA Calldata Layout:
 //

| Bytes | Description |
|--------------|------------------------------------|
| [0, 20) | Game creator address |
| [20, 52) | Root claim |
| [52, 84) | Parent block hash at creation time |
| [84, 84 + n) | Extra data (opaque) |

 //
 proxy_ = IDisputeGame(address(impl).clone(abi.encodePacked(msg.sender, _rootClaim, parentHash,
 _extraData)));
}
```

### 2. Clone 에서 Immutable Args 읽기

Clone 은 calldata 끝에 붙은 immutable args 를 \_getArgXXX 함수로 읽습니다:

```
// packages/contracts-bedrock/src/dispute/v2/FaultDisputeGameV2.sol

/// @notice Getter for the root claim.
/// @dev `clones-with-immutable-args` argument #2
/// @return rootClaim_ The root claim of the DisputeGame.
function rootClaim() public pure returns (Claim rootClaim_) {
 rootClaim_ = Claim.wrap(_getArgBytes32(20)); // ← 오프셋 20 에서 32 바이트 읽기
}

/// @notice Getter for the parent hash of the L1 block when the dispute game was created.
/// @dev `clones-with-immutable-args` argument #3
/// @return l1Head_ The parent hash of the L1 block when the dispute game was created.
function l1Head() public pure returns (Hash l1Head_) {
 l1Head_ = Hash.wrap(_getArgBytes32(52)); // ← 오프셋 52 에서 32 바이트 읽기
}

/// @notice Getter for the game type.
/// @dev `clones-with-immutable-args` argument #4
/// @return gameType_ The type of proof system being used.
function gameType() public pure returns (GameType gameType_) {
 gameType_ = GameType.wrap(_getArgUint32(84)); // ← 오프셋 84 에서 4 바이트 읽기
}
```

### 3. Clone 에 저장되는 상태 변수

```
// packages/contracts-bedrock/src/dispute/v2/FaultDisputeGameV2.sol

/// @notice The starting timestamp of the game
Timestamp public createdAt; // ← 이 게임의 생성 시간

/// @notice The timestamp of the game's global resolution.
Timestamp public resolvedAt; // ← 이 게임의 종료 시간

/// @notice Returns the current status of the game.
GameStatus public status; // ← IN_PROGRESS, DEFENDER_WINS 등

/// @notice Flag for the `initialize` function to prevent re-initialization.
bool internal initialized; // ← 초기화 여부

/// @notice Flag for whether or not the L2 block number claim has been invalidated
bool public l2BlockNumberChallenged; // ← 챌린지 여부

/// @notice An append-only array of all claims made during the dispute game.
ClaimData[] public claimData; // ← 이 게임의 모든 claim 들

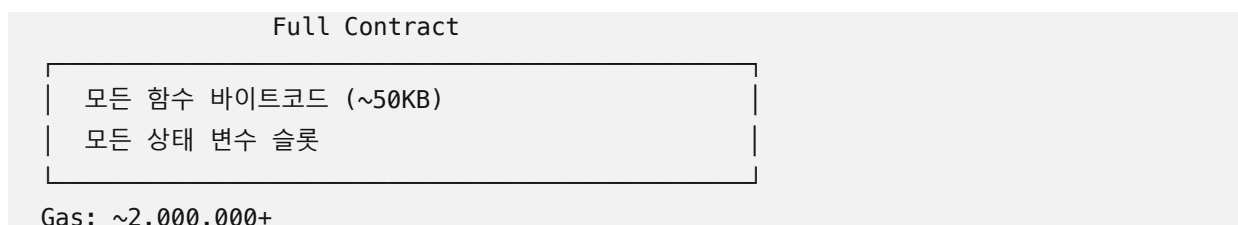
/// @notice Credited balances for winning participants.
mapping(address => uint256) public normalModeCredit; // ← 승리자 보상 잔액

/// @notice A mapping to allow for constant-time lookups of existing claims.
mapping(Hash => bool) public claims; // ← claim 존재 여부

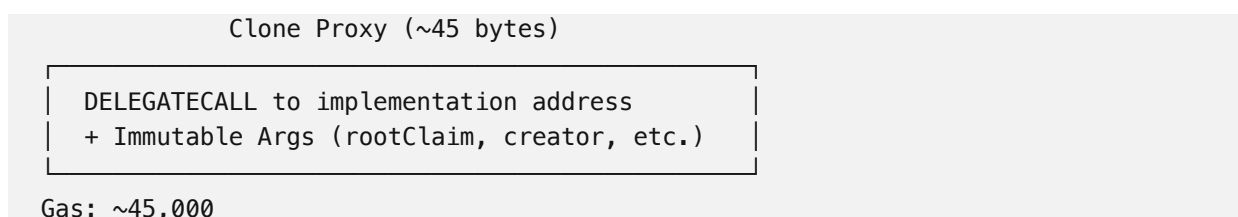
/// @notice A mapping of subgames rooted at a claim index
mapping(uint256 => uint256[]) public subgames; // ← 서브게임 구조
```

### 메모리 레이아웃 비교

#### 일반 Contract 배포



#### Clone (Minimal Proxy)



## 실제 작동 흐름 예시

1. op-proposer 가 create(CANNON, 0xABC..., extraData) 호출  
↓
2. DisputeGameFactory:
  - impl = gameImpls[CANNON] (FaultDisputeGameV2 주소)
  - proxy = impl.clone(abi.encodePacked(
    - msg.sender, // creator: op-proposer 주소
    - 0xABC..., // rootClaim: L2 output root
    - blockhash(...), // l1Head
    - extraData
 ))
 ↓
3. 새 Clone 생성됨 (주소: 0x1234...)
  - 바이트코드: ~45 bytes (delegatecall 로직 + immutable args)
  - 상태 변수: 비어있음 (initialize 전)
 ↓
4. proxy.initialize() 호출
  - Clone 의 상태 변수에 초기값 설정
  - createdAt = block.timestamp
  - status = IN\_PROGRESS
 ↓
5. 게임 진행 중 (7 일간)
  - Clone 에 claimData[] 추가
  - Clone 에 credits 누적
  - 로직은 모두 Implementation 에서 실행
 ↓
6. resolve() 호출
  - status = DEFENDER\_WINS 또는 CHALLENGER\_WINS
  - resolvedAt = block.timestamp

## 요약

구분	Implementation	Clone
배포	1 번	매 게임마다
바이트코드	~50KB	~45 bytes
로직	✅ 모든 함수	❌ delegatecall 만
상태 변수	❌ 사용 안함	✅ 각 게임별 저장
Immutable Args	❌	✅ (rootClaim, creator 등)
Gas	~2M	~45K

메인 루프: 폴링 → 제안 여부 → L1 전송

PollInterval(기본 6 초 등)마다 FetchL2OOOutput 또는 \*\*FetchDGFOutput\*\* 으로 "지금 제출할지"를 판단하고, shouldPropose 이면 proposeOutput → sendTransaction 으로 이어진다.

```

// op-proposer/proposer/driver.go

// loop is responsible for creating & submitting the next outputs
// The loop regularly polls the L2 chain to infer whether to make the next proposal.
func (l *L2OutputSubmitter) loop() {
 defer l.wg.Done()
 defer l.Log.Info("loop returning")
 ctx := l.ctx
 ticker := time.NewTicker(l.Cfg.PollInterval)
 defer ticker.Stop()
 for {
 select {
 case <-ticker.C:
 // prioritize quit signal
 select {
 case <-l.done:
 return
 default:
 }

 // A note on retrying: the outer ticker already runs on a short
 // poll interval, which has a default value of 6 seconds. So no
 // retry logic is needed around proposal fetching here.
 var proposal source.Proposal
 var shouldPropose bool
 var err error
 if l.dgfContract == nil {
 proposal, shouldPropose, err = l.FetchL2OOOutput(ctx)
 } else {
 proposal, shouldPropose, err = l.FetchDGFOutput(ctx)
 }
 if err != nil {
 l.Log.Warn("Error getting proposal", "err", err)
 continue
 } else if !shouldPropose {
 // debug logging already in Fetch(DGF|L2OO)Output
 continue
 }

 l.proposeOutput(ctx, proposal)
 }
 }
}

```

DGF 경로 요지: ProposalInterval 기준 시각 이전에 동일 프로포저·게임 타입으로 게임을 만든 적이 있으면 잠시 쉬고, finalized(또는 설정 시 safe) L2 블록 번호에 대한 출력을 가져온 뒤, 직전에 올린 claim 과 output root 가 같으면 중복 제출을 건너뛴다.

```

// op-proposer/proposer/driver.go

func (l *L2OutputSubmitter) FetchDGFOutput(ctx context.Context) (source.Proposal, bool, error) {
 cutoff := time.Now().Add(-l.Cfg.ProposalInterval)
 proposedRecently, proposalTime, claim, err := l.dgfContract.HasProposedSince(ctx, l.Txmgr.From(), cutoff,
l.Cfg.DisputeGameType)
 if err != nil {
 return source.Proposal{}, false, fmt.Errorf("could not check for recent proposal: %w", err)
 }

 if proposedRecently {
 l.Log.Debug("Duration since last game not past proposal interval", "duration", time.Since(proposalTime))
 return source.Proposal{}, false, nil
 }

 // Fetch the current L2 heads
 currentBlockNumber, err := l.FetchCurrentBlockNumber(ctx)
 if err != nil {
 return source.Proposal{}, false, fmt.Errorf("could not fetch current block number: %w", err)
 }

 if currentBlockNumber == 0 {
 l.Log.Info("Skipping proposal for genesis block")
 return source.Proposal{}, false, nil
 }

 output, err := l.FetchOutput(ctx, currentBlockNumber)
 if err != nil {
 return source.Proposal{}, false, fmt.Errorf("could not fetch output at current block number %d: %w",
currentBlockNumber, err)
 }

 if claim == output.Root {
 l.Log.Debug("Skipping proposal: output root unchanged since last proposed game",
"last_proposed_root", claim, "output_root", output.Root)
 return source.Proposal{}, false, nil
 }

 l.Log.Info("No proposals found for at least proposal interval, submitting proposal now", "proposalInterval",
l.Cfg.ProposalInterval)

 return output, true, nil
}

```

```

// op-proposer/proposer/driver.go
// FetchCurrentBlockNumber gets the current block number from the [L2OutputSubmitter]'s [RollupClient]. If
the `AllowNonFinalized` configuration
// option is set, it will return the safe head block number, and if not, it will return the finalized head block
number.
func (l *L2OutputSubmitter) FetchCurrentBlockNumber(ctx context.Context) (uint64, error) {
 status, err := l.ProposalSource.SyncStatus(ctx)
 if err != nil {
 return 0, fmt.Errorf("getting sync status: %w", err)
 }

 // Use either the finalized or safe head depending on the config. Finalized head is default & safer.
 if l.Cfg.AllowNonFinalized {
 return status.SafeL2, nil
 }
 return status.FinalizedL2, nil
}

func (l *L2OutputSubmitter) FetchOutput(ctx context.Context, block uint64) (source.Proposal, error) {
 output, err := l.ProposalSource.ProposalAtSequenceNum(ctx, block)
 if err != nil {
 return source.Proposal{}, fmt.Errorf("fetching output at block %d: %w", block, err)
 }
 if onum := output.SequenceNum; onum != block { // sanity check, e.g. in case of bad RPC caching
 return source.Proposal{}, fmt.Errorf("output block number %d mismatches requested %d",
output.SequenceNum, block)
 }
 return output, nil
}

```

---

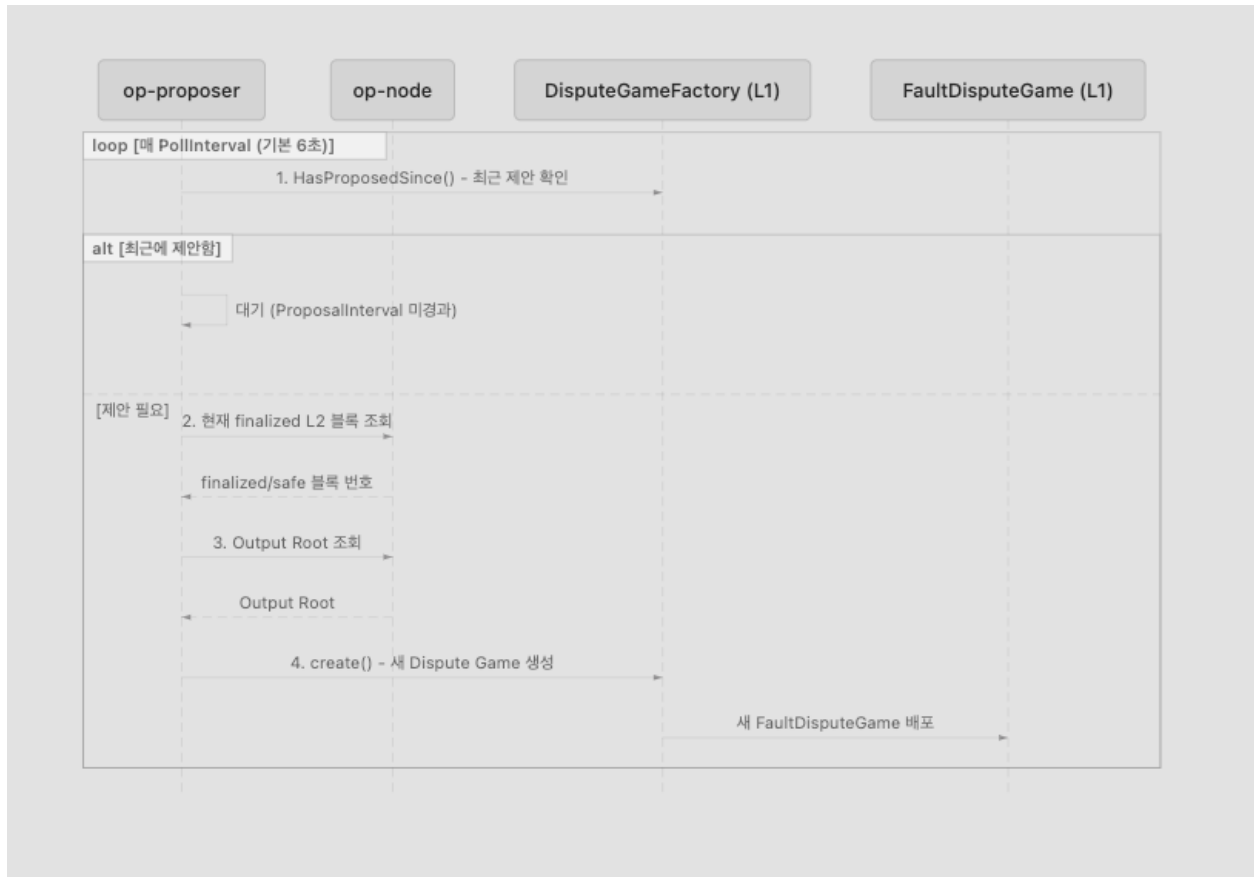
## op-proposer 에서 DisputeGameFactory 의 역할과 작동 순서

### 역할 개요

DisputeGameFactory 는 L1 에 배포된 스마트 컨트랙트로, op-proposer 가 L2 의 상태(output root)를 L1 에 제안할 때 새로운 Dispute Game 을 생성하는 팩토리 역할을 합니다.

기존 L2OutputOracle 과 달리, 각 제안은 독립적인 Dispute Game 컨트랙트로 생성되어 Fault Proof 시스템을 통해 검증됩니다.

## 작동 순서 (Flow)



## 단계별 상세 설명

### 1 단계: 제안 필요 여부 확인 (FetchDGFOutput)

```

// :op-proposer/proposer/driver.go
func (l *L2OutputSubmitter) FetchDGFOutput(ctx context.Context) (source.Proposal, bool, error) {
 cutoff := time.Now().Add(-l.Cfg.ProposalInterval)
 proposedRecently, proposalTime, claim, err := l.dgfContract.HasProposedSince(ctx, l.Txmgr.From(), cutoff,
 l.Cfg.DisputeGameType)
 // ...
 if proposedRecently {
 l.Log.Debug("Duration since last game not past proposal interval", "duration", time.Since(proposalTime))
 return source.Proposal{}, false, nil
 }
}

```

- ProposalInterval (예: 1 시간) 이후 제안한 적이 없는지 확인
- DisputeGameFactory 의 gameAtIndex()를 역순으로 검색하여 동일 proposer 의 게임 존재 여부 확인

### 2 단계: 현재 L2 블록 조회 (FetchCurrentBlockNumber)

```

// op-proposer/proposer/driver.go
func (l *L2OutputSubmitter) FetchCurrentBlockNumber(ctx context.Context) (uint64, error) {
 status, err := l.ProposalSource.SyncStatus(ctx)
}

```

```
// ...
if l.Cfg.AllowNonFinalized {
 return status.SafeL2, nil
}
return status.FinalizedL2, nil
}
```

- op-node 에서 finalized 또는 safe L2 블록 번호를 가져옴
- 기본값은 finalized (더 안전함)

### 3 단계: Output Root 조회 (FetchOutput)

```
// op-proposer/proposer/driver.go
func (l *L2OutputSubmitter) FetchOutput(ctx context.Context, block uint64) (source.Proposal, error) {
 output, err := l.ProposalSource.ProposalAtSequenceNum(ctx, block)
 // ...
}
```

- 해당 블록의 Output Root 를 op-node 에서 가져옴
- Output Root 는 L2 상태의 암호학적 커밋먼트

### 4 단계: Dispute Game 생성 트랜잭션 전송 (sendTransaction)

```
// op-proposer/proposer/driver.go
if l.Cfg.DisputeGameFactoryAddr != nil {
 candidate, err := l.ProposeL2OutputDGFTxCandidate(ctx, output)
 if err != nil {
 return err
 }
 receipt, err = l.Txmgr.Send(ctx, candidate)
 // ...
}
```

Go 래퍼에서 트랜잭션 생성:

```
// op-proposer/contracts/disputegamefactory.go
func (f *DisputeGameFactory) ProposalTx(ctx context.Context, gameType uint32, outputRoot common.Hash,
l2BlockNum uint64) (txmgr.TxCandidate, error) {
 // initBond 조회
 result, err := f.caller.SingleCall(cCtx, rpcblock.Latest, f.contract.Call(methodInitBonds, gameType))
 initBond := result.GetBigInt(0)

 // create() 호출 데이터 생성
 call := f.contract.Call(methodCreateGame, gameType, outputRoot,
common.BigToHash(big.NewInt(int64(l2BlockNum))).Bytes())
 candidate, err := call.ToTxCandidate()
 candidate.Value = initBond // 초기 보증금 첨부
 return candidate, err
}
```

## 5 단계: DisputeGameFactory 컨트랙트에서 게임 생성

```
// packages/contracts-bedrock/src/dispute/DisputeGameFactory.sol
function create(
 GameType _gameType,
 Claim _rootClaim,
 bytes calldata _extraData
) external payable returns (IDisputeGame proxy_) {
 // 구현체 가져오기
 IDisputeGame impl = gameImpls[_gameType];

 // 보증금 확인
 if (msg.value != initBonds[_gameType]) revert IncorrectBondAmount();

 // Clone 방식으로 새 게임 컨트랙트 배포
 proxy_ = IDisputeGame(address(impl).clone(...));
 proxy_.initialize{ value: msg.value }();

 // 저장 및 이벤트 발생
 _disputeGames[uuid] = id;
 _disputeGameList.push(id);
 emit DisputeGameCreated(address(proxy_), _gameType, _rootClaim);
}
```

## 핵심 컴포넌트 요약

컴포넌트	역할
op-proposer	주기적으로 L2 상태를 폴링하고 제안 트랜잭션 전송
DisputeGameFactory	새로운 Dispute Game 컨트랙트 인스턴스 생성
FaultDisputeGame	생성된 개별 게임, challenger 가 이의 제기 가능
op-challenger	잘못된 제안에 대해 이의 제기 및 게임 해결

## 메인 루프 요약

```
// op-proposer/proposer/driver.go
func (l *L2OutputSubmitter) loop() {
 ticker := time.NewTicker(l.Cfg.PollInterval) // 기본 6 초
 for {
 select {
 case <-ticker.C:
 if l.dgfContract == nil {
 proposal, shouldPropose, err = l.FetchL2OOOutput(ctx) // Legacy
 } else {
 proposal, shouldPropose, err = l.FetchDGFOOutput(ctx) // Fault Proof
 }
 }
 }
}
```

```

 if shouldPropose {
 l.proposeOutput(ctx, proposal) // 트랜잭션 전송
 }
 case <-l.done:
 return
 }
}
}

```



Output root 형성: 왜 op-node RPC 인가

Output root 는 버전·stateRoot·messagePasserStorageRoot·blockHash 등을 정해진 규칙으로 직렬화한 뒤 Keccak 한 값이다.

```

// op-service/eth/output.go
type OutputV0 struct {
 StateRoot Bytes32 `json:"stateRoot"`
 MessagePasserStorageRoot Bytes32 `json:"messagePasserStorageRoot"`
 BlockHash common.Hash `json:"blockHash"`
}

```

```

func (o *OutputV0) Marshal() []byte {
 var buf [128]byte
 version := o.Version()
 copy(buf[:32], version[:])
 copy(buf[32:], o.StateRoot[:])
 copy(buf[64:], o.MessagePasserStorageRoot[:])
 copy(buf[96:], o.BlockHash[:])
 return buf[:]
}

// OutputRoot returns the keccak256 hash of the marshaled L2 output
func OutputRoot(output Output) Bytes32 {
 marshaled := output.Marshal()
 return Bytes32(crypto.Keccak256Hash(marshaled))
}

```

프로포저 쪽 RollupProposalSource 는 롤업 JSON-RPC optimism\_outputAtBlock 로 위 응답을 받아 Proposal 으로 포장한다.

```

// op-proposer/proposer/source/source_rollup.go
func (r *RollupProposalSource) ProposalAtSequenceNum(ctx context.Context, blockNum uint64) (Proposal, error) {
 client, err := r.provider.RollupClient(ctx)
 if err != nil {
 return Proposal{}, fmt.Errorf("failed to select active rollup client: %w", err)
 }
 output, err := client.OutputAtBlock(ctx, blockNum)
 if err != nil {
 return Proposal{}, err
 }

 if output.Version != supportedL2OutputVersion {
 return Proposal{}, fmt.Errorf("unsupported l2 output version: %v, supported: %v", output.Version, supportedL2OutputVersion)
 }
 return Proposal{
 Root: common.Hash(output.OutputRoot),
 SequenceNum: output.BlockRef.Number,
 CurrentL1: output.Status.CurrentL1.ID(),
 Legacy: LegacyProposalData{
 HeadL1: output.Status.HeadL1,
 SafeL2: output.Status.SafeL2,
 FinalizedL2: output.Status.FinalizedL2,
 BlockRef: output.BlockRef,
 },
 },

```

```
}, nil
}
```

```
// op-service/sources/rollupclient.go
func (r *RollupClient) OutputAtBlock(ctx context.Context, blockNum uint64) (*eth.OutputResponse, error) {
 var output *eth.OutputResponse
 err := r.rpc.CallContext(ctx, &output, "optimism_outputAtBlock", hexutil.Uint64(blockNum))
 return output, err
}
```

op-node 는 해당 블록 ref 와 동기화 상태를 잡고, EL 에서 OutputV0AtBlock 등으로 구성요소를 읽은 뒤 OutputRoot 를 채운다.

```
// op-node/node/api.go
func (n *nodeAPI) OutputAtBlock(ctx context.Context, number hexutil.Uint64) (*eth.OutputResponse, error) {
 ref, status, err := n.dr.BlockRefWithStatus(ctx, uint64(number))
 if err != nil {
 return nil, fmt.Errorf("failed to get L2 block ref with sync status: %w", err)
 }

 // OutputV0AtBlock uses the WithdrawalsRoot in the block header as the value for the
 // output MessagePasserStorageRoot, if Isthmus hard fork has activated.
 output, err := n.client.OutputV0AtBlock(ctx, ref.Hash())
 if err != nil {
 return nil, fmt.Errorf("failed to get L2 output at block %s: %w", ref, err)
 }

 return ð.OutputResponse{
 Version: output.Version(),
 OutputRoot: eth.OutputRoot(output),
 BlockRef: ref,
 WithdrawalStorageRoot: common.Hash(output.MessagePasserStorageRoot),
 StateRoot: common.Hash(output.StateRoot),
 Status: status,
 }, nil
}
```

Output Root 는 다음 3 가지를 keccak256 해시한 값입니다:

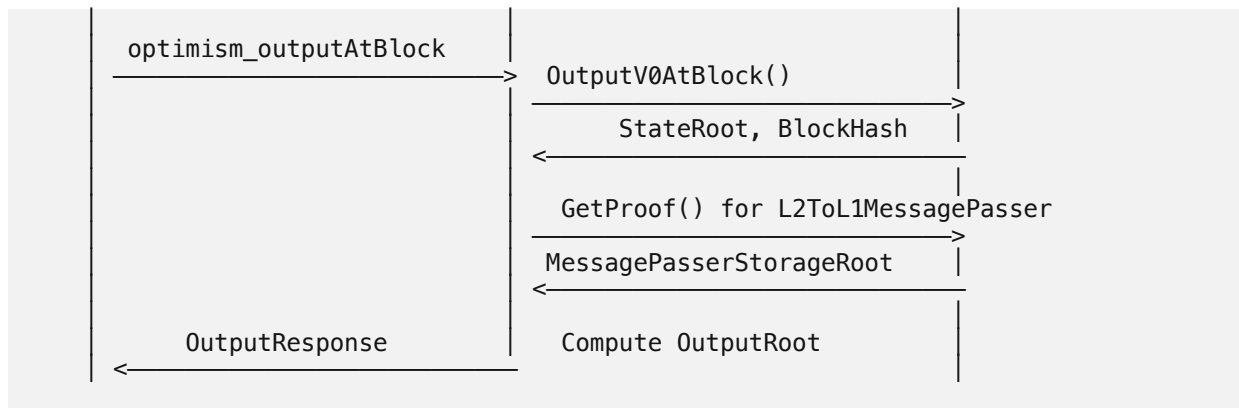
1. StateRoot: L2 블록의 전체 상태 트리 루트
2. MessagePasserStorageRoot: L2ToL1MessagePasser 컨트랙트의 스토리지 루트 (출금 정보)
3. BlockHash: L2 블록 해시

RPC 호출 흐름

op-proposer

op-node

op-geth (L2)



## 7 일 후 변경되는 값과 프로세스

Optimism의 Fault Proof 시스템에서 7 일(604,800 초)은 중요한 보안 기간입니다. 크게 두 가지 지연 시간이 있습니다:

### 1. Proof Maturity Delay (증명 성숙 지연)

```
// packages/contracts-bedrock/src/L1/OptimismPortal2.sol

/// @notice The delay between when a withdrawal is proven and when it may be finalized.
uint256 internal immutable PROOF_MATURITY_DELAY_SECONDS;
```

#### 변경되는 값

Withdrawal의 finalize 가능 여부: proveWithdrawalTransaction 호출 후 7 일이 지나면  
finalizeWithdrawalTransaction 호출 가능

#### 검증 로직

```
// packages/contracts-bedrock/src/L1/OptimismPortal2.sol

// A proven withdrawal must wait at least `PROOF_MATURITY_DELAY_SECONDS` before finalizing.
if (block.timestamp - provenWithdrawal.timestamp <= PROOF_MATURITY_DELAY_SECONDS) {
 revert OptimismPortal_ProofNotOldEnough();
}
```

### 2. Dispute Game Finality Delay (분쟁 게임 확정 지연)

```
// packages/contracts-bedrock/src/dispute/AnchorStateRegistry.sol

/// @notice The dispute game finality delay in seconds.
uint256 internal immutable DISPUTE_GAME_FINALITY_DELAY_SECONDS;
```

#### 변경되는 값

Dispute Game의 finalized 상태: resolve() 호출 후 7 일이 지나면 게임이 "finalized" 상태로 전환

#### 검증 로직

```
// packages/contracts-bedrock/src/dispute/AnchorStateRegistry.sol
```

```

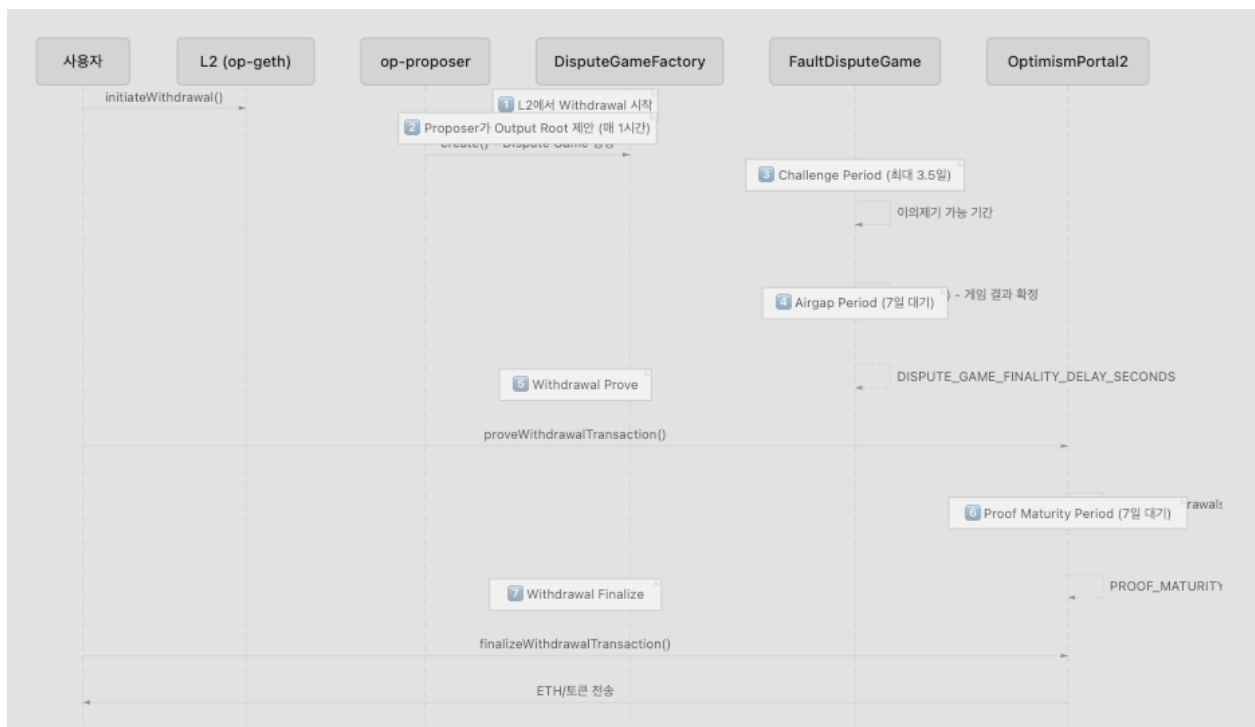
/// @notice Returns whether a game is finalized.
/// @param _game The game to check.
/// @return Whether the game is finalized.
function isGameFinalized(IDisputeGame _game) public view returns (bool) {
 // Game must be resolved.
 if (!isGameResolved(_game)) {
 return false;
 }

 // Game must be beyond the "airgap period" - time since resolution must be at least
 // "dispute game finality delay" seconds in the past.
 if (block.timestamp - _game.resolvedAt().raw() <= DISPUTE_GAME_FINALITY_DELAY_SECONDS) {
 return false;
 }

 return true;
}

```

## 전체 Withdrawal 프로세스 타임라인



## 핵심 상태 변화 요약

시점	이벤트	변경되는 상태
T+0	Dispute Game 생성	status = IN_PROGRESS

T+~3.5 일	Game Resolve	status = DEFENDER_WINS, resolvedAt = timestamp
T+~10.5 일	Game Finalized	isGameFinalized() = true
T+~10.5 일	Withdrawal Prove	provenWithdrawals[hash].timestamp = now
T+~17.5 일	Withdrawal Finalize 가능	finalizedWithdrawals[hash] = true

## 7 일 대기의 목적

1. 보안 버퍼: 잘못된 output root 가 제안되었을 경우 Guardian 이 개입할 시간 확보
2. Challenge 기회: 악의적인 claim 에 대해 challenger 가 이의를 제기할 추가 시간
3. Airgap 보호: Dispute Game 결과 확정 후에도 추가적인 안전 마진 제공

mainnet 설정에서 확인 가능:

```
{
 "faultGameWithdrawalDelay": 604800,
 "proofMaturityDelaySeconds": 604800
}
```

## op-proposer 가 제출하는 트랜잭션

op-proposer 는 L2 상태를 L1 에 제출하는 역할을 합니다. 두 가지 모드가 있습니다:

### 1. Legacy 모드: L2OutputOracle.proposeL2Output()

```
// op-proposer/proposer/driver.go
func proposeL2OutputTxData(abi *abi.ABI, output source.Proposal) ([]byte, error) {
 return abi.Pack(
 "proposeL2Output",
 output.Root,
 new(big.Int).SetUint64(output.SequenceNum),
 output.CurrentL1.Hash,
 new(big.Int).SetUint64(output.CurrentL1.Number))
}
```

트랜잭션 파라미터:

파라미터	설명
outputRoot	L2 상태의 해시 (StateRoot + MessagePasserStorageRoot + BlockHash)
l2BlockNumber	해당 output 이 속한 L2 블록 번호
l1BlockHash	참조한 L1 블록 해시
l1BlockNumber	참조한 L1 블록 번호

## 2. Fault Proof 모드: DisputeGameFactory.create()

```
// op-proposer/contracts/disputefactory.go
func (f *DisputeGameFactory) ProposalTx(ctx context.Context, gameType uint32, outputRoot common.Hash,
l2BlockNum uint64) (txmgr.TxCandidate, error) {
 cCtx, cancel := context.WithTimeout(ctx, f.networkTimeout)
 defer cancel()
 result, err := f.caller.SingleCall(cCtx, rpcblock.Latest, f.contract.Call(methodInitBonds, gameType))
 if err != nil {
 return txmgr.TxCandidate{}, fmt.Errorf("failed to fetch init bond: %w", err)
 }
 initBond := result.GetBigInt(0)
 call := f.contract.Call(methodCreateGame, gameType, outputRoot,
common.BigToHash(big.NewInt(int64(l2BlockNum))).Bytes())
 candidate, err := call.ToTxCandidate()
 if err != nil {
 return txmgr.TxCandidate{}, err
 }
 candidate.Value = initBond
 return candidate, err
}
```

### 트랜잭션 파라미터:

파라미터	설명
gameType	Dispute Game 종류 (예: Cannon, Permissioned 등)
rootClaim	제안하는 L2 output root
extraData	L2 블록 번호 (bytes 로 인코딩)
value	Init Bond (보증금, ETH)

### 트랜잭션 제출 흐름

```
// op-proposer/proposer/driver.go

// loop is responsible for creating & submitting the next outputs
// The loop regularly polls the L2 chain to infer whether to make the next proposal.
func (l *L2OutputSubmitter) loop() {
 defer l.wg.Done()
 defer l.Log.Info("loop returning")
 ctx := l.ctx
 ticker := time.NewTicker(l.Cfg.PollInterval)
 defer ticker.Stop()
 for {
```

```

select {
case <-ticker.C:
 // prioritize quit signal
 select {
 case <-l.done:
 return
 default:
 }

 // A note on retrying: the outer ticker already runs on a short
 // poll interval, which has a default value of 6 seconds. So no
 // retry logic is needed around proposal fetching here.
 var proposal source.Proposal
 var shouldPropose bool
 var err error
 if l.dgfContract == nil {
 proposal, shouldPropose, err = l.FetchL2OOOutput(ctx)
 } else {
 proposal, shouldPropose, err = l.FetchDGFOutput(ctx)
 }
 if err != nil {
 l.Log.Warn("Error getting proposal", "err", err)
 continue
 } else if !shouldPropose {
 // debug logging already in Fetch(DGF|L2OO)Output
 continue
 }

 l.proposeOutput(ctx, proposal)
case <-l.done:
 return
}
}

```

흐름 요약:

#### op-proposer Loop

1. PollInterval (기본 6 초) 마다 체크
  - ↓
2. op-node 에서 SyncStatus 조회
  - FinalizedL2 또는 SafeL2 블록 번호 확인
  - ↓
3. 제출할 output 이 있는지 확인
  - L200: NextBlockNumber() 호출
  - DGF: HasProposedSince() 호출

↓  
4. op-node 에서 OutputAtBlock() 호출  
- OutputRoot 계산 결과 수신

↓  
5. L1 에 트랜잭션 제출  
- L200: proposeL2Output()  
- DGF: create() + initBond 전송

## Bond 제출 방법과 금액

op-proposer 는 create() 호출 직전에 온체인에서 현재 bond 금액을 읽어와서 msg.value 로 ETH 를 전송합니다. create()는 payable 함수입니다. Bond 는 calldata 가 아니라 msg.value(ETH 송금)으로 전달합니다.

```
DisputeGameFactory.create(gameType, rootClaim, extraData)
├─ msg.value = initBonds[gameType] ← 정확히 일치해야 함 (초과/부족 모두 revert)
├─ proxy_.initialize{ value: msg.value }()
└─ WETH.deposit{ value: msg.value }() ← DelayedWETH 컨트랙트에 예치
```

### 요약표

항목	내용
금액 결정	DisputeGameFactory.initBonds(gameType) 온체인 조회
설정 권한	owner 만 setInitBond() 호출 가능
전달 방법	create() 호출 시 msg.value 로 ETH 동봉
검증	컨트랙트가 msg.value != initBonds[gameType]이면 IncorrectBondAmount revert
예치처	FaultDisputeGame → DelayedWETH.deposit()
반환 시점	게임 resolve 후 claimCredit() 호출 시 출금

ex. GameType 이 CANNON(id=0)이고 Optimism mainnet 의 경우 0.08ETH 로 설정되어 있음

## op-challenger

---

### 설명

L2(Layer 2) 체인, 특히 Optimistic Rollup(예: Optimism, Arbitrum) 구조에서 \*\*Challenger(챌린저)\*\*는 네트워크의 보안과 무결성을 유지하는 핵심적인 감시자 역할을 수행합니다.

Optimistic Rollup 은 이름 그대로 "모든 트랜잭션이 올바르다"고 낙관적으로 가정하고 먼저 L1(Ethereum)에 기록하기 때문에, 누군가는 이 기록이 실제로 맞는지 검증해야 합니다. 이때 Challenger 가 다음과 같은 중요한 업무를 담당합니다.

1. 트랜잭션 감시 및 검증

Challenger 는 L2 에서 발생하는 모든 트랜잭션을 실시간으로 모니터링합니다. L2 시퀀서(Sequencer)가 L1 에 제출한 상태 루트(State Root)가 실제 트랜잭션 결과와 일치하는지 직접 계산하여 대조합니다.

2. 사기 입증 (Fraud Proof) 개시

만약 시퀀서가 잘못된 트랜잭션을 포함시키거나 상태를 조작하여 허위 데이터를 L1 에 올린 것을 발견하면, Challenger 는 즉시 결함 증명(Fault Proof) 또는 사기 증명(Fraud Proof) 절차를 시작합니다. 이를 통해 해당 데이터에 이의를 제기(Challenge)합니다

3. 분쟁 해결 (Dispute Game) 참여

이의 제기가 발생하면 L1 스마트 컨트랙트 상에서 시퀀서와 Challenger 간의 "진실 공방"이 벌어집니다.

- 이진 탐색(Bisection Game) 방대한 데이터 중 정확히 어느 지점에서 계산이 틀렸는지 찾아내기 위해 데이터를 반씩 나누어가며 검증하는 과정을 거칩니다.
- 최종 판결: 최종적으로 잘못된 지점이 특정되면 L1 에서 해당 연산을 직접 실행하여 누구의 말이 맞는지 판결합니다.

4. 경제적 징벌 및 보상

Challenger 시스템은 경제적 인센티브로 작동합니다.

- 시퀀서 처벌: 시퀀서의 부정이 확정되면 시퀀서가 예치(Staking)한 담보금을 몰수(Slashing)합니다.
- Challenger 보상: 부정을 잡아낸 Challenger 는 몰수된 담보금의 일부를 보상으로 받아 수익을 창출합니다. 반대로 Challenger 가 틀린 주장을 했을 경우 자신의 담보금을 잃게 됩니다.

### 요약하자면

Challenger 는 "신뢰하지 말고 검증하라(Don't trust, verify)"는 블록체인의 철학을 실천하는 주체입니다. 단 한 명의 정직한 Challenger 만 존재해도 L2 체인의 보안이 유지될 수 있기 때문에, Optimistic Rollup 의 안전성을 보장하는 최후의 보루라고 할 수 있습니다.

---

## op-challenger 의 동작 흐름

### op-challenger

1. DisputeGameFactory 에서 게임 목록 조회
2. 각 게임의 rootClaim 과 claims 로드
3. 자체 TraceProvider 로 "올바른 값" 계산
4. 계산된 값과 게임의 claim 비교
5. 불일치 시 attack 또는 defend 수행

### 1 단계: 게임 모니터링

```
// op-challenger/game/monitor.go
func (m *gameMonitor) progressGames(ctx context.Context, blockHash common.Hash, blockNumber uint64)
error {
 minGameTimestamp := clock.MinCheckedTimestamp(m.clock, m.gameWindow)
 games, err := m.source.GetGamesAtOrAfter(ctx, blockHash, minGameTimestamp)
 if err != nil {
 return fmt.Errorf("failed to load games: %w", err)
 }
}
```

- DisputeGameFactory 에서 최근 게임 목록 조회
- 각 게임의 proxy 주소와 메타데이터 획득

### 2 단계: 게임에서 Claims 로드

```
// op-challenger/game/fault/contracts/gamefactory.go
func (f *DisputeGameFactoryContract) GetGame(ctx context.Context, idx uint64, block rpcblock.Block)
(gameTypes.GameMetadata, error) {
 defer f.metrics.StartContractRequest("GetGame")()
 result, err := f.multiCaller.SingleCall(ctx, block, f.contract.Call(methodGameAtIndex, new(big.Int).SetUint64(idx)))
 if err != nil {
 return gameTypes.GameMetadata{}, fmt.Errorf("failed to load game %v: %w", idx, err)
 }
 return f.decodeGame(idx, result), nil
}
```

- 각 proxy 에서 claimData[] 배열 로드
- rootClaim (output root), extraData (L2 블록 번호) 등 획득

### 3 단계: 올바른 값 계산 (핵심!)

```
// op-challenger/game/fault/trace/outputs/provider.go
// OutputTraceProvider is a [types.TraceProvider] implementation that uses
// output roots for given L2 Blocks as a trace.
type OutputTraceProvider struct {
 types.PrestateProvider
 logger log.Logger
 rollupProvider OutputRollupClient // ← op-node RPC 클라이언트
```

```

l2Client utils.L2HeaderSource
prestateBlock uint64
poststateBlock uint64
l1Head eth.BlockID
gameDepth types.Depth
}

```

```

// op-challenger/game/fault/trace/outputs/provider.go
func (o *OutputTraceProvider) Get(ctx context.Context, pos types.Position) (common.Hash, error) {
 outputBlock, err := o.HonestBlockNumber(ctx, pos)
 if err != nil {
 return common.Hash{}, err
 }
 return o.outputAtBlock(ctx, outputBlock)
}

```

```

// op-challenger/game/fault/trace/outputs/provider.go
func (o *OutputTraceProvider) outputAtBlock(ctx context.Context, block uint64) (common.Hash, error) {
 output, err := o.rollupProvider.OutputAtBlock(ctx, block) // ← op-node 에 요청!
 if err != nil {
 return common.Hash{}, fmt.Errorf("failed to fetch output at block %v: %w", block, err)
 }
 return common.Hash(output.OutputRoot), nil
}

```

핵심: op-challenger 는 자체 op-node 에 연결하여 동일한 L2 블록에 대해 직접 output root 를 계산합니다!

#### 4 단계: 비교 및 불일치 감지

```

// op-challenger/game/fault/solver/solver.go

// agreeWithClaim returns true if the claim is correct according to the internal [TraceProvider].
func (s *claimSolver) agreeWithClaim(ctx context.Context, game types.Game, claim types.Claim) (bool, error) {
 ourValue, err := s.trace.Get(ctx, game, claim, claim.Position)
 return bytes.Equal(ourValue[:], claim.Value[:]), err // ← 직접 계산한 값과 claim 비교!
}

```

#### 5 단계: 공격 또는 방어

```

// op-challenger/game/fault/solver/solver.go

// NextMove returns the next move to make given the current state of the game.
func (s *claimSolver) NextMove(ctx context.Context, claim types.Claim, game types.Game, honestClaims
*honestClaimTracker) (*types.Claim, error) {
 // ... 생략 ...
}

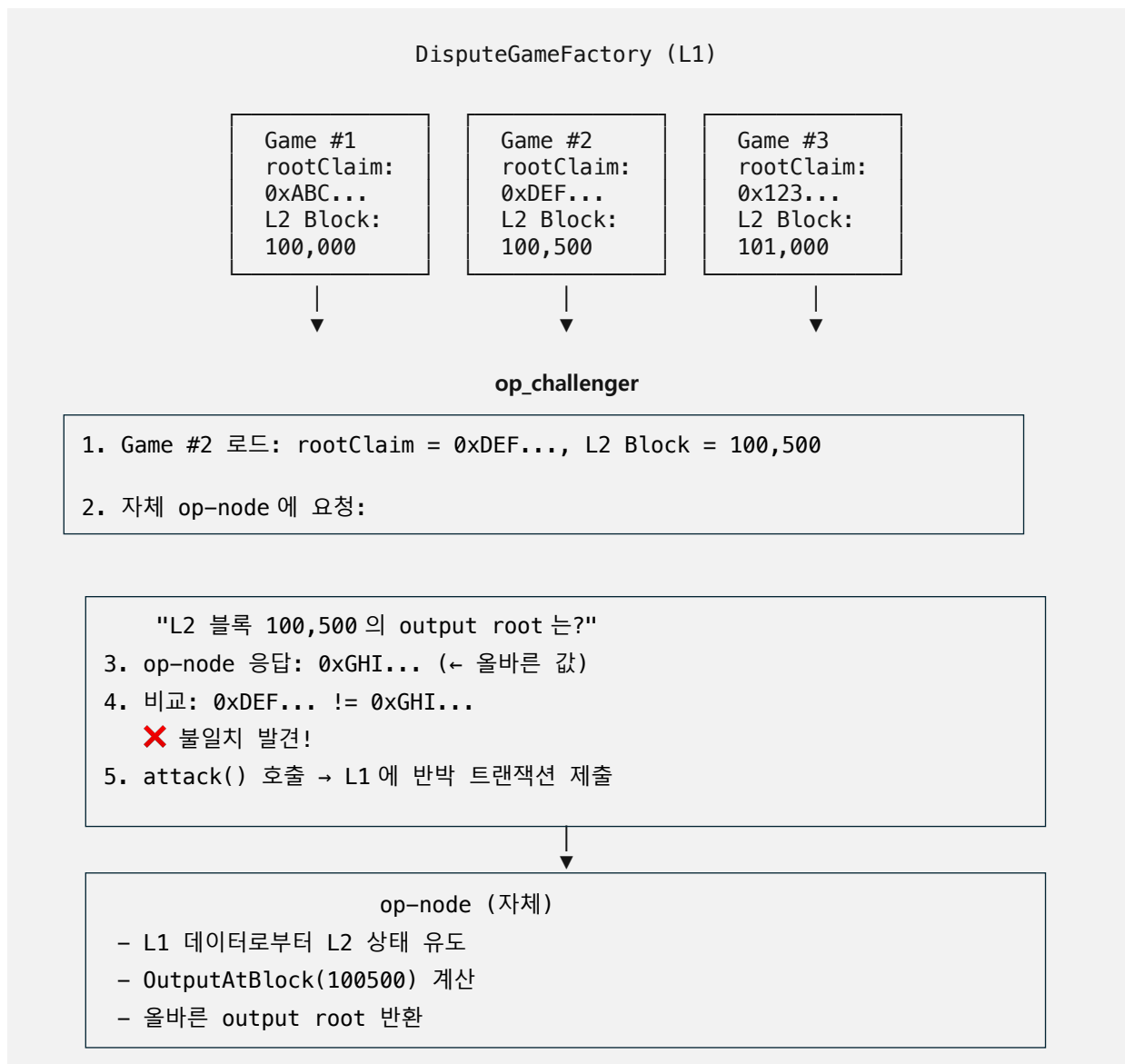
```

```

if agree, err := s.agreeWithClaim(ctx, game, claim); err != nil {
 return nil, err
} else if agree {
 return s.defend(ctx, game, claim) // ← 동의하면 방어
} else {
 return s.attack(ctx, game, claim) // ← 불일치하면 공격!
}
}

```

## 전체 흐름 시각화



## 요약

단계	설명
1. 게임 발견	Factory 에서 최근 생성된 proxy 목록 조회
2. Claims 로드	각 proxy 에서 rootClaim, claimData[] 로드

3. 올바른 값 계산	자체 op-node 에서 동일 L2 블록의 output root 계산
4. 비교	ourValue == claim.Value 체크
5. 대응	불일치 시 attack(), 일치 시 defend() 또는 무시

핵심 포인트: op-challenger 는 L1 데이터를 기반으로 독립적으로 L2 상태를 유도하여, 제출된 claim 이 올바른지 검증합니다!

## Claim 구조 이해 및 생성과정

Claim 은 각 블록마다 생성되지 않고, Dispute Game 의 Bisection(이분 탐색) 과정에서 생성됩니다.

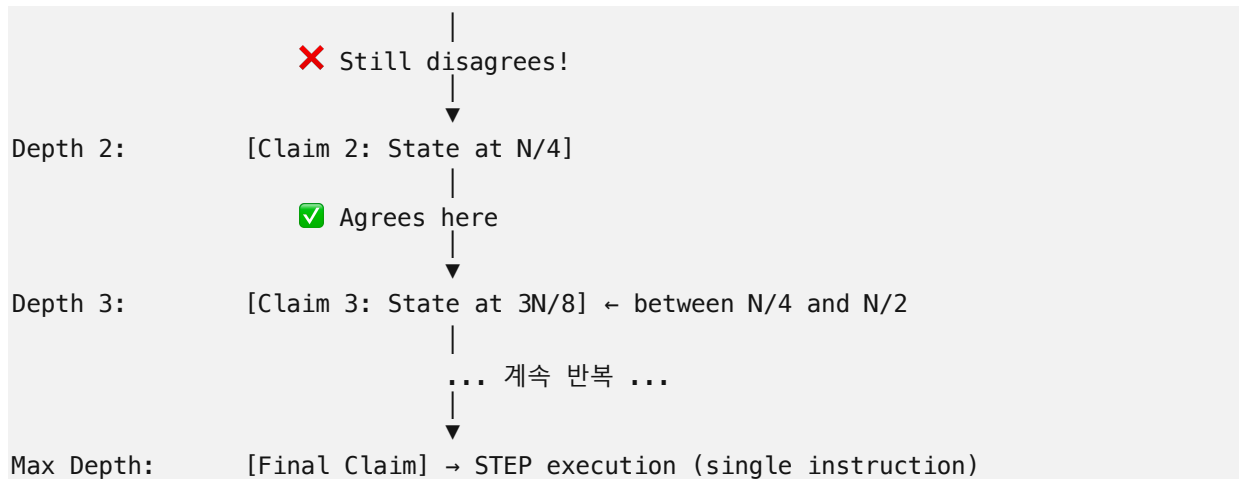
```
// op-challenger/game/fault/types/types.go
// ClaimData is the core of a claim. It must be unique inside a specific game.
type ClaimData struct {
 Value common.Hash // ← 특정 위치의 state root
 Bond *big.Int // ← 보증금
 Position // ← 게임 트리에서의 위치
}
```

```
// op-challenger/game/fault/types/types.go
// Claim extends ClaimData with information about the relationship between two claims.
// It uses ClaimData to break cyclicity without using pointers.
// If the position of the game is Depth 0, IndexAtDepth 0 it is the root claim
// and the Parent field is empty & meaningless.
type Claim struct {
 ClaimData
 CountedBy common.Address // ← 누가 반박했는지
 Claimant common.Address // ← 누가 제출했는지
 Clock Clock // ← 체스 클럭
 ContractIndex int // ← 컨트랙트 내 인덱스
 ParentContractIndex int // ← 부모 claim 인덱스
}
```

## Bisection (이분 탐색) 과정

Claim 은 \*\*dispute 가 발생할 때만\*\* 생성됩니다:

```
Depth 0 (Root): [Claim 0: Output Root for Block N]
 |
 X Challenger disagrees!
 ↓
Depth 1: [Claim 1: State at N/2] ← Bisection point
```



## 실제 예시

Dispute Game 1 개 (L2 블록 100,000 ~ 101,000):

시나리오: 1,000 개 블록 범위에서 블록 100,500 에 오류 발생

Depth 0: rootClaim (블록 101,000 의 output root)  
→ Proposer 가 제출 (1 개 claim)

Depth 1: 블록 100,500 상태 (중간점)  
→ Challenger 가 attack (1 개 claim)

Depth 2: 블록 100,250 상태  
→ Proposer 가 defend (1 개 claim)

... (약 30 depth 까지 bisection) ...

Max Depth: 단일 instruction 실행  
→ step() 호출로 최종 결정

총 Claim 수: 약 30~60 개 (depth \* 2 정도)

총 L2 블록: 1,000 개

## Claim 생성 조건

조건	Claim 생성
게임 생성 시	1 개 (rootClaim)
Challenger 가 attack	1 개 추가
Proposer 가 defend	1 개 추가
아무도 반박 안 함	0 개 추가

## 왜 블록마다 생성하지 않는가?

이유	설명
Gas 효율성	1,000 개 블록 → 30 개 claim 만 필요 ( $\log_2(1000) \approx 10$ )
Challenge Period	7 일 동안 dispute 없으면 0 개 추가 claim
대부분 정상	정상 운영 시 dispute 자체가 드물게 발생

## 요약

질문	답변
Claim 이 블록마다 생성?	❌ 아니요
Claim 생성 시점	Dispute 발생 시 (attack/defend)
1 개 게임당 Claim 수	약 30~60 개 (bisection depth 에 따라)
Dispute 없을 때	1 개 (rootClaim 만)

비율 예시:

- L2 블록 범위: 100,000 개
- Dispute 시 Claim 수: ~60 개
- 비율: 0.0006% (매우 효율적!)

---

## Metadata 와 claimData[]를 로드하는 방법

### 1. rootClaim, L2 블록 번호 등 메타데이터 로드

```
// op-challenger/game/fault/contracts/faultdisputegame.go
// GetExtendedMetadata returns the game's L1 head, L2 block number, root claim, status, max clock duration,
// and is L2 block number challenged.
func (f *FaultDisputeGameContractLatest) GetExtendedMetadata(ctx context.Context, block rpcblock.Block)
(GameMetadata, error) {
 defer f.metrics.StartContractRequest("GetExtendedMetadata")()
 results, err := f.multiCaller.Call(ctx, block,
 f.contract.Call(methodL1Head), // ← L1 블록 해시
 f.contract.Call(methodL2BlockNumber), // ← L2 블록 번호 (extraData)
 f.contract.Call(methodRootClaim), // ← Output Root
 f.contract.Call(methodStatus), // ← 게임 상태
 f.contract.Call(methodMaxClockDuration),
 f.contract.Call(methodL2BlockNumberChallenged),
 f.contract.Call(methodL2BlockNumberChallenger),
)
 // ... 결과 파싱 ...
 return GameMetadata{
```

```

L1Head: l1Head,
L2SequenceNum: l2BlockNumber, // ← extraData 에서 온 값
RootClaim: rootClaim, // ← 제안된 output root
Status: status,
// ...
}, nil
}

```

## 2. 모든 Claims 로드 (claimData[])

```

// op-challenger/game/fault/contracts/faultdisputegame.go
func (f *FaultDisputeGameContractLatest) GetAllClaims(ctx context.Context, block rpcblock.Block) ([]types.Claim,
error) {
 defer f.metrics.StartContractRequest("GetAllClaims")()
 // claimDataLen()으로 claim 개수 조회 후, 각 claim 을 순회
 results, err := batching.ReadArray(ctx, f.multiCaller, block,
 f.contract.Call(methodClaimCount), // ← claimData.length
 func(i *big.Int) *batching.ContractCall {
 return f.contract.Call(methodClaim, i) // ← claimData[i]
 })
 if err != nil {
 return nil, fmt.Errorf("failed to load claims: %w", err)
 }

 var claims []types.Claim
 for idx, result := range results {
 claims = append(claims, f.decodeClaim(result, idx))
 }
 return claims, nil
}

```

## 3. Claim 데이터 디코딩

```

// op-challenger/game/fault/contracts/faultdisputegame.go
func (f *FaultDisputeGameContractLatest) decodeClaim(result *batching.CallResult, contractIndex int)
types.Claim {
 parentIndex := result.GetUint32(0) // ← 부모 claim 인덱스
 counteredBy := result.GetAddress(1) // ← 누가 반박했는지
 claimant := result.GetAddress(2) // ← 누가 제출했는지
 bond := result.GetBigInt(3) // ← 보증금
 claim := result.GetHash(4) // ← claim 값 (state root)
 position := result.GetBigInt(5) // ← 게임 트리에서 위치
 clock := result.GetBigInt(6) // ← 체스 클럭
 return types.Claim{
 ClaimData: types.ClaimData{
 Value: claim,

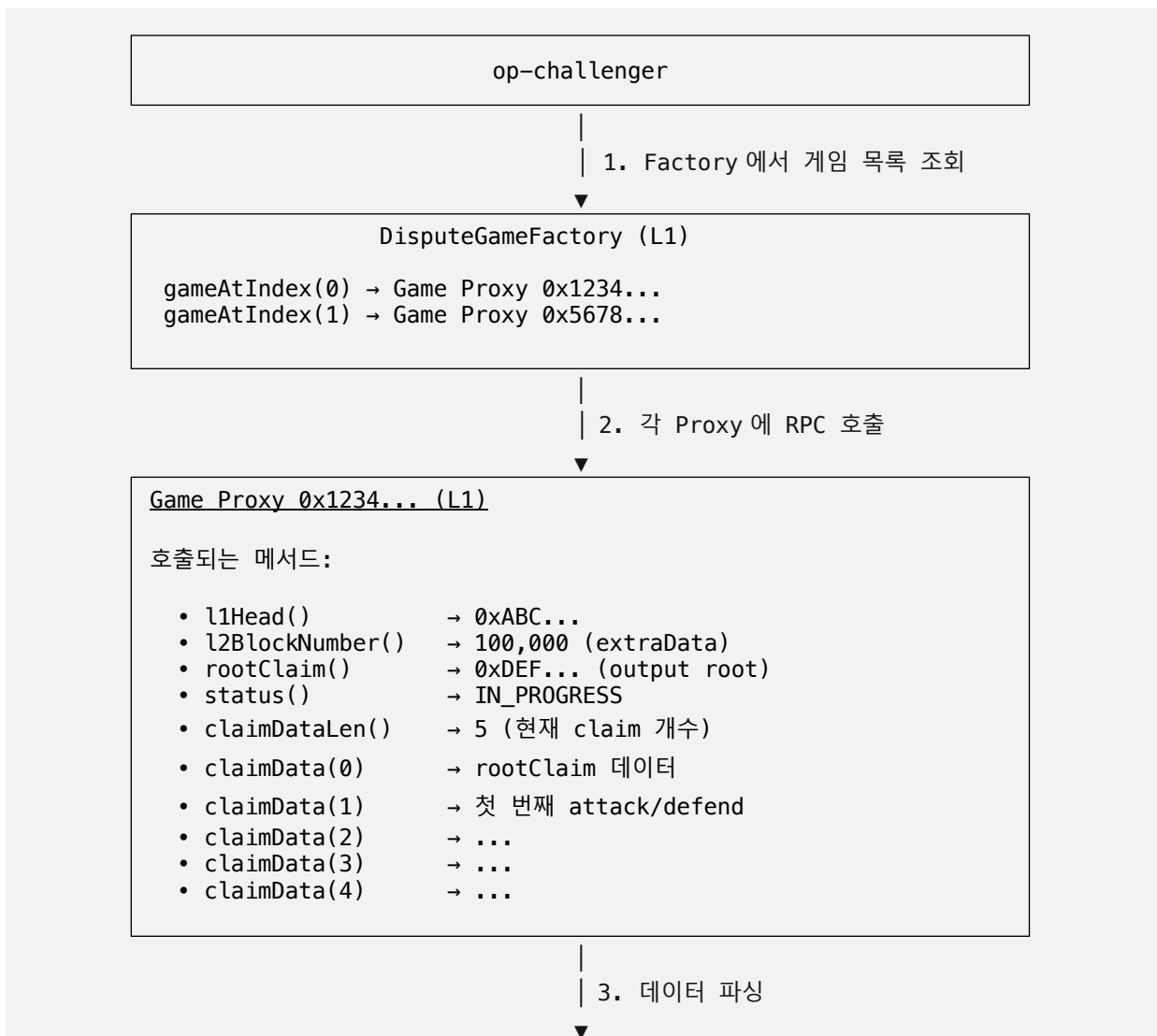
```

```

 Position: types.NewPositionFromGIndex(position),
 Bond: bond,
},
CounteredBy: counteredBy,
Claimant: claimant,
Clock: decodeClock(clock),
ContractIndex: contractIndex,
ParentContractIndex: int(parentIndex),
}
}

```

## 전체 흐름 시각화



#### op-challenger 메모리

```
GameMetadata:
 L1Head: 0xABC...
 L2SequenceNum: 100,000
 RootClaim: 0xDEF...

Claims[]:
 [0] Value: 0xDEF, Position: 1, Claimant: Proposer
 [1] Value: 0x123, Position: 2, Claimant: Challenger
 [2] Value: 0x456, Position: 4, Claimant: Proposer
 ...
```

|  
| 4. 올바른 값과 비교 후 대응

▼  
attack() 또는 defend() 호출

### Solidity 컨트랙트 측 (참고)

FaultDisputeGameV2 컨트랙트에서 해당 데이터가 저장되는 곳:

```
// rootClaim 은 immutable args 에서 가져옴 (Clone 생성 시 전달)
function rootClaim() public pure returns (Claim rootClaim_) {
 rootClaim_ = Claim.wrap(_getArgBytes32(20));
}

// L2 블록 번호도 extraData 에서 가져옴
function l2BlockNumber() public pure returns (uint256 l2BlockNumber_) {
 l2BlockNumber_ = _getArgUint256(88);
}

// claimData 는 상태 변수 배열
ClaimData[] public claimData;
```

### 요약

단계	호출	반환값
메타데이터	rootClaim()	Output Root (0xDEF...)
메타데이터	l2BlockNumber()	L2 블록 번호 (100,000)
메타데이터	l1Head()	L1 블록 해시
메타데이터	status()	IN_PROGRESS / DEFENDER_WINS / ...
Claims	claimDataLen()	claim 개수
Claims	claimData(i)	i 번째 claim 데이터

## 챌린지 게임 엔진 Cannon

Cannon 은 OP Stack 의 Fault Proof 시스템에서 핵심적인 역할을 하는 온체인 검증 가능한 MIPS VM 기반 인터랙티브 챌린지 게임 엔진으로서 Dispute Game(분쟁 게임) 에서 두 당사자(Proposer vs Challenger) 간의 상태 불일치를 단일 명령어 수준까지 좁혀 온체인에서 검증할 수 있게 해주는 도구입니다.

Cannon 은 두 가지 컴포넌트의 조합입니다:

컴포넌트	위치	역할
mipsevm (Go)	cannon/mipsevm/	오프체인 MIPS64 에뮬레이터. 수백만 단계를 실행하고 특정 단계의 Witness(증명 데이터) 생성
MIPS64.sol (Solidity)	packages/contracts-bedrock/src/cannon/	온체인 MIPS 에뮬레이터. 단 하나의 명령어를 L1 EVM 에서 직접 실행하여 최종 판결

### 전체 흐름

0 단계: L2 Output Root 제출 (Proposer) -> Dispute Game 시작 (FaultDisputeGame)

1 단계: 이분법 분쟁 게임 (Bisection Game)

op-proposer 가 잘못된 output root 를 제출했다고 가정하면, op-challenger 와 수백만 개의 실행 단계 중 "어디서 처음 달라졌는지"를 이진 탐색으로 찾아 나갑니다.  $O(\log N)$  라운드 만에 단 하나의 MIPS 명령어로 범위가 좁혀집니다.

2 단계: op-program 이 실행 추적 생성

op-program 은 MIPS 바이너리로 컴파일된 Go 프로그램입니다. 이 프로그램은 L1 데이터로부터 L2 output root 를 결정론적으로 재연산합니다. Cannon CLI 가 이 프로그램을 에뮬레이션하면서 각 단계의 상태 스냅샷과 증명 데이터를 생성합니다.

3 단계: 단일 명령어 온체인 실행

분쟁 대상 명령어가 하나로 좁혀지면 FaultDisputeGame.sol 의 step()이 호출됩니다:

```
bool validStep = VM.step(_stateData, _proof, uuid.raw()) == postState.claim.raw();
```

MIPS64.sol 이 해당 명령어를 직접 실행하고, 결과가 주장과 다르면 틀린 쪽이 패배합니다.

### Witness: 하나의 명령어 실행에 필요한 3 가지 데이터

1. Packed State → 압축된 VM 상태 (PC, 레지스터, memRoot 등)
2. Memory Proof → Binary Merkle Tree 60 개 bytes32 (고정 크기 1,920 bytes)
3. Preimage Data → L1/L2 블록 데이터를 검증 가능하게 제공하는 PreimageOracle

## 핵심 설계 철학

“거대한 계산 전체를 증명하는 것이 아니라, 의견이 갈리는 단 하나의 명령어만 증명한다.”

이 덕분에 op-program(실행 엔진)을 교체하더라도 MIPS 로 컴파일만 가능하면 동일한 증명 시스템을 그대로 사용할 수 있는 미래 지향적(future-proof) 설계입니다.

---

## op-challenger 의 Resolve 프로세스

### Resolve 프로세스의 역할

- 상태 확정 (Status Finalization): 게임의 '체스 시계(Chess Clock)'가 만료되었을 때, 해당 게임이 CHALLENGER\_WINS 인지 DEFENDER\_WINS 인지 최종 상태를 L1 컨트랙트에 기록
- 자금 인출 권한 (Withdrawal Validity): 사용자가 L2 에서 L1 으로 자금을 출금할 때, 그 기반이 되는 '상태 루트'가 유효하다는 것이 이 resolve 과정을 통해 증명되어야만 출금이 승인
- 본드(Bond) 분배: 챌린저나 방어자가 게임에 참여하기 위해 예치했던 담보금(Bond)을 승자에게 지급하거나 패자의 자금을 몰수하는 프로세스를 트리거

### 1. Agent 가 주기적으로 Resolve 시도

```
// op-challenger/game/fault/agent.go
// Returns true if the game is resolvable (regardless of whether it was actually resolved)
func (a *Agent) tryResolve(ctx context.Context) bool {
 if err := a.resolveClaims(ctx); err != nil {
 a.log.Error("Failed to resolve claims", "err", err)
 return false
 }
 if a.selective {
 // Never resolve games in selective mode as it won't unlock any bonds for us.
 // Assume the game is still in progress or the player wouldn't have told us to act.
 return false
 }
 status, err := a.responder.CallResolve(ctx)
 if err != nil || status == gameTypes.GameStatusInProgress {
 return false
 }
 a.log.Info("Resolving game")
 if err := a.responder.Resolve(); err != nil {
 a.log.Error("Failed to resolve the game", "err", err)
 }
 return true
}
```

## 2. Responder 가 실제 트랜잭션 전송

```

`go
// op-challenger/game/fault/responder/responder.go
// Resolve executes a resolve transaction to resolve a fault dispute game.
func (r *FaultResponder) Resolve() error {
 candidate, err := r.contract.ResolveTx()
 if err != nil {
 return err
 }

 return r.sender.SendAndWaitSimple("resolve game", candidate)
}

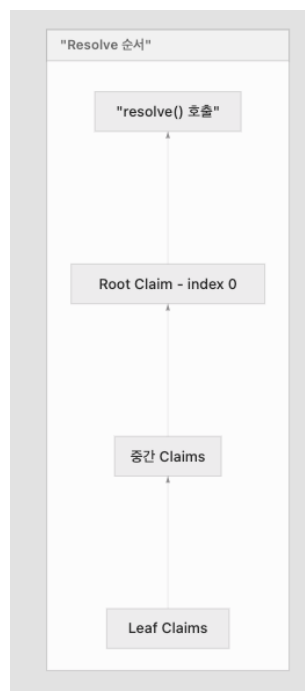
```

### Resolve 호출 조건

resolve()가 성공하려면 다음 조건이 필요합니다:

조건	설명
status == IN_PROGRESS	게임이 아직 진행 중이어야 함
resolvedSubgames[0] == true	Root claim 이 먼저 resolve 되어야 함
Clock 만료	Challenge 기간이 종료되어야 함

### Resolve 순서 (Bottom-up)



8. resolveClaim(claimIdx): 개별 claim 을 bottom-up 으로 resolve
9. resolve(): 모든 subgame 이 resolve 된 뒤 최종 게임 결과 확정

## 요약

호출 주체	설명
op-challenger (자동)	주기적으로 resolvable 한 게임을 감지하고 자동 resolve
누구나 (수동)	CLI 또는 직접 컨트랙트 호출로 resolve 가능
인센티브	Resolve 를 통해 Bond 회수 가능 → 경제적 동기

## 수동 호출 방법

CLI 를 통해 수동으로도 호출

```
// op-challenger/cmd/resolve.go

package main

import (
 "context"
 "fmt"

 "github.com/ethereum-optimism/optimism/op-challenger/flags"
 "github.com/ethereum-optimism/optimism/op-challenger/game/fault/contracts"
 oplog "github.com/ethereum-optimism/optimism/op-service/log"
 "github.com/ethereum-optimism/optimism/op-service/txmgr"
 "github.com/urfave/cli/v2"
)

func Resolve(ctx *cli.Context) error {
 contract, txMgr, err := NewContractWithTxMgr[contracts.FaultDisputeGameContract](ctx,
AddrFromFlag(GameAddressFlag.Name), contracts.NewFaultDisputeGameContract)
 if err != nil {
 return fmt.Errorf("failed to create dispute game bindings: %w", err)
 }

 tx, err := contract.ResolveTx()
 if err != nil {
 return fmt.Errorf("failed to create resolve tx: %w", err)
 }

 rct, err := txMgr.Send(context.Background(), tx)
 if err != nil {
 return fmt.Errorf("failed to send tx: %w", err)
 }

 fmt.Printf("Sent resolve tx with status: %v, hash: %s\n", rct.Status, rct.TxHash.String())
}
```

```
return nil
}
```

# CLI 명령어

```
op-challenger resolve --game-address <GAME_ADDRESS> --l1-eth-rpc <L1_RPC>
```

TypeScript 예 의한 호출

```
/**
 * FaultDisputeGame Resolve Example (TypeScript / viem)
 *
 * 일반 사용자가 직접 FaultDisputeGame 을 resolve 하는 예제입니다.
 * op-challenger agent 없이 누구나 호출 가능합니다.
 *
 * 흐름:
 * 1. 게임 상태·클릭 만료 여부 확인
 * 2. 모든 claim 을 bottom-up 순서로 resolveClaim()
 * 3. 최종 resolve() 호출
 */

import {
 createPublicClient,
 createWalletClient,
 http,
 parseAbi,
 type Address,
 type Hash,
} from "viem";
import { privateKeyToAccount } from "viem/accounts";
import { mainnet } from "viem/chains"; // 실제 환경에 맞게 교체 (e.g. optimism)

// -----
// 설정
// -----

const L1_RPC_URL = process.env.L1_RPC_URL ?? "http://localhost:8545";
const PRIVATE_KEY = (process.env.PRIVATE_KEY ?? "0x" + "a".repeat(64)) as Hash;
const GAME_ADDRESS = (process.env.GAME_ADDRESS ??
 "0xYourGameAddressHere") as Address;

/** resolveClaim 한 번에 처리할 최대 자식 수 (컨트랙트 가스 한도 고려) */
const MAX_CHILDREN_PER_CALL = 10n;

// -----
// ABI (필요한 함수만 발췌)
// -----
```

```

const FAULT_DISPUTE_GAME_ABI = parseAbi([
 // — view —
 "function status() view returns (uint8)",
 "function claimDataLen() view returns (uint256 len_)",
 "function claimData(uint256) view returns (uint32 parentIndex, address counteredBy, address claimant,
uint128 bond, bytes32 claim, uint128 position, uint128 clock)",
 "function resolvedSubgames(uint256) view returns (bool)",
 "function getNumToResolve(uint256 _claimIndex) view returns (uint256 numRemainingChildren_)",
 "function subgames(uint256, uint256) view returns (uint256)",
 "function maxClockDuration() view returns (uint64)",
 "function clockExtension() view returns (uint64)",
 "function createdAt() view returns (uint64)",
 "function resolutionCheckpoints(uint256) view returns (bool initialCheckpointComplete, uint32 subgameIndex,
uint128 leftmostPosition, address counteredBy)",
 // — write —
 "function resolveClaim(uint256 _claimIndex, uint256 _numToResolve) nonpayable",
 "function resolve() nonpayable returns (uint8 status_)",
 // — events —
 "event Resolved(uint8 indexed status)",
]);

//
// GameStatus 열거형
//
enum GameStatus {
 InProgress = 0,
 ChallengerWins = 1,
 DefenderWins = 2,
}

const STATUS_LABEL: Record<number, string> = {
 [GameStatus.InProgress]: "InProgress",
 [GameStatus.ChallengerWins]: "ChallengerWins",
 [GameStatus.DefenderWins]: "DefenderWins",
};

//
// 클라이언트 초기화
//
const account = privateKeyToAccount(PRIVATE_KEY);

const publicClient = createPublicClient({
 chain: mainnet,
 transport: http(L1_RPC_URL),
});

```

```

});

const walletClient = createWalletClient({
 account,
 chain: mainnet,
 transport: http(L1_RPC_URL),
});

// -----
// 헬퍼 함수
// -----

/** 현재 게임 상태 조회 */
async function getGameStatus(): Promise<GameStatus> {
 const status = await publicClient.readContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "status",
 });
 return status as GameStatus;
}

/** 클릭 만료 여부 확인 (maxClockDuration + clockExtension 경과 후 resolve 가능) */
async function isClockExpired(): Promise<boolean> {
 const [createdAt, maxClockDuration, clockExtension] = await Promise.all([
 publicClient.readContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "createdAt",
 }),
 publicClient.readContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "maxClockDuration",
 }),
 publicClient.readContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "clockExtension",
 }),
]);
}

const nowSec = BigInt(Math.floor(Date.now() / 1000));
// 실제 컨트랙트는 각 claim 의 clock 기반으로 판단하지만,

```

```

// 게임 전체로는 createdAt + maxClockDuration * 2 이후 보수적으로 안전.
const expiresAt = BigInt(createdAt) + BigInt(maxClockDuration) + BigInt(clockExtension);
return nowSec >= expiresAt;
}

/**
 * Bottom-up 순서로 resolve 가 필요한 claim 인덱스 배열을 반환합니다.
 * 리프(자식 없는 claim)부터 루트(index=0)까지 역순으로 정렬합니다.
 */
async function buildResolveOrder(totalClaims: bigint): Promise<bigint[]> {
 const order: bigint[] = [];

 for (let i = totalClaims - 1n; i >= 0n; i--) {
 const isResolved = await publicClient.readContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "resolvedSubgames",
 args: [i],
 });

 if (!isResolved) {
 order.push(i);
 }
 }

 return order; // 이미 역순(리프 → 루트)
}

/**
 * 단일 claim 을 resolveClaim 으로 처리합니다.
 * 남은 자식이 많을 경우 여러 번 나눠서 호출합니다.
 */
async function resolveOneClaim(claimIdx: bigint): Promise<void> {
 while (true) {
 const numRemaining = await publicClient.readContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "getNumToResolve",
 args: [claimIdx],
 });

 if (numRemaining === 0n) {
 // 이미 처리 완료 (혹은 자식 없음) → 최종 resolveClaim 1 회 호출
 const tx = await walletClient.writeContract({

```

```

 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "resolveClaim",
 args: [claimIdx, 0n], // _numToResolve=0 → 컨트랙트가 내부 처리
 });
 console.log(` resolveClaim(${claimIdx}, 0) tx: ${tx}`);
 await publicClient.waitForTransactionReceipt({ hash: tx });
 break;
}

// 남은 자식을 MAX_CHILDREN_PER_CALL 단위로 분할 처리
const batch = numRemaining > MAX_CHILDREN_PER_CALL
 ? MAX_CHILDREN_PER_CALL
 : numRemaining;

const tx = await walletClient.writeContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "resolveClaim",
 args: [claimIdx, batch],
});
console.log(` resolveClaim(${claimIdx}, ${batch}) tx: ${tx}`);
await publicClient.waitForTransactionReceipt({ hash: tx });
}
}

// -----
// 메인 로직
// -----

async function main() {
 console.log("=== FaultDisputeGame Manual Resolve ===");
 console.log(`Game address : ${GAME_ADDRESS}`);
 console.log(`Caller      : ${account.address}`);

 // ----- 1. 게임 상태 확인 -----
 const status = await getGameStatus();
 console.log(`\n[1] Game status: ${STATUS_LABEL[status]} (${status})`);

 if (status !== GameStatus.InProgress) {
 console.log("이미 resolve 된 게임입니다. 종료합니다.");
 return;
 }
}

```

```

// — 2. 클럭 만료 확인 —————
const expired = await isClockExpired();
console.log(`[2] Clock expired: ${expired}`);

if (!expired) {
 console.log(
 "아직 dispute 기간이 끝나지 않았습니다. 나중에 다시 시도하세요."
);
 return;
}

// — 3. claim 수 확인 —————
const totalClaims = await publicClient.readContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "claimDataLen",
});
console.log(`[3] Total claims: ${totalClaims}`);

// — 4. resolveClaim (bottom-up) —————
console.log(`\n[4] Resolving claims bottom-up...`);
const resolveOrder = await buildResolveOrder(totalClaims);
console.log(
 `  Unresolved claim indices (leaf→root): [${resolveOrder.join(", ")}]`
);

for (const idx of resolveOrder) {
 console.log(`\n  Processing claim #${idx}...`);
 await resolveOneClaim(idx);
 console.log(`  claim #${idx} resolved ✓`);
}

// — 5. 최종 resolve() —————
console.log(`\n[5] Calling resolve() to finalize the game...`);
const resolveTx = await walletClient.writeContract({
 address: GAME_ADDRESS,
 abi: FAULT_DISPUTE_GAME_ABI,
 functionName: "resolve",
});
console.log(`  resolve() tx: ${resolveTx}`);

const receipt = await publicClient.waitForTransactionReceipt({
 hash: resolveTx,
});

```

```

console.log(` receipt status: ${receipt.status}`);

// — 6. 결과 확인 —————
const finalStatus = await getGameStatus();
console.log(
 `\\n[6] Final game status: ${STATUS_LABEL[finalStatus]} (${finalStatus})`
);

if (finalStatus === GameStatus.DefenderWins) {
 console.log("결과: 제안자(Proposer) 승리 — L2 출력 루트가 유효합니다.");
} else if (finalStatus === GameStatus.ChallengerWins) {
 console.log("결과: 챌린저(Challenger) 승리 — L2 출력 루트가 무효입니다.");
}

console.log("\\n=== Done ===");
}

main().catch((err) => {
 console.error("오류 발생:", err);
 process.exit(1);
});

```

# Cannon: MIPS 온체인 에뮬레이터

## Cannon 의 역할과 위치

Cannon 은 Stack 의 Fault Proof 시스템에서 핵심적인 역할을 하는 온체인 검증 가능한 MIPS VM 기반 인터랙티브 챌린지 게임 엔진으로서 Dispute Game(분쟁 게임) 에서 두 당사자(Proposer vs Challenger) 간의 상태 불일치를 단일 명령어 수준까지 좁혀 온체인에서 검증할 수 있게 해주는 도구이다.

## OP Stack 에서 Cannon 이 필요한 이유

OP Stack 은 낙관적 롤업(Optimistic Rollup) 방식을 사용합니다. op-proposer 가 제출한 L2 output root 를 처음에는 그냥 믿고(낙관적으로) 넘어가되, 7 일간의 챌린지 기간 동안 누구든 그 값이 틀렸다고 이의를 제기할 수 있다.

문제는, "이 output root 가 정말 잘못됐다"는 것을 어떻게 L1 이더리움 온체인에서 객관적으로 증명하느냐입니다. L2 블록 전체를 L1 에서 다시 실행하는 것은 가스비가 터무니없이 비쌉니다.

Cannon 은 이 문제를 해결합니다. 수백만 개의 실행 단계 중 논쟁이 되는 단 하나의 MIPS 명령어만 L1 에서 실행하여 진실을 판결한다.

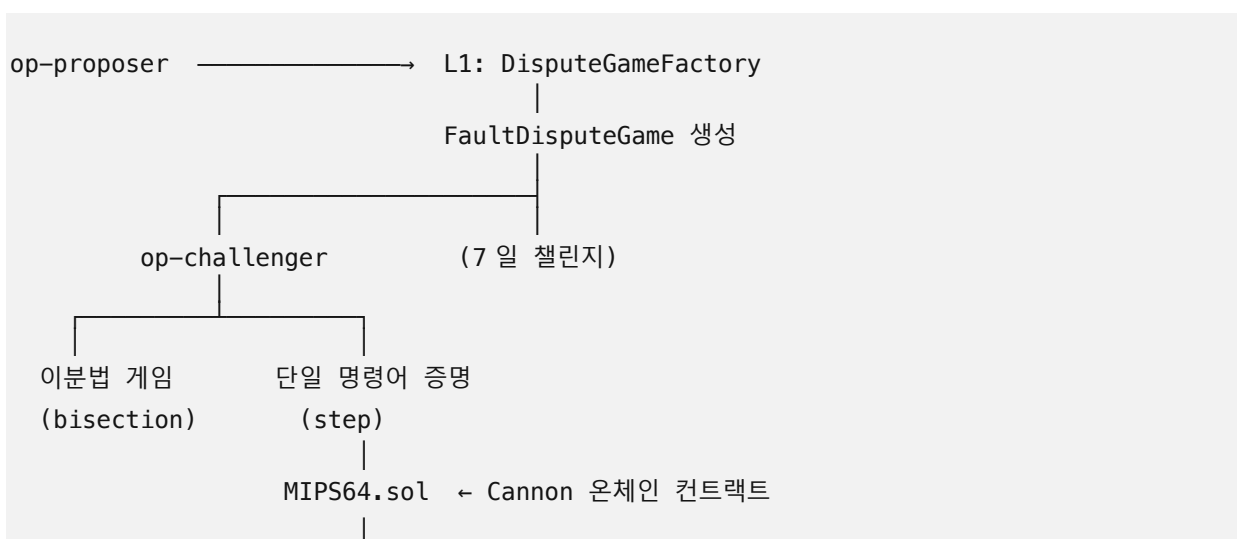
## Cannon 의 한 줄 요약

Cannon = 온체인 MIPS 명령어 에뮬레이터 + 오프체인 증명 생성기

Cannon README 의 유명한 재귀적 설명:

```
It's Go code
...that runs an EVM
...emulating a MIPS machine
...running compiled Go code
...that runs an EVM
```

## OP Stack 내 위치

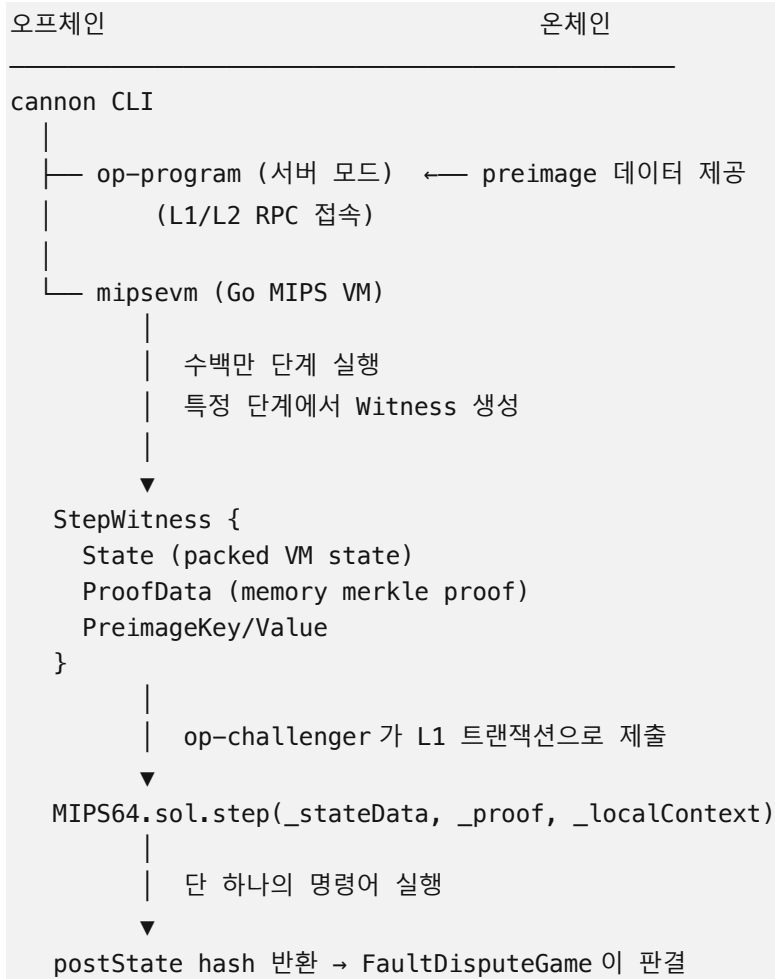


## 전체 아키텍처 개요

Cannon 은 세 개의 주요 컴포넌트로 구성됩니다.

컴포넌트	위치	역할
mipsevm (Go)	cannon/mipsevm/	오프체인 MIPS 에뮬레이터. 실행 추적과 증명 데이터 (Witness) 생성
op-program (Go → MIPS ELF)	op-program/	MIPS 위에서 돌아가는 폴트 증명 프로그램. L1 데이터로 L2 output root 를 재연산
MIPS64.sol (Solidity)	packages/contracts-bedrock/src/cannon/	온체인 MIPS 에뮬레이터. 단 하나의 명령어를 L1 EVM에서 실행하여 판결

이 세 컴포넌트가 협력하는 방식:



---

## 이분법 분쟁 게임 (Bisection Game)

### 왜 이분법인가?

op-proposer 가 주장하는 output root(= 수백만 단계의 실행 결과)가 틀렸다고 할 때, op-challenger 는 어느 단계에서 처음 잘못됐는지를 찾아야 합니다.

수백만 단계를 하나씩 검사하는 것은 불가능하므로, 이진 탐색(binary search) 방식으로 범위를 좁힙니다.

### 게임 구조 (FaultDisputeGame)

FaultDisputeGame 은 2 단계 트리 구조를 가집니다:

```
전체 게임 트리
├─ 상위(Top): output root 이분법 (splitDepth 까지)
│ └─ L2 output root 단위로 좁혀나감
└─ 하위(Bottom): 실행 이분법 (splitDepth ~ maxGameDepth)
 └─ 개별 MIPS 명령어 단위로 좁혀나감
 └─ 리프(leaf) 도달 시 → step() 호출
```

FaultDisputeGame.sol 의 핵심 파라미터:

```
struct GameConstructorParams {
 GameType gameType;
 Claim absolutePrestate; // 프로그램 초기 상태 해시
 uint256 maxGameDepth; // 전체 트리 최대 깊이
 uint256 splitDepth; // output 이분법 / 실행 이분법 분리 지점
 Duration clockExtension;
 Duration maxClockDuration; // 7 일
 IBigStepper vm; // MIPS64.sol (Cannon)
 ...
}
```

### 게임 진행 흐름

- [1] proposer: output root O 를 claim 으로 제출
- [2] challenger: O 가 틀렸다고 attack
- [3] 이진 탐색:  
proposer/challenger 가 번갈아 가며 attack/defend  
각 claim 은 트리의 특정 위치(gindex)에 해당하는 상태 해시  
disagreement 범위를 반으로 계속 줄임
- [4] 단일 명령어 단계 도달:  
challenger 가 step() 호출  
→ MIPS64.sol 이 해당 명령어를 직접 실행  
→ 결과 비교 → 틀린 쪽 패배

## Position (gindex) 좌표계

각 claim 은 이진 트리에서 gindex(generalized index)로 표현됩니다:

```
depth 0: 1 (루트)
depth 1: 2 3
depth 2: 4 5 6 7
...
```

op-challenger 의 position.go 에서 이 좌표계를 관리합니다.

## step() 함수 — 최종 판결

트리의 리프에 도달하면 FaultDisputeGame.sol 의 step()이 호출됩니다:

```
function step(
 uint256 _claimIndex,
 bool _isAttack,
 bytes calldata _stateData, // 해당 단계의 VM 상태 (packed)
 bytes calldata _proof // 메모리 merkle proof
)
public
virtual
{
 // ...
 // prestate claim 과 제공된 stateData 가 일치하는지 확인
 if (keccak256(_stateData) << 8 != preStateClaim.raw() << 8) revert InvalidPrestate();

 // localContext: preimage oracle 격리를 위한 UUID
 Hash uuid = _findLocalContext(_claimIndex);

 // VM(MIPS64.sol)이 명령어를 실행하고 post-state 반환
 bool validStep = VM.step(_stateData, _proof, uuid.raw()) == postState.claim.raw();

 // 기대 결과와 실제 결과가 일치하면 → 잘못된 claim 이 반증됨
 bool parentPostAgree = (parentPos.depth() - postState.position.depth()) % 2 == 0;
 if (parentPostAgree == validStep) revert ValidStep();

 // 틀린 쪽 claim 을 countered 로 표시
 parent.counteredBy = msg.sender;
}
```

---

## op-program: 폴트 증명 프로그램

### 역할

op-program 은 MIPS 아키텍처용으로 컴파일된 Go 프로그램입니다. 이 프로그램이 하는 일은 딱 하나입니다:

주어진 L1 헤드 블록과 L2 시작 상태로부터, 특정 L2 블록 번호의 output root 를 재연산하여 제안된 output root 와 일치하는지 확인한다.

이 프로그램은 결정론적(deterministic)입니다. 동일한 입력에 대해 항상 동일한 실행 추적(trace)을 생성합니다. 이것이 온체인에서 검증 가능하게 만드는 핵심 조건입니다.

## 두 가지 실행 모드

### op-program

Client 모드 (MIPS 위에서 실행)

- MIPS ELF 바이너리로 컴파일됨
- cannon 에 의해 에뮬레이션됨
- preimage 는 파일 디스크립터(fd)를 통해 요청

Host(Server) 모드 (네이티브 Go 로 실행)

- L1/L2 RPC 에 접속하여 preimage 데이터 제공
- client 프로세스의 preimage 요청을 처리

```
// client/program.go
func Main(useInterop bool) {
 preimageOracle := preimage.ClientPreimageChannel() // fd 기반 채널
 preimageHinter := preimage.ClientHinterChannel()
 if err := RunProgram(logger, preimageOracle, preimageHinter, config);
 errors.Is(err, claim.ErrClaimNotValid) {
 os.Exit(1) // claim 무효 → 종료코드 1
 } else if err != nil {
 os.Exit(2) // 실행 오류 → 종료코드 2
 } else {
 os.Exit(0) // claim 유효 → 정상 종료
 }
}
```

## 절대 사전상태 (Absolute Prestate)

op-program 의 MIPS ELF 바이너리를 Cannon 에 로드하면 초기 VM 상태 (state.bin.gz)가 생성됩니다. 이 초기 상태의 해시가 absolutePrestate 이며, FaultDisputeGame 생성 시 파라미터로 등록됩니다.

모든 참여자가 동일한 absolutePrestate 를 사용해야 분쟁이 의미를 가집니다.

```
op-program 을 MIPS 바이너리로 빌드 후 초기 상태 생성
cannon load-elf --type singlethreaded-2 W
--path=./op-program/bin/op-program-client.elf
→ state.bin.gz (absolutePrestate), meta.json 생성
```

---

## 오프체인 에뮬레이터: mipsevm

### 구조

cannon/mipsevm/는 오프체인에서 MIPS 를 에뮬레이션하는 Go 패키지입니다.

mipsevm/

```
|─ iface.go ← FPVMState, FPVM 인터페이스 정의
|─ oracle.go ← PreimageOracle 인터페이스
|─ witness.go ← StepWitness (증명 데이터 구조체)
|─ state.go ← CpuScalars, VmStatus
|─ arch/ ← 64 비트 / 32 비트 아키텍처 상수
|─ exec/
| |─ mips_instructions.go ← MIPS 명령어 실행 로직
| |─ mips_syscalls.go ← Linux syscall 에뮬레이션
|─ memory/ ← Binary merkle tree 기반 메모리
|─ multithreaded/ ← 멀티스레드 VM (MIPS64 기본)
└─ program/
 └─ load.go ← ELF 바이너리 로더
```

### 핵심 인터페이스

```
// FPVM: 한 단계씩 실행하는 VM 인터페이스
type FPVM interface {
 GetState() FPVMState
 Step(includeProof bool) (*StepWitness, error) // 핵심!
 CheckInfiniteLoop() bool
 LastPreimage() (key [32]byte, value []byte, offset arch.Word)
 Traceback()
}

// FPVMState: VM 의 완전한 상태
type FPVMState interface {
 GetMemory() *memory.Memory
 GetPC() arch.Word
 GetStep() uint64
 GetExited() bool
 GetExitCode() uint8
 EncodeWitness() (witness []byte, hash common.Hash)
 CreateVM(...) FPVM
}

// PreimageOracle: preimage 데이터 공급자
type PreimageOracle interface {
 Hint(v []byte) // "이런 preimage 가 필요할 것" 힌트
```

```

 GetPreimage(k [32]byte) []byte // key 로 preimage 조회
}

```

## 메모리 구조: Binary Merkle Tree

Cannon 의 메모리는 이진 머클 트리로 표현됩니다:

```

깊이 59, 리프 크기 32 bytes
→ 2^59 * 32 bytes = 2^64 bytes (전체 64 비트 주소 공간)

노드 결합: node = keccak256(left ++ right)
빈 서브트리: zero_hash[d] = keccak256(zero_hash[d-1], zero_hash[d-1])
zero_hash[0] = bytes32(0)

```

메모리 증명 형식: 60 개의 bytes32 연결 배열

- proof[0]: 리프 값 (32 bytes)
- proof[1..59]: 루트까지의 형제 노드들 (bottom-up 순서)

이 방식의 장점:

- 프루프 크기가 고정적 (60 \* 32 = 1920 bytes)
- 읽기 프루프로 쓰기 후 새 루트도 계산 가능 (같은 경로를 재사용)

## StepWitness: 하나의 명령어 실행에 필요한 모든 데이터

```

type StepWitness struct {
 State []byte // 압축된 VM 상태 (packed state)
 StateHash common.Hash // 상태 해시

 ProofData []byte // 메모리 머클 증명

 PreimageKey [32]byte // 접근한 preimage 키 (없으면 zero)
 PreimageValue []byte // preimage 값 (8 바이트 길이 접두사 포함)
 PreimageOffset arch.Word
}

```

---

## 온체인 에뮬레이터: MIPS64.sol

### 역할

MIPS64.sol 은 IBigStepper 인터페이스를 구현하는 온체인 MIPS 에뮬레이터입니다. FaultDisputeGame 이 최종 판결을 위해 호출합니다.

```

interface IBigStepper {
 function step(

```

```

 bytes calldata _stateData, // packed VM 상태
 bytes calldata _proof, // 메모리 merkle proof
 bytes32 _localContext // preimage oracle 격리용 UUID
) external returns (bytes32 postState_); // 실행 후 상태 해시

function oracle() external view returns (IPreimageOracle oracle_);
}

```

## VM 상태 구조

```

contract MIPS64 is ISemver {
 // 스레드 단위 상태 (총 298 bytes)
 struct ThreadState {
 uint64 threadID;
 uint8 exitCode;
 bool exited;
 uint64 pc; // 프로그램 카운터
 uint64 nextPC; // 다음 PC (delay slot 처리용)
 uint64 lo; // 곱셈/나눗셈 보조 레지스터
 uint64 hi;
 uint64[32] registers; // MIPS 32 개 범용 레지스터
 }

 // 전역 VM 상태
 struct State {
 bytes32 memRoot; // 메모리 binary merkle tree 루트
 bytes32 preimageKey; // 마지막으로 접근한 preimage 키
 uint64 preimageOffset; // preimage 읽기 오프셋
 uint64 heap; // 힙 포인터
 // ... 스레드 스택 등
 }
}

```

## step() 실행 흐름

```

step(_stateData, _proof, _localContext)
|
├─ [1] _stateData 언패킹 → State 복원
|
├─ [2] _proof 로 현재 명령어 메모리 검증
| (memRoot 와 proof 로 PC 위치의 명령어 인증)
|
├─ [3] 명령어 디코딩 (MIPS64Instructions.sol)
| R-type / I-type / J-type / syscall 분류
|
├─ [4] 명령어 실행
| └─ 일반 ALU 연산: 레지스터 읽기/쓰기

```

```

| └─ 메모리 접근: proof 로 읽기 후 쓰기, memRoot 갱신
| └─ syscall: PreimageOracle 접근, 메모리 할당, 스레드 제어
|
└─ [5] 새 State 직렬화 → postState 해시 계산
|
└─ [6] postState_ 반환 → FaultDisputeGame 이 판결에 사용

```

## 지원하는 MIPS64 명령어셋

MIPS64 Release 1 (MIPS64r1) 전체를 지원합니다. Go 컴파일러가 출력하는 명령어 집합에 맞춰져 있습니다:

- 산술/논리 연산: ADD, SUB, AND, OR, XOR, SLL, SRL, SRA 등
- 메모리 접근: LW, SW, LD, SD, LB, SB 등 (8 바이트 정렬 보장)
- 분기/점프: BEQ, BNE, J, JAL, JR, JALR
- Syscall: Linux 최소 서브셋 (메모리 할당, 파일 I/O, 종료, 스레드)

## 관련 라이브러리 컨트랙트

```

packages/contracts-bedrock/src/cannon/
├─ MIPS64.sol ← 메인: IBigStepper 구현
├─ PreimageOracle.sol ← preimage 저장/조회
├─ PreimageKeyLib.sol ← preimage 키 지역화
└─ libraries/
 ├─ MIPS64Instructions.sol ← 명령어 디코딩 및 실행
 ├─ MIPS64Memory.sol ← Binary merkle memory 읽기/쓰기
 ├─ MIPS64State.sol ← CpuScalars, 버전별 기능 플래그
 ├─ MIPS64Arch.sol ← 아키텍처 상수 (word size 등)
 ├─ MIPS64Syscalls.sol ← syscall 처리
 ├─ CannonTypes.sol ← LPP 메타데이터 packed type
 └─ CannonErrors.sol ← revert 에러 정의

```

## Witness 데이터의 3 가지 구성요소

온체인에서 하나의 MIPS 명령어를 실행하려면 다음 세 가지 데이터가 필요합니다.

### (1) Packed State (압축 VM 상태)

모든 온체인 명령어 실행마다 제공됩니다. VM 전체 상태를 직렬화한 바이트 배열입니다.

크기가 작아서 증명 없이 전체를 제공하는 것이 더 효율적입니다:

- PC, NextPC, LO, HI (CPU 스칼라)
- 32 개 레지스터 - memRoot (메모리 merkle 루트 — 전체 메모리 대신 이것만 제공)
- preimageKey, preimageOffset –
- heap, exitCode, exited

## (2) Memory Proof (메모리 머클 증명)

각 명령어는 최대 다음 메모리 접근을 수행합니다:

- 읽기 1: PC 위치의 명령어 (4 bytes)
- 읽기 2: 로드/syscall 메모리 읽기 (8 bytes, 8 바이트 정렬)
- 쓰기 1: 스토어/syscall 메모리 쓰기 (마지막 읽기와 동일 주소)

각 메모리 접근마다 60 개의 bytes<sup>32</sup> 로 구성된 증명:

- proof[0] = 리프 값 (32 bytes)
- proof[1..59] = 루트까지 형제 노드들 (bottom-up)

쓰기 후 새 memRoot 는 읽기에 사용한 형제 노드들을 재활용하여 계산합니다.

## (3) Pre-image Data (사전 이미지 데이터)

syscall 을 통해서만 접근합니다. op-program 은 특정 파일 디스크립터(fd)에 read/write 시스템 콜을 보냄으로써 preimage oracle 과 통신합니다:

fd	역할
fd 5	preimage 읽기 (hint 포함)
fd 6	preimage 쓰기 (key 제출)

오프체인에서는 op-program 의 Host 모드가 L1/L2 RPC 를 통해 preimage 를 제공하고, 온체인에서는 PreimageOracle.sol 이 미리 등록된 preimage 를 제공합니다.

---

## Preimage Oracle

### 역할

Cannon 의 MIPS 프로그램이 "외부 데이터"를 필요로 할 때 사용하는 메커니즘입니다. L2 블록 데이터, L1 블록 헤더, 트랜잭션 영수증 등 L1 에서 검증 가능한 모든 데이터를 요청할 수 있습니다.

### Key 타입

키 타입	prefix	설명
Keccak256	0x01	데이터의 keccak256 해시로 조회
Keccak256 Commitment	0x02	EIP-4844 blob 커밋먼트
Local	0x01 (localized)	분쟁 게임별 로컬 컨텍스트
Precompile	0x06	EVM precompile 결과

## Local Context (지역화)

여러 분쟁 게임이 동시에 진행될 때 preimage 데이터가 섞이지 않도록, \_localContext(분쟁 게임별 UUID)로 preimage 키를 지역화(localize)합니다:

```
function localize(bytes32 _key, bytes32 _localContext)
 internal pure returns (bytes32 localizedKey_)
{
 // 키를 local context 와 조합하여 게임별 고유 키 생성
}
```

## 대용량 Preimage (LPP: Large Preimage Proposal)

preimage 가 단일 트랜잭션에 담기 어려울 만큼 크면(예: blob 데이터), 분할 제출 후 keccak256 트리로 증명합니다:

- PreimageOracle.sol 의 keccak tree depth: 16
- 챌린지 기간 + 본드(bond) 메커니즘으로 올바른 preimage 보장

---

## op-challenger 와의 연동

### CannonTraceProvider

op-challenger 는 CannonTraceProvider 를 통해 Cannon 의 오프체인 실행 결과를 분쟁 게임에 공급합니다:

```
// game/fault/trace/cannon/provider.go

type CannonTraceProvider struct {
 dir string // 증명 파일 저장 디렉터리
 prestate string // absolutePrestate 경로
 generator utils.ProofGenerator // cannon CLI 실행기
 gameDepth types.Depth
 stateConverter vm.StateConverter
}

// 특정 게임 위치(position)의 상태 해시 조회
func (p *CannonTraceProvider) Get(ctx context.Context, pos types.Position)
(common.Hash, error) {
 traceIndex := pos.TraceIndex(p.gameDepth)
 proof, _ := p.loadProof(ctx, traceIndex.Uint64())
 return proof.ClaimValue, nil
}

// step() 호출에 필요한 3 가지 데이터 반환
func (p *CannonTraceProvider) GetStepData(ctx context.Context, pos types.Position) (
 stateData []byte, // packed VM state
 proofData []byte, // memory merkle proof
 oracleData *PreimageOracleData, // preimage 데이터
 err error,
)
```

```

) {
 proof, _ := p.loadProof(ctx, traceIndex.Uint64())
 return proof.StateData, proof.ProofData, oracleData, nil
}

```

## 이분법 게임 → 단일 step 트랜잭션 흐름

```

[1] game_solver.go: 현재 게임 상태 분석
 | 리프 도달 확인
 ▼
[2] solver.go: AttemptStep() 호출
 | GetStepData()로 stateData + proof 획득
 ▼
[3] types.Action{Type: ActionTypeStep, ...} 생성
 ▼
[4] responder.go: StepTx() 호출
 |
 ▼
[5] faultdisputegame.go:
 r.contract.StepTx(claimIndex, isAttack, preState, proofData)
 |
 ▼
[6] L1 트랜잭션 전송:
 FaultDisputeGame.step(_claimIndex, _isAttack, _stateData, _proof)
 |
 ▼
[7] MIPS64.sol.step() 실행 → postState 해시 반환
 |
 ▼
[8] FaultDisputeGame: 올바른 쪽 승리, 틀린 claim countered 처리

```

## Split Trace Provider

분쟁 게임이 splitDepth 위에서는 output root 단위로, 아래에서는 MIPS 명령어 단위로 다르게 동작합니다:

```

// game/fault/trace/split/split.go
// depth <= splitDepth: OutputRootProvider (L2 블록 단위)
// depth > splitDepth: CannonTraceProvider (MIPS 명령어 단위)

```

## Cannon CLI 사용 흐름

### 전체 흐름

```

1. op-program 빌드 (MIPS ELF 바이너리 생성)
cd op-program && make op-program

2. Cannon CLI 빌드
cd cannon && make cannon

```

```
3. ELF → 초기 VM 상태 변환
./bin/cannon load-elf \
 --type singlethreaded-2 \
 --path=../op-program/bin/op-program-client.elf
→ state.bin.gz (absolutePrestate), meta.json 생성

4. Cannon 에뮬레이션 실행
./bin/cannon run \
 --input ./state.bin.gz \
 --proof-at '<TRACE_INDEX>' # 이 단계에서 witness 생성
 --stop-at '<STOP_INDEX>' # 이 단계에서 중단
 --snapshot-at '%1000000000' # N 단계마다 스냅샷
 -- \
 ../op-program/bin/op-program \
 --network op-mainnet \
 --l1 <L1_RPC> \
 --l2 <L2_RPC> \
 --l1.head <L1_HEAD_HASH> \
 --l2.claim <L2_OUTPUT_ROOT_CLAIM> \
 --l2.head <L2_HEAD_HASH> \
 --l2.blocknumber <L2_BLOCK_NUMBER> \
 --l2.outputroot <L2_OUTPUT_ROOT> \
 --server # op-program 을 서버 모드(preimage 제공자)로 실행
```

## 산출 파일

파일	내용
state.bin.gz	초기/중간 VM 상태 스냅샷
meta.json	디버그 심볼 (함수명 ↔ 주소 매핑)
proofs/<N>.json.gz	N 번째 단계의 Witness (StateData + ProofData + Preimage)

## 요약: Cannon 의 핵심 아이디어

개념	설명
결정론성	동일 입력 → 동일 MIPS 실행 추적. 온체인과 오프체인이 항상 일치
이분법	수백만 단계 분쟁을 $O(\log N)$ 라운드로 단일 명령어까지 좁힘
단일 명령어 증명	최종적으로 단 하나의 MIPS 명령어만 L1 에서 실행 → 가스 최소화
Binary Merkle Memory	전체 메모리 대신 60 개 bytes32 증명으로 임의 메모리 접근 인증
Preimage Oracle	MIPS 프로그램이 L1/L2 블록 데이터를 검증 가능하게 접근하는 채널
확장성	op-program 만 교체하면 다른 실행 엔진도 동일한 증명 시스템 사용 가능

Cannon 의 궁극적 설계 철학: "거대한 계산 전체를 증명하는 것이 아니라, 의견이 갈리는 단 하나의 명령어만 증명한다."

# Withdraw

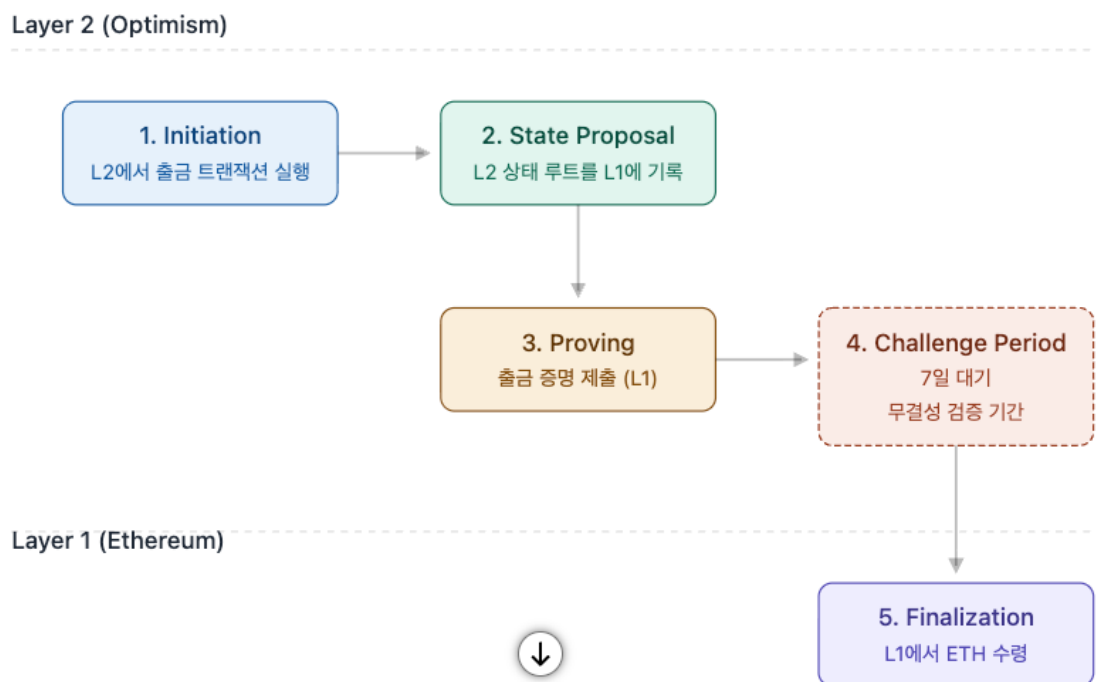
## 개요

Optimism(OP Stack)에서 L2 의 ETH 를 L1 으로 출금하는 과정은 '낙관적 가설(Optimistic Rollup)'에 기반합니다. 보안을 위해 약 7 일간의 분쟁 기간(Challenge Period)을 포함하며, 크게 5 단계의 절차를 거칩니다

## 출금 프로세스 5 단계 요약

전체 과정은 L2 에서 시작하여 L1 에서 두 번의 트랜잭션을 실행함으로써 완료됩니다

OP Stack 출금 프로세스 (L2 -> L1)



## 관련 OP Stack 구성요소

구성요소	출금과의 관계
L2ToL1MessagePasser (L2)	출금 시작, sentMessages 스토리지, MessagePassed 이벤트
L2StandardBridge (L2)	사용자가 흔히 쓰는 withdraw 등 래퍼
OptimismPortal2 (L1)	proveWithdrawalTransaction, finalizeWithdrawalTransaction
DisputeGameFactory (L1)	Output Root 를 담은 Dispute Game 생성; 증명 시 게임 인덱스·root claim 연동



- 순차 검색 (gameAtIndex): 게임 수가 적을 때 간단히 사용.
- 배치 검색 (findLatestGames): 특정 범위 내에서 역순 검색.

## [5 단계] L1 에서의 최종 확정 및 수령 (Finalization)

- 모든 검증과 대기 시간이 끝난 후 자산을 실제로 수령하는 단계입니다.
- 대기 조건: 증명 제출 후 7 일간의 분쟁 기간이 경과해야 합니다.
- 실행: 사용자가 L1 에서 finalizeWithdrawal 트랜잭션을 실행합니다.
- 결과: OptimismPortal 이 락업된 ETH 를 사용자의 L1 지갑으로 전송하며 프로세스가 종료됩니다.

## 기술적 심화 및 참고 사항

### 핵심 컨트랙트 요약

체인	컨트랙트 명	역할	주소
L2	L2ToL1MessagePasser	출금 시작 및 메시지 저장	0x420000...016
L2	L2StandardBridge	사용자 친화적 출금 인터페이스	0x420000...010
L1	OptimismPortal2	출금 증명 및 완료 처리	(배포 시 결정)
L1	DisputeGameFactory	Output Root 게시 및 검증	(배포 시 결정)

### L2 블록 번호 관계 (중요 조건)

출금 증명 시 Game L2 Block Number  $\geq$  Withdrawal L2 Block Number 조건을 만족해야 합니다.

- 출금 블록: initiateWithdrawal 이 호출된 블록.
- 게임 블록: Dispute Game 이 증명하는 시점의 상태 블록.
- 이유: 게임이 커버하는 블록 번호가 출금 블록보다 크거나 같아야 해당 출금 정보가 포함된 상태를 증명할 수 있기 때문입니다.

## 핵심소스 코드

### 1. [L2] 출금 시작 (Initiate Withdrawal)

- 컨트랙트 L2ToL1MessagePasser(L2 predeploy: 0x4200000000000000000000000000000000000000000000000000000000000000) 의 initiateWithdrawal() 함수를 실행하면 MessagePassed 이벤트 발생한다는 게 중요하름이다.
- 사용자가 initiateWithdrawal() 함수를 실행하는 데는 3 가지 방법이 있다. 그 구조를 보면 다음과 같다.

#### L2 (Optimism)

##### 1. 사용자가 직접 ETH 전송

```
User → L2ToL1MessagePasser.receive()
 ↳ initiateWithdrawal(msg.sender, 100000, "")
 ↳ emit MessagePassed(...) ✓
```

## 2. 사용자가 함수 직접 호출

```
User → L2ToL1MessagePasser.initiateWithdrawal(target, gas, data)
 ↳ emit MessagePassed(...) ✓
```

## 3. L2StandardBridge 를 통한 출금

```
User → L2StandardBridge.withdraw(token, amount, ...)
 ↳ L2ToL1MessagePasser.initiateWithdrawal(...)
 ↳ emit MessagePassed(...) ✓
```

사용자는 이 txHash 를 보관해야 한다. 이 txHash 의 receipt 안에는 MessagePassed Event 가 저장되어 있는데, 이 이벤트안에는 다음 단계에서 사용할 정보가 있기 때문이다.

실제 initiateWithdrawal 함수를 살펴보면 주요 동작은 다음과 같다

- WithdrawalTransaction 구조체를 생성 (nonce, sender, target, value, gasLimit, data 포함)
- 해시를 계산하여 sentMessages 매핑에 true 로 저장
- MessagePassed 이벤트 발생
- 메시지 nonce 증가

```
// packages > contracts-bedrock > src > L2 > L2ToL1MessagePasser.sol

/// @notice Sends a message from L2 to L1.
/// @param _target Address to call on L1 execution.
/// @param _gasLimit Minimum gas limit for executing the message on L1.
/// @param _data Data to forward to L1 target.
function initiateWithdrawal(address _target, uint256 _gasLimit, bytes memory _data) public payable virtual {
 bytes32 withdrawalHash = Hashing.hashWithdrawal(
 Types.WithdrawalTransaction({
 nonce: messageNonce(),
 sender: msg.sender,
 target: _target,
 value: msg.value,
 gasLimit: _gasLimit,
 data: _data
 })
);

 sentMessages[withdrawalHash] = true;

 emit MessagePassed(messageNonce(), msg.sender, _target, msg.value, _gasLimit, _data, withdrawalHash);

 unchecked {
 ++msgNonce;
 }
}
```

```

}

/// @notice Allows users to withdraw ETH by sending directly to this contract.
receive() external payable {
 initiateWithdrawal(msg.sender, RECEIVE_DEFAULT_GAS_LIMIT, bytes(""));
}
...

``solidity
// packages > contracts-bedrock > src > L2 > L2StandardBridge.sol

function withdraw(
 address _l2Token,
 uint256 _amount,
 uint32 _minGasLimit,
 bytes calldata _extraData
)
 external
 payable
 virtual
 onlyEOA
{
 _initiateWithdrawal(_l2Token, msg.sender, msg.sender, _amount, _minGasLimit, _extraData);
}

```

## 2. [L1] 출금 증명 (Prove Withdrawal)

L2 블록이 L1 에 게시된 후, 사용자는 L1 의 OptimismPortal2 컨트랙트에서 출금을 증명해야 합니다.

```

// packages/contracts-bedrock/src/L1/OptimismPortal2.sol

function proveWithdrawalTransaction(
 Types.WithdrawalTransaction memory _tx,
 uint256 _disputeGameIndex,
 Types.OutputRootProof calldata _outputRootProof,
 bytes[] calldata _withdrawalProof
)
 external
{
 // ... 시스템 일시 정지 확인, 타겟 주소 안전성 검증 ...

 // Dispute Game Factory 에서 dispute game proxy 가져오기
 (, IDisputeGame disputeGameProxy) = disputeGameFactory().gameAtIndex(_disputeGameIndex);
}

```

```

// 게임 유효성 검증...

// Output Root 가 올바르게 생성되었는지 검증
if (disputeGameProxy.rootClaim().raw() != Hashing.hashOutputRootProof(_outputRootProof)) {
 revert OptimismPortal_InvalidOutputRootProof();
}

// withdrawal hash 계산
bytes32 withdrawalHash = Hashing.hashWithdrawal(_tx);

// L2ToL1MessagePasser 컨트랙트의 storage slot 계산
bytes32 storageKey = keccak256(
 abi.encode(
 withdrawalHash,
 uint256(0) // withdrawals 매핑이 첫 번째 슬롯에 있음
)
);

// Merkle Trie 증명으로 L2 에서 출금이 실제로 발생했는지 검증
if (
 SecureMerkleTrie.verifyInclusionProof({
 _key: abi.encode(storageKey),
 _value: hex"01",
 _proof: _withdrawalProof,
 _root: _outputRootProof.messagePasserStorageRoot
 }) == false
) {
 revert OptimismPortal_InvalidMerkleProof();
}

// 출금을 증명됨으로 표시
provenWithdrawals[withdrawalHash][msg.sender] =
 ProvenWithdrawal({ disputeGameProxy: disputeGameProxy, timestamp: uint64(block.timestamp) });
}

```

증명에 필요한 데이터:

```

// op-node/withdrawals/utlis.go

// ProvenWithdrawalParameters is the set of parameters to pass to the ProveWithdrawalTransaction
// and FinalizeWithdrawalTransaction functions
type ProvenWithdrawalParameters struct {
 Nonce *big.Int
 Sender common.Address
}

```

```

Target common.Address
Value *big.Int
GasLimit *big.Int
L2OutputIndex *big.Int
Data []byte
OutputRootProof bindings.TypesOutputRootProof
WithdrawalProof [][]byte // List of trie nodes to prove L2 storage
}

```

증명 생성 과정:

```

// op-node/withdrawals/utls.go
func GetWithdrawalProof(ctx context.Context, proofCl ProofClient, ev
*bindings.L2ToL1MessagePasserMessagePassed, l2Header *types.Header) ([][]byte, common.Hash, error) {
 // 출금 해시 생성 및 검증
 withdrawalHash, err := WithdrawalHash(ev)
 // ...

 // 해시에서 storage slot 계산
 slot := StorageSlotOfWithdrawalHash(withdrawalHash)

 // L2 노드에서 storage proof 가져오기
 p, err := proofCl.GetProof(ctx, predeploys.L2ToL1MessagePasserAddr, []string{slot.String()}, l2Header.Number)
 // ...

 // 증명 검증
 err = VerifyProof(l2Header.Root, p)
 // ...

 // 컨트랙트가 기대하는 형식으로 인코딩
 trieNodes := make([][]byte, len(p.StorageProof[0].Proof))
 for i, s := range p.StorageProof[0].Proof {
 trieNodes[i] = common.FromHex(s)
 }
 return trieNodes, p.StorageHash, nil
}

```

### 3 최종 확정 (Finalize Withdrawal)

Challenge Period(약 7 일)가 지난 후, 사용자는 출금을 완료할 수 있습니다.

```

// packages/contracts-bedrock/src/L1/OptimismPortal2.sol

function finalizeWithdrawalTransaction(Types.WithdrawalTransaction memory _tx) external {
 finalizeWithdrawalTransactionExternalProof(_tx, msg.sender);
}

```

```

function finalizeWithdrawalTransactionExternalProof(
 Types.WithdrawalTransaction memory _tx,
 address _proofSubmitter
)
{
 public

 // 시스템 일시 정지 확인
 _assertNotPaused();

 // 재진입 공격 방지
 if (l2Sender != Constants.DEFAULT_L2_SENDER) {
 revert OptimismPortal_NoReentrancy();
 }

 // 출금 해시 계산
 bytes32 withdrawalHash = Hashing.hashWithdrawal(_tx);

 // 출금 완료 가능 여부 확인 (증명 성숙 기간 등)
 checkWithdrawal(withdrawalHash, _proofSubmitter);

 // 재실행 방지를 위해 완료됨으로 표시
 finalizedWithdrawals[withdrawalHash] = true;

 // ETHLockbox 에서 ETH 잠금 해제 (사용 시)
 if (_isUsingLockbox()) {
 if (_tx.value > 0) ethLockbox.unlockETH(_tx.value);
 }

 // L2 sender 설정
 l2Sender = _tx.sender;

 // 타겟 컨트랙트 호출 (ETH 전송)
 bool success = SafeCall.callWithMinGas(_tx.target, _tx.gasLimit, _tx.value, _tx.data);

 // L2 sender 리셋
 l2Sender = Constants.DEFAULT_L2_SENDER;

 emit WithdrawalFinalized(withdrawalHash, success);
}

```

완료 조건 확인:

```

// packages/contracts-bedrock/src/L1/OptimismPortal2.sol
function checkWithdrawal(bytes32 _withdrawalHash, address _proofSubmitter) public view {

```

```

ProvenWithdrawal memory provenWithdrawal = provenWithdrawals[_withdrawalHash][_proofSubmitter];
IDisputeGame disputeGameProxy = provenWithdrawal.disputeGameProxy;

// 이미 완료된 출금인지 확인
if (finalizedWithdrawals[_withdrawalHash]) {
 revert OptimismPortal_AlreadyFinalized();
}

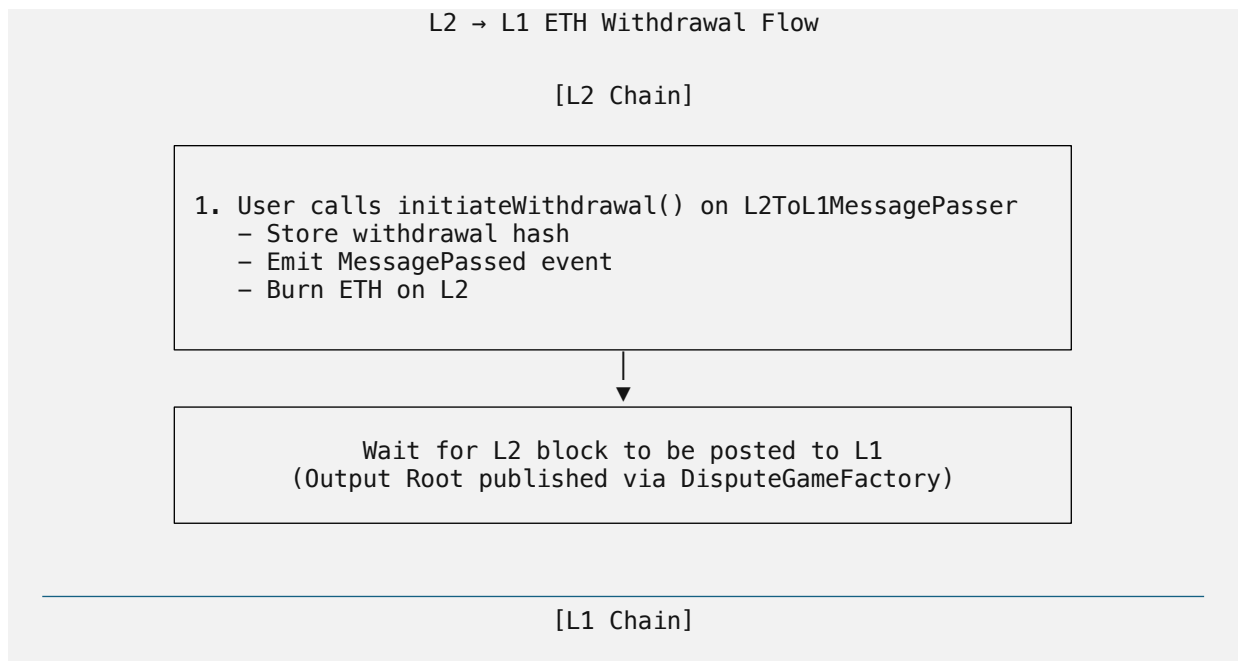
// 증명되지 않은 출금인지 확인
if (provenWithdrawal.timestamp == 0) {
 revert OptimismPortal_Unproven();
}

// 증명 성숙 기간(PROOF_MATURITY_DELAY_SECONDS) 경과 여부 확인
if (block.timestamp - provenWithdrawal.timestamp <= PROOF_MATURITY_DELAY_SECONDS) {
 revert OptimismPortal_ProofNotOldEnough();
}

// root claim 이 유효한지 확인
if (!anchorStateRegistry.isGameClaimValid(disputeGameProxy)) {
 revert OptimismPortal_InvalidRootClaim();
}
}

```

#### 4 요약 다이어그램



```

2. proveWithdrawalTransaction()
 on OptimismPortal2
 - Verify output root proof
 - Verify Merkle inclusion proof
 - Store proven withdrawal

```

Wait for Challenge Period (~7 days)  
(PROOF\_MATURITY\_DELAY\_SECONDS)

```

3. finalizeWithdrawalTransaction() on OptimismPortal2
 - Check proof maturity
 - Unlock ETH from Lockbox
 - Send ETH to target address

```

## proveWithdrawalTransaction 4 개 인자 획득 방법

### 인자 개요

```

function proveWithdrawalTransaction(
 Types.WithdrawalTransaction memory _tx, // 1. 출금 트랜잭션 데이터
 uint256 _disputeGameIndex, // 2. Dispute Game 인덱스
 Types.OutputRootProof calldata _outputRootProof, // 3. Output Root 증명
 bytes[] calldata _withdrawalProof // 4. 스토리지 Merkle 증명
)

```

### 1. \_tx (WithdrawalTransaction) - L2 트랜잭션 영수증에서 파싱

L2 에서 initiateWithdrawal 호출 시 발생한 MessagePassed 이벤트를 파싱하여 획득합니다.

구조체 정의:

```

// packages/contracts-bedrock/src/libraries/Types.sol

/// @notice Struct representing a withdrawal transaction.
/// @custom:field nonce Nonce of the withdrawal transaction
/// @custom:field sender Address of the sender of the transaction.
/// @custom:field target Address of the recipient of the transaction.
/// @custom:field value Value to send to the recipient.
/// @custom:field gasLimit Gas limit of the transaction.
/// @custom:field data Data of the transaction.
struct WithdrawalTransaction {
 uint256 nonce;
 address sender;
 address target;
}

```

```

uint256 value;
uint256 gasLimit;
bytes data;
}

```

획득 방법 (Go 코드):

```

// op-node/withdrawals/utils.go

// ParseMessagePassed parses MessagePassed events from
// a transaction receipt. It does not support multiple withdrawals
// per receipt.
func ParseMessagePassed(receipt *types.Receipt) (*bindings.L2ToL1MessagePasserMessagePassed, error) {
 events, err := ParseMessagesPassed(receipt)
 if err != nil {
 return nil, err
 }
 return events[0], nil
}

// ParseMessagesPassed parses MessagePassed events from
// a transaction receipt. It supports multiple withdrawals
// per receipt.
func ParseMessagesPassed(receipt *types.Receipt) ([]*bindings.L2ToL1MessagePasserMessagePassed, error) {
 contract, err := bindings.NewL2ToL1MessagePasser(common.Address{}, nil)
 if err != nil {
 return nil, err
 }

 var events []*bindings.L2ToL1MessagePasserMessagePassed
 for _, log := range receipt.Logs {
 if len(log.Topics) == 0 || log.Topics[0] != MessagePassedTopic {
 continue
 }

 ev, err := contract.ParseMessagePassed(*log)
 if err != nil {
 return nil, fmt.Errorf("failed to parse log: %w", err)
 }
 events = append(events, ev)
 }
 // ...
}

```

JavaScript/TypeScript 예시:

```

``typescript
// L2 트랜잭션 영수증에서 MessagePassed 이벤트 파싱
const l2Receipt = await l2Provider.getTransactionReceipt(l2TxHash);
const l2ToL1MessagePasser = new ethers.Contract(
 "0x4200",
 l2ToL1MessagePasserABI,
 l2Provider
);

// MessagePassed 이벤트 찾기
const messagePassedLog = l2Receipt.logs.find(
 log => log.topics[0] === l2ToL1MessagePasser.interface.getEventTopic("MessagePassed")
);
const parsed = l2ToL1MessagePasser.interface.parseLog(messagePassedLog);

const _tx = {
 nonce: parsed.args.nonce,
 sender: parsed.args.sender,
 target: parsed.args.target,
 value: parsed.args.value,
 gasLimit: parsed.args.gasLimit,
 data: parsed.args.data
};

```

## 2. \_disputeGameIndex - DisputeGameFactory 에서 조회

최신 dispute game 을 찾아 해당 인덱스를 사용합니다.

획득 방법 (Go 코드):

```

// op-node/withdrawals/utls.go
// FindLatestGame finds the latest game in the DisputeGameFactory contract.
func FindLatestGame(ctx context.Context, disputeGameFactoryContract *bindings.DisputeGameFactoryCaller,
optimismPortal2Contract *bindingspreview.OptimismPortal2Caller)
(*bindings.IDisputeGameFactoryGameSearchResult, error) {
 respectedGameType, err := optimismPortal2Contract.RespectedGameType(&bind.CallOpts{})
 if err != nil {
 return nil, fmt.Errorf("failed to get respected game type: %w", err)
 }

 gameCount, err := disputeGameFactoryContract.GameCount(&bind.CallOpts{})
 if err != nil {
 return nil, fmt.Errorf("failed to get game count: %w", err)
 }
 if gameCount.Cmp(common.Big0) == 0 {
 return nil, errors.New("no games")
 }
}

```

```

}

searchStart := new(big.Int).Sub(gameCount, common.Big1)
latestGames, err := disputeGameFactoryContract.FindLatestGames(&bind.CallOpts{}, respectedGameType,
searchStart, common.Big1)
if err != nil {
 return nil, fmt.Errorf("failed to get latest games: %w", err)
}
if len(latestGames) == 0 {
 return nil, errors.New("no latest games")
}

latestGame := latestGames[0]
return &latestGame, nil // latestGame.Index 가 _disputeGameIndex
}

```

JavaScript/TypeScript 예시:

```

const disputeGameFactory = new ethers.Contract(
 disputeGameFactoryAddress,
 DisputeGameFactoryABI,
 I1Provider
);

const optimismPortal = new ethers.Contract(
 optimismPortalAddress,
 OptimismPortal2ABI,
 I1Provider
);

// 존중되는 게임 타입 조회
const respectedGameType = await optimismPortal.respectedGameType();

// 게임 수 조회
const gameCount = await disputeGameFactory.gameCount();

// 최신 게임 찾기 (출금 블록 이후의 게임이어야 함)
const latestGames = await disputeGameFactory.findLatestGames(
 respectedGameType,
 gameCount.sub(1),
 1
);

const _disputeGameIndex = latestGames[0].index;

```

### 3. \_outputRootProof (OutputRootProof) - L2 블록 헤더에서 생성

L2 블록 헤더와 L2ToL1MessagePasser 컨트랙트의 storage root 를 조합하여 생성합니다.

구조체 정의:

```
// packages/contracts-bedrock/src/libraries/Types.sol
/// @notice Struct representing the elements that are hashed together to generate an output root
/// which itself represents a snapshot of the L2 state.
/// @custom:field version Version of the output root.
/// @custom:field stateRoot Root of the state trie at the block of this output.
/// @custom:field messagePasserStorageRoot Root of the message passer storage trie.
/// @custom:field latestBlockhash Hash of the block this output was generated from.
struct OutputRootProof {
 bytes32 version;
 bytes32 stateRoot;
 bytes32 messagePasserStorageRoot;
 bytes32 latestBlockhash;
}
```

획득 방법 (Go 코드):

```
// op-node/withdrawals/utls.go
func ProveWithdrawalParametersForEvent(ctx context.Context, proofCl ProofClient, ev
*bindings.L2ToL1MessagePasserMessagePassed, l2Header *types.Header, l2OutputIndex *big.Int)
(ProvenWithdrawalParameters, error) {
 withdrawalProof, storageRoot, err := GetWithdrawalProof(ctx, proofCl, ev, l2Header)
 if err != nil {
 return ProvenWithdrawalParameters{}, err
 }
 return ProvenWithdrawalParameters{
 // ...
 OutputRootProof: bindings.TypesOutputRootProof{
 Version: [32]byte{}, // Empty for version 1
 StateRoot: l2Header.Root, // L2 상태 트라이 루트
 MessagePasserStorageRoot: storageRoot, // L2ToL1MessagePasser 스토리지 루트
 LatestBlockhash: l2Header.Hash(), // L2 블록 해시
 },
 WithdrawalProof: withdrawalProof,
 }, nil
}
```

JavaScript/TypeScript 예시:

```
// L2 블록 헤더 조회
const l2BlockNumber = /* dispute game 의 extraData 에서 추출 */;
const l2Block = await l2Provider.getBlock(l2BlockNumber);
```

```
// L2ToL1MessagePasser 의 storage proof 조회
const proof = await l2Provider.send("eth_getProof", [
 "0x4200", // L2ToL1MessagePasser
 [storageSlot], // withdrawal hash 의 storage slot
 l2BlockNumber
]);

const _outputRootProof = {
 version: ethers.constants.HashZero, // 0x00...00 (version 1)
 stateRoot: l2Block.stateRoot, // L2 상태 루트
 messagePasserStorageRoot: proof.storageHash, // L2ToL1MessagePasser 스토리지 루트
 latestBlockhash: l2Block.hash // L2 블록 해시
};
```

storageHash : 컨트랙트의 모든 storage 상태를 하나의 해시로 요약 Output Root 계산 시 :  
messagePasserStorageRoot 로 사용

```
Output Root = keccak256(
 version,
 stateRoot,
 messagePasserStorageRoot, ← storageHash 가 여기 들어감!
 blockHash
)
```

출금 증명에서의 사용

```
proveWithdrawalTransaction()

_outputRootProof.messagePasserStorageRoot ←
eth_getProof 응답: storageHash ————— 일치해야 함

_withdrawalProof ← storageProof[0].proof

SecureMerkleTrie.verifyInclusionProof({
 _key: storageSlot,
 _value: hex"01", // true
 _proof: _withdrawalProof, // storage proof
 _root: messagePasserStorageRoot // storage hash
})
```

#### 4. \_withdrawalProof - Merkle Trie 증명

L2ToL1MessagePasser 컨트랙트의 스토리지에서 출금 해시가 저장되어 있음을 증명하는 Merkle proof 입니다.

획득 방법 (Go 코드):

```
// op-node/withdrawals/utls.go
func GetWithdrawalProof(ctx context.Context, proofCl ProofClient, ev
*bindings.L2ToL1MessagePasserMessagePassed, l2Header *types.Header) ([]byte, common.Hash, error) {
 // 출금 해시 계산
 withdrawalHash, err := WithdrawalHash(ev)
 // ...

 // storage slot 계산: keccak256(withdrawalHash ++ 0)
 slot := StorageSlotOfWithdrawalHash(withdrawalHash)

 // L2 노드에서 storage proof 조회
 p, err := proofCl.GetProof(ctx, predeploys.L2ToL1MessagePasserAddr, []string{slot.String()}, l2Header.Number)
 // ...

 // 증명 검증
 err = VerifyProof(l2Header.Root, p)
 // ...

 // trie nodes 를 bytes[] 형식으로 변환
 trieNodes := make([]byte, len(p.StorageProof[0].Proof))
 for i, s := range p.StorageProof[0].Proof {
 trieNodes[i] = common.FromHex(s)
 }
 return trieNodes, p.StorageHash, nil
}

func VerifyProof(stateRoot common.Hash, proof *gethclient.AccountResult) error {
 balance, overflow := uint256.FromBig(proof.Balance)
 if overflow {
 return fmt.Errorf("proof balance overflows uint256: %d", proof.Balance)
 }
 err := VerifyAccountProof(
 stateRoot,
 proof.Address,
 types.StateAccount{
 Nonce: proof.Nonce,
 Balance: balance,
 Root: proof.StorageHash,
```

```

 CodeHash: proof.CodeHash[:],
},
proof.AccountProof,
)
if err != nil {
 return fmt.Errorf("failed to validate account: %w", err)
}
for i, storageProof := range proof.StorageProof {
 err = VerifyStorageProof(proof.StorageHash, storageProof)
 if err != nil {
 return fmt.Errorf("failed to validate storage proof %d: %w", i, err)
 }
}
return nil
}

```

```

// op-geth/ethclient/ethclient/ethclient.go

// AccountResult is the result of a GetProof operation.
type AccountResult struct {
 Address common.Address `json:"address"`
 AccountProof []string `json:"accountProof"`
 Balance *big.Int `json:"balance"`
 CodeHash common.Hash `json:"codeHash"`
 Nonce uint64 `json:"nonce"`
 StorageHash common.Hash `json:"storageHash"`
 StorageProof []StorageResult `json:"storageProof"`
}

```

Storage Slot 계산:

```

// op-node/withdrawals/utils.go

// StorageSlotOfWithdrawalHash determines the storage slot of the L2ToL1MessagePasser contract to look at
// given a WithdrawalHash
func StorageSlotOfWithdrawalHash(hash common.Hash) common.Hash {
 // The withdrawals mapping is the 0th storage slot in the L2ToL1MessagePasser contract.
 // To determine the storage slot, use keccak256(withdrawalHash ++ p)
 // Where p is the 32 byte value of the storage slot and ++ is concatenation
 buf := make([]byte, 64)
 copy(buf, hash[:])
 return crypto.Keccak256Hash(buf)
}

```

JavaScript/TypeScript 예시:

```

// 1. Withdrawal Hash 계산
const withdrawalHash = ethers.utils.keccak256(
 ethers.utils.defaultAbiCoder.encode(
 ["uint256", "address", "address", "uint256", "uint256", "bytes"],
 [_tx.nonce, _tx.sender, _tx.target, _tx.value, _tx.gasLimit, _tx.data]
)
);

// 2. Storage Slot 계산 (keccak256(withdrawalHash ++ 0))
const storageSlot = ethers.utils.keccak256(
 ethers.utils.concat([
 withdrawalHash,
 ethers.utils.zeroPad("0x00", 32) // slot 0
])
);

// 3. eth_getProof 호출
const proof = await l2Provider.send("eth_getProof", [
 "0x42000", // L2ToL1MessagePasser
 [storageSlot],
 ethers.utils.hexValue(l2BlockNumber)
]);

// 4. 증명 데이터 추출
const _withdrawalProof = proof.storageProof[0].proof;

```

eth\_getProof 응답구조

```

{
 "accountProof": ["0x...", "0x...", ...],
 "balance": "0x...",
 "codeHash": "0x...",
 "nonce": "0x...",
 "storageHash": "0xabc123...", // ← _outputRootProof.messagePasserStorageRoot
 "storageProof": [
 {
 "key": "0x...",
 "value": "0x01", // true
 "proof": [// ← _withdrawalProof
 "0xf90211a0...", // trie node 1 (branch)
 "0xf90211a0...", // trie node 2
 "0xf8518080...", // trie node 3 (leaf)
 ...
]
 }
]
}

```

```
]
}
```

## 5. 전체 흐름 요약

### proveWithdrawalTransaction 인자 획득

1. `_tx (WithdrawalTransaction)`
  - └ L2 트랜잭션 영수증 → `MessagePassed` 이벤트 파싱
    - `nonce, sender, target, value, gasLimit, data`
2. `_disputeGameIndex`
  - └ `DisputeGameFactory.findLatestGames()` 호출
    - `respectedGameType` 에 해당하는 최신 게임 인덱스
3. `_outputRootProof`
  - └ `version: 0x00...00` (고정)
  - └ `stateRoot: L2 블록 헤더의 stateRoot`
  - └ `messagePasserStorageRoot: eth_getProof 의 storageHash`
  - └ `latestBlockhash: L2 블록 해시`
4. `_withdrawalProof`
  - └ `eth_getProof(L2ToL1MessagePasser, [storageSlot], blockNumber)`
    - `storageSlot = keccak256(withdrawalHash ++ 0)`
    - 반환값의 `storageProof[0].proof` 배열

## 6. 통합 예시 (Go 코드)

```
// op-node/withdrawals/utlis.go

// ProveWithdrawalParametersFaultProofs calls ProveWithdrawalParametersForBlock with the most recent L2
// output after the latest game.
func ProveWithdrawalParametersFaultProofs(ctx context.Context, proofCI ProofClient, l2ReceiptCI ReceiptClient,
l2HeaderCI HeaderClient, txHash common.Hash, disputeGameFactoryContract
*bindings.DisputeGameFactoryCaller, optimismPortal2Contract *bindingspreview.OptimismPortal2Caller)
(ProvenWithdrawalParameters, error) {
 latestGame, err := FindLatestGame(ctx, disputeGameFactoryContract, optimismPortal2Contract)
 if err != nil {
 return ProvenWithdrawalParameters{}, fmt.Errorf("failed to find latest game: %w", err)
 }

 l2BlockNumber := new(big.Int).SetBytes(latestGame.ExtraData[0:32])
 l2OutputIndex := latestGame.Index
 // Fetch the block header from the L2 node
 l2Header, err := l2HeaderCI.HeaderByNumber(ctx, l2BlockNumber)
 if err != nil {
```

```

 return ProvenWithdrawalParameters{}, fmt.Errorf("failed to get l2Block: %w", err)
}
return ProveWithdrawalParametersForBlock(ctx, proofCl, l2ReceiptCl, txHash, l2Header, l2OutputIndex)
}

```

## 7. 실제 사용 예시

(JavaScript/TypeScript)

```

import { ethers } from 'ethers';

async function getWithdrawalProof(
 l2Provider: ethers.Provider,
 withdrawalTxHash: string,
 l2BlockNumber: number
) {
 // 1. 트랜잭션 영수증에서 MessagePassed 이벤트 가져오기
 const receipt = await l2Provider.getTransactionReceipt(withdrawalTxHash);
 const messagePassedEvent = parseMessagePassedEvent(receipt.logs);

 // 2. withdrawalHash 계산
 const withdrawalHash = ethers.keccak256(
 ethers.AbiCoder.defaultAbiCoder().encode(
 ['uint256', 'address', 'address', 'uint256', 'uint256', 'bytes'],
 [
 messagePassedEvent.nonce,
 messagePassedEvent.sender,
 messagePassedEvent.target,
 messagePassedEvent.value,
 messagePassedEvent.gasLimit,
 messagePassedEvent.data
]
)
);

 // 3. storage slot 계산
 const storageSlot = ethers.keccak256(
 ethers.AbiCoder.defaultAbiCoder().encode(
 ['bytes32', 'uint256'],
 [withdrawalHash, 0] // sentMessages 는 slot 0
)
);

 // 4. eth_getProof 호출
 const L2_TO_L1_MESSAGE_PASSER = '0x4200';

```

```

const proof = await l2Provider.send('eth_getProof', [
 L2_TO_L1_MESSAGE_PASSER,
 [storageSlot],
 ethers.toBeHex(l2BlockNumber)
]);

// 5. proof 반환
return {
 withdrawalProof: proof.storageProof[0].proof, // bytes[]
 storageRoot: proof.storageHash
};
}

```

## Dispute Game 인덱스 조회 방법

### 방법 1: gameAtIndex()로 순차 검색

DisputeGameFactory 는 모든 생성된 게임을 배열에 저장하므로, 인덱스로 직접 접근할 수 있습니다.

```

// packages/contracts-bedrock/src/dispute/DisputeGameFactory.sol

/// @notice `gameAtIndex` returns the dispute game contract address and its creation timestamp
/// at the given index. Each created dispute game increments the underlying index.
/// Reverts if the provided index does not correspond to an existing dispute game.
/// @param _index The index of the dispute game.
/// @return gameType_ The type of the DisputeGame - used to decide the proxy implementation.
/// @return timestamp_ The timestamp of the creation of the dispute game.
/// @return proxy_ The clone of the `DisputeGame` created with the given parameters.
function gameAtIndex(uint256 _index)
 external
 view
 returns (GameType gameType_, Timestamp timestamp_, IDisputeGame proxy_)
{
 (GameType gameType, Timestamp timestamp, address proxy) = _disputeGameList[_index].unpack();
 (gameType_, timestamp_, proxy_) = (gameType, timestamp, IDisputeGame(proxy));
}

```

JavaScript 예시: L2 블록 번호로 적절한 게임 찾기

```

async function findGameForL2Block(
 disputeGameFactory: Contract,
 optimismPortal: Contract,
 targetL2BlockNumber: BigNumber
): Promise<Bignumber> {
 const respectedGameType = await optimismPortal.respectedGameType();

```

```

const gameCount = await disputeGameFactory.gameCount();

// 역순으로 검색하여 targetL2BlockNumber 를 커버하는 첫 번째 게임 찾기
for (let i = gameCount.sub(1); i.gte(0); i = i.sub(1)) {
 const [gameType, timestamp, gameProxy] = await disputeGameFactory.gameAtIndex(i);

 // 존중되는 게임 타입인지 확인
 if (gameType !== respectedGameType) continue;

 // 게임의 L2 블록 번호 가져오기
 const gameContract = new ethers.Contract(gameProxy, FaultDisputeGameABI, provider);
 const extraData = await gameContract.extraData();
 const gameL2BlockNumber = ethers.BigNumber.from(extraData.slice(0, 66)); // 첫 32 bytes

 // 이 게임이 출금 L2 블록을 커버하는지 확인
 if (gameL2BlockNumber.gte(targetL2BlockNumber)) {
 return i; // 이 인덱스가 _disputeGameIndex
 }
}

throw new Error("No suitable game found");
}

```

## 방법 2: findLatestGames()를 특정 범위로 호출

```

// packages/contracts-bedrock/src/dispute/DisputeGameFactory.sol

/// @notice Finds the `_n` most recent `GameId`s of type `_gameType` starting at `_start`. If there are less than
/// `_n` games of type `_gameType` starting at `_start`, then the returned array will be shorter than `_n`.
/// @param _gameType The type of game to find.
/// @param _start The index to start the reverse search from.
/// @param _n The number of games to find.
function findLatestGames(
 GameType _gameType,
 uint256 _start,
 uint256 _n
)
 external
 view
 returns (GameSearchResult[] memory games_)
{
 // ...
 // Perform a reverse linear search for the `_n` most recent games of type `_gameType`.
 for (uint256 i = _start; i >= 0 && i <= _start;) {
 // ...
 }
}

```

```

 games_[games_.length - 1] = GameSearchResult({
 index: i, // ← 게임 인덱스
 metadata: id,
 timestamp: timestamp,
 rootClaim: rootClaim,
 extraData: extraData // ← L2 블록 번호 포함
 });
 // ...
 }
}

```

JavaScript 예시: 대량 검색

```

async function findGameForL2BlockBatch(
 disputeGameFactory: Contract,
 optimismPortal: Contract,
 targetL2BlockNumber: BigNumber
): Promise<Bignumber> {
 const respectedGameType = await optimismPortal.respectedGameType();
 const gameCount = await disputeGameFactory.gameCount();

 // 한 번에 100 개씩 검색
 const batchSize = 100;
 let startIndex = gameCount.sub(1);

 while (startIndex.gte(0)) {
 const games = await disputeGameFactory.findLatestGames(
 respectedGameType,
 startIndex,
 batchSize
);

 for (const game of games) {
 // extraData 에서 L2 블록 번호 추출
 const gameL2BlockNumber = ethers.BigNumber.from(
 "0x" + game.extraData.slice(2, 66) // 첫 32 bytes
);

 if (gameL2BlockNumber.gte(targetL2BlockNumber)) {
 return game.index; // _disputeGameIndex
 }
 }

 if (games.length < batchSize) break;
 startIndex = startIndex.sub(batchSize);
 }
}

```

```

}

throw new Error("No suitable game found");
}

```

### 방법 3: DisputeGameCreated 이벤트 로그 검색

가장 효율적인 방법은 이벤트 로그를 사용하는 것입니다.

```

// packages/contracts-bedrock/src/dispute/DisputeGameFactory.sol

/// @notice Emitted when a new dispute game is created
/// @param disputeProxy The address of the dispute game proxy
/// @param gameType The type of the dispute game proxy's implementation
/// @param rootClaim The root claim of the dispute game
event DisputeGameCreated(address indexed disputeProxy, GameType indexed gameType, Claim indexed rootClaim);

```

#### JavaScript 예시: 이벤트 로그 검색

```

``typescript
async function findGameByEvent(
 disputeGameFactory: Contract,
 optimismPortal: Contract,
 targetL2BlockNumber: BigNumber,
 fromL1Block: number,
 toL1Block: number
): Promise<BigNumber> {
 const respectedGameType = await optimismPortal.respectedGameType();

 // DisputeGameCreated 이벤트 필터
 const filter = disputeGameFactory.filters.DisputeGameCreated(
 null, // disputeProxy - any
 respectedGameType, // gameType - 존중되는 타입만
 null // rootClaim - any
);

 // 이벤트 쿼리
 const events = await disputeGameFactory.queryFilter(filter, fromL1Block, toL1Block);

 // 각 게임을 확인하여 적절한 것 찾기
 for (const event of events.reverse()) {
 const gameProxy = event.args.disputeProxy;
 const gameContract = new ethers.Contract(gameProxy, FaultDisputeGameABI, provider);

 // L2 블록 번호 확인

```

```

const extraData = await gameContract.extraData();
const gameL2BlockNumber = ethers.BigNumber.from("0x" + extraData.slice(2, 66));

if (gameL2BlockNumber.gte(targetL2BlockNumber)) {
 // games() 함수로 인덱스 가져오기 (또는 별도 조회)
 const rootClaim = await gameContract.rootClaim();

 // 인덱스는 이벤트 순서로 추정하거나 gameAtIndex 로 확인 필요
 // 또는 게임 주소를 직접 사용
 return findGameIndexByProxy(disputeGameFactory, gameProxy);
}

throw new Error("No suitable game found");
}

async function findGameIndexByProxy(
 factory: Contract,
 targetProxy: string
): Promise<BigNumber> {
 const count = await factory.gameCount();
 for (let i = count.sub(1); i.gte(0); i = i.sub(1)) {
 const [, proxy] = await factory.gameAtIndex(i);
 if (proxy.toLowerCase() === targetProxy.toLowerCase()) {
 return i;
 }
 }
 throw new Error("Game not found");
}

```

#### 방법 4: games() 함수 사용 (UUID 로 조회)

게임의 gameType, rootClaim, extraData 를 알고 있다면 직접 조회 가능합니다.

```

// packages/contracts-bedrock/src/dispute/DisputeGameFactory.sol

/// @notice `games` queries an internal mapping that maps the hash of
/// `gameType ++ rootClaim ++ extraData` to the deployed `DisputeGame` clone.
/// @dev `++` equates to concatenation.
/// @param _gameType The type of the DisputeGame - used to decide the proxy implementation
/// @param _rootClaim The root claim of the DisputeGame.
/// @param _extraData Any extra data that should be provided to the created dispute game.
/// @return proxy_ The clone of the `DisputeGame` created with the given parameters.
/// Returns `address(0)` if nonexistent.
/// @return timestamp_ The timestamp of the creation of the dispute game.

```

```
function games(
 GameType _gameType,
 Claim _rootClaim,
 bytes calldata _extraData
)
external
view
returns (IDisputeGame proxy_, Timestamp timestamp_)
```

## 방법 비교

방법	장점	단점	사용 시기
gameAtIndex() 순차 검색	간단함	가스 비용 높음 (많은 호출)	게임 수가 적을 때
findLatestGames() 배치	RPC 호출 감소	여전히 L2 블록 확인 필요	중간 규모 검색
DisputeGameCreated 이벤트 로그 검색	매우 효율적	아카이브 노드 필요할 수 있음	대규모/오래된 게임 검색
games() 직접 조회	O(1) 조회	UUID 미리 알아야 함	정확한 게임 정보 있을 때

## 실제 구현 예시 (Go 코드)

op-devstack 에서 L2 블록을 커버하는 게임을 찾는 로직:

```
// op-devstack/dsl/bridge.go

func (b *StandardBridge) forGamePublished(l2BlockNumber *big.Int) disputeGame {
 respectedGameType := b.RespectedGameType()
 l2SequenceNumber := l2BlockNumber.Uint64()
 // ...

 var game bindings.DisputeGame
 var gameSeqNum uint64
 var gameIndex *big.Int
 b.require.Eventuallyf(func() bool {
 var err error
 game, gameIndex, err = b.findLatestGame(respectedGameType)
 if err != nil {
 b.log.Warn("No game of required type found", "err", err)
 return false
 }
 })
 gameContract := bindings.NewBindings[bindings.FaultDisputeGame](/* ... */)
 seqNum, err := contractio.Read(gameContract.L2SequenceNumber(), b.ctx)
 // ...
 gameSeqNum = seqNum.Uint64()
```

```

// 게임의 L2 블록이 출금 L2 블록보다 크거나 같아야 함
return gameSeqNum >= l2SequenceNumber
}, 90*time.Second, 100*time.Millisecond, "...")

return disputeGame{
 Index: gameIdIndex, // ← _disputeGameIndex
 Address: game.Proxy,
 L2BlockNumber: gameBlockNum,
 // ...
}
}

```

```

/// @notice `gameAtIndex` returns the dispute game contract address and its creation timestamp
/// at the given index. Each created dispute game increments the underlying index.
/// Reverts if the provided index does not correspond to an existing dispute game.
/// @param _index The index of the dispute game.
/// @return gameType_ The type of the DisputeGame - used to decide the proxy implementation.
/// @return timestamp_ The timestamp of the creation of the dispute game.
/// @return proxy_ The clone of the `DisputeGame` created with the given parameters.
function gameAtIndex(uint256 _index)
 external
 view
 returns (GameType gameType_, Timestamp timestamp_, IDisputeGame proxy_)
{
 (GameType gameType, Timestamp timestamp, address proxy) = _disputeGameList[_index].unpack();
 (gameType_, timestamp_, proxy_) = (gameType, timestamp, IDisputeGame(proxy));
}

```

```

async function findGameForL2Block(
 disputeGameFactory: Contract,
 optimismPortal: Contract,
 targetL2BlockNumber: BigNumber
): Promise<BiggerNumber> {
 const respectedGameType = await optimismPortal.respectedGameType();
 const gameCount = await disputeGameFactory.gameCount();

 // 역순으로 검색하여 targetL2BlockNumber 를 커버하는 첫 번째 게임 찾기
 for (let i = gameCount.sub(1); i.gte(0); i = i.sub(1)) {
 const [gameType, timestamp, gameProxy] = await disputeGameFactory.gameAtIndex(i);

 // 존중되는 게임 타입인지 확인
 if (gameType !== respectedGameType) continue;
 }
}

```

```

// 게임의 L2 블록 번호 가져오기
const gameContract = new ethers.Contract(gameProxy, FaultDisputeGameABI, provider);
const extraData = await gameContract.extraData();
const gameL2BlockNumber = ethers.BigNumber.from(extraData.slice(0, 66)); // 첫 32 bytes

// 이 게임이 출금 L2 블록을 커버하는지 확인
if (gameL2BlockNumber.gte(targetL2BlockNumber)) {
 return i; // 이 인덱스가 _disputeGameIndex
}
}

throw new Error("No suitable game found");
}

```

```

/// @notice Finds the `_n` most recent `GameId`s of type `_gameType` starting at `_start`. If there are less than
/// `_n` games of type `_gameType` starting at `_start`, then the returned array will be shorter than `_n`.
/// @param _gameType The type of game to find.
/// @param _start The index to start the reverse search from.
/// @param _n The number of games to find.
function findLatestGames(
 GameType _gameType,
 uint256 _start,
 uint256 _n
)
 external
 view
 returns (GameSearchResult[] memory games_)
{
 // ...
 // Perform a reverse linear search for the `_n` most recent games of type `_gameType`.
 for (uint256 i = _start; i >= 0 && i <= _start;) {
 // ...
 games_[games_.length - 1] = GameSearchResult({
 index: i, // ← 게임 인덱스
 metadata: id,
 timestamp: timestamp,
 rootClaim: rootClaim,
 extraData: extraData // ← L2 블록 번호 포함
 });
 // ...
 }
}

```

```

async function findGameForL2BlockBatch(
 disputeGameFactory: Contract,
 optimismPortal: Contract,
 targetL2BlockNumber: BigNumber
): Promise<Bignumber> {
 const respectedGameType = await optimismPortal.respectedGameType();
 const gameCount = await disputeGameFactory.gameCount();

 // 한 번에 100 개씩 검색
 const batchSize = 100;
 let startIndex = gameCount.sub(1);

 while (startIndex.gte(0)) {
 const games = await disputeGameFactory.findLatestGames(
 respectedGameType,
 startIndex,
 batchSize
);

 for (const game of games) {
 // extraData 에서 L2 블록 번호 추출
 const gameL2BlockNumber = ethers.BigNumber.from(
 "0x" + game.extraData.slice(2, 66) // 첫 32 bytes
);

 if (gameL2BlockNumber.gte(targetL2BlockNumber)) {
 return game.index; // _disputeGameIndex
 }
 }

 if (games.length < batchSize) break;
 startIndex = startIndex.sub(batchSize);
 }

 throw new Error("No suitable game found");
}

```

```

/// @notice Emitted when a new dispute game is created
/// @param disputeProxy The address of the dispute game proxy
/// @param gameType The type of the dispute game proxy's implementation
/// @param rootClaim The root claim of the dispute game
event DisputeGameCreated(address indexed disputeProxy, GameType indexed gameType, Claim indexed
rootClaim);

```

```

async function findGameByEvent(
 disputeGameFactory: Contract,
 optimismPortal: Contract,
 targetL2BlockNumber: BigNumber,
 fromL1Block: number,
 toL1Block: number
): Promise<BigNumber> {
 const respectedGameType = await optimismPortal.respectedGameType();

 // DisputeGameCreated 이벤트 필터
 const filter = disputeGameFactory.filters.DisputeGameCreated(
 null, // disputeProxy - any
 respectedGameType, // gameType - 존중되는 타입만
 null // rootClaim - any
);

 // 이벤트 쿼리
 const events = await disputeGameFactory.queryFilter(filter, fromL1Block, toL1Block);

 // 각 게임을 확인하여 적절한 것 찾기
 for (const event of events.reverse()) {
 const gameProxy = event.args.disputeProxy;
 const gameContract = new ethers.Contract(gameProxy, FaultDisputeGameABI, provider);

 // L2 블록 번호 확인
 const extraData = await gameContract.extraData();
 const gameL2BlockNumber = ethers.BigNumber.from("0x" + extraData.slice(2, 66));

 if (gameL2BlockNumber.gte(targetL2BlockNumber)) {
 // games() 함수로 인덱스 가져오기 (또는 별도 조회)
 const rootClaim = await gameContract.rootClaim();

 // 인덱스는 이벤트 순서로 추정하거나 gameAtIndex 로 확인 필요
 // 또는 게임 주소를 직접 사용
 return findGameIndexByProxy(disputeGameFactory, gameProxy);
 }
 }

 throw new Error("No suitable game found");
}

async function findGameIndexByProxy(
 factory: Contract,

```

```

 targetProxy: string
): Promise<BigNumber> {
 const count = await factory.gameCount();
 for (let i = count.sub(1); i.gte(0); i = i.sub(1)) {
 const [, proxy] = await factory.gameAtIndex(i);
 if (proxy.toLowerCase() === targetProxy.toLowerCase()) {
 return i;
 }
 }
 throw new Error("Game not found");
 }

```

```

/// @notice `games` queries an internal mapping that maps the hash of
/// `gameType ++ rootClaim ++ extraData` to the deployed `DisputeGame` clone.
/// @dev `++` equates to concatenation.
/// @param _gameType The type of the DisputeGame - used to decide the proxy implementation
/// @param _rootClaim The root claim of the DisputeGame.
/// @param _extraData Any extra data that should be provided to the created dispute game.
/// @return proxy_ The clone of the `DisputeGame` created with the given parameters.
/// Returns `address(0)` if nonexistent.
/// @return timestamp_ The timestamp of the creation of the dispute game.
function games(
 GameType _gameType,
 Claim _rootClaim,
 bytes calldata _extraData
)
 external
 view
 returns (IDisputeGame proxy_, Timestamp timestamp_)

```

```

if (disputeGameProxy.status() == GameStatus.CHALLENGER_WINS) {
 revert OptimismPortal_InvalidDisputeGame();
}

```

```

if (!anchorStateRegistry.isGameRespected(disputeGameProxy)) {
 revert OptimismPortal_InvalidDisputeGame();
}

```

## 중요 고려사항

- 출금 L2 블록 ≤ 게임 L2 블록: 게임이 커버하는 L2 블록이 출금이 발생한 블록보다 크거나 같아야 합니다.

- 게임 상태 확인: 게임이 CHALLENGER\_WINS 상태가 아닌지 확인해야 합니다.

```
if (disputeGameProxy.status() == GameStatus.CHALLENGER_WINS) {
 revert OptimismPortal_InvalidDisputeGame();
}
```

- Blacklist 확인: 블랙리스트에 등록된 게임인지 확인해야 합니다.

```
if (!anchorStateRegistry.isGameRespected(disputeGameProxy)) {
 revert OptimismPortal_InvalidDisputeGame();
}
```

- 재증명 가능: 같은 출금을 다른 게임으로 다시 증명할 수 있으므로, 더 최신의 유효한 게임을 사용하는 것이 좋습니다.

## Blocknumber: L2ToL1MessagePasser.initiateWithdrawal vs extraData

### 1. 출금 L2 블록 번호 (Withdrawal Receipt 에서)

이것은 사용자가 L2ToL1MessagePasser.initiateWithdrawal()을 호출한 트랜잭션이 포함된 L2 블록 번호입니다.

```
// 사용자의 출금 트랜잭션 영수증
const withdrawalReceipt = await l2Provider.getTransactionReceipt(withdrawalTxHash);
const withdrawalL2BlockNumber = withdrawalReceipt.blockNumber; // 예: 1000
```

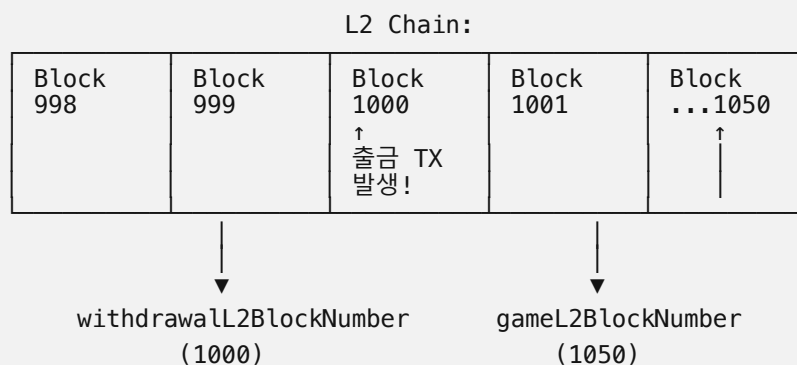
### 2. Dispute Game 의 L2 블록 번호 (extraData 에서)

이것은 Dispute Game 이 커밋(증명)하는 L2 상태의 블록 번호입니다. 이 블록의 상태 루트가 L1 에 제출됩니다.

```
// Dispute Game 이 커버하는 L2 블록
const gameL2BlockNumber = /* extraData 에서 추출 */; // 예: 1050
```

### 3. 관계

시간 흐름 →



← 게임이 커버해야 함 →

## 핵심 조건

gameL2BlockNumber >= withdrawalL2BlockNumber 이어야 합니다!

왜냐하면:

- Dispute Game 은 특정 L2 블록 시점의 전체 상태를 커밋합니다
- 출금이 블록 1000 에서 발생했다면, 블록 1000 이후의 상태 루트에 해당 출금 정보가 포함되어 있습니다
- 따라서 게임이 커버하는 블록 번호가 출금 블록 번호보다 같거나 커야 증명이 가능합니다

## 코드에서의 증거

```
// op-devstack/dsl/bridge.go

func (b *StandardBridge) forGamePublished(l2BlockNumber *big.Int) disputeGame {
 // l2BlockNumber = 출금이 발생한 L2 블록
 l2SequenceNumber := l2BlockNumber.Uint64()
 // ...

 b.require.Eventuallyf(func() bool {
 game, gameIndex, err = b.findLatestGame(respectedGameType)
 // ...

 // gameSeqNum = 게임이 커버하는 L2 블록
 seqNum, err := contractio.Read(gameContract.L2SequenceNumber(), b.ctx)
 gameSeqNum = seqNum.Uint64()

 // 게임의 L2 블록이 출금 L2 블록보다 크거나 같아야 함
 return gameSeqNum >= l2SequenceNumber // ← 핵심 조건!
 }, ...)
}
```

## 실제 증명 과정

```
// 1. 출금 트랜잭션 영수증에서 출금 블록 번호 확인
const withdrawalReceipt = await l2Provider.getTransactionReceipt(withdrawalTxHash);
const withdrawalL2Block = withdrawalReceipt.blockNumber; // 예: 1000

// 2. 출금 블록보다 크거나 같은 L2 블록을 커버하는 Dispute Game 찾기
const gameL2Block = 1050; // 예: extraData 에서 추출한 값
assert(gameL2Block >= withdrawalL2Block); // 필수 조건!

// 3. 해당 게임의 L2 블록(1050)에서 상태 증명 생성
const l2Header = await l2Provider.getBlock(gameL2Block); // 블록 1050
const proof = await l2Provider.send("eth_getProof", [
```

```

L2ToL1MessagePasserAddress,
[storageSlot],
gameL2Block // ← 1050 (게임의 블록)
]);

// 4. 이 증명은 블록 1050 시점의 상태에서
// 블록 1000 에 기록된 출금이 존재함을 증명

```

보통 Dispute Game 은 주기적으로 생성되므로, 출금 후 다음 게임이 생성될 때까지 기다려야 합니다.

```

// 사용자의 출금 트랜잭션 영수증
const withdrawalReceipt = await l2Provider.getTransactionReceipt(withdrawalTxHash);
const withdrawalL2BlockNumber = withdrawalReceipt.blockNumber; // 예: 1000
...

``typescript
// Dispute Game 이 커버하는 L2 블록
const gameL2BlockNumber = /* extraData 에서 추출 */; // 예: 1050

```

## 요약

항목	출금 L2 블록	게임 L2 블록
출처	출금 트랜잭션 영수증	Dispute Game extraData
의미	출금 TX 가 포함된 블록	게임이 증명하는 상태의 블록
예시	1000	1050
관계	-	>= 출금 L2 블록 이어야 함
증명 생성 시 사용	MessagePassed 이벤트 파싱용	eth_getProof 호출 시 블록 번호

즉, 두 블록 번호가 같을 필요는 없고, 게임의 L2 블록이 출금 블록보다 크거나 같으면 됩니다.

## eth\_getProof 응답의 storageHash 와 storageProof

### 전체 응답 구조

```

{
 // 계정(Account) 증명
 "accountProof": ["0x...", ...], // State Trie 의 Merkle 경로
 "address": "0x4200...0016", // 조회한 컨트랙트 주소
 "balance": "0x...", // ETH 잔액
 "codeHash": "0x...", // 컨트랙트 코드 해시
 "nonce": "0x1", // 계정 nonce

 // Storage 증명 (핵심!)

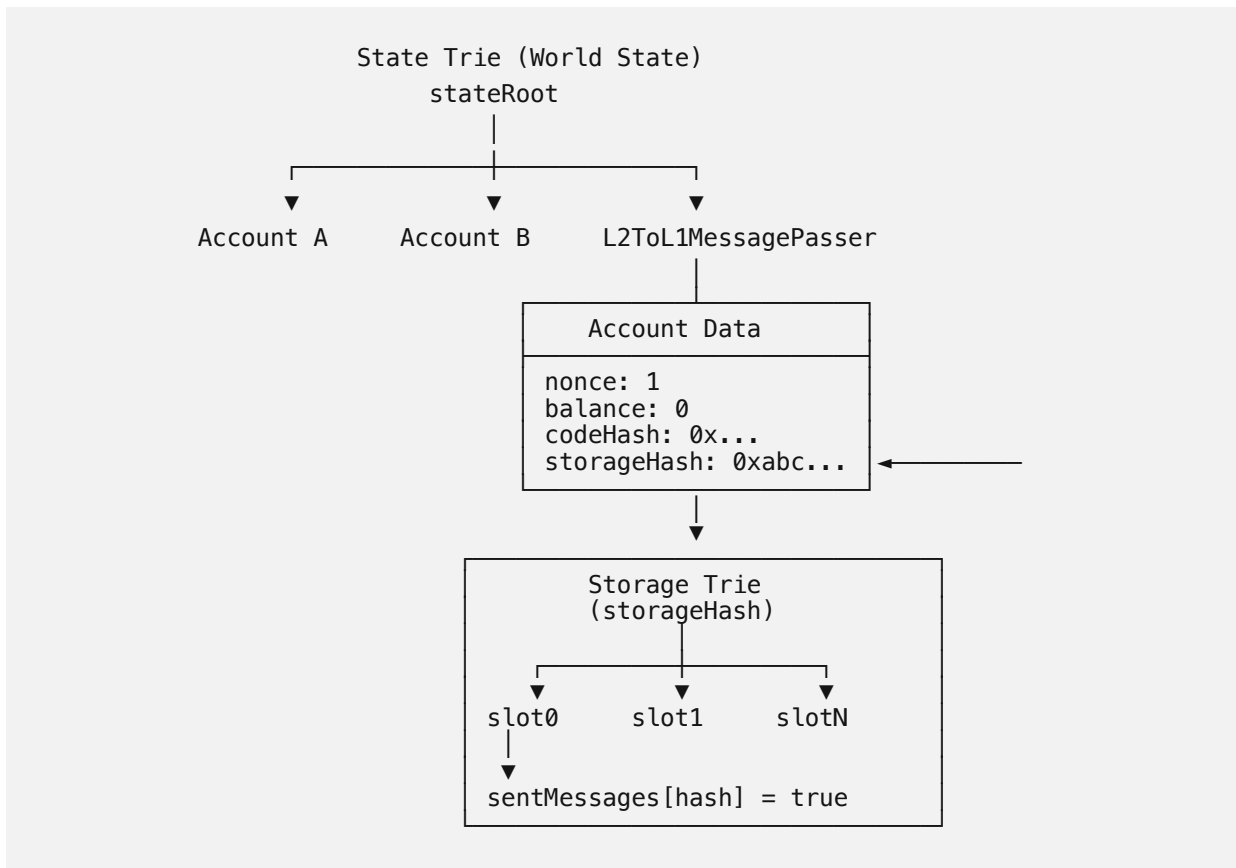
```

```

"storageHash": "0xabc123...", // Storage Trie Root
"storageProof": [// Storage 값들의 증명
{
 "key": "0x...", // storage slot
 "value": "0x01", // 저장된 값
 "proof": ["0x...", ...] // Storage Trie 의 Merkle 경로
}
]
}

```

## Ethereum 의 트라이 구조



### 1. storageHash (= storageRoot)

정의: 해당 컨트랙트의 Storage Trie 의 루트 해시

```

// op-service/eth/account_proof.go
type AccountResult struct {
 // ...
 StorageHash common.Hash `json:"storageHash"` // Storage Trie Root
 // ...
}

```

용도:

- 컨트랙트의 모든 storage 상태를 하나의 해시로 요약
- Output Root 계산 시 messagePasserStorageRoot 로 사용

```
Output Root = keccak256(
 version,
 stateRoot,
 messagePasserStorageRoot, ← storageHash 가 여기 들어감!
 blockHash
)
```

## 2. storageProof

정의: 특정 storage slot 의 값을 증명하는 Merkle Patricia Trie 경로

```
// op-service/eth/account_proof.go
type StorageProofEntry struct {
 Key StorageKey `json:"key"` // storage slot (예: 0x123...)
 Value hexutil.Big `json:"value"` // 저장된 값 (예: 0x01 = true)
 Proof []hexutil.Bytes `json:"proof"` // Merkle 경로 (trie 노드들)
}
```

예시 (출금 증명):

```
{
 "key": "0x5e8e...abc", // keccak256(withdrawalHash || 0)
 "value": "0x1", // true (출금이 기록됨)
 "proof": [
 "0xf90211a0...", // Branch node (root)
 "0xf90211a0...", // Branch node
 "0xf8518080...", // Extension node
 "0xe2a0...", // Leaf node (value = 0x01)
]
}
```

## Proof 검증 과정

```
// op-service/eth/account_proof.go

func (res *AccountResult) Verify(stateRoot common.Hash) error {
 // Storage proof 검증
 for i, entry := range res.StorageProof {
 // 1. proof 노드들을 메모리 DB 에 로드
 db := memorydb.New()
 for j, encodedNode := range entry.Proof {
 nodeKey := encodedNode
 if len(encodedNode) >= 32 {
 nodeKey = crypto.Keccak256(encodedNode)
 }
 }
 }
}
```

```

 }
 db.Put(nodeKey, encodedNode)
 }

 // 2. storage key 를 keccak256 으로 해싱 (SecureMerkleTrie)
 path := crypto.Keccak256(entry.Key)

 // 3. storageHash 를 root 로 하여 proof 검증
 val, err := trie.VerifyProof(res.StorageHash, path, db)

 // 4. 검증된 값과 주장된 값 비교
 comparison, _ := rlp.EncodeToBytes(entry.Value.ToInt().Bytes())
 if !bytes.Equal(val, comparison) {
 return error // 불일치!
 }
}
// ...
}

```

## 출금 증명에서의 사용

```

proveWithdrawalTransaction()

_outputRootProof.messagePasserStorageRoot ←
eth_getProof 응답: storageHash ────────────┘ 일치해야 함

_withdrawalProof ← storageProof[0].proof

SecureMerkleTrie.verifyInclusionProof({
 _key: storageSlot,
 _value: hex"01", // true
 _proof: _withdrawalProof, // storage proof
 _root: messagePasserStorageRoot // storage hash
})

```

## 요약

필드	설명	출금 증명에서의 역할
storageHash	컨트랙트 Storage Trie 의 루트 해시	_outputRootProof.messagePasserStorageRoot 와 일치해야 함
storageProof[].key	조회한 storage slot	keccak256(withdrawalHash $\mathbb{H}(\mathbb{H}(0))$ )

storageProof[].value	해당 slot 의 값	0x01 (true = 출금 기록됨)
storageProof[].proof	Merkle 경로 (trie 노드 배열)	_withdrawalProof 로 전달

핵심: storageHash 가 루트이고, storageProof 가 그 트라이 안에서 특정 key-value 쌍의 존재를 증명하는 Merkle 경로입니다.

## Withdrawal 에 해당하는 DisputeGame 을 찾는 방법

### 1. extraData 에 L2 블록 번호가 저장됨

각 DisputeGame 생성 시 extraData 의 처음 32 바이트에 L2 블록 번호가 인코딩됩니다:

```
extraData = abi.encode(l2BlockNumber) // 첫 32 바이트 = L2 블록 번호
```

```
// op-node/withdrawals/utils.go
```

```
l2BlockNumber := new(big.Int).SetBytes(latestGame.ExtraData[0:32])
```

```
l2OutputIndex := latestGame.Index
```

### 2. 현재 SDK 의 접근 방식: “최신 게임 사용”

현재 op-node 의 구현은 가장 최신의 게임을 찾아서 사용합니다:

```
// op-node/withdrawals/utils.go
```

```
// FindLatestGame finds the latest game in the DisputeGameFactory contract.
```

```
func FindLatestGame(ctx context.Context, disputeGameFactoryContract, optimismPortal2Contract) {
 respectedGameType, _ := optimismPortal2Contract.RespectedGameType()
 gameCount, _ := disputeGameFactoryContract.GameCount()
```

```
 searchStart := new(big.Int).Sub(gameCount, common.Big1)
```

```
 latestGames, _ := disputeGameFactoryContract.FindLatestGames(respectedGameType, searchStart, 1)
```

```
 return &latestGames[0], nil
```

```
}
```

중요: 이 방식은 출금 트랜잭션이 최신 게임의 L2 블록보다 이전에 발생했다면 문제없이 작동합니다.

### 3. 오래된 출금을 위한 검색 방법

특정 L2 블록에 해당하는 DisputeGame 을 찾으려면:

```
/**
```

```
* 특정 L2 블록을 포함하는 DisputeGame 찾기
```

```

*
* 조건: DisputeGame 의 l2BlockNumber >= withdrawalBlockNumber
*/
async function findGameForWithdrawal(withdrawalL2BlockNumber) {
 const gameCount = await disputeGameFactory.gameCount();
 const respectedGameType = await optimismPortal.respectedGameType();

 // 방법 1: 이진 검색 (효율적)
 let left = 0n;
 let right = gameCount - 1n;
 let resultGame = null;

 while (left <= right) {
 const mid = (left + right) / 2n;
 const game = await disputeGameFactory.gameAtIndex(mid);

 // gameType 필터링
 if (game.gameType !== respectedGameType) {
 // 건너뛰기 - 실제로는 더 복잡한 로직 필요
 right = mid - 1n;
 continue;
 }

 const gameProxy = await IDisputeGame.attach(game.proxy);
 const extraData = await gameProxy.extraData();
 const gameL2BlockNumber = BigInt(extraData.slice(0, 66)); // 첫 32 바이트

 if (gameL2BlockNumber >= withdrawalL2BlockNumber) {
 resultGame = { index: mid, l2Block: gameL2BlockNumber, proxy: game.proxy };
 right = mid - 1n; // 더 오래된 적합한 게임 찾기
 } else {
 left = mid + 1n;
 }
 }

 return resultGame;
}

// 방법 2: findLatestGames 를 활용한 역순 검색
async function findGameByLinearSearch(withdrawalL2BlockNumber, batchSize = 100) {
 const gameCount = await disputeGameFactory.gameCount();
 const respectedGameType = await optimismPortal.respectedGameType();

 for (let start = gameCount - 1n; start >= 0n; start -= BigInt(batchSize)) {

```

```

const games = await disputeGameFactory.findLatestGames(
 respectedGameType,
 start,
 batchSize
);

for (const game of games) {
 const gameL2Block = BigInt('0x' + game.extraData.slice(2, 66));

 // 출금 블록을 포함하는 가장 오래된 게임 찾기
 if (gameL2Block >= withdrawalL2BlockNumber) {
 return {
 index: game.index,
 l2Block: gameL2Block,
 rootClaim: game.rootClaim
 };
 }
}

throw new Error('No suitable game found');
}

```

#### 4. 시각화: 게임 선택 로직

DisputeGame 목록 (시간순)

Index 0:	L2 Block 1000	└─	
Index 1:	L2 Block 2000	└─	
Index 2:	L2 Block 3000	└─	← Withdrawal TX (Block 2500)는 Index 2 (Block 3000) 또는 그 이후 사용 가능
Index 3:	L2 Block 4000	└─	
Index 4:	L2 Block 5000	└─	
...			
Index N:	L2 Block XXXX (Latest)		

선택 조건:

- 1. GameType == respectedGameType
- 2. Game.l2BlockNumber >= Withdrawal.l2BlockNumber
- 3. Game 이 CHALLENGER\_WINS 가 아닐 것 (유효한 상태)
- 4. Game 이 Proper & Respected 상태일 것

#### 5. 실제 운영에서의 고려사항

문제	해결 방법
오래된 게임 검색 비용	이진 검색 또는 캐싱/인덱싱 서비스 사용

게임 수가 매우 많음	오프체인 인덱서 (The Graph 등) 활용
게임이 무효화됨	해당 블록을 포함하는 다른 유효한 게임 선택
매우 오래된 출금	Anchor State Registry 에서 finalized 상태 확인

## 6. 권장 구현 패턴

```

async function findBestGameForWithdrawal(withdrawalTxHash) {
 // 1. 출금 트랜잭션의 L2 블록 번호 확인
 const receipt = await l2Provider.getTransactionReceipt(withdrawalTxHash);
 const withdrawalBlockNumber = receipt.blockNumber;

 // 2. 최신 게임부터 확인 (대부분의 경우 최신 게임이 적합)
 const latestGame = await findLatestGame();
 const latestGameL2Block = BigInt(latestGame.extraData.slice(0, 66));

 if (latestGameL2Block >= withdrawalBlockNumber) {
 // 최신 게임 사용 가능 - 가장 일반적인 케이스
 return latestGame;
 }

 // 3. 오래된 출금인 경우 - 적합한 게임 검색
 console.log('Searching for older game covering block:', withdrawalBlockNumber);

 // 3a. 먼저 인덱서/캐시 확인 (있는 경우)
 // 3b. 없으면 on-chain 검색
 return await findGameByBinarySearch(withdrawalBlockNumber);
}

```

## 7. 결론

항목	설명
extraData	각 DisputeGame 의 L2 블록 번호 저장
검색 방법	findLatestGames() 또는 gameAtIndex()로 역순 검색
효율화	이진 검색, 오프체인 인덱서, 캐싱 활용
실무 권장	대부분 최신 게임 사용 가능; 오래된 경우만 검색 필요

현재 코드베이스에는 특정 L2 블록을 위한 전용 검색 함수가 없으며, 필요시 위와 같은 로직을 직접 구현해야 합니다. 대부분의 경우 사용자가 출금 후 며칠 내에 증명을 제출하므로 최신 게임을 사용하면 됩니다.

## respectedGameType

respectedGameType 은 OptimismPortal2 가 출금 증명에 사용할 수 있는 유일한 GameType 입니다.

## 1. 정의와 역할

```
// packages/contracts-bedrock/src/dispute/AnchorStateRegistry.sol

/// @notice The respected game type.
GameType public respectedGameType;
```

항목	설명
저장 위치	AnchorStateRegistry 컨트랙트
설정 권한	Guardian 만 설정 가능
목적	출금에 사용할 수 있는 Dispute Game 타입 제한

## 2. 출금에서의 검증

proveWithdrawalTransaction 에서 DisputeGame 이 respectedGameType 인지 확인합니다:

```
// packages/contracts-bedrock/src/dispute/AnchorStateRegistry.sol

/// @notice Determines whether a game is of a respected game type.
/// @param _game The game to check.
/// @return Whether the game is of a respected game type.
function isGameRespected(IDisputeGame _game) public view returns (bool) {
 return _game.wasRespectedGameTypeWhenCreated();
}
...

`OptimismPortal2.proveWithdrawalTransaction`에서:
```solidity
// Game must have been respected game type when created.
if (!anchorStateRegistry.isGameRespected(disputeGameProxy)) {
    revert OptimismPortal_InvalidDisputeGame();
}
```

3. 왜 필요한가?

보안 및 운영상의 이유

1. 버전 관리
 - └ 새 VM 이나 증명 시스템 도입 시 단계적 전환 가능
2. 보안 강화
 - └ 취약한 GameType 발견 시 즉시 비활성화 가능
 - └ Guardian 이 respectedGameType 변경으로 안전한 타입만 허용

3. Permissioned → Permissionless 전환
 - └ Stage 0: PERMISSIONED_CANNON (ID=1) 사용
 - └ Stage 1: CANNON (ID=0)으로 전환

4. Guardian 에 의한 설정

```
// packages/contracts-bedrock/src/dispute/AnchorStateRegistry.sol

/// @notice Allows the Guardian to set the respected game type.
/// @param _gameType The new respected game type.
function setRespectedGameType(GameType _gameType) external {
    // Only the Guardian can set the respected game type.
    _assertOnlyGuardian();

    // Set the respected game type.
    respectedGameType = _gameType;
    emit RespectedGameTypeSet(_gameType);
}
```

5. 실제 네트워크의 respectedGameType

네트워크	일반적인 respectedGameType
OP Mainnet (Stage 1)	CANNON (0) 또는 PERMISSIONED_CANNON (1)
Testnet	네트워크 설정에 따라 다름
Interop 지원 체인	SUPER_CANNON (4) 또는 SUPER_PERMISSIONED_CANNON (5)

6. 코드에서 확인하는 방법

```
// JavaScript 로 respectedGameType 확인
const anchorStateRegistry = new ethers.Contract(
  ANCHOR_STATE_REGISTRY_ADDRESS,
  ['function respectedGameType() view returns (uint32)'],
  I1Provider
);

const respectedGameType = await anchorStateRegistry.respectedGameType();
console.log('Respected Game Type:', respectedGameType);
// 예: 0 = CANNON, 1 = PERMISSIONED_CANNON
```

요약

항목	내용
정의	출금에 사용 가능한 유일한 Dispute Game 타입

설정	Guardian 이 setRespectedGameType() 호출
검증	proveWithdrawalTransaction 에서 isGameRespected() 확인
목적	보안 관리, 버전 전환, Permissioned/Permissionless 제어

interop

op_stack 에서 상호운용성(Interoperability)는 서로 다른 L2 체인끼리 데이터를 매끄럽게 주고받고 자산을 이동할 수 있게 하는 기술 프레임워크를 말하는 데, Optimism 이 지향하는 슈퍼체인(Superchain)비전의 핵심요소로 여러 개의 독립적인 op_stack 체인들이 파편화되지 않고 통합된 생태계를 형성하도록 돕습니다.

설명

11. 핵심 목표

기존의 L2 간 통신은 이더리움 메인넷(L1)을 거쳐야 했기에 비용이 비싸고 시간이 오래 걸렸으나, Interop의 목표는 L1 을 거치지 않고 낮은 지연시간으로 체인간 메시지를 전달하여 사용자 경험을 개선하는 것이다.

12. 주요 구성 요소 및 작동 원리

OP Supervisor: 모든 Interop 클러스터 내 체인들의 로그 이벤트를 감시하고 데이터베이스화하는 새로운 서비스로서 특정 체인에서 발생한 메시지가 유효한지 검증하는 역할

3. 메시지 전달 방식

- Source chain(출발지) : 메시지를 시작하는 트랜잭션을 실행하고 로그(Initiating Message)를 남깁니다.
- Destination chain(목적지) : Cross2Inbox 라는 시스템 계약을 통해 출발지 체인의 메시지를 식별하고 실행(Execution Message)합니다.
- 보안 모델: Interop 은 '의존성세트 (Dependency Set)'를 통해 작동하며, 각 체인은 서로의 상태를 신뢰할 수 있는 메커니즘을 공유

4. 주요 이점

- 자산 이동의 효율화: ETH 나 ERC-20 토큰을 브릿지 과정 없이 네이티브하게 민팅(Mint) 및 소각(Burn)방식으로 이동시킬 수 있어 유동성 파편화 문제를 해결한다.
- 수평적 확장성: 하나의 앱이 여러 체인에 걸쳐 데이터를 구성할 수 있어, 트래픽이 몰릴 때 체인을 추가하는 방식으로 성능을 확장하기 용이함
- 통합된 사용자 경험: 사용자는 자신이 어떤 특정 L2 체인을 쓰고 있는 지 의식하지 않고도 슈퍼체인 내의 다양한 앱을 이용할 수 있다.

Identifier — 메시지 포인터

메시지를 참조하기 위한 구조체로, "어떤 체인의 어떤 블록 몇 번째 로그인지"를 정확히 가리킵니다.

```
// packages/contracts-bedrock/src/L2/CrossL2Inbox.sol
```

```
struct Identifier {  
    address origin;
```

```

uint256 blockNumber;
uint256 logIndex;
uint256 timestamp;
uint256 chainId;
}

```

메시지 전달 방식: Source → Destination

1. Source Chain: Initiating Message (로그를 남김)

Source chain 의 컨트랙트가 트랜잭션을 실행하면 EVM 로그(이벤트)를 남깁니다. 이 로그 자체가 Initiating Message 입니다. 별도의 “보내는” 컨트랙트는 없고, L2 블록에 기록된 모든 로그가 잠재적 메시지가 됩니다.

2. Destination Chain: CrossL2Inbox.validateMessage (Executing Message)

목적지 체인에서 누군가가 CrossL2Inbox.validateMessage 를 호출하면 메시지를 “실행”합니다.

```

// packages/contracts-bedrock/src/L2/CrossL2Inbox.sol

function validateMessage(Identifier calldata _id, bytes32 _msgHash) external {
    bytes32 checksum = calculateChecksum(_id, _msgHash);
    (bool isWarm,) = _isWarm(checksum);
    if (!isWarm) revert NotInAccessList();

    emit ExecutingMessage(_msgHash, _id);
}

```

핵심 보안 메커니즘 — Access List 위밍 확인:

트랜잭션의 EIP-2930 Access List 에 checksum 슬롯이 미리 포함되어 있어야만 sload 가스 비용이 낮아집니다(isWarm = true). op-node 가 트랜잭션 실행 전에 메시지 유효성을 미리 검증하고 Access List 를 구성합니다.

```

// packages/contracts-bedrock/src/L2/CrossL2Inbox.sol

function _isWarm(bytes32 _slot) internal view returns (bool isWarm_, uint256 value_) {
    assembly {
        let startGas := gas()
        value_ := sload(_slot)
        let endGas := gas()
        // If the gas cost of the 'sload' is below than the threshold, the slot is warm.
        isWarm_ := iszero(gt(sub(startGas, endGas), _WARM_READ_THRESHOLD))
    }
}

```

Checksum 계산 — 메시지 위·변조 방지:

```
// packages/contracts-bedrock/src/L2/CrossL2Inbox.sol

function calculateChecksum(Identifier memory _id, bytes32 _msgHash) public pure returns (bytes32 checksum_)
{
    // ...
    bytes32 logHash = keccak256(abi.encodePacked(_id.origin, _msgHash));
    // ...
    bytes32 idPacked = bytes32(abi.encodePacked(uint96(0), blockNumber, timestamp, logIndex));
    bytes32 idLogHash = keccak256(abi.encodePacked(logHash, idPacked));
    bytes32 bareChecksum = keccak256(abi.encodePacked(idLogHash, _id.chainId));
    checksum_ = (bareChecksum & _MSB_MASK) | _TYPE_3_MASK;
}
```

origin + msgHash + blockNumber + timestamp + logIndex + chainId 를 모두 묶어 해시하므로, 한 필드라도 틀리면 체크섬이 달라져 NotInAccessList 에러가 납니다.

op_supervisor

역할

op-supervisor 는 크로스체인 메시지 안전성을 모니터링하고 결정하는 서비스입니다. Optimism 의 네이티브 상호운용성(interoperability)을 위한 핵심 컴포넌트로, "superchain backend" 역할을 합니다.

목적

멀티체인 환경에서 크로스체인 메시지가 안전한지 빠르게 판단하여, 인터럽 트랜잭션의 유효성을 보장하고 체인 간 통신의 안전성을 유지합니다. Failsafe 기능을 통해 문제 발생 시 크로스체인 메시지 수락을 중단할 수도 있습니다.

주요 기능

- 블록 안전성 레벨 추적
 - unsafe: 낙관적으로 처리된 블록
 - safe: 유효한 의존성으로부터 재현 가능한 블록
 - finalized: 되돌릴 수 없는 유효한 의존성으로부터 재현 가능한 블록
- 크로스체인 의존성 검증
 - local unsafe → cross unsafe: L2 간 의존성 충족 확인
 - local safe → cross safe: DA(Data Availability) + L2 의존성 모두 충족
 - cross safe → finalized: 의존성이 되돌릴 수 없게 됨
- op-node 와의 상호작용

- Indexing Mode: op-node 의 derivation 프로세스 일부를 supervisor 가 제어. 각 체인당 최소 1 개의 indexing 노드 필요
- Following Mode: op-node 가 독립적으로 derivation 수행하면서 주기적으로 cross-heads 업데이트

데이터베이스 관리

- Log DB: 체인별 로그 이벤트 저장 (L2 의존성 정보)
- Local-safe DB: L2 블록이 어떤 L1 블록에서 derive 되었는지 저장
- Cross-safe DB: L2 블록이 언제 cross-safe 가 되었는지 저장

특징

1. 블록 안전성 레벨 (Safety Level)

```
// op-supervisor/supervisor/types/types.go
```

```
const (
    Finalized SafetyLevel = "finalized"
    CrossSafe  SafetyLevel = "safe"
    LocalSafe  SafetyLevel = "local-safe"
    CrossUnsafe SafetyLevel = "cross-unsafe"
    LocalUnsafe SafetyLevel = "unsafe"
```

레벨	의미
LocalUnsafe	시퀀서가 만든 최신 블록, 아직 아무것도 검증되지 않음
CrossUnsafe	L2 간 의존성이 충족된 것을 확인 (DA 미검증)
LocalSafe	L1 DA 에서 재현 가능, 단 cross-L2 의존성 미검증
CrossSafe	L1 DA + cross-L2 의존성 모두 충족 (RPC "safe")
Finalized	cross-safe 이고, 의존성이 finalized L1 에서만 유래

2. 데이터베이스 구조 — ChainsDB

세 가지 DB 가 ChainsDB 하나에 통합 관리됩니다.

```
// op-supervisor/supervisor/backend/db/db.go

// ChainsDB is a database that stores logs and derived-from data for multiple chains.
type ChainsDB struct {
    // unsafe info: the sequence of block seals and events
    logDBs locks.RWMap[eth.ChainID, LogStorage]

    // cross-unsafe: how far we have processed the unsafe data.
    crossUnsafe locks.RWMap[eth.ChainID, *locks.RWValue[types.BlockSeal]]
```

```
// local-safe: index of what we optimistically know about L2 blocks being derived from L1
localDBs locks.RWMap[eth.ChainID, DerivationStorage]

// cross-safe: index of L2 blocks we know to only have cross-L2 valid dependencies
crossDBs locks.RWMap[eth.ChainID, DerivationStorage]

// finalized: the L1 finality progress.
finalizedL1 locks.RWValue[eth.L1BlockRef]
```

필드	역할
logDBs	Log DB: 체인별 로그(이벤트) 해시 저장. Executing Message 포함.
localDBs	Local-safe DB: L2 블록이 어떤 L1 블록에서 derive 되었는지
crossDBs	Cross-safe DB: L2 블록이 언제 cross-safe 가 되었는지
crossUnsafe	Cross-unsafe 헤드 추적
finalizedL1	L1 finality로부터 L2 finality 계산

3. Executing Message 디코딩 — Log DB 로 저장

op-supervisor 의 LogProcessor 가 블록의 모든 로그를 순회하면서, CrossL2Inbox 주소에서 발생한 로그만 Executing Message 로 디코딩합니다.

```
// op-supervisor/supervisor/backend/processors/executing_message.go

func MessageFromLog(l *ethTypes.Log) (*types.Message, error) {
    if l.Address != params.InteropCrossL2InboxAddress {
        return nil, nil // CrossL2Inbox 주소가 아니면 무시
    }
    if len(l.Topics) != 2 { // topics: event-id and payload-hash
        return nil, nil
    }
    if l.Topics[0] != types.ExecutingMessageEventTopic {
        return nil, nil
    }
    var msg types.Message
    if err := msg.DecodeEvent(l.Topics, l.Data); err != nil {
        return nil, fmt.Errorf("invalid executing message: %w", err)
    }
    return &msg, nil
}
```

4. 의존성 세트 (Dependency Set) & Link 검사

각 체인이 다른 체인의 메시지를 받아들일 수 있는지 판단하는 규칙입니다.

```
// op-supervisor/supervisor/backend/depset/links.go

func (lc *LinkCheckerImpl) CanExecute(execlnChain eth.ChainID,
    execlnTimestamp uint64, initChainID eth.ChainID, initTimestamp uint64) bool {
    // 두 체인이 모두 dependency set 에 속하는지
    if !lc.cfg.HasChain(execlnChain) {
        return false
    }
    if !lc.cfg.HasChain(initChainID) {
        return false
    }
    // 실행 체인이 interop 활성화 상태인지
    if !lc.cfg.IsInterop(execlnChain, execlnTimestamp) {
        return false
    }
    // 메시지가 미래에서 온 것은 불가 (타임스탬프 순서 보장)
    if initTimestamp > execlnTimestamp {
        return false
    }
    // 메시지 만료 시간 체크
    expiresAt := safemath.SaturatingAdd(initTimestamp, lc.cfg.MessageExpiryWindow())
    if expiresAt < execlnTimestamp {
        return false
    }
    return true
}
```

5 가지 거절 조건:

- 1) 두 체인 중 하나라도 dependency set 밖에 있으면 거절
- 2) 실행 체인이 아직 interop 미활성화면 거절
- 3) Interop 활성화 첫 블록에서는 거절
- 4) 메시지 타임스탬프가 실행 타임스탬프보다 미래면 거절 (순서 역전 방지)
- 5) 메시지가 만료 시간을 초과하면 거절

5. Cross-safe 검증 파이프라인

LocalSafe → CrossSafe 승격의 핵심 로직으로, 세 단계를 순서대로 통과해야 합니다.

```
// op-supervisor/supervisor/backend/cross/safe_update.go

func scopedCrossSafeUpdate(h reads.Handle, logger log.Logger, chainID eth.ChainID, d CrossSafeDeps, linker
    depset.LinkChecker) (update types.DerivedBlockRefPair, err error) {
    candidate, err := d.CandidateCrossSafe(chainID) // 1. 승격 후보 블록 선택
    // ...
}
```

```

// 2. HazardSet 구성: 이 블록이 의존하는 모든 블록 수집
hazards, err := CrossSafeHazards(d, linker, logger, chainID, candidate.Source.ID(),
types.BlockSealFromRef(candidate.Derived))
// ...

// 3. Frontier 체크: 의존 블록들이 현재 L1 범위 내에 있는지
if err := HazardSafeFrontierChecks(d, candidate.Source.ID(), hazards); err != nil {
    return candidate, ...
}

// 4. 사이클 체크: 순환 의존성이 없는지 (DAG 위상정렬)
if err := HazardCycleChecks(d, candidate.Derived.Time, hazards); err != nil {
    return candidate, ...
}

// 5. 모두 통과하면 cross-safe 로 승격
if err := d.UpdateCrossSafe(chainID, candidate.Source, candidate.Derived); err != nil {
    return candidate, ...
}
return candidate, nil
}

```

HazardSet 구성: 재귀적 의존성 추적

```

// op-supervisor/supervisor/backend/cross/hazard_set.go
func (h *HazardSet) build(deps HazardDeps, linker depset.LinkChecker, logger log.Logger, chainID eth.ChainID,
block types.BlockSeal) error {
    stack := []potentialHazard{{chainID: chainID, block: block}}

    for len(stack) > 0 {
        // ...
        // 블록 내 모든 Executing Message 를 열어서 확인
        opened, _, execMsgs, err := deps.OpenBlock(destChainID, candidate.Number)
        // ...

        for _, msg := range execMsgs {
            // Log DB 에서 initiating log 가 실제로 존재하는지 확인
            includedIn, err := deps.Contains(msg.ChainID, q)
            // ...

            if msg.Timestamp < candidate.Timestamp {
                // 과거 메시지: 해당 블록이 이미 cross-valid 인지만 확인
                h.checkMessageWithOlderTimestamp(...)
            } else if msg.Timestamp == candidate.Timestamp {
                // 같은 타임스탬프: 해당 체인 블록도 hazard set 에 추가하고 재귀 탐색
                h.entries[msg.ChainID] = includedIn
            }
        }
    }
}

```

```

        stack = append(stack, potentialHazard{chainID: msg.ChainID, block: includedIn})
    } else {
        // 미래 메시지 참조: 타임스탬프 불변식 위반
        return fmt.Errorf("... breaks timestamp invariant: %w", types.ErrConflict)
    }
}
}
}
return nil
}

```

전체 흐름 요약 (코드 레벨)

[Source Chain 트랜잭션]

- └ EVM 로그 발생 (Initiating Message)

[Destination Chain 트랜잭션]

- └ CrossL2Inbox.validateMessage(_id, _msgHash)
 - └ calculateChecksum() → 체크섬 계산
 - └ _isWarm() → Access List 검증 (EIP-2930)
- └ emit ExecutingMessage(_msgHash, _id)

[op-supervisor: 블록 수신]

- └ LogProcessor.ProcessLogs()
 - └ MessageFromLog() → CrossL2Inbox 로그 디코딩
 - └ Log DB 에 저장

[op-supervisor: Cross-safe 승격]

- └ CrossSafeUpdate()
 - └ CandidateCrossSafe() → 후보 블록 선택
 - └ HazardSet.build() → 의존 블록 재귀 수집
 - └ LinkChecker.CanExecute() → 체인/타임스탬프 규칙 확인
 - └ HazardSafeFrontierChecks() → L1 범위 내 확인
 - └ HazardCycleChecks() → DAG 위상정렬로 사이클 방지
 - └ UpdateCrossSafe() → Cross-safe DB 업데이트

Anvil (L1) + OpStack (L2) installation

Setup

folder structure

```
rollup/
├─
├─── deployer/      # op-deployer files and contracts
├─
├─── sequencer/     # op-geth and op-node
├─
├─── batcher/       # op-batcher configuration
├─
├─── proposer/      # op-proposer setup
├─
└─── challenger/    # op-challenger files
```

use external docker network

```
docker network create op-stack
```

anvil keys

```
docker logs anvil -f
```

```

      _ _
     ( ) | |
  _ _ _ _ _ _ _ _ | |
 / _ | | _ W W W / / | | |
 | ( | | | | W V / | | |
 W _ | | | | W / | | |
```

1.5.1-stable (b0a9dd9ced 2025-12-22T11:38:41.548294378Z)

<https://github.com/foundry-rs/foundry>

Available Accounts

=====

- (0) 0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266 (1000000.000000000000000000 ETH)
- (1) 0x70997970C51812dc3A010C7d01b50e0d17dc79C8 (1000000.000000000000000000 ETH)
- (2) 0x3C44CdDdB6a900fa2b585dd299e03d12FA4293BC (1000000.000000000000000000 ETH)

(3) 0x90F79bf6EB2c4f870365E785982E1f101E93b906 (1000000.000000000000000000 ETH)
(4) 0x15d34AAf54267DB7D7c367839AAf71A00a2C6A65 (1000000.000000000000000000 ETH)
(5) 0x9965507D1a55bcC2695C58ba16FB37d819B0A4dc (1000000.000000000000000000 ETH)
(6) 0x976EA74026E726554dB657fA54763abd0C3a0aa9 (1000000.000000000000000000 ETH)
(7) 0x14dC79964da2C08b23698B3D3cc7Ca32193d9955 (1000000.000000000000000000 ETH)
(8) 0x23618e81E3f5cdF7f54C3d65f7FBc0aBf5B21E8f (1000000.000000000000000000 ETH)
(9) 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 (1000000.000000000000000000 ETH)

Private Keys

=====

(0) 0xac0974bec39a17e36ba4a6b4d238ff944bacb478cbed5efcae784d7bf4f2ff80
(1) 0x59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
(2) 0x5de4111afa1a4b94908f83103eb1f1706367c2e68ca870fc3fb9a804cdab365a
(3) 0x7c852118294e51e653712a81e05800f419141751be58f605c371e15141b007a6
(4) 0x47e179ec197488593b187f80a00eb0da91f1b9d0b13f8733639f19c30a34926a
(5) 0x8b3a350cf5c34c9194ca85829a2df0ec3153be0318b5e2d3348e872092edffba
(6) 0x92db14e403b83dfe3df233f83dfa3a0d7096f21ca9b0d6d6b8d88b2b4ec1564e
(7) 0x4bbbf85ce3377467afe5d46f804f221813b2bb87f24d81f60f1fcd6f7cbf4356
(8) 0xdbda1821b80551c9d65939329250298aa3472ba22feea921c0cf5d620ea67b97
(9) 0x2a871d0798f97d79848a013d4936a73bf4cc922c825d33c1cf7073dff6d409c6

Wallet

=====

Mnemonic: test test test test test test test test test test test junk

Derivation path: m/44'/60'/0'/0/

Chain ID

=====

11155111

Base Fee

=====

1000000000

Gas Limit

=====

30000000

Genesis Timestamp

=====

```
1770789499
```

Genesis Number

```
=====
```

```
0
```

Listening on 0.0.0.0:8545

Block Number: 1

Block Hash: 0xa7978d259de8fa4e37bbd5fe6bcb35f84e34cf5c155d65ae881c1a7b4f5a6cdf

Block Time: "Wed, 11 Feb 2026 05:58:31 +0000"

op-deployer

op-deployer version

```
./bin/op-deployer --version  
op-deployer version 0.4.7-21a33f07-2025-12-08T21:53:13Z
```

download

link: <https://github.com/ethereum-optimism/optimism/releases/tag/op-deployer%2Fv0.4.7>

workspace

```
cd /rollup/deployer
```

Step 1. .env file

```
DEPLOYER_L1_CHAIN_ID=11155111  
DEPLOYER_L2_CHAIN_IDS=111  
DEPLOYER_WORKDIR=.deployer  
DEPLOYER_INTENT_TYPE=standard-overrides  
L1_RPC_URL=http://127.0.0.1:8545  
DEPLOYER_L1_RPC_URL=http://127.0.0.1:8545  
PRIVATE_KEY=ac0974bec39a17e36ba4a6b4d238ff944bacb478cbcd5efcae784d7bf4f2ff80  
DEPLOYER_PRIVATE_KEY=ac0974bec39a17e36ba4a6b4d238ff944bacb478cbcd5efcae784d7bf4f2ff80  
DEPLOYER_ADDRESS=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
```

init

```
source .env
```

Step 2. init deployer

command

```
./op-deployer init ₩
--l1-chain-id 11155111 ₩
--l2-chain-ids 111 ₩
--workdir .deployer ₩
--intent-type standard-overrides
```

output

Successfully initialized op-deployer intent **in** directory: **.deployer**

result

- **.deployer/intent.toml**

```
configType = "standard-overrides"
l1ChainID = 11155111
opcmAddress = "0x3bb6437aba031afbf9cb3538fa064161e2bf2d78"
fundDevAccounts = false
l1ContractsLocator = "embedded"
l2ContractsLocator = "embedded"

[[chains]]
  id = "0x0000000000000000000000000000000000000000000000000000000000000006f"
  baseFeeVaultRecipient = "0x000000000000000000000000000000000000000000000000"
  l1FeeVaultRecipient = "0x000000000000000000000000000000000000000000000000"
  sequencerFeeVaultRecipient = "0x000000000000000000000000000000000000000000000"
  eip1559DenominatorCanyon = 250
  eip1559Denominator = 50
  eip1559Elasticity = 6
  gasLimit = 60000000
  operatorFeeScalar = 0
  operatorFeeConstant = 0
  [chains.roles]
    l1ProxyAdminOwner = "0x1eb2ffc903729a0f03966b917003800b145f56e2"
    l2ProxyAdminOwner = "0x2fc3ffc903729a0f03966b917003800b145f67f3"
    systemConfigOwner = "0x000000000000000000000000000000000000000000000000"
    unsafeBlockSigner = "0x000000000000000000000000000000000000000000000000"
    batcher = "0x000000000000000000000000000000000000000000000000"
    proposer = "0x000000000000000000000000000000000000000000000000"
    challenger = "0xfd1d2e729ae8eee2e146c033bf4400fe75284301"
```

- **.deployer/state.json**

```
{
  "version": 1,
  "create2Salt": "0x0000000000000000000000000000000000000000000000000000000000000000",
  "appliedIntent": null,
```

```

"prestateManifest": null,
"superchainContracts": null,
"superchainRoles": null,
"implementationsDeployment": null,
"opChainDeployments": null,
"l1StateDump": null,
"DeploymentCalldata": null
}

```

Step 3. bootstrap deployer

proxy

- command

```

./op-deployer bootstrap proxy ₩
--l1-rpc-url $L1_RPC_URL ₩
--private-key $DEPLOYER_PRIVATE_KEY ₩
--proxy-owner $DEPLOYER_ADDRESS ₩
--outfile ./deployer/deployments/11155111.proxy.json

```

- output

```

INFO [01-13|15:26:39.750] Initialized path database      readonly=true cache=0.00B buffer=0.00B
history=0
INFO [01-13|15:26:39.781] transaction broadcasted        id=fee033..cbe805 nonce=0
INFO [01-13|15:26:39.782] Publishing transaction          service=transactor tx=7ab76e..bce1fb nonce=0
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=629,678
INFO [01-13|15:26:39.782] Transaction successfully published service=transactor tx=7ab76e..bce1fb
nonce=0 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=629,678 tx=7ab76e..bce1fb
INFO [01-13|15:26:47.787] Transaction confirmed          service=transactor tx=7ab76e..bce1fb
block=b7ab53..181a74:218 effectiveGasPrice=5,000,000,007
INFO [01-13|15:26:47.787] transaction confirmed          id=fee033..cbe805 completed=1 total=1
hash=0x7ab76e219f655320e41591b27331f2dfbcc92518c56bfc6bb672e12036bce1fb nonce=0
creation=0x5FbDB2315678afecb367f032d93f642f64180aa3
INFO [01-13|15:26:47.787] deployed new ERC-1967 proxy

```

- result

deployments/11155111.proxy.json file

```

{
  "Proxy": "0x5fbdb2315678afecb367f032d93f642f64180aa3"
}

```

superchain

- command

```

./op-deployer bootstrap superchain ₩
--l1-rpc-url $L1_RPC_URL ₩
--private-key $DEPLOYER_PRIVATE_KEY ₩
--superchain-proxy-admin-owner $DEPLOYER_ADDRESS ₩

```

```
--protocol-versions-owner    $DEPLOYER_ADDRESS ₩
--guardian                   $DEPLOYER_ADDRESS ₩
--outfile ./deployer/deployments/11155111.superchain.json
```

- output

```
INFO [01-13|15:29:29.193] Initialized path database      readonly=true cache=0.00B buffer=0.00B
history=0
WARN [01-13|15:29:29.360] Fault
addr=0x0165878A594ca255338adfa4d48449f69242Eb8F label=ProtocolVersionsProxy err="execution reverted"
depth=3 op=253 byte4=0x38d38c97
WARN [01-13|15:29:29.360] callframe                      depth=1 byte4=0xff07a313
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=DeploySuperchain
WARN [01-13|15:29:29.360] callframe                      depth=2 byte4=0x38d38c97
addr=0x0165878A594ca255338adfa4d48449f69242Eb8F callsite= label=ProtocolVersionsProxy
WARN [01-13|15:29:29.360] callframe                      depth=3 byte4=0x38d38c97
addr=0x0165878A594ca255338adfa4d48449f69242Eb8F callsite= label=ProtocolVersionsProxy
WARN [01-13|15:29:29.360] Revert
addr=0x0165878A594ca255338adfa4d48449f69242Eb8F label=ProtocolVersionsProxy err="execution reverted"
revertData=0x depth=2
WARN [01-13|15:29:29.360] Fault
addr=0x0165878A594ca255338adfa4d48449f69242Eb8F label=ProtocolVersionsProxy err="execution reverted"
depth=2 op=253 byte4=0x38d38c97
WARN [01-13|15:29:29.360] callframe                      depth=1 byte4=0xff07a313
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=DeploySuperchain
WARN [01-13|15:29:29.360] callframe                      depth=2 byte4=0x38d38c97
addr=0x0165878A594ca255338adfa4d48449f69242Eb8F callsite= label=ProtocolVersionsProxy
WARN [01-13|15:29:29.360] Revert
addr=0x0165878A594ca255338adfa4d48449f69242Eb8F label=ProtocolVersionsProxy err="execution reverted"
revertData=0x depth=1
INFO [01-13|15:29:29.363] transaction broadcasted        id=85d742..ec1f3c nonce=1
INFO [01-13|15:29:29.363] Publishing transaction         service=transactor tx=6e1130..ed62d5 nonce=1
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=1,780,244
INFO [01-13|15:29:29.364] Transaction successfully published service=transactor tx=6e1130..ed62d5
nonce=1 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=1,780,244 tx=6e1130..ed62d5
INFO [01-13|15:29:29.364] transaction broadcasted        id=2c9450..82b775 nonce=0
INFO [01-13|15:29:29.364] Publishing transaction         service=transactor tx=a67016..88502f nonce=2
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=1,034,708
INFO [01-13|15:29:29.365] Transaction successfully published service=transactor tx=a67016..88502f
nonce=2 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=1,034,708 tx=a67016..88502f
INFO [01-13|15:29:29.365] transaction broadcasted        id=ad0144..8a434b nonce=0
INFO [01-13|15:29:29.365] Publishing transaction         service=transactor tx=fcb911..d3ce38 nonce=3
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=755,742
INFO [01-13|15:29:29.366] Transaction successfully published service=transactor tx=fcb911..d3ce38
nonce=3 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=755,742 tx=fcb911..d3ce38
INFO [01-13|15:29:29.366] transaction broadcasted        id=c60321..75800e nonce=4
```

INFO [01-13|15:29:29.366] Publishing transaction service=transactor tx=2d9c9f..d7dac9 nonce=4
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=629,678

INFO [01-13|15:29:29.367] Transaction successfully published service=transactor tx=2d9c9f..d7dac9
nonce=4 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=629,678 tx=2d9c9f..d7dac9

INFO [01-13|15:29:29.367] transaction broadcasted id=c5e641..801594 nonce=5

INFO [01-13|15:29:29.367] Publishing transaction service=transactor tx=d8025b..514bdf nonce=5
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=104,856

INFO [01-13|15:29:29.367] Transaction successfully published service=transactor tx=d8025b..514bdf
nonce=5 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=104,856 tx=d8025b..514bdf

INFO [01-13|15:29:29.368] transaction broadcasted id=a63569..c492a0 nonce=6

INFO [01-13|15:29:29.368] Publishing transaction service=transactor tx=05e83b..2677ea nonce=6
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=629,678

INFO [01-13|15:29:29.368] Transaction successfully published service=transactor tx=05e83b..2677ea
nonce=6 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=629,678 tx=05e83b..2677ea

INFO [01-13|15:29:29.369] transaction broadcasted id=983ea7..6bea37 nonce=7

INFO [01-13|15:29:29.369] Publishing transaction service=transactor tx=652233..591664 nonce=7
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=193,480

INFO [01-13|15:29:29.369] Transaction successfully published service=transactor tx=652233..591664
nonce=7 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=193,480 tx=652233..591664

INFO [01-13|15:29:29.370] transaction broadcasted id=a55877..b87048 nonce=8

INFO [01-13|15:29:29.370] Publishing transaction service=transactor tx=4f7824..a82b4b nonce=8
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=30914

INFO [01-13|15:29:29.370] Transaction successfully published service=transactor tx=4f7824..a82b4b
nonce=8 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=30914 tx=4f7824..a82b4b

INFO [01-13|15:29:37.368] Transaction confirmed service=transactor tx=a67016..88502f
block=e62d02..808597:235 effectiveGasPrice=5,000,000,007

INFO [01-13|15:29:37.368] Transaction confirmed service=transactor tx=2d9c9f..d7dac9
block=e62d02..808597:235 effectiveGasPrice=5,000,000,007

INFO [01-13|15:29:37.368] Transaction confirmed service=transactor tx=6e1130..ed62d5
block=e62d02..808597:235 effectiveGasPrice=5,000,000,007

INFO [01-13|15:29:37.368] transaction confirmed id=85d742..ec1f3c completed=1 total=8
hash=0x6e113035f9a76bf4350a68dd946d88bcc08c436478ad9472c4a72ea550ed62d5 nonce=1
creation=0xe7f1725E7734CE288F8367e1Bb143E90bb3F0512

INFO [01-13|15:29:37.368] transaction confirmed id=2c9450..82b775 completed=2 total=8
hash=0xa67016cccd09605b41ee2bb57dd96c7a4314935d80d32f7e9e761a27bfe88502f nonce=0
creation=0x00

INFO [01-13|15:29:37.368] Transaction confirmed service=transactor tx=fcb911..d3ce38
block=e62d02..808597:235 effectiveGasPrice=5,000,000,007

INFO [01-13|15:29:37.368] transaction confirmed id=ad0144..8a434b completed=3 total=8
hash=0xfcb91105f3f93fdd8904280bc2f79b8d310b60311c7ad353e4066ad123d3ce38 nonce=0
creation=0x00

INFO [01-13|15:29:37.368] transaction confirmed id=c60321..75800e completed=4 total=8
hash=0x2d9c9fa935f945078e6b52354698b088801dd0a121dc4457dddc0b2519d7dac9 nonce=4
creation=0xDc64a140Aa3E981100a9becA4E685f962f0cF6C9

```

INFO [01-13|15:29:37.369] Transaction confirmed          service=transactor tx=d8025b..514bdf
block=e62d02..808597:235 effectiveGasPrice=5,000,000,007
INFO [01-13|15:29:37.369] transaction confirmed          id=c5e641..801594 completed=5 total=8
hash=0xd8025ba160dff213d5779a60cb7ae325eb1ff7b9fa1f88f1bae3d8595f514bdf nonce=5
creation=0x0000000000000000000000000000000000000000000000000000000000000000
INFO [01-13|15:29:37.369] Transaction confirmed          service=transactor tx=05e83b..2677ea
block=e62d02..808597:235 effectiveGasPrice=5,000,000,007
INFO [01-13|15:29:37.370] transaction confirmed          id=a63569..c492a0 completed=6 total=8
hash=0x05e83b643869bdb1c722634ee2ab1830b783802c97ecc6b5b70a6782bc2677ea nonce=6
creation=0x0165878A594ca255338adfa4d48449f69242Eb8F
INFO [01-13|15:29:37.370] Transaction confirmed          service=transactor tx=652233..591664
block=e62d02..808597:235 effectiveGasPrice=5,000,000,007
INFO [01-13|15:29:37.371] transaction confirmed          id=983ea7..6bea37 completed=7 total=8
hash=0x6522330e9320cd6e9dd74267884c4b86060bd8eaf5bfd66778080e9b9b591664 nonce=7
creation=0x0000000000000000000000000000000000000000000000000000000000000000
INFO [01-13|15:29:37.371] Transaction confirmed          service=transactor tx=4f7824..a82b4b
block=e62d02..808597:235 effectiveGasPrice=5,000,000,007
INFO [01-13|15:29:37.371] transaction confirmed          id=a55877..b87048 completed=8 total=8
hash=0x4f78246ae757d854d9b02e35035a314964915ab43ddd045c61e9a43cfca82b4b nonce=8
creation=0x0000000000000000000000000000000000000000000000000000000000000000
INFO [01-13|15:29:37.371] deployed superchain configuration

```

- result

deployments/11155111.superchain.json file

```

{
  "protocolVersionsImplAddress": "0x37e15e4d6dffa9e5e320ee1ec036922e563cb76c",
  "protocolVersionsProxyAddress": "0x0165878a594ca255338adfa4d48449f69242eb8f",
  "superchainConfigImplAddress": "0xce28685eb204186b557133766eca00334eb441e4",
  "superchainConfigProxyAddress": "0xdc64a140aa3e981100a9beca4e685f962f0cf6c9",
  "proxyAdminAddress": "0xe7f1725e7734ce288f8367e1bb143e90bb3f0512"
}

```

implementations

- load variables

```
export $(jq -r 'to_entries | map("\W(.key)=\W(.value)") | .[]' ./deployer/deployments/11155111.superchain.json)
```

- command

```

./op-deployer bootstrap implementations \W
--l1-rpc-url $L1_RPC_URL \W
--private-key $DEPLOYER_PRIVATE_KEY \W
--superchain-config-proxy $superchainConfigProxyAddress \W
--protocol-versions-proxy $protocolVersionsProxyAddress \W
--upgrade-controller $proxyAdminAddress \W
--superchain-proxy-admin $proxyAdminAddress \W
--challenger $DEPLOYER_ADDRESS \W
--outfile ./deployer/deployments/11155111.impl.json

```

- output

```
INFO [01-13|15:31:16.110] Initialized path database          readonly=true cache=0.00B buffer=0.00B
history=0
INFO [01-13|15:31:16.580] Running chain assertions on the DelayedWETH implementation at
0x33Dadc2d1aA9BB613A7AE6B28425eA00D44c6998 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.580] Running chain assertions on the DisputeGameFactory implementation at
0x33D1e8571a85a538ed3D5A4d88f46C112383439D sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.581] Running chain assertions on the L1CrossDomainMessenger implementation at
0x22D12E0FAebD62d429514A65EBAe32dd316c12D6 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
WARN [01-13|15:31:16.582] Fault
addr=0x22D12E0FAebD62d429514A65EBAe32dd316c12D6 label=L1CrossDomainMessengerImpl err="execution
reverted" depth=2 op=253 byte4=0x3e47158c
WARN [01-13|15:31:16.582] callframe                                depth=1 byte4=0xd194827c
addr=0xfdaF657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=DeployImplementations
WARN [01-13|15:31:16.582] callframe                                depth=2 byte4=0x3e47158c
addr=0x22D12E0FAebD62d429514A65EBAe32dd316c12D6 callsite= label=L1CrossDomainMessengerImpl
WARN [01-13|15:31:16.582] Revert
addr=0x22D12E0FAebD62d429514A65EBAe32dd316c12D6 label=L1CrossDomainMessengerImpl err="execution
reverted" revertData=0x54e433cd depth=1
INFO [01-13|15:31:16.582] Running chain assertions on the L1ERC721Bridge implementation at
0x7f1d12fB2911EB095278085f721e644C1f675696 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
WARN [01-13|15:31:16.583] Fault
addr=0x7f1d12fB2911EB095278085f721e644C1f675696 label=L1ERC721BridgeImpl err="execution
reverted" depth=2 op=253 byte4=0x3e47158c
WARN [01-13|15:31:16.583] callframe                                depth=1 byte4=0xd194827c
addr=0xfdaF657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=DeployImplementations
WARN [01-13|15:31:16.583] callframe                                depth=2 byte4=0x3e47158c
addr=0x7f1d12fB2911EB095278085f721e644C1f675696 callsite= label=L1ERC721BridgeImpl
WARN [01-13|15:31:16.583] Revert
addr=0x7f1d12fB2911EB095278085f721e644C1f675696 label=L1ERC721BridgeImpl err="execution
reverted" revertData=0x54e433cd depth=1
INFO [01-13|15:31:16.583] Running chain assertions on the L1StandardBridge implementation at
0xe32B192fb1DcA88fCB1C56B3ACb429e32238aDCb sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.584] Running chain assertions on the MIPS at
0x07BAbE08EE4D07dBA236530183B24055535A7011 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.584] Running chain assertions on the OPContractsManager at
0x5cbe1A8ED6E4Cb9e36bB831dD26C0d9416e78CEb sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.597] Running chain assertions on the OptimismMintableERC20Factory implementation at
0x5493f4677A186f64805fe7317D6993ba4863988F sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.598] Running chain assertions on the OptimismPortal2 implementation at
0x381E729FF983FA4BCed820e7b922d79bF653B999 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.599] Running chain assertions on the ETHLockbox implementation at
0x784d2F03593A42A6E4676A012762F18775ecbBe6 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.599] Running chain assertions on the SystemConfig implementation at
```

```

0x2bFE4A5Bd5A41e9d848d843ebCDFa15954e9A557 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:31:16.605] transaction broadcasted          id=f96f6b..f8939a nonce=0
INFO [01-13|15:31:16.605] Publishing transaction            service=transactor tx=7167fe..266880 nonce=9
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=3,177,256
INFO [01-13|15:31:16.606] Transaction successfully published  service=transactor tx=7167fe..266880
nonce=9 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=3,177,256 tx=7167fe..266880
.....

```

- result

deployments/11155111.impl.json file

```

{
  "opcmAddress": "0x5cbe1a8ed6e4cb9e36bb831dd26c0d9416e78ceb",
  "opcmContractsContainerAddress": "0x10596fe51861c5d3865568278270b1df050ee547",
  "opcmGameTypeAdderAddress": "0x721bb337577cb1d5ce8f546dbd8f2e23d2e0b251",
  "opcmDeployerAddress": "0x5437b18b3be1e70737731fce2ee76fc2329dae3a",
  "opcmUpgraderAddress": "0xf0b65d6a9f2b96d5692e22e0d3a5b143b7b98152",
  "opcmInteropMigratorAddress": "0xf50ee8e05c7fd2d2b054469898b6ad64d7dc3966",
  "opcmStandardValidatorAddress": "0x91d5c73b3994d0a285e0c0c27287e720ddd2c54b",
  "delayedWETHImplAddress": "0x33dadcd21aa9bb613a7ae6b28425ea00d44c6998",
  "optimismPortalImplAddress": "0x381e729ff983fa4bcd820e7b922d79bf653b999",
  "optimismPortalInteropImplAddress": "0xb0eb854fd842e0e564d49d2fe6b2ac25d035523c",
  "ethLockboxImplAddress": "0x784d2f03593a42a6e4676a012762f18775ecbbe6",
  "preimageOracleSingletonAddress": "0x1fb8cdfc6831fc866ed9c51af8817da5c287add3",
  "mipsSingletonAddress": "0x07babe08ee4d07dba236530183b24055535a7011",
  "systemConfigImplAddress": "0x2bfe4a5bd5a41e9d848d843ebcdfa15954e9a557",
  "I1CrossDomainMessengerImplAddress": "0x22d12e0faebd62d429514a65ebae32dd316c12d6",
  "I1ERC721BridgeImplAddress": "0x7f1d12fb2911eb095278085f721e644c1f675696",
  "I1StandardBridgeImplAddress": "0xe32b192fb1dca88fcb1c56b3acb429e32238adcb",
  "optimismMintableERC20FactoryImplAddress": "0x5493f4677a186f64805fe7317d6993ba4863988f",
  "disputeGameFactoryImplAddress": "0x33d1e8571a85a538ed3d5a4d88f46c112383439d",
  "anchorStateRegistryImplAddress": "0xeb69cc681e8d4a557b30dffbad85affd47a2cf2e",
  "superchainConfigImplAddress": "0xce28685eb204186b557133766eca00334eb441e4",
  "protocolVersionsImplAddress": "0x37e15e4d6dfa9e5e320ee1ec036922e563cb76c"
}

```

Step 4. update the intent.json

intent.json file

```

configType = "standard-overrides"
I1ChainID = 11155111
opcmAddress = "0x5cbe1a8ed6e4cb9e36bb831dd26c0d9416e78ceb"
fundDevAccounts = false
I1ContractsLocator = "embedded"
I2ContractsLocator = "embedded"

```

```
[[chains]]
  id = "0x000000000000000000000000000000000000000000000000000000000000006f"
  baseFeeVaultRecipient = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  l1FeeVaultRecipient = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  sequencerFeeVaultRecipient = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  eip1559DenominatorCanyon = 250
  eip1559Denominator = 50
  eip1559Elasticity = 6
  gasLimit = 60000000
  operatorFeeScalar = 0
  operatorFeeConstant = 0
[chains.roles]
  l1ProxyAdminOwner = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  l2ProxyAdminOwner = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  systemConfigOwner = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  unsafeBlockSigner = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  batcher = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  proposer = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
  challenger = "0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266"
```

Step 5. apply deployer

command

```
./op-deployer apply ₩
--workdir .deployer ₩
--l1-rpc-url $L1_RPC_URL ₩
--private-key $DEPLOYER_PRIVATE_KEY
```

output

```
INFO [01-13|15:50:18.692] Initialized path database          readonly=true cache=0.00B buffer=0.00B
history=0
INFO [01-13|15:50:18.832] initializing pipeline              stage=init strategy=live
INFO [01-13|15:50:18.859] superchain deployment not needed   stage=deploy-superchain
INFO [01-13|15:50:18.860] implementations deployment not needed stage=deploy-implementations
INFO [01-13|15:50:18.860] deploying OP chain using local allocs stage=deploy-opchain
id=0x000000000000000000000000000000000000000000000000000000000000006f
INFO [01-13|15:50:18.945] Running chain assertions on the DisputeGameFactory proxy at
0x8E1E505274F5054200B3D26d7EBc75722038cE8f sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:50:18.945] Running chain assertions on the L1CrossDomainMessenger proxy at
0x65BaD6D5A135448E224A8Ae96733F0A49c2563f3 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
WARN [01-13|15:50:18.945] Fault
addr=0x1903e2aD975aA384d3458F9eE2e88810972fc97a label=optimismMintableERC20FactoryProxy
err="execution reverted" depth=3 op=253 byte4=0x38d38c97
WARN [01-13|15:50:18.945] callframe                          depth=1 byte4=0xfc4dcacb
addr=0x3dbCf1D6840A85cfaa89395c0626D7a4FB2D1D47 callsite= label=DeployOPChain
```

```

WARN [01-13|15:50:18.945] callframe                depth=2 byte4=0x38d38c97
addr=0x1903e2aD975aA384d3458F9eE2e88810972fc97a callsite= label=optimismMintableERC20FactoryProxy
WARN [01-13|15:50:18.945] callframe                depth=3 byte4=0x38d38c97
addr=0x1903e2aD975aA384d3458F9eE2e88810972fc97a callsite= label=optimismMintableERC20FactoryProxy
WARN [01-13|15:50:18.945] Revert
addr=0x1903e2aD975aA384d3458F9eE2e88810972fc97a label=optimismMintableERC20FactoryProxy
err="execution reverted" revertData=0x depth=2
WARN [01-13|15:50:18.945] Fault
addr=0x1903e2aD975aA384d3458F9eE2e88810972fc97a label=optimismMintableERC20FactoryProxy
err="execution reverted" depth=2 op=253 byte4=0x38d38c97
WARN [01-13|15:50:18.945] callframe                depth=1 byte4=0xfc4dcacb
addr=0x3dbCf1D6840A85cfaa89395c0626D7a4FB2D1D47 callsite= label=DeployOPChain
WARN [01-13|15:50:18.945] callframe                depth=2 byte4=0x38d38c97
addr=0x1903e2aD975aA384d3458F9eE2e88810972fc97a callsite= label=optimismMintableERC20FactoryProxy
WARN [01-13|15:50:18.945] Revert
addr=0x1903e2aD975aA384d3458F9eE2e88810972fc97a label=optimismMintableERC20FactoryProxy
err="execution reverted" revertData=0x depth=1
INFO [01-13|15:50:18.945] Running chain assertions on the OptimismPortal2 proxy at
0xf499C82fA80d3D482885dFBb6771811c74BD6b99 sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:50:18.946] Running chain assertions on the SystemConfig proxy at
0x6431661ca383e59a05CC9a3800eCB60932f41ddF sender=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
INFO [01-13|15:50:18.971] transaction broadcasted      id=6fa164..1561c6 nonce=42
INFO [01-13|15:50:18.971] Publishing transaction        service=transactor tx=3efc5e..d9089a nonce=42
gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=15,701,690
INFO [01-13|15:50:18.972] Transaction successfully published service=transactor tx=3efc5e..d9089a
nonce=42 gasTipCap=5,000,000,000 gasFeeCap=7,000,000,000 gasLimit=15,701,690 tx=3efc5e..d9089a
INFO [01-13|15:50:27.975] Transaction confirmed          service=transactor tx=3efc5e..d9089a
block=6f9565..965976:360 effectiveGasPrice=5,000,000,007
INFO [01-13|15:50:27.975] transaction confirmed          id=6fa164..1561c6 completed=1 total=1
hash=0x3efc5e62ff8bf125501b80e44bed1a8cbf29af87cb320f0ab9c718de01d9089a nonce=42
creation=0x0000000000000000000000000000000000000000000000000000000000000000
INFO [01-13|15:50:27.977] alt-da deployment not needed   stage=deploy-alt-da
INFO [01-13|15:50:27.978] additional dispute games deployment not needed stage=deploy-additional-dispute-
games
INFO [01-13|15:50:27.978] generating L2 genesis          stage=generate-l2-genesis
id=0x0000000000000000000000000000000000000000000000000000000000000006f
INFO [01-13|15:50:28.160] Setting MultiCall3 preinstall code at:
0xcA11bde05977b3631167028862bE2a173976CA11
sender=0xaE0bDc4eEAC5E950B67C6819B118761CaAF61946
INFO [01-13|15:50:28.160] Setting Create2Deployer preinstall code at:
0x13b0D85CcB8bF860b6b79AF3029fCA081AE9beF2 sender=0xaE0bDc4eEAC5E950B67C6819B118761CaAF61946
INFO [01-13|15:50:28.160] Setting Safe_v130 preinstall code at:
0x69f4D1788e39c87893C980c06EdF4b7f686e2938 sender=0xaE0bDc4eEAC5E950B67C6819B118761CaAF61946
INFO [01-13|15:50:28.160] Setting SafeL2_v130 preinstall code at:

```

[illegible]

result

- `.deployer/state.json` file

```
{
  "version": 1,
  "create2Salt": "0xceb6ddf6e54ebca9c3ffaa30c763e3b68834ba3a20c0e2721672d7d9be58f4ba",
  "appliedIntent": {
```

```

configType": "standard-overrides",
"l1ChainID": 11155111,
"opcmAddress": "0x5cbe1a8ed6e4cb9e36bb831dd26c0d9416e78ceb",
"superchainConfigProxy": null,
"superchainRoles": null,
"fundDevAccounts": false,
"l1ContractsLocator": "embedded",
"l2ContractsLocator": "embedded",
"chains": [
{
  "id": "0x0000000000000000000000000000000000000000000000000000000000000000",
  "baseFeeVaultRecipient": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
  "l1FeeVaultRecipient": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
  "sequencerFeeVaultRecipient": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
  "eip1559DenominatorCanyon": 250,
  "eip1559Denominator": 50,
  "eip1559Elasticity": 6,
  "gasLimit": 60000000,
  "roles": {
    "l1ProxyAdminOwner": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
    "l2ProxyAdminOwner": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
    "systemConfigOwner": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
    "unsafeBlockSigner": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
    "batcher": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
    "proposer": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
    "challenger": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266"
  },
  "deployOverrides": null,
  "dangerousAltDAConfig": {
    "useAltDA": false,
    "daCommitmentType": "",
    "daChallengeWindow": 0,
    "daResolveWindow": 0,
    "daBondSize": 0,
    "daResolverRefundPercentage": 0
  },
  "dangerousAdditionalDisputeGames": null
}
],
"globalDeployOverrides": null,
"l1DevGenesisParams": null
},
"prestateManifest": null,
"interopDepSet": {

```

```

"dependencies": {
  "111": {}
},
"superchainContracts": {
  "SuperchainProxyAdminImpl": "0xe7f1725e7734ce288f8367e1bb143e90bb3f0512",
  "SuperchainConfigProxy": "0xdc64a140aa3e981100a9beca4e685f962f0cf6c9",
  "SuperchainConfigImpl": "0xce28685eb204186b557133766eca00334eb441e4",
  "ProtocolVersionsProxy": "0x0165878a594ca255338adfa4d48449f69242eb8f",
  "ProtocolVersionsImpl": "0x37e15e4d6dffa9e5e320ee1ec036922e563cb76c"
},
"superchainRoles": {
  "SuperchainProxyAdminOwner": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
  "SuperchainGuardian": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
  "ProtocolVersionsOwner": "0xf39fd6e51aad88f6f4ce6ab8827279cfff92266",
  "Challenger": "0x0000000000000000000000000000000000000000"
},
"implementationsDeployment": {
  "OpcmImpl": "0x5cbe1a8ed6e4cb9e36bb831dd26c0d9416e78ceb",
  "OpcmContractsContainerImpl": "0x0000000000000000000000000000000000000000",
  "OpcmGameTypeAdderImpl": "0x0000000000000000000000000000000000000000",
  "OpcmDeployerImpl": "0x0000000000000000000000000000000000000000",
  "OpcmUpgraderImpl": "0x0000000000000000000000000000000000000000",
  "OpcmInteropMigratorImpl": "0x0000000000000000000000000000000000000000",
  "OpcmStandardValidatorImpl": "0x0000000000000000000000000000000000000000",
  "DelayedWethImpl": "0x33dadcd2d1aa9bb613a7ae6b28425ea00d44c6998",
  "OptimismPortalImpl": "0x381e729ff983fa4bcd820e7b922d79bf653b999",
  "OptimismPortalInteropImpl": "0xb0eb854fd842e0e564d49d2fe6b2ac25d035523c",
  "EthLockboxImpl": "0x784d2f03593a42a6e4676a012762f18775ecbbe6",
  "PreimageOracleImpl": "0x1fb8cdfc6831fc866ed9c51af8817da5c287add3",
  "MipsImpl": "0x07babe08ee4d07dba236530183b24055535a7011",
  "SystemConfigImpl": "0x2bfe4a5bd5a41e9d848d843ebcdfa15954e9a557",
  "L1CrossDomainMessengerImpl": "0x22d12e0faebd62d429514a65ebae32dd316c12d6",
  "L1Erc721BridgeImpl": "0x7f1d12fb2911eb095278085f721e644c1f675696",
  "L1StandardBridgeImpl": "0xe32b192fb1dca88fcb1c56b3acb429e32238adcb",
  "OptimismMintableErc20FactoryImpl": "0x5493f4677a186f64805fe7317d6993ba4863988f",
  "DisputeGameFactoryImpl": "0x33d1e8571a85a538ed3d5a4d88f46c112383439d",
  "AnchorStateRegistryImpl": "0x0000000000000000000000000000000000000000"
},
"opChainDeployments": [
  {
    "id": "0x000000000000000000000000000000000000000000000000000000000000006f",
    "OpChainProxyAdminImpl": "0x3cf14c436a7a83336c06e69288f29b4e92acb15b",
    "OptimismPortalProxy": "0xf499c82fa80d3d482885dfbb6771811c74bd6b99",

```

```

    "AddressManagerImpl": "0x33c0cf2f6b6b452ab0e68218253f7780d26d690b",
    "L1Erc721BridgeProxy": "0x8e7d0ad620e66b494fca6ae413c311d3555e04",
    "SystemConfigProxy": "0x6431661ca383e59a05cc9a3800ecb60932f41ddf",
    "OptimismMintableErc20FactoryProxy": "0x1903e2ad975aa384d3458f9ee2e88810972fc97a",
    "L1StandardBridgeProxy": "0xc428cfc3442d445c71ef7bf2e34a9f5f23d6477f",
    "L1CrossDomainMessengerProxy": "0x65bad6d5a135448e224a8ae96733f0a49c2563f3",
    "EthLockboxProxy": "0xdba25897de81d5b8523fada951ff67140b0a9894",
    "DisputeGameFactoryProxy": "0x8e1e505274f5054200b3d26d7ebc75722038ce8f",
    "AnchorStateRegistryProxy": "0x13e7134ee3fa91689468ef40746b366885e7c5fb",
    "FaultDisputeGameImpl": "0x00000000000000000000000000000000",
    "PermissionedDisputeGameImpl": "0x19a9b6f4aff0099d14e2804a0c2e9e8a7b507d",
    "DelayedWethPermissionedGameProxy": "0xd0ff6c648d698f6c3ad893f8b134a417f6227c85",
    "DelayedWethPermissionlessGameProxy": "0x00000000000000000000000000000000",
    "AltDAChallengeProxy": "0x00000000000000000000000000000000",
    "AltDAChallengeImpl": "0x00000000000000000000000000000000",
    "L2OutputOracleProxy": "0x00000000000000000000000000000000",
    "additionalDisputeGames": null,
    "allocs": "H4slAAAAAAAAA/+y9.....cxD//+pPTf6d/l/bnv/6Y/zV//vXn3/+z/AR//u///n8AAAD//600NT0+kl4A",
    "startBlock": {
      "hash": "0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976",
      "parentHash": "0xe3dd5cca51bb98ab8dbb3b53f86d8b353ab22ade06ceb0299184333cf09943ec",
      "number": "0x168",
      "timestamp": "0x6965eb33"
    }
  },
  "l1StateDump": null,
  "DeploymentCalldata": null
}

```

Step 6. generate the genesis.json

command

```
./op-deployer inspect genesis --workdir .deployer 111 > .deployer/genesis.json
```

- result
new genesis.json file under .deployer

Step 7. generate the rollup.json

command

```
./op-deployer inspect rollup --workdir .deployer 111 > .deployer/rollup.json
```

- result
new rollup.json file under .deployer

Step for proposer. set dispute game implementation

Notes.

- op-deployer manage add-game-type command performs this step
- However, due to errors, on direct commands, this guide shows the manual setup

add-game-type command

```
./op-deployer manage add-game-type -h
```

NAME:

op-deployer manage add-game-type - adds a **new** game type to the chain

USAGE:

op-deployer manage add-game-type [command options]

OPTIONS:

--l1-rpc-url value RPC URL **for** the L1 chain. **Must** be **set for** live chains. **Must** be blank **for** chains deploying to local allocs files. [\$L1_RPC_URL]

--artifacts-locator value Locator **for** artifacts. (**default:** "embedded")
[\$DEPLOYER_ARTIFACTS_LOCATOR]

--l1-proxy-admin-owner-address value Address to use **for as** the proxy admin owner. **Not** compatible **with** the --workdir flag. [\$DEPLOYER_PROXY_ADMIN_OWNER]

--opcm-impl-address value Address **of** the OPCM implementation contract. **Not** compatible **with** the --workdir flag. [\$DEPLOYER_OPCM_IMPL_ADDRESS]

--system-config-proxy-address value Address **of** the SystemConfig proxy contract. **Not** compatible **with** the --workdir flag. [\$DEPLOYER_SYSTEM_CONFIG_PROXY_ADDRESS]

--op-chain-proxy-admin-address value Address **of** the OP Chain's proxy admin on L1. **Not** compatible with the --workdir flag. [\$DEPLOYER_OP_CHAIN_PROXY_ADMIN_ADDRESS]

--delayed-weth-proxy-address value Address **of** the DelayedWETHProxy contract to include **as** part **of this** game type. **If** not specified, one will be deployed **for** you by the OPCM.
[\$DEPLOYER_DELAYED_WETH_PROXY_ADDRESS]

--dispute-game-type value Numeric type identifier **for** the dispute game. (**default:** 1)
[\$DEPLOYER_DISPUTE_GAME_TYPE]

--dispute-absolute-prestate value The absolute prestate hash **for** the dispute game. **Defaults** to the standard value. (**default:** "0x038512e02c4c3f7bdaec27d00edf55b7155e0905301e1a88083e4e0a6764d54c")
[\$DEPLOYER_DISPUTE_ABSOLUTE_PRESTATE]

--dispute-max-game-depth value Maximum depth **of** the dispute game **tree** (value **as** string). **Defaults** to the standard value. (**default:** 73) [\$DEPLOYER_DISPUTE_MAX_GAME_DEPTH]

--dispute-split-depth value Depth at which the dispute game tree **splits** (value **as** string). **Defaults** to the standard value. (**default:** 30) [\$DEPLOYER_DISPUTE_SPLIT_DEPTH]

--dispute-clock-extension value Clock extension **in** seconds **for** dispute game timing. **Defaults** to the standard value. (**default:** 10800) [\$DEPLOYER_DISPUTE_CLOCK_EXTENSION]

--dispute-max-clock-duration value Maximum clock duration **in** seconds **for** dispute game timing. **Defaults** to the standard value. (**default:** 302400) [\$DEPLOYER_DISPUTE_MAX_CLOCK_DURATION]

--initial-bond value Initial bond amount required **for** the dispute **game** (value **as** string, **in** wei).

```

Defaults to 1 ETH. (default: "1000000000000000000") [$DEPLOYER_INITIAL_BOND]
--vm-address value Address of the VM contract used by the dispute game.
[$DEPLOYER_VM_ADDRESS]
--permissionless Boolean indicating if the dispute game should be deployed in
permissionless mode. (default: false) [$DEPLOYER_PERMISSIONED]
--salt-mixer value String value for the salt mixer, used in CREATE2 address calculation. Default
to keccak256("op-stack-contract-impls-salt-v0"). (default:
"89fca2352a158519d2daabf7e53686272e828ddbff9487204546d918490b2ecf") [$DEPLOYER_SALT_MIXER]
--workdir value Path to a working directory containing a state file. Addresses will be
retrieved from the state file, and cannot be specified on the command line. [$DEPLOYER_WORKDIR]
--l2-chain-id value Chain ID of the L2 network to retrieve from state. Must be specified when
--workdir is set. [$DEPLOYER_CHAIN_ID]
--log.level value The lowest log level that will be output (default: INFO)
[$DEPLOYER_LOG_LEVEL]
--log.format value Format the log output. Supported formats: 'text', 'terminal', 'logfmt', 'json',
'json-pretty', (default: text) [$DEPLOYER_LOG_FORMAT]
--log.color Color the log output if in terminal mode (default: false)
[$DEPLOYER_LOG_COLOR]
--log.pid Show pid in the log (default: false) [$DEPLOYER_LOG_PID]
--help, -h show help

```

errors

- op-deployer error

[illegible]

[illegible]

```
// from GameDisputeFactory ABI
```

- implementation error

```
INFO [01-15|14:41:56.176] Initialized path database          readonly=true cache=0.00B buffer=0.00B
history=0
WARN [01-15|14:41:56.427] Fault
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 label=DummyCaller err="execution reverted" depth=4
op=253 byte4=0x1661a2e9
WARN [01-15|14:41:56.427] callframe          depth=1 byte4=0xa590c30b
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=AddGameType
WARN [01-15|14:41:56.427] callframe          depth=2 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] callframe          depth=3 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] callframe          depth=4 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] Revert
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 label=DummyCaller err="execution reverted"
revertData=0x depth=3
WARN [01-15|14:41:56.427] Fault
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 label=DummyCaller err="execution reverted" depth=3
op=253 byte4=0x1661a2e9
WARN [01-15|14:41:56.427] callframe          depth=1 byte4=0xa590c30b
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=AddGameType
WARN [01-15|14:41:56.427] callframe          depth=2 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] callframe          depth=3 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] Revert
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffb92266 label=DummyCaller err="execution reverted"
```

```

revertData=0x depth=2
WARN [01-15|14:41:56.427] Fault
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 label=AddGameType err="execution reverted"
depth=1 op=253 byte4=0xa590c30b
WARN [01-15|14:41:56.427] callframe depth=1 byte4=0xa590c30b
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=AddGameType
WARN [01-15|14:41:56.427] Revert
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 label=AddGameType err="execution reverted"
revertMsg="AddGameType: addGameType failed" depth=0
panic
panic: revision id 1 cannot be reverted [recovered]
    panic: revision id 1 cannot be reverted

goroutine 1 [running]:
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*Host).Call.func1()
    /home/circleci/project/op-chain-ops/script/script.go:362 +0x1de
panic({0xd5ee40?, 0xc000194dd0?})
    /data/mise-data/installs/go/1.23.8/src/runtime/panic.go:791 +0x132
github.com/ethereum/go-ethereum/core/state.(*journal).RevertToSnapshot(0xc00026d600, 0x1, 0xc00001e600)
    /home/circleci/go/pkg/mod/github.com/ethereum-optimism/op-geth@v1.101511.1-
dev.1.0.20250710181308-c6e05723600e/core/state/journal.go:91 +0x10e
github.com/ethereum/go-ethereum/core/state.(*StateDB).RevertToSnapshot(0xc0001bd8c0?, 0xc000390d78?)
    /home/circleci/go/pkg/mod/github.com/ethereum-optimism/op-geth@v1.101511.1-
dev.1.0.20250710181308-c6e05723600e/core/state/statedb.go:757 +0x25
github.com/ethereum-optimism/optimism/op-chain-
ops/script/forking.(*ForkableState).RevertToSnapshot(0xc00013ec00?, 0xc000035600?)
    /home/circleci/project/op-chain-ops/script/forking/state.go:370 +0x1e
github.com/ethereum/go-ethereum/core/vm.(*EVM).Call(0xc00001e480, {0x44, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0,
0x0, ...}, ...)
    /home/circleci/go/pkg/mod/github.com/ethereum-optimism/op-geth@v1.101511.1-
dev.1.0.20250710181308-c6e05723600e/core/vm/evm.go:263 +0xb6e
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*wrappedEVM).Call(0x1007f65affd?, {0x44, 0x0,
0x0, 0x0, 0x0, 0x0, 0x0, 0x0, ...}, ...)
    /home/circleci/project/op-chain-ops/script/evm.go:42 +0x48
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*Host).Call(0xc00068aa80, {0x44, 0x0, 0x0, 0x0,
0x0, 0x0, 0x0, 0x0, ...}, ...)
    /home/circleci/project/op-chain-ops/script/script.go:375 +0x17a
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*forgeScriptBackendImpl).Call(0xc00080cc28,
{0xfd, 0xaf, 0x65, 0x7f, 0x71, 0x46, 0xc1, 0xab, 0x9d, ...}, ...)
    /home/circleci/project/op-chain-ops/script/deploy.go:189 +0xda
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*forgeScriptImpl).Call(0xc000257ad0,
{0xc000368c80, 0x264, 0x280})
    /home/circleci/project/op-chain-ops/script/deploy.go:257 +0x198
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*deployScriptWithoutOutputImpl[...]).Call(...)

```

```

/home/circleci/project/op-chain-ops/script/deploy.go:286
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*deployScriptWithoutOutputImpl[...]).run(0x47f2ec0, {{0xf3, 0x9f, 0xd6, 0xe5, 0x1a, 0xad, 0x88, 0xf6, 0xf4, ...}, ...})
/home/circleci/project/op-chain-ops/script/deploy.go:307 +0x2b1
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*deployScriptWithOutputImpl[...]).Run(_ {{0xf3, 0x9f, 0xd6, 0xe5, 0x1a, 0xad, 0x88, 0xf6, 0xf4, ...}, ...})
/home/circleci/project/op-chain-ops/script/deploy.go:337 +0xde
github.com/ethereum-optimism/optimism/op-deployer/pkg/deployer/manage.AddGameType({_, _}, {{0x7ffc96bd2678, 0x16}, {0x47f83d8, 0xc000251f40}, 0xc00013ae70, {0xc0001a79e0, 0x20}, {0xf3, ...}, ...})
/home/circleci/project/op-deployer/pkg/deployer/manage/add_game_type.go:277 +0x6dc
github.com/ethereum-optimism/optimism/op-deployer/pkg/deployer/manage.AddGameTypeCLI(0xc000367900)
/home/circleci/project/op-deployer/pkg/deployer/manage/add_game_type.go:163 +0x738
github.com/urfave/cli/v2.(*Command).Run(0x52183c0, 0xc000367900, {0xc000163b90, 0x9, 0x9})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/command.go:276 +0x7e2
github.com/urfave/cli/v2.(*Command).Run(0xc00047f080, 0xc000367840, {0xc00036cfa0, 0xa, 0xa})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/command.go:269 +0xa65
github.com/urfave/cli/v2.(*Command).Run(0xc00047f340, 0xc0003675c0, {0xc0001c2000, 0xb, 0xb})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/command.go:269 +0xa65
github.com/urfave/cli/v2.(*App).RunContext(0xc0000fe800, {0x47ebed8, 0x5255a20}, {0xc0001c2000, 0xb, 0xb})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/app.go:333 +0x5a5
github.com/urfave/cli/v2.(*App).Run(0xc00003fe00?, {0xc0001c2000?, 0x0?, 0x68?})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/app.go:307 +0x2f
main.main()
/home/circleci/project/op-deployer/cmd/op-deployer/main.go:25 +0xa8

```

Get the GameFactoryAddress

```

GAME_FACTORY_ADDRESS=$(cat state.json | jq -r '.opChainDeployments[0].DisputeGameFactoryProxy')
echo "DisputeGameFactory Address: $GAME_FACTORY_ADDRESS"

// result
DisputeGameFactory Address: 0x8e1e505274f5054200b3d26d7ebc75722038ce8f

```

Get the implementation address

```

GAME_FACTORY_IMPL=$(cat state.json | jq -r '.opChainDeployments[0].PermissionedDisputeGameImpl')
echo "Game Factory Implementation Address: $GAME_FACTORY_IMPL"

// result
Game Factory Implementation Address: 0x19a9b6f4afaff0099d14e2804a0c2e9e8a7b507d

```

Make sure the contract owner is correct

```

L1=http://127.0.0.1:38545
cast call $GAME_FACTORY_ADDRESS "owner()(address)" --rpc-url $L1

// result
0xf39fd6e51aad88f6f4ce6ab8827279cfffb92266

```

set implementation tx

```
cast send ₩
--rpc-url $L1_RPC_URL ₩
--private-key $DEPLOYER_PRIVATE_KEY ₩
$GAME_FACTORY_ADDRESS ₩
"setImplementation(uint32,address)" ₩
0 ₩
$GAME_FACTORY_IMPL
```

result

[illegible]

check the registration

```
cast call --rpc-url $L1_RPC_URL \W
0x8e1e505274f5054200b3d26d7ebc75722038ce8f \W
"gameImpls(uint32)(address)" \W
0
```

```
// result
```

```
0x19a9B6f4AfAfF0099d14e2804A0C2e9e8a7B507D
```

sequencer

workspace

```
cd /rollup/sequencer
```

Step 1. .env file

```
# used inside docker
```

```
L1_RPC_URL=http://host.docker.internal:8545
```

```
# beacon not available on Anvil
```

```
# so, disabling beacon url
```

```
# L1_BEACON_URL=https://ethereum-sepolia-beacon-api.publicnode.com
```

```
PRIVATE_KEY=ac0974bec39a17e36ba4a6b4d238ff944bacb478cbcd5efcae784d7bf4f2ff80
```

```
# Only needed if you want P2P to work over internet
```

```
P2P_ADVERTISE_IP=127.0.0.1
```

- init

```
source .env
```

Step 2. copy the files from deployer

command

```
cp ../deployer/.deployer/genesis.json .
```

```
cp ../deployer/.deployer/rollup.json .
```

Step 3. create jwt

command

```
openssl rand -hex 32 > jwt.txt
```

```
chmod 600 jwt.txt
```

jwt.txt file

```
6e9a1103d096b03da8943f463574471e90e3fdb165020e27c58b50225d408fca
```

- Note. the jwt file will be used to auth between op-node and op-geth

Step 4. init docker compose

docker-compose.yml file

```
networks:
  op-stack:
    external: true

services:
  op-geth:
    image: us-docker.pkg.dev/oplabs-tools-artifacts/images/op-geth:v1.101511.1
    container_name: sequencer-op-geth
    networks:
      - op-stack
    volumes:
      - ../workspace
    working_dir: /workspace
    ports:
      - "18545:8545"
      - "8546:8546"
      - "8551:8551"
    command:
      - "--datadir=/workspace/op-geth-data"
      - "--http"
      - "--http.addr=0.0.0.0"
      - "--http.port=8545"
      - "--ws"
      - "--ws.addr=0.0.0.0"
      - "--ws.port=8546"
      - "--authrpc.addr=0.0.0.0"
      - "--authrpc.port=8551"
      - "--authrpc.jwtsecret=/workspace/jwt.txt"
      - "--syncmode=full"
      - "--gcmode=archive"
      - "--rollup.disabletxpoolgossip=true"
      - "--http.vhosts="
      - "--http.corsdomain="
      - "--http.api=eth,net,web3,debug,txpool,admin"
      - "--ws.origins="
      - "--ws.api=eth,net,web3,debug,txpool,admin"
      - "--authrpc.vhosts="

  op-node:
    image: us-docker.pkg.dev/oplabs-tools-artifacts/images/op-node:v1.13.5
    container_name: sequencer-node
    depends_on:
```

```

- op-geth
networks:
- op-stack
volumes:
- ./workspace
working_dir: /workspace
ports:
- "8547:8547"
- "9222:9222"
environment:
  L1_RPC_URL: "${L1_RPC_URL}"
  PRIVATE_KEY: "${PRIVATE_KEY}"
  P2P_ADVERTISE_IP: "${P2P_ADVERTISE_IP}"
command:
- "op-node"
- "--l1=${L1_RPC_URL}"
- "--l1.trustrpc=true"
- "--l2=http://sequencer-op-geth:8551"
- "--l2.jwt-secret=/workspace/jwt.txt"
- "--rollup.config=/workspace/rollup.json"
- "--sequencer.enabled=true"
- "--sequencer.stopped=false"
- "--sequencer.max-safe-lag=3600"
- "--verifier.l1-confs=1"
- "--p2p.listen.ip=0.0.0.0"
- "--p2p.listen.tcp=9222"
- "--p2p.listen.udp=9222"
- "--p2p.advertise.ip=${P2P_ADVERTISE_IP}"
- "--p2p.advertise.tcp=9222"
- "--p2p.advertise.udp=9222"
- "--p2p.sequencer.key=${PRIVATE_KEY}"
- "--rpc.addr=0.0.0.0"
- "--rpc.port=8547"
- "--rpc.enable-admin"
- "--log.level=info"
- "--log.format=json"

```

Step 5. init op-geth state

command

```

docker run --rm W
-v $(pwd):/workspace W
-w /workspace W
us-docker.pkg.dev/oplabs-tools-artifacts/images/op-geth:v1.101511.1 W
init --datadir=./op-geth-data --state.scheme=hash ./genesis.json

```

output

```
INFO [01-13|07:15:31.664] Maximum peer count          ETH=50 total=50
INFO [01-13|07:15:31.667] Smartcard socket not found, disabling  err="stat /run/pcscd/pcscd.comm: no such
file or directory"
INFO [01-13|07:15:31.677] Set global gas cap                cap=50,000,000
INFO [01-13|07:15:31.679] Initializing the KZG library        backend=gokzg
INFO [01-13|07:15:31.681] Defaulting to pebble as the backing database
INFO [01-13|07:15:31.681] Allocated cache and file handles    database=/workspace/op-geth-
data/geth/chaindata cache=16.00MiB handles=16
INFO [01-13|07:15:31.724] Opened ancient database             database=/workspace/op-geth-
data/geth/chaindata/ancient/chain readonly=false
INFO [01-13|07:15:31.724] State scheme set by user           scheme=hash
INFO [01-13|07:15:31.724] Writing custom genesis block
ERROR[01-13|07:15:31.725] Head block is not reachable
INFO [01-13|07:15:31.813] Persisted trie from memory database  nodes=3146 size=454.59KiB
time=14.581542ms gcnodes=0 gcsiz=0.00B gctime=0s livenodes=0 livesize=0.00B
INFO [01-13|07:15:31.965] Successfully wrote genesis state     database=chaindata hash=896904..b94fa9
```

result

- new folder op-geth-data created

Step 6. run the nodes

command

```
docker compose up -d
```

output

```
[+] Running 2/2
✓ Container sequencer-op-geth Started
0.2s
✓ Container sequencer-node Started
```

result

- op-node logs

```
docker logs sequencer-node | head -n 100
```

```
{"t":"2026-01-13T07:30:38.976140257Z","lvl":"info","msg":"Not opted in to ProtocolVersions signal loading,
disabling ProtocolVersions contract now."}
{"t":"2026-01-13T07:30:39.011368257Z","lvl":"info","msg":"Using default bootnodes, none provided."}
{"t":"2026-01-13T07:30:39.03429059Z","lvl":"info","msg":"No persisted sequencer state loaded"}
{"t":"2026-01-13T07:30:39.034751465Z","lvl":"info","msg":"Rollup
Config","l2_chain_id":"111","l2_network":"unknown
L2","l1_chain_id":"11155111","l1_network":"sepolia","l2_start_time":1768287027,"l2_block_hash":"0x8969049a7433
2a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9","l2_block_number":0,"l1_block_hash":"0x6f9565e4
94373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976","l1_block_number":360,"regolith_time":"@
genesis","canyon_time":"@ genesis","delta_time":"@ genesis","ecotone_time":"@ genesis","fjord_time":"@
```

```

genesis","granite_time":"@ genesis","holocene_time":"@ genesis","isthmus_time":"@ genesis","jovian_time":"(not
configured)","interop_time":"(not configured)"}
{"t":"2026-01-13T07:30:39.034861424Z","lvl":"info","msg":"Initializing rollup node","version":"v1.13.5-7a0ab04e-
1752261210"}
{"t":"2026-01-13T07:30:39.047075924Z","lvl":"warn","msg":"This endpoint is required for the Ecotone upgrade,
but is missing, and configured to be ignored. The node may be unable to retrieve EIP-4844 blobs data."}
{"t":"2026-01-13T07:30:39.050551507Z","lvl":"warn","msg":"No interop RPC configured, falling back to legacy
sync mode."}
{"t":"2026-01-13T07:30:39.053237215Z","lvl":"info","msg":"loaded new runtime config
values!","p2p_seq_address":"0xf39Fd6e51aad88F6F4ce6aB8827279cfffB92266"}
{"t":"2026-01-13T07:30:39.067857715Z","lvl":"info","msg":"started p2p
host","addrs":["/ip4/127.0.0.1/tcp/9222","/ip4/172.19.0.3/tcp/9222"],"peerID":"16Uiu2HAkxTs77rJkCRu6DCdZMje2
qHFs5qVdGFPFRYSYUTCroXv"}
{"t":"2026-01-13T07:30:39.070903132Z","lvl":"info","msg":"New local node
record","seq":"1768289439068","id":"18173cc1c8f20d2adaecce19467a90dbd78f46cadcb332c43146fb70f607e62","
ip":"127.0.0.1","udp":"9222","tcp":"9222"}
{"t":"2026-01-13T07:30:39.070939424Z","lvl":"info","msg":"started discovery service","p2p":"discv5","enr":"enr:-
Jy4QLGcWfWNJIMTqqxj5utgPXyFG1EJl9JxRtYWOtWR0njFijZSHR3GDvSxf4gxSD0RJ68WUvU6ucp98iUW400xuGA
Zu2Q41cgmlkgnY0gmlwhH8AAAGHb3BzdGFja4JvAIlzZWwMjU2azGhAi0izY3KIP_XM32HKLrj1EGkpTZVpEyYqXS7
hwJlOEbtRg3RjcllkBoN1ZHCCJAY","id":"18173cc1c8f20d2adaecce19467a90dbd78f46cadcb332c43146fb70f607e
62"}
{"t":"2026-01-13T07:30:39.073074924Z","lvl":"info","msg":"registered API","route":"","namespace":"optimism"}
{"t":"2026-01-13T07:30:39.07313834Z","lvl":"info","msg":"registered API","route":"","namespace":"opp2p"}
{"t":"2026-01-13T07:30:39.073149465Z","lvl":"info","msg":"P2P RPC enabled"}
{"t":"2026-01-13T07:30:39.073166257Z","lvl":"info","msg":"registered API","route":"","namespace":"admin"}
{"t":"2026-01-13T07:30:39.073168174Z","lvl":"info","msg":"Admin RPC enabled"}
{"t":"2026-01-13T07:30:39.073169299Z","lvl":"info","msg":"Starting JSON-RPC server"}
{"t":"2026-01-13T07:30:39.084491465Z","lvl":"info","msg":"Started RPC server","endpoint":"http://[::]:8547"}
{"t":"2026-01-13T07:30:39.084517924Z","lvl":"info","msg":"Started JSON-RPC server","addr":"[::]:8547"}
{"t":"2026-01-13T07:30:39.08452034Z","lvl":"info","msg":"metrics disabled"}
{"t":"2026-01-13T07:30:39.084528715Z","lvl":"info","msg":"Starting execution engine driver"}
{"t":"2026-01-13T07:30:39.084532465Z","lvl":"info","msg":"Starting
driver","sequencerEnabled":true,"sequencerStopped":false,"recoverMode":false}
{"t":"2026-01-13T07:30:39.085009632Z","lvl":"info","msg":"Starting sequencing, without known pre-state"}
{"t":"2026-01-13T07:30:39.085022882Z","lvl":"info","msg":"Sequencer has been started","next action":"2026-01-
13T07:30:39.085021674Z"}
{"t":"2026-01-13T07:30:39.085027049Z","lvl":"info","msg":"Rollup node started"}
{"t":"2026-01-13T07:30:39.085041424Z","lvl":"info","msg":"State loop started"}
{"t":"2026-01-13T07:30:39.085061465Z","lvl":"info","msg":"Scheduled sequencer action","delta":"-39.125µs"}
{"t":"2026-01-13T07:30:39.085901299Z","lvl":"warn","msg":"Deriver system is resetting","err":"reset: cannot
continue derivation until Engine has been reset"}
{"t":"2026-01-13T07:30:39.085927507Z","lvl":"error","msg":"Sequencer encountered reset signal, aborting
work","err":"reset: cannot continue derivation until Engine has been reset"}
{"t":"2026-01-13T07:30:39.08750259Z","lvl":"info","msg":"Loaded current L2

```

```

heads","unsafe":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","safe":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","finalized":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","unsafe_origin":"0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360","safe_origin":"0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360"}
{"t":"2026-01-13T07:30:39.088327757Z","lvl":"info","msg":"Walking back L1Block by number","curr":"0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360","next":"0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360","l2block":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0"}
{"t":"2026-01-13T07:30:39.088342757Z","lvl":"info","msg":"Hit finalized L2 head, returning immediately","unsafe":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","safe":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","finalized":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","unsafe_origin":"0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360","safe_origin":"0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360"}
{"t":"2026-01-13T07:30:39.088377924Z","lvl":"info","msg":"Current hardfork version detected","forkName":"isthmus"}
{"t":"2026-01-13T07:30:39.088401424Z","lvl":"info","msg":"Reset of Engine is completed","local_unsafe":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","cross_unsafe":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","local_safe":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","cross_safe":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","finalized":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0"}
{"t":"2026-01-13T07:30:39.090539132Z","lvl":"info","msg":"Confirming pipeline reset"}
{"t":"2026-01-13T07:30:39.09056209Z","lvl":"info","msg":"Engine reset confirmed, sequencer may continue","next":true}
{"t":"2026-01-13T07:30:39.091461049Z","lvl":"info","msg":"Scheduled sequencer action","delta":"1.9990945s"}
{"t":"2026-01-13T07:30:41.09154605Z","lvl":"info","msg":"Started sequencing new block","parent":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","l1Origin":"0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360"}
{"t":"2026-01-13T07:30:41.093125425Z","lvl":"info","msg":"Scheduled sequencer action","delta":"-2.042µs"}
{"t":"2026-01-13T07:30:41.102135091Z","lvl":"info","msg":"Sequencer sealed block","payloadID":"0x03c0b6f9e2a923fa","block":"0x02585dfac7011e9db157f03161033a409b5ff1e0a489a6ae8d91eba6ef7edac7:1","parent":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0","txs":1,"time":1768287029}
{"t":"2026-01-13T07:30:41.102305341Z","lvl":"info","msg":"Publishing signed execution payload on p2p","id":"0x02585dfac7011e9db157f03161033a409b5ff1e0a489a6ae8d91eba6ef7edac7:1"}
{"t":"2026-01-13T07:30:41.104007925Z","lvl":"warn","msg":"payload is too old","topic":"blocksV4","timestamp":1768287029}
{"t":"2026-01-13T07:30:41.10402705Z","lvl":"warn","msg":"failed to publish newly created block","id":"0x02585dfac7011e9db157f03161033a409b5ff1e0a489a6ae8d91eba6ef7edac7:1","hash":"0x02585dfac7011e9db157f03161033a409b5ff1e0a489a6ae8d91eba6ef7edac7","err":"validation failed"}
{"t":"2026-01-13T07:30:41.104974633Z","lvl":"info","msg":"Sequencer inserted block","block":"0x02585dfac7011e9db157f03161033a409b5ff1e0a489a6ae8d91eba6ef7edac7:1","parent":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0"}

```

```
049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9:0"}
{"t":"2026-01-13T07:30:41.10653805Z","lvl":"info","msg":"Inserted new L2 unsafe
block","hash":"0x02585dfac7011e9db157f03161033a409b5ff1e0a489a6ae8d91eba6ef7edac7","number":1,"state_r
oot":"0x1967a55373a1e7a4bd7c1920163af18d0aba15bc65589f00fee5be58fa52e09","timestamp":1768287029,"p
arent":"0x8969049a74332a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9","prev_randao":"0x60a09df
7bf7fa1ec68f476dfad20f5c6e6936a89f6f8bd9d2d027ff3bc4d14e5","fee_recipient":"0x4200000000000000000000
00000000000000011","txs":1,"build_time":"10.542ms","insert_time":"4.343ms","total_time":"14.886ms","mgas":0.16
2696,"mgasps":10.92921943906535}
{"t":"2026-01-13T07:30:41.10662205Z","lvl":"info","msg":"Scheduled sequencer action","delta":"-25.041µs"}
{"t":"2026-01-13T07:30:41.106648591Z","lvl":"info","msg":"Started sequencing new
block","parent":"0x02585dfac7011e9db157f03161033a409b5ff1e0a489a6ae8d91eba6ef7edac7:1","l1Origin":"0x6f
9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360"}
{"t":"2026-01-13T07:30:41.109140758Z","lvl":"info","msg":"Scheduled sequencer action","delta":"-1.25µs"}
...
```

lifecycle

```
{
  "t": "2026-01-13T07:40:16.953602427Z",
  "lvl": "info",
  "msg": "Sequencer inserted block",
  "block": "0xe4a42e32ef0d7ee01e36531b2358d9c291a6bf371bc52f92e5ced9308a2134a3:1495",
  "parent": "0xbad98090d7d0567e529a40b64fabdd4255bb4844f560430d78c0e61cff96ebe2:1494"
}

{
  "t": "2026-01-13T07:40:16.954870969Z",
  "lvl": "info",
  "msg": "Inserted new L2 unsafe block",
  "hash": "0xe4a42e32ef0d7ee01e36531b2358d9c291a6bf371bc52f92e5ced9308a2134a3",
  "number": 1495,
  "state_root": "0xd5682d8df879b97bb9caefbc297ef36a914a643f71198a178aff691c3aa30f14",
  "timestamp": 1768290017,
  "parent": "0xbad98090d7d0567e529a40b64fabdd4255bb4844f560430d78c0e61cff96ebe2",
  "prev_randao": "0xaf54fb3de68b08a8414f414d17204c62ec32df13083167ce121f9e73943854ad",
  "fee_recipient": "0x4200000000000000000000000000000000000000000000000000000000000000",
  "txs": 1, "build_time": "1.949s",
  "insert_time": "3.371ms",
  "total_time": "1.953s",
  "mgas": 0.046084,
  "mgasps": 0.023594604308023213
}

{
  "t": "2026-01-13T07:40:16.954940636Z",
  "lvl": "info",
  "msg": "Scheduled sequencer action",
  "delta": "45.061031ms"
```

```

}
{
  "t":"2026-01-13T07:40:17.000892386Z",
  "lvl":"info",
  "msg":"Started sequencing new block",
  "parent":"0xe4a42e32ef0d7ee01e36531b2358d9c291a6bf371bc52f92e5ced9308a2134a3:1495",
  "l1Origin":"0x80113a6c943642fecb581e34244a10c29259cbb8ca01dc7c6cc476a5c2d16a22:654"
}
{
  "t":"2026-01-13T07:40:17.002575094Z",
  "lvl":"info",
  "msg":"Scheduled sequencer action",
  "delta":"1.947426323s"
}
{
  "t":"2026-01-13T07:40:18.951447303Z",
  "lvl":"info",
  "msg":"Sequencer sealed block",
  "payloadID":"0x039a09350da2daa6",
  "block":"0x6f169d289aafb4c914c3a5346fd0723b7d1426a2287d807d7051f11f142b4c66:1496",
  "parent":"0xe4a42e32ef0d7ee01e36531b2358d9c291a6bf371bc52f92e5ced9308a2134a3:1495",
  "txs":1,
  "time":1768290019
}
{
  "t":"2026-01-13T07:40:18.951586178Z",
  "lvl":"info",
  "msg":"Publishing signed execution payload on p2p",
  "id":"0x6f169d289aafb4c914c3a5346fd0723b7d1426a2287d807d7051f11f142b4c66:1496"
}
}

```

- op-geth logs

```

...
INFO [01-13|07:34:26.953] Chain head was updated      number=1320 hash=c3a41b..7a761e
root=5ecbfa..be2ef5 elapsed="375.959µs"
INFO [01-13|07:34:27.001] Starting work on payload      id=0x038895d1122234d1
INFO [01-13|07:34:27.003] Updated payload              id=0x038895d1122234d1 number=1321
hash=428c5c..d15f3d txs=1 withdrawals=0 gas=46096 fees=0 root=aa7495..aad2af elapsed=1.141ms
INFO [01-13|07:34:28.950] Stopping work on payload      id=0x038895d1122234d1 reason=delivery
elapsed=1.948s
INFO [01-13|07:34:28.953] Imported new potential chain segment    number=1321 hash=428c5c..d15f3d
blocks=1 txs=1 mgas=0.046 elapsed=1.803ms   mgasps=25.562 snapdiffs=664.74KiB triedirty=0.00B
INFO [01-13|07:34:28.954] Chain head was updated      number=1321 hash=428c5c..d15f3d
root=aa7495..aad2af elapsed="267.834µs"
INFO [01-13|07:34:29.001] Starting work on payload      id=0x039e4389e6706d27

```

INFO [01-13|07:34:29.002] Updated payload id=0x039e4389e6706d27 number=1322
hash=be4f9c..87e67b txs=1 withdrawals=0 gas=51684 fees=0 root=e4d44b..e99e62 elapsed="379.375µs"

INFO [01-13|07:34:30.950] Stopping work on payload id=0x039e4389e6706d27 reason=delivery
elapsed=1.948s

INFO [01-13|07:34:30.953] Imported **new** potential chain segment number=1322 hash=be4f9c..87e67b
blocks=1 txs=1 mgas=0.052 elapsed=1.695ms mgasps=30.479 snapdiffs=665.33KiB triedirty=0.00B

INFO [01-13|07:34:30.954] Chain head was updated number=1322 hash=be4f9c..87e67b
root=e4d44b..e99e62 elapsed="371.458µs"

INFO [01-13|07:34:31.002] Starting work on payload id=0x03d73c8a5e96ed11

INFO [01-13|07:34:31.002] Updated payload id=0x03d73c8a5e96ed11 number=1323
hash=10c8f3..6531cf txs=1 withdrawals=0 gas=48884 fees=0 root=7475ab..80233f elapsed="512.542µs"

INFO [01-13|07:34:31.788] Looking **for** peers peercount=0 tried=87 **static**=0

INFO [01-13|07:34:32.951] Stopping work on payload id=0x03d73c8a5e96ed11 reason=delivery
elapsed=1.949s

INFO [01-13|07:34:32.955] Imported **new** potential chain segment number=1323 hash=10c8f3..6531cf
blocks=1 txs=1 mgas=0.049 elapsed=2.482ms mgasps=19.694 snapdiffs=665.88KiB triedirty=0.00B

INFO [01-13|07:34:32.956] Chain head was updated number=1323 hash=10c8f3..6531cf
root=7475ab..80233f elapsed="337.75µs"

INFO [01-13|07:34:33.002] Starting work on payload id=0x0320e6fd7c5a5360

INFO [01-13|07:34:33.002] Updated payload id=0x0320e6fd7c5a5360 number=1324
hash=509776..75d117 txs=1 withdrawals=0 gas=46096 fees=0 root=d11d24..09e45a elapsed="604.792µs"

INFO [01-13|07:34:34.950] Stopping work on payload id=0x0320e6fd7c5a5360 reason=delivery
elapsed=1.948s

INFO [01-13|07:34:34.953] Imported **new** potential chain segment number=1324 hash=509776..75d117
blocks=1 txs=1 mgas=0.046 elapsed=1.609ms mgasps=28.638 snapdiffs=666.36KiB triedirty=0.00B

INFO [01-13|07:34:34.953] Chain head was updated number=1324 hash=509776..75d117
root=d11d24..09e45a elapsed="225.417µs"

INFO [01-13|07:34:35.001] Starting work on payload id=0x03e9215040a39ec3

INFO [01-13|07:34:35.002] Updated payload id=0x03e9215040a39ec3 number=1325
hash=043f97..a6cd72 txs=1 withdrawals=0 gas=46096 fees=0 root=08878e..38dc52 elapsed="468.291µs"

INFO [01-13|07:34:36.950] Stopping work on payload id=0x03e9215040a39ec3 reason=delivery
elapsed=1.948s

INFO [01-13|07:34:36.953] Imported **new** potential chain segment number=1325 hash=043f97..a6cd72
blocks=1 txs=1 mgas=0.046 elapsed=1.759ms mgasps=26.198 snapdiffs=666.84KiB triedirty=0.00B

INFO [01-13|07:34:36.954] Chain head was updated number=1325 hash=043f97..a6cd72
root=08878e..38dc52 elapsed="388.709µs"

INFO [01-13|07:34:37.001] Starting work on payload id=0x03d976a34da8b034

INFO [01-13|07:34:37.002] Updated payload id=0x03d976a34da8b034 number=1326
hash=7195de..eb7789 txs=1 withdrawals=0 gas=46096 fees=0 root=5fc09f..3ca0e0 elapsed="319µs"

INFO [01-13|07:34:38.950] Stopping work on payload id=0x03d976a34da8b034 reason=delivery
elapsed=1.949s

INFO [01-13|07:34:38.953] Imported **new** potential chain segment number=1326 hash=7195de..eb7789
blocks=1 txs=1 mgas=0.046 elapsed=2.212ms mgasps=20.837 snapdiffs=667.33KiB triedirty=0.00B

INFO [01-13|07:34:38.954] Chain head was updated number=1326 hash=7195de..eb7789

```

root=5fc09f..3ca0e0 elapsed="250.583µs"
INFO [01-13|07:34:39.002] Starting work on payload          id=0x038c4f4188948d0e
INFO [01-13|07:34:39.003] Updated payload          id=0x038c4f4188948d0e number=1327
hash=f4cf0e..9b2993 txs=1 withdrawals=0 gas=46096 fees=0 root=2818fa..574f54 elapsed="622.083µs"
INFO [01-13|07:34:40.950] Stopping work on payload      id=0x038c4f4188948d0e reason=delivery
elapsed=1.948s
INFO [01-13|07:34:40.952] Imported new potential chain segment    number=1327 hash=f4cf0e..9b2993
blocks=1 txs=1 mgas=0.046 elapsed=1.418ms    mgasps=32.493 snapdiffs=667.81KiB triedirty=0.00B

// lifecycle
INFO [01-13|07:34:40.953] Chain head was updated          number=1327 hash=f4cf0e..9b2993
root=2818fa..574f54 elapsed="244.75µs"
INFO [01-13|07:34:41.001] Starting work on payload          id=0x039bbebef224fe74
INFO [01-13|07:34:41.002] Updated payload          id=0x039bbebef224fe74 number=1328
hash=e540e3..fe5aac txs=1 withdrawals=0 gas=51684 fees=0 root=07a050..240322 elapsed="363.25µs"
INFO [01-13|07:34:41.908] Looking for peers          peercount=0 tried=73 static=0
INFO [01-13|07:34:42.950] Stopping work on payload      id=0x039bbebef224fe74 reason=delivery
elapsed=1.949s
INFO [01-13|07:34:42.952] Imported new potential chain segment    number=1328 hash=e540e3..fe5aac
blocks=1 txs=1 mgas=0.052 elapsed=1.286ms    mgasps=40.164 snapdiffs=668.40KiB triedirty=0.00B
...

```

batcher

workspace

```
cd /rollup/sequencer
```

Step 1. copy the files from deployer

- command

```
cp ../deployer/.deployer/state.json .
```

Step 2. extract BatchInbox address

command

```
BATCH_INBOX_ADDRESS=$(cat state.json | jq -r '.opChainDeployments[0].SystemConfigProxy')
echo "BatchInbox Address: $BATCH_INBOX_ADDRESS"
```

output

```
BatchInbox Address: 0x6431661ca383e59a05cc9a3800ecb60932f41ddf
```

Step 3. .env file

```
OP_BATCHER_L1_ETH_RPC=http://host.docker.internal:8545
```

```
OP_BATCHER_PRIVATE_KEY=ac0974bec39a17e36ba4a6b4d238ff944bacb478cbcd5efcae784d7bf4f2ff80
```

```
# L2 Configuration - Should match your sequencer setup
```

```
OP_BATCHER_L2_ETH_RPC=http://sequencer-op-geth:8545
```

```
OP_BATCHER_ROLLUP_RPC=http://sequencer-node:8547
```

```
# Contract addresses - Extract from your op-deployer output
```

```
# OP_BATCHER_BATCH_INBOX_ADDR=0x6431661ca383e59a05cc9a3800ecb60932f41ddf
```

```
# Batcher configuration
```

```
OP_BATCHER_POLL_INTERVAL=1s
```

```
OP_BATCHER_SUB_SAFETY_MARGIN=6
```

```
OP_BATCHER_NUM_CONFIRMATIONS=1
```

```
OP_BATCHER_SAFE_ABORT_NONCE_TOO_LOW_COUNT=3
```

```
OP_BATCHER_MAX_CHANNEL_DURATION=1
```

```
OP_BATCHER_DATA_AVAILABILITY_TYPE=calldata
```

```
OP_BATCHER_THROTTLE_THRESHOLD=0
```

```
# RPC configuration
```

```
OP_BATCHER_RPC_PORT=8548
```

- init

```
source .env
```

Step 4. init docker compose

docker-compose.yml file

```
networks:
  op-stack:
    external: true

services:
  op-batcher:
    image: us-docker.pkg.dev/oplabs-tools-artifacts/images/op-batcher:v1.13.2
    container_name: op-batcher
    networks:
      - op-stack
    volumes:
      - ../workspace
    working_dir: /workspace
    env_file:
      - .env
    ports:
      - "8548:8548"
    command: >
      op-batcher
```

```
--rpc.addr=0.0.0.0
--rpc.enable-admin
--resubmission-timeout=30s
--log.level=info
--log.format=json
restart: unless-stopped
```

Step 5. run the batcher

command

```
docker compose up -d
```

result

- op-batcher logs

```
docker logs op-batcher | head -n 100
```

```
{"t":"2026-01-13T08:31:23.126758305Z","lvl":"info","msg":"Initializing Batch Submitter"}
{"t":"2026-01-13T08:31:23.132546305Z","lvl":"info","msg":"Rollup
Config","l2_chain_id":"111","l2_network":"unknown
L2","l1_chain_id":"11155111","l1_network":"sepolia","l2_start_time":"1768287027","l2_block_hash":"0x8969049a7433
2a5bb7212e4e5b793c0129f5515f0fec7acdb945b3648bb94fa9","l2_block_number":0,"l1_block_hash":"0x6f9565e4
94373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976","l1_block_number":360,"regolith_time":"@
genesis","canyon_time":"@ genesis","delta_time":"@ genesis","ecotone_time":"@ genesis","fjord_time":"@
genesis","granite_time":"@ genesis","holocene_time":"@ genesis","isthmus_time":"@ genesis","jovian_time":"(not
configured)","interop_time":"(not configured)"}
{"t":"2026-01-13T08:31:23.141341847Z","lvl":"warn","msg":"Ecotone upgrade is active, but batcher is not
configured to use Blobs!"}
{"t":"2026-01-13T08:31:23.141464889Z","lvl":"info","msg":"Initialized channel-
config","da_type":"calldata","use_alt_da":false,"max_frame_size":119999,"target_num_frames":1,"compressor":"sha
dow","compression_algo":"zlib","batch_type":0,"max_channel_duration":1,"channel_timeout":50,"sub_safety_margi
n":6}
{"t":"2026-01-13T08:31:23.141483805Z","lvl":"info","msg":"Metrics disabled"}
{"t":"2026-01-13T08:31:23.14204818Z","lvl":"info","msg":"registered API","route":"","namespace":"admin"}
{"t":"2026-01-13T08:31:23.142079597Z","lvl":"info","msg":"registered API","route":"","namespace":"txmgr"}
{"t":"2026-01-13T08:31:23.14208593Z","lvl":"info","msg":"Admin RPC enabled"}
{"t":"2026-01-13T08:31:23.142087389Z","lvl":"info","msg":"Starting JSON-RPC server"}
{"t":"2026-01-13T08:31:23.15415893Z","lvl":"info","msg":"Started RPC server","endpoint":"http://[::]:8548"}
{"t":"2026-01-13T08:31:23.154182639Z","lvl":"info","msg":"Starting batcher","notSubmittingOnStart":false}
{"t":"2026-01-13T08:31:23.154186139Z","lvl":"info","msg":"Starting Batch Submitter"}
{"t":"2026-01-13T08:31:23.154189639Z","lvl":"info","msg":"Clearing state"}
{"t":"2026-01-13T08:31:23.15562943Z","lvl":"info","msg":"Clearing state with safe L1
origin","origin":"0x6f9565e494373f654c6c01164a6b1c3e4cf5d5b165d33bc6f6a462407e965976:360"}
{"t":"2026-01-13T08:31:23.155656305Z","lvl":"info","msg":"State cleared"}
{"t":"2026-01-13T08:31:23.155659722Z","lvl":"warn","msg":"Throttling loop is DISABLED due to 0 throttle-
threshold. This should not be disabled in prod."}
{"t":"2026-01-13T08:31:23.155668889Z","lvl":"info","msg":"Batch Submitter started"}
```

```

{"t":"2026-01-13T08:31:23.155990972Z","lvl":"info","msg":"Starting receipts processing loop"}
{"t":"2026-01-13T08:31:24.158140708Z","lvl":"info","msg":"no blocks in
state","syncStatus.headL1":"11ed06..43d008:964","syncStatus.currentL1":"e81dbc..f3f159:963","syncStatus.localSafeL2":"896904..b94fa9:0","syncStatus.safeL2":"896904..b94fa9:0","syncStatus.unsafeL2":"bcee5c..626c08:3028","syncActions":{"SyncActions{blocksToPrune: 0, channelsToPrune: 0, clearState: nil, blocksToLoad: [1, 3028]}}"}
{"t":"2026-01-13T08:31:24.15909525Z","lvl":"info","msg":"Loading range of multiple blocks into
state","start":1,"end":3028}
{"t":"2026-01-13T08:31:24.162396292Z","lvl":"info","msg":"Added L2 block to local
state","block":"0x02585dfac7011e9db157f03161033a409b5ff1e0a489a6ae8d91eba6ef7edac7:1","tx_count":1,"time":1768287029}
{"t":"2026-01-13T08:31:24.162914208Z","lvl":"info","msg":"Added L2 block to local
state","block":"0x20dcc77da7dfbb8ad78da59bbc288f191a0bc1d7ed56dd31f73e75942899a027:2","tx_count":1,"time":1768287031}
{"t":"2026-01-13T08:31:24.163407167Z","lvl":"info","msg":"Added L2 block to local
state","block":"0xf93d97f348767a5ffc7e0f6f736f48523b1f17023257502185ecf2b87cd770d5:3","tx_count":1,"time":1768287033}
{"t":"2026-01-13T08:31:24.163888292Z","lvl":"info","msg":"Added L2 block to local
state","block":"0x91b6678cecb77f72210479ee2cbaed9bf3f11e49e3c99f72ced5609d7c005feb:4","tx_count":1,"time":1768287035}
{"t":"2026-01-13T08:31:24.16473675Z","lvl":"info","msg":"Added L2 block to local
state","block":"0x16934e28f311af7e5c319445ef616148f9f539270cfb72bbbfd183bc0d279d04:5","tx_count":1,"time":1768287037}
{"t":"2026-01-13T08:31:24.165267Z","lvl":"info","msg":"Added L2 block to local
state","block":"0xe5a3b83d4149fabb9baff92b641dc8ba7c6e66dbc9f8c2e59353d846aa0fd59e:6","tx_count":1,"time":1768287039}

```

lifecycle

```

{
  "t":"2026-01-13T08:33:13.158061759Z",
  "lvl":"info",
  "msg":"Added L2 block to local state",
  "block":"0x1c2409df40a8a9497db5de4c3d961850ae7ea89c7ed448d5d63ac9bfb8be85f4:3083",
  "tx_count":1,
  "time":1768293193
}
{
  "t":"2026-01-13T08:33:15.158061759Z",
  "lvl":"info",
  "msg":"Added L2 block to local state",
  "block":"0x1c2409df40a8a9497db5de4c3d961850ae7ea89c7ed448d5d63ac9bfb8be85f4:3083",
  "tx_count":1,
  "time":1768293193
}
...

```

```

{
  "t": "2026-01-13T08:33:13.584949426Z",
  "lvl": "info",
  "msg": "Transaction confirmed",
  "service": "batcher",
  "tx": "0x01d5e8bccb57f0c786a5a4e81e96d77732a677988cd5436fc864f96896b0e2d",
  "block": "0x4f5138d4be621040dddc9660a9f93dd1fba845309abefc58819acd6d5690e40a:976",
  "effectiveGasPrice": "1000000007"
}
{
  "t": "2026-01-13T08:33:13.585113759Z",
  "lvl": "info", "msg": "Handling receipt",
  "id": "072a691c545bbcb7dba5075ab56d7a42:0"
}
{
  "t": "2026-01-13T08:33:13.585264009Z",
  "lvl": "info",
  "msg": "Transaction confirmed",
  "tx_id": "072a691c545bbcb7dba5075ab56d7a42:0",
  "tx": "0x01d5e8bccb57f0c786a5a4e81e96d77732a677988cd5436fc864f96896b0e2d",
  "block": "0x4f5138d4be621040dddc9660a9f93dd1fba845309abefc58819acd6d5690e40a:976"
}
{
  "t": "2026-01-13T08:33:13.585288301Z",
  "lvl": "info",
  "msg": "Channel is fully submitted",
  "id": "072a691c545bbcb7dba5075ab56d7a42",
  "min_inclusion_block": 976,
  "max_inclusion_block": 976
}
{
  "t": "2026-01-13T08:33:13.591876884Z",
  "lvl": "info",
  "msg": "Publishing transaction",
  "service": "batcher",
  "tx": "0x6b9c0a12721da080c72c31e00fbd79d2a504d40a003512c0bd5e4c04ce69279f",
  "nonce": 52,
  "gasTipCap": "1000000000",
  "gasFeeCap": "3000000000",
  "gasLimit": 35040
}
{
  "t": "2026-01-13T08:33:13.592971259Z",
  "lvl": "info",

```

```

    "msg": "Created channel",
    "id": "e64dd270f8e0041317b4a096d5f61450",
    "l1Head": "0x4f5138d4be621040dddc9660a9f93dd1fba845309abefc58819acd6d5690e40a:976",
    "blocks_pending": 6,

    "l1OriginLastSubmittedChannel": "0x3a6206b7f319daf971596ae7aeb468f34cd1cf600f1cd39fcf12c8463e6b1cd0:970",
    "batch_type": 0,
    "compression_algo": "zlib",
    "target_num_frames": 1,
    "max_frame_size": 119999,
    "use_blobs": false
  }
  {
    "t": "2026-01-13T08:33:13.593177676Z",
    "lvl": "info",
    "msg": "Transaction successfully published",
    "service": "batcher",
    "tx": "0x6b9c0a12721da080c72c31e00fbd79d2a504d40a003512c0bd5e4c04ce69279f",
    "nonce": 52,
    "gasTipCap": "1000000000",
    "gasFeeCap": "3000000000",
    "gasLimit": 35040,
    "tx": "0x6b9c0a12721da080c72c31e00fbd79d2a504d40a003512c0bd5e4c04ce69279f"
  }
  {
    "t": "2026-01-13T08:33:13.593229426Z",
    "lvl": "info",
    "msg": "Channel closed",
    "id": "e64dd270f8e0041317b4a096d5f61450",
    "blocks_pending": 0,
    "num_frames": 1,
    "input_bytes": 480,
    "output_bytes": 356,
    "oldest_l1_origin": "0x3a6206b7f319daf971596ae7aeb468f34cd1cf600f1cd39fcf12c8463e6b1cd0:970",
    "l1_origin": "0xa58f72113115ef64d3af2cd1dfba1a142c07bda991a8ce95403892c2ef23093a:971",
    "oldest_l2": "0x93f552aa3582ae165d8d02d3c265464b4dc5cab876cd2ade036d916d540c83c0:3078",
    "latest_l2": "0x1c2409df40a8a9497db5de4c3d961850ae7ea89c7ed448d5d63ac9bfb8be85f4:3083",
    "full_reason": "channel full: max channel duration reached",
    "compr_ratio": 0.7416666666666667
  }
  {
    "t": "2026-01-13T08:33:13.593261092Z",
    "lvl": "info",
    "msg": "Building Calldata transaction candidate",

```

```

    "size":357
  }
  {
    "t":"2026-01-13T08:33:15.158824968Z",
    "lvl":"info",
    "msg":"Added L2 block to local state",
    "block":"0x4d470f43e8789c01e0191d7f42b60893ea8695bed1ba967450dc87f4a1ea800f:3084",
    "tx_count":1,
    "time":1768293195
  }
}

```

proposer

workspace

```
cd /rollup/proposer
```

Step 1. copy the files from deployer

command

```
cp ../deployer/.deployer/state.json .
```

Step 2 [skip if already done]. set dispute game implementation

Notes.

- op-deployer manage add-game-type command performs this step
- However, due to errors, on direct commands, this guide shows the manual setup

add-game-type command

```
./op-deployer manage add-game-type -h
```

NAME:

op-deployer manage add-game-type - adds a **new** game type to the chain

USAGE:

op-deployer manage add-game-type [command options]

OPTIONS:

--l1-rpc-url value RPC URL **for** the L1 chain. **Must** be **set for** live chains. **Must** be blank **for** chains deploying to local allocs files. [\$L1_RPC_URL]

--artifacts-locator value Locator **for** artifacts. (**default:** "embedded")
[\$DEPLOYER_ARTIFACTS_LOCATOR]

--l1-proxy-admin-owner-address value Address to use **for as** the proxy admin owner. **Not** compatible **with** the --workdir flag. [\$DEPLOYER_PROXY_ADMIN_OWNER]

--opcm-impl-address value Address **of** the OPCM implementation contract. **Not** compatible **with**

the `--workdir` flag. [`$DEPLOYER_OPCM_IMPL_ADDRESS`]

`--system-config-proxy-address` value Address **of** the SystemConfig proxy contract. **Not** compatible **with** the `--workdir` flag. [`$DEPLOYER_SYSTEM_CONFIG_PROXY_ADDRESS`]

`--op-chain-proxy-admin-address` value Address **of** the OP Chain's proxy admin on L1. **Not** compatible with the `--workdir` flag. [`$DEPLOYER_OP_CHAIN_PROXY_ADMIN_ADDRESS`]

`--delayed-weth-proxy-address` value Address **of** the DelayedWETHProxy contract to include **as** part **of** this game type. **If** not specified, one will be deployed **for** you by the OPCM. [`$DEPLOYER_DELAYED_WETH_PROXY_ADDRESS`]

`--dispute-game-type` value Numeric type identifier **for** the dispute game. (**default:** 1) [`$DEPLOYER_DISPUTE_GAME_TYPE`]

`--dispute-absolute-prestate` value The absolute prestate hash **for** the dispute game. **Defaults** to the standard value. (**default:** "0x038512e02c4c3f7bdaec27d00edf55b7155e0905301e1a88083e4e0a6764d54c") [`$DEPLOYER_DISPUTE_ABSOLUTE_PRESTATE`]

`--dispute-max-game-depth` value Maximum depth **of** the dispute game tree (value **as** string). **Defaults** to the standard value. (**default:** 73) [`$DEPLOYER_DISPUTE_MAX_GAME_DEPTH`]

`--dispute-split-depth` value Depth at which the dispute game tree **splits** (value **as** string). **Defaults** to the standard value. (**default:** 30) [`$DEPLOYER_DISPUTE_SPLIT_DEPTH`]

`--dispute-clock-extension` value Clock extension **in** seconds **for** dispute game timing. **Defaults** to the standard value. (**default:** 10800) [`$DEPLOYER_DISPUTE_CLOCK_EXTENSION`]

`--dispute-max-clock-duration` value Maximum clock duration **in** seconds **for** dispute game timing. **Defaults** to the standard value. (**default:** 302400) [`$DEPLOYER_DISPUTE_MAX_CLOCK_DURATION`]

`--initial-bond` value Initial bond amount required **for** the dispute game (value **as** string, **in** wei). **Defaults** to 1 ETH. (**default:** "1000000000000000000") [`$DEPLOYER_INITIAL_BOND`]

`--vm-address` value Address **of** the VM contract used by the dispute game. [`$DEPLOYER_VM_ADDRESS`]

`--permissionless` Boolean indicating **if** the dispute game should be deployed **in** permissionless mode. (**default:** false) [`$DEPLOYER_PERMISSIONED`]

`--salt-mixer` value String value **for** the salt mixer, used **in** CREATE2 address calculation. **Default** to keccak256("op-stack-contract-impls-salt-v0"). (**default:** "89fca2352a158519d2daabf7e53686272e828ddbff9487204546d918490b2ecf") [`$DEPLOYER_SALT_MIXER`]

`--workdir` value Path to a working directory containing a state file. **Addresses** will be retrieved **from** the state file, and cannot be specified on the command line. [`$DEPLOYER_WORKDIR`]

`--l2-chain-id` value Chain ID **of** the L2 network to retrieve **from** state. **Must** be specified when `--workdir` is **set**. [`$DEPLOYER_CHAIN_ID`]

`--log.level` value The lowest log level that will be **output** (**default:** INFO) [`$DEPLOYER_LOG_LEVEL`]

`--log.format` value Format the log output. **Supported** formats: 'text', 'terminal', 'logfmt', 'json', 'json-pretty', (**default:** text) [`$DEPLOYER_LOG_FORMAT`]

`--log.color` Color the log output **if in** terminal **mode** (**default:** false) [`$DEPLOYER_LOG_COLOR`]

`--log.pid` Show pid **in** the log (**default:** false) [`$DEPLOYER_LOG_PID`]

`--help, -h` show help

errors

- op-deployer error

reason (from contract ABI)

```
|-----+-----+-----|  
|error   | NotImplemented(GameType) | 0x031c6de4  
|-----+-----+-----|
```

- ```
./op-deployer manage add-game-type W
--workdir .deployer W
--l2-chain-id 111 W
--l1-rpc-url $L1_RPC_URL W
--dispute-game-type 0

INFO [01-15|14:41:56.176] Initialized path database readonly=true cache=0.00B buffer=0.00B
history=0
WARN [01-15|14:41:56.427] Fault
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266 label=DummyCaller err="execution reverted" depth=4
op=253 byte4=0x1661a2e9
```

```

WARN [01-15|14:41:56.427] callframe depth=1 byte4=0xa590c30b
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=AddGameType
WARN [01-15|14:41:56.427] callframe depth=2 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffB92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] callframe depth=3 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffB92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] callframe depth=4 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffB92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] Revert
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffB92266 label=DummyCaller err="execution reverted"
revertData=0x depth=3
WARN [01-15|14:41:56.427] Fault
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffB92266 label=DummyCaller err="execution reverted" depth=3
op=253 byte4=0x1661a2e9
WARN [01-15|14:41:56.427] callframe depth=1 byte4=0xa590c30b
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=AddGameType
WARN [01-15|14:41:56.427] callframe depth=2 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffB92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] callframe depth=3 byte4=0x1661a2e9
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffB92266 callsite= label=DummyCaller
WARN [01-15|14:41:56.427] Revert
addr=0xf39Fd6e51aad88F6F4ce6aB8827279cffB92266 label=DummyCaller err="execution reverted"
revertData=0x depth=2
WARN [01-15|14:41:56.427] Fault
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 label=AddGameType err="execution reverted"
depth=1 op=253 byte4=0xa590c30b
WARN [01-15|14:41:56.427] callframe depth=1 byte4=0xa590c30b
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 callsite= label=AddGameType
WARN [01-15|14:41:56.427] Revert
addr=0xfdAf657f7146c1AB9db4d70cEd5FDEa7dC4906c0 label=AddGameType err="execution reverted"
revertMsg="AddGameType: addGameType failed" depth=0
panic
panic: revision id 1 cannot be reverted [recovered]
 panic: revision id 1 cannot be reverted

goroutine 1 [running]:
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*Host).Call.func1()
 /home/circleci/project/op-chain-ops/script/script.go:362 +0x1de
panic({0xd5ee40?, 0xc000194dd0?})
 /data/mise-data/installs/go/1.23.8/src/runtime/panic.go:791 +0x132
github.com/ethereum/go-ethereum/core/state.(*journal).RevertToSnapshot(0xc00026d600, 0x1, 0xc00001e600)
 /home/circleci/go/pkg/mod/github.com/ethereum-optimism/op-geth@v1.101511.1-
dev.1.0.20250710181308-c6e05723600e/core/state/journal.go:91 +0x10e
github.com/ethereum/go-ethereum/core/state.(*StateDB).RevertToSnapshot(0xc0001bd8c0?, 0xc000390d78?)

```

```

/home/circleci/go/pkg/mod/github.com/ethereum-optimism/op-geth@v1.101511.1-
dev.1.0.20250710181308-c6e05723600e/core/state/statedb.go:757 +0x25
github.com/ethereum-optimism/optimism/op-chain-
ops/script/forking.(*ForkableState).RevertToSnapshot(0xc00013ec00?, 0xc000035600?)
/home/circleci/project/op-chain-ops/script/forking/state.go:370 +0x1e
github.com/ethereum/go-ethereum/core/vm.(*EVM).Call(0xc00001e480, {0x44, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0, 0x0,
0x0, ...}, ...)
/home/circleci/go/pkg/mod/github.com/ethereum-optimism/op-geth@v1.101511.1-
dev.1.0.20250710181308-c6e05723600e/core/vm/evm.go:263 +0xb6e
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*wrappedEVM).Call(0x1007f65affd?, {0x44, 0x0,
0x0, 0x0, 0x0, 0x0, 0x0, 0x0, ...}, ...)
/home/circleci/project/op-chain-ops/script/evm.go:42 +0x48
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*Host).Call(0xc00068aa80, {0x44, 0x0, 0x0, 0x0,
0x0, 0x0, 0x0, 0x0, ...}, ...)
/home/circleci/project/op-chain-ops/script/script.go:375 +0x17a
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*forgeScriptBackendImpl).Call(0xc00080cc28,
{0xfd, 0xaf, 0x65, 0x7f, 0x71, 0x46, 0xc1, 0xab, 0x9d, ...}, ...)
/home/circleci/project/op-chain-ops/script/deploy.go:189 +0xda
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*forgeScriptImpl).Call(0xc000257ad0,
{0xc000368c80, 0x264, 0x280})
/home/circleci/project/op-chain-ops/script/deploy.go:257 +0x198
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*deployScriptWithoutOutputImpl[...]).Call(...)
/home/circleci/project/op-chain-ops/script/deploy.go:286
github.com/ethereum-optimism/optimism/op-chain-
ops/script.(*deployScriptWithoutOutputImpl[...]).run(0x47f2ec0, {{0xf3, 0x9f, 0xd6, 0xe5, 0x1a, 0xad, 0x88, 0xf6,
0xf4, ...}, ...})
/home/circleci/project/op-chain-ops/script/deploy.go:307 +0x2b1
github.com/ethereum-optimism/optimism/op-chain-ops/script.(*deployScriptWithOutputImpl[...]).Run(_ {{0xf3,
0x9f, 0xd6, 0xe5, 0x1a, 0xad, 0x88, 0xf6, 0xf4, ...}, ...})
/home/circleci/project/op-chain-ops/script/deploy.go:337 +0xde
github.com/ethereum-optimism/optimism/op-deployer/pkg/deployer/manage.AddGameType({_ , _},
{{0x7ffc96bd2678, 0x16}, {0x47f83d8, 0xc000251f40}, 0xc00013ae70, {0xc0001a79e0, 0x20}, {0xf3, ...}, ...})
/home/circleci/project/op-deployer/pkg/deployer/manage/add_game_type.go:277 +0x6dc
github.com/ethereum-optimism/optimism/op-deployer/pkg/deployer/manage.AddGameTypeCLI(0xc000367900)
/home/circleci/project/op-deployer/pkg/deployer/manage/add_game_type.go:163 +0x738
github.com/urfave/cli/v2.(*Command).Run(0x52183c0, 0xc000367900, {0xc000163b90, 0x9, 0x9})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/command.go:276 +0x7e2
github.com/urfave/cli/v2.(*Command).Run(0xc00047f080, 0xc000367840, {0xc00036cfa0, 0xa, 0xa})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/command.go:269 +0xa65
github.com/urfave/cli/v2.(*Command).Run(0xc00047f340, 0xc0003675c0, {0xc0001c2000, 0xb, 0xb})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/command.go:269 +0xa65
github.com/urfave/cli/v2.(*App).RunContext(0xc0000fe800, {0x47ebcd8, 0x5255a20}, {0xc0001c2000, 0xb, 0xb})
/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/app.go:333 +0x5a5
github.com/urfave/cli/v2.(*App).Run(0xc00003fe00?, {0xc0001c2000?, 0x0?, 0x68?})

```

```

/home/circleci/go/pkg/mod/github.com/urfave/cli/v2@v2.27.6/app.go:307 +0x2f
main.main()
/home/circleci/project/op-deployer/cmd/op-deployer/main.go:25 +0xa8

```

### Get the GameFactoryAddress

```

GAME_FACTORY_ADDRESS=$(cat state.json | jq -r '.opChainDeployments[0].DisputeGameFactoryProxy')
echo "DisputeGameFactory Address: $GAME_FACTORY_ADDRESS"

// result
DisputeGameFactory Address: 0x8e1e505274f5054200b3d26d7ebc75722038ce8f

```

### Get the implementation address

```

GAME_FACTORY_IMPL=$(cat state.json | jq -r '.opChainDeployments[0].PermissionedDisputeGameImpl')
echo "Game Factory Implementation Address: $GAME_FACTORY_IMPL"

// result
Game Factory Implementation Address: 0x19a9b6f4afaff0099d14e2804a0c2e9e8a7b507d

```

### Make sure the contract owner is correct

```

L1=http://127.0.0.1:38545
cast call $GAME_FACTORY_ADDRESS "owner()(address)" --rpc-url $L1

// result
0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266

```

### set implementation tx

```

cast send ₩
--rpc-url $L1_RPC_URL ₩
--private-key $DEPLOYER_PRIVATE_KEY ₩
$GAME_FACTORY_ADDRESS ₩
"setImplementation(uint32,address)" ₩
0 ₩
$GAME_FACTORY_IMPL

```

### result

```

blockHash 0x6466851c02546ea5abf6d211e80af312c1d606d993365a89eff02fe14200c5f2
blockNumber 16919
contractAddress
cumulativeGasUsed 52816
effectiveGasPrice 1000000007
from 0xf39Fd6e51aad88F6F4ce6aB8827279cFfB92266
gasUsed 52816
logs
[{"address":"0x8e1e505274f5054200b3d26d7ebc75722038ce8f","topics":["0xff513d80e2c7fa487608f70a618dfbc0c
f415699dc69588c747e8c71566c88de","0x000000000000000000000000000000000019a9b6f4afaff0099d14e2804a0c2e9e8a7b
507d","0x00"],"data":"0x","blockHash":
"0x6466851c02546ea5abf6d211e80af312c1d606d993365a89eff02fe14200c5f2","blockNumber":"0x4217","blockTi

```



```
OP_PROPOSER_PRIVATE_KEY=ac0974bec39a17e36ba4a6b4d238ff944bacb478cbcd5efcae784d7bf4f2ff80

OP Stack proposer configuration (optional - defaults provided)
OP_PROPOSER_PROPOSAL_INTERVAL=3600s
OP_PROPOSER_GAME_TYPE=0
OP_PROPOSER_POLL_INTERVAL=20s
OP_PROPOSER_ALLOW_NON_FINALIZED=true
OP_PROPOSER_WAIT_NODE_SYNC=true
```

*init*

```
source .env
```

## Step 5. init docker compose

*docker-compose.yml file*

```
networks:
 op-stack:
 external: true

services:
 op-proposer:
 image: us-docker.pkg.dev/oplabs-tools-artifacts/images/op-proposer:v1.10.0
 container_name: op-proposer
 networks:
 - op-stack
 volumes:
 - ./workspace
 working_dir: /workspace
 ports:
 - "8560:8560"
 env_file:
 - .env
 command: >
 op-proposer
 --rpc.port=8560
 --log.level=info
 --log.format=json
 restart: unless-stopped
```

## Step 6. run the proposer

*command*

```
docker compose up -d
```

*result*

```
docker logs op-proposer -f
```

```

{"t":"2026-01-15T05:14:47.535915052Z","lvl":"info","msg":"Initializing L2Output Submitter"}
{"t":"2026-01-15T05:14:47.563864136Z","lvl":"info","msg":"Metrics disabled"}
{"t":"2026-01-15T05:14:47.572581219Z","lvl":"info","msg":"Connected to
DisputeGameFactory","address":"0x8E1E505274F5054200B3D26d7EBc75722038cE8f","version":"1.2.0"}
{"t":"2026-01-15T05:14:47.574690552Z","lvl":"info","msg":"Starting JSON-RPC server"}
{"t":"2026-01-15T05:14:47.586028469Z","lvl":"info","msg":"Started RPC server","endpoint":"http://[::]:8560"}
{"t":"2026-01-15T05:14:47.586054094Z","lvl":"info","msg":"Starting Proposer"}
{"t":"2026-01-15T05:14:47.586056594Z","lvl":"info","msg":"Starting Proposer"}
{"t":"2026-01-15T05:14:47.591726969Z","lvl":"info","msg":"rollup current L1 block still behind target,
retrying","current_l1":"0x22616f69c00e008784e535a2b2874f23d7d1ece105d5492af778fe009d4a1f1e:17061","targ
et_l1":17064}
{"t":"2026-01-15T05:14:59.595846085Z","lvl":"info","msg":"rollup current L1 block still behind target,
retrying","current_l1":"0xbb28bb4c4f3212b82ee82bcde5c7e6c694f9a3d35bec21045091d376c3248ba:17063","tar
get_l1":17064}
{"t":"2026-01-15T05:15:11.598397383Z","lvl":"info","msg":"rollup current L1 block target
reached","current_l1":"0xaae44679690c2dda52b0ae958a2943f80f2816af075b54e284a8b93060a9d798:17064","tar
get_l1":17064}
{"t":"2026-01-15T05:15:11.598438758Z","lvl":"info","msg":"Proposer started"}
{"t":"2026-01-15T05:15:31.605541628Z","lvl":"info","msg":"No proposals found for at least proposal interval,
submitting proposal now","proposalInterval":"1h0m0s"}
{"t":"2026-01-15T05:15:31.605574045Z","lvl":"info","msg":"Proposing output
root","output":"0x18bd7b560c145413035b1610275ac9cdd31dafc269ddb7a85e1d41fb385e5589","block":83531}
{"t":"2026-01-15T05:15:31.615378295Z","lvl":"info","msg":"Publishing
transaction","service":"proposer","tx":"0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6
def6","nonce":13433,"gasTipCap":"1000000000","gasFeeCap":"3000000000","gasLimit":400481}
{"t":"2026-01-15T05:15:31.616697211Z","lvl":"info","msg":"Transaction successfully
published","service":"proposer","tx":"0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6d
ef6","nonce":13433,"gasTipCap":"1000000000","gasFeeCap":"3000000000","gasLimit":400481,"tx":"0x3add71648a
4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6def6"}
{"t":"2026-01-15T05:17:07.619419756Z","lvl":"info","msg":"Transaction
confirmed","service":"proposer","tx":"0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6d
ef6","block":"0x976dacf11f44a5f169228d2ca26d0245dd856f8fded547b0d4b5dd2f94de55a:17069","effectiveGasP
rice":"1000000007"}
{"t":"2026-01-15T05:17:07.619597298Z","lvl":"info","msg":"Proposer tx successfully
published","tx_hash":"0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6def6","l1blocknu
m":17066,"l1blockhash":"0x51fc5498582f3f933c09e7976b754f6214ac9af22f0177e9346a9ddef7200242"}

```

- lifecycle

```

{
 "t":"2026-01-15T05:15:11.598438758Z",
 "lvl":"info",
 "msg":"Proposer started"
}
{
 "t":"2026-01-15T05:15:31.605541628Z",

```

```

 "lvl": "info",
 "msg": "No proposals found for at least proposal interval, submitting proposal now",
 "proposalInterval": "1h0m0s"
 }
 {
 "t": "2026-01-15T05:15:31.605574045Z",
 "lvl": "info",
 "msg": "Proposing output root",
 "output": "0x18bd7b560c145413035b1610275ac9cdd31dafc269ddb7a85e1d41fb385e5589",
 "block": 83531
 }
 {
 "t": "2026-01-15T05:15:31.615378295Z",
 "lvl": "info",
 "msg": "Publishing transaction",
 "service": "proposer",
 "tx": "0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6def6",
 "nonce": 13433,
 "gasTipCap": "1000000000",
 "gasFeeCap": "3000000000",
 "gasLimit": 400481
 }
 {
 "t": "2026-01-15T05:15:31.616697211Z",
 "lvl": "info",
 "msg": "Transaction successfully published",
 "service": "proposer",
 "tx": "0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6def6",
 "nonce": 13433,
 "gasTipCap": "1000000000",
 "gasFeeCap": "3000000000",
 "gasLimit": 400481,
 "tx": "0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6def6"
 }
 {
 "t": "2026-01-15T05:17:07.619419756Z",
 "lvl": "info",
 "msg": "Transaction confirmed",
 "service": "proposer",
 "tx": "0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6def6",
 "block": "0x976dacf11f44a5f169228d2ca26d0245dd856f8fded547b0d4b5dd2f94de55a:17069",
 "effectiveGasPrice": "1000000007"
 }
 {

```

```
{
 "t": "2026-01-15T05:17:07.619597298Z",
 "lvl": "info",
 "msg": "Proposer tx successfully published",
 "tx_hash": "0x3add71648a4ba59cdf15fb6197e7404b2ee831ccc709534d6d377837eae6def6",
 "l1blocknum": 17066,
 "l1blockhash": "0x51fc5498582f3f933c09e7976b754f6214ac9af22f0177e9346a9ddef7200242"
}
```

---

## challenger

### workspace

cd /rollup/challenger

### Step 1. copy the files from deployer

#### command

```
cp ../deployer/.deployer/rollup.json .
cp ../deployer/.deployer/genesis.json .
cp ../deployer/.deployer/state.json .
```

### Step 2. extract GameFactory address

#### command

```
GAME_FACTORY_ADDRESS=$(cat state.json | jq -r '.opChainDeployments[0].DisputeGameFactoryProxy')
echo "DisputeGameFactory Address: $GAME_FACTORY_ADDRESS"
```

#### output

```
DisputeGameFactory Address: 0x8e1e505274f5054200b3d26d7ebc75722038ce8f
```

### Step 3. generate absolute prestate

#### clone the optimism repo

```
git clone https://github.com/ethereum-optimism/optimism.git
cd optimism

git checkout op-deployer/v0.4.7
```

#### copy the genesis and rollup

```
cp ~/Desktop/research/opstack/deployer/.deployer/rollup.json op-program/chainconfig/configs/111-rollup.json
cp ~/Desktop/research/opstack/deployer/.deployer/genesis.json op-program/chainconfig/configs/111-genesis-
l2.json
```

#### generate prestate

```
make reproducible-prestate
```

```
// result
```

```

...
----- Production Prestates -----

Cannon64 Absolute prestate hash:
0x038325f39e0e4db3292d31b249133b501a369c9368c86d87d74aac9fb1b50d2f

----- Experimental Prestates -----
...

```

#### *get the files*

```

cd op-program/bin
mv prestate-mt64.bin.gz 0x038325f39e0e4db3292d31b249133b501a369c9368c86d87d74aac9fb1b50d2f.bin.gz

// copy this file to challenger directory
cp 0x038325f39e0e4db3292d31b249133b501a369c9368c86d87d74aac9fb1b50d2f.bin.gz
~/Desktop/research/opstack/challenger

```

#### Step 4. .env file

```

OP_CHALLENGER_L1_RPC_URL=https://sepolia.infura.io/v3/YOUR_ACTUAL_INFURA_KEY
OP_CHALLENGER_L1_ETH_RPC=https://eth-sepolia.g.alchemy.com/v2/r2EhlwzzhVy1dWeYGgig9
OP_CHALLENGER_L1_BEACON=https://ethereum-sepolia-beacon-api.publicnode.com
OP_CHALLENGER_PRIVATE_KEY=482cd19fe6b4652d57923881c07e9f5ddf278771bdb52239d3014738d11d4ca0

L2 Configuration - Replace with your actual node endpoints
OP_CHALLENGER_L2_ETH_RPC=http://op-geth:8545
OP_CHALLENGER_ROLLUP_RPC=http://sequencer-node:8547

OP Stack challenger configuration (optional - defaults provided)
OP_CHALLENGER_GAME_FACTORY_ADDRESS=0xfa0b1aaa8d042fa4a02832d6721fba84c1050000
OP_CHALLENGER_CANNON_L2_GENESIS=genesis.json
OP_CHALLENGER_CANNON_ROLLUP_CONFIG=rollup.json

PRESTATE_HASH=0x038325f39e0e4db3292d31b249133b501a369c9368c86d87d74aac9fb1b50d2f

Prestate configuration - Replace with the file from 'make reproducible-prestate'
OP_CHALLENGER_CANNON_PRESTATE=0x038325f39e0e4db3292d31b249133b501a369c9368c86d87d74aac9fb1
b50d2f.bin.gz

```

#### *init*

```
source .env
```

#### Step 5. init docker compose

```

networks:
 op-stack:

```

```
external: true
```

```
services:
```

```
 challenger:
```

```
 image: us-docker.pkg.dev/oplabs-tools-artifacts/images/op-challenger:v1.5.1
```

```
 container_name: op-challenger
```

```
 networks:
```

```
 - op-stack
```

```
 user: "1000"
```

```
 volumes:
```

```
 - ./challenger-data:/data
```

```
 - ./rollup.json:/workspace/rollup.json:ro
```

```
 - ./genesis-l2.json:/workspace/genesis-l2.json:ro
```

```
 - ./prestate-proof-mt64.json:/workspace/prestate-proof.json:ro
```

```
 - /${PRESTATE_HASH}.bin.gz:/workspace/${PRESTATE_HASH}.bin.gz:ro
```

```
 env_file:
```

```
 - .env
```

```
 command: >
```

```
 op-challenger run-trace
```

```
 --trace-type=cannon
```

```
 --datadir=/data
```

```
 --log.level=info
```

```
 --log.format=json
```

```
 restart: unless-stopped
```

## Step 6. run the node

```
docker-compose up -d
```

*result*

=====

## CLI

---

### Fund L2

#### workspace

~/rollup/deployer

```
L1_RPC_URL=http://127.0.0.1:38545
```

```
L2_RPC_URL=http://127.0.0.1:18545
```

## get Optimism Portal address

```
export OPTIMISM_PORTAL=$(jq -r '.opChainDeployments[0].OptimismPortalProxy' ./deployer/state.json)
echo $OPTIMISM_PORTAL
```

## get L1 standard bridge address

```
export L1_STANDARD_BRIDGE=$(jq -r '.opChainDeployments[0].L1StandardBridgeProxy' ./deployer/state.json)
echo $L1_STANDARD_BRIDGE
```

## get new priv key with balance on L1 (which is not being used by L2)

```
PRIV_KEY=59c6995e998f97a5a0044966f0945389dc9e86dae88c7a8412f4603b6b78690d
ADDR=0x70997970C51812dc3A010C7d01b50e0d17dc79C8
```

*// Note*

address belongs to the **private** key

## deposit through smart contract

```
cast send ₩
--rpc-url $L1_RPC_URL ₩
--private-key $PRIV_KEY ₩
--value 1ether ₩
$OPTIMISM_PORTAL ₩
"depositTransaction(address,uint256,uint64,bool,bytes)" ₩
$ADDR ₩
0 ₩
200000 ₩
false ₩
0x
```

## result

```
cast send ₩
--rpc-url $L1_RPC_URL ₩
--private-key $PRIV_KEY ₩
--value 1ether ₩
$OPTIMISM_PORTAL ₩
"depositTransaction(address,uint256,uint64,bool,bytes)" ₩
$ADDR ₩
0 ₩
200000 ₩
false ₩
0x

// output
blockHash 0xa7be715e76a7bd89727d0208cc9e8903dd0f6796f07fe968987f223c1ae888ab
```

[illegible]

## confirm

```
cast balance $ADDR --rpc-url $L2_RPC_URL
```

```
// result
```

100000000000000000000

tx on L2

initial balance

```
cast balance 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 --rpc-url $L2_RPC_URL
```

```
// result
```

0

comand

```
cast send ₩
--rpc-url $L2_RPC_URL ₩
--private-key $PRIV ₩
0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 ₩
--value 0.5ether
```

result

|                     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|---------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| blockHash           | 0x6183b8b3dfd3fe1d9fc4ff5f6f7e11da0fa8f91944c09ed36f69df1cd864a70b                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| blockNumber         | 1841                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| contractAddress     |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| cumulativeGasUsed   | 67096                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| effectiveGasPrice   | 646612                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |
| from                | 0x70997970C51812dc3A010C7d01b50e0d17dc79C8                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| gasUsed             | 21000                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                        |
| logs                | []                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| logsBloom           | 0x0000000000000000000000000000000000000000000000000000000000000000000000000000000000000<br>0000000000000000000000000000000000000000000000000000000000000000000000000000000000000<br>0000000000000000000000000000000000000000000000000000000000000000000000000000000000000<br>0000000000000000000000000000000000000000000000000000000000000000000000000000000000000<br>0000000000000000000000000000000000000000000000000000000000000000000000000000000000000<br>0000000000000000000000000000000000000000000000000000000000000000000000000000000000000<br>root |
| status              | 1 (success)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                  |
| transactionHash     | 0x84656bc1c1647077753682862588cec3b5c62d17413808c4aebdbf01b63942b1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                           |
| transactionIndex    | 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| type                | 2                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| blobGasPrice        |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| blobGasUsed         |                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
| to                  | 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                   |
| l1BaseFeeScalar     | 1368                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                         |
| l1BlobBaseFee       | 1                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                            |
| l1BlobBaseFeeScalar | 810949                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                       |

|            |      |
|------------|------|
| l1Fee      | 96   |
| l1GasPrice | 7    |
| l1GasUsed  | 1600 |

## after balance

```
cast balance 0xa0Ee7A142d267C1f36714E4a8F75612F20a79720 --rpc-url $L2_RPC_URL
```

```
// result
```

```
500000000000000000
```